

Babel

Code

Version 26.13.137466
2026/10/05

Javier Bezos
Current maintainer

Johannes L. Braams
Original author

Localization and
internationalization

Unicode

T_EX

LuaT_EX

pdfT_EX

XeT_EX

Contents

1	Identification and loading of required files	3
2	locale directory	3
3	Tools	3
3.1	A few core definitions	8
3.2	$\LaTeX$: babel.sty (start)	8
3.3	base	10
3.4	key=value options and other general option	10
3.5	Post-process some options	12
3.6	Plain: babel.def (start)	13
4	babel.sty and babel.def (common)	14
4.1	Selecting the language	16
4.2	Errors	24
4.3	More on selection	25
4.4	Short tags	26
4.5	Compatibility with language.def	26
4.6	Hooks	27
4.7	Setting up language files	28
4.8	Shorthands	30
4.9	Language attributes	39
4.10	Support for saving and redefining macros	40
4.11	French spacing	42
4.12	Hyphens	42
4.13	Multiencoding strings	44
4.14	Tailor captions	49
4.15	Making glyphs available	50
4.15.1	Quotation marks	50
4.15.2	Letters	52
4.15.3	Shorthands for quotation marks	52
4.15.4	Umlauts and tremas	53
4.16	Layout	55
4.17	Load engine specific macros	55
4.18	Creating and modifying languages	55
4.19	Main loop in ‘provide’	63
4.20	Processing keys in ini	68
4.21	French spacing (again)	74
4.22	Handle language system	75
4.23	Numerals	76
4.24	Casing	77
4.25	Getting info	78
4.26	BCP 47 related commands	79
5	Adjusting the Babel behavior	80
5.1	Cross referencing macros	83
5.2	Layout	85
5.3	Marks	87
5.4	Other packages	88
5.4.1	ifthen	88
5.4.2	varioref	88
5.4.3	hhline	89
5.5	Encoding and fonts	89
5.6	Basic bidi support	91
5.7	Local Language Configuration	94
5.8	Language options	95

6	Tagging	99
7	The kernel of Babel	99
8	Error messages	100
9	Loading hyphenation patterns	104
10	luatex + xetex: common stuff	108
11	Hooks for XeTeX and LuaTeX	111
11.1	XeTeX	111
11.2	Support for interchar	113
11.3	Layout	115
11.4	8-bit TeX	116
11.5	LuaTeX	117
11.6	Southeast Asian scripts	124
11.7	CJK line breaking	125
11.8	Arabic justification	127
11.9	Common stuff	131
11.10	Automatic fonts and ids switching	132
11.11	Bidi	139
11.12	Layout	141
11.13	Lua: transforms	150
11.14	Lua: Auto bidi with basic and basic-r	160
12	Data for CJK	173
13	The ‘nil’ language	173
14	Calendars	174
14.1	Islamic	175
14.2	Hebrew	176
14.3	Persian	180
14.4	Coptic and Ethiopic	181
14.5	Julian	181
14.6	Buddhist	182
15	Support for Plain TeX (plain.def)	183
15.1	Not renaming hyphen.tex	183
15.2	Emulating some L ^A T _E X features	184
15.3	General tools	184
15.4	Encoding related macros	188
16	Acknowledgements	191

The babel package is being developed incrementally, which means parts of the code are under development and therefore incomplete. Only documented features are considered complete. In other words, use babel in real documents only as documented (except, of course, if you want to explore and test them).

1. Identification and loading of required files

The babel package after unpacking consists of the following files:

babel.sty is the $\LaTeX$ package, which set options and load language styles.

babel.def is loaded by Plain.

switch.def defines macros to set and switch languages (it loads part babel.def).

plain.def is not used, and just loads babel.def, for compatibility.

hyphen.cfg is the file to be used when generating the formats to load hyphenation patterns.

There some additional tex, def and lua files.

The babel installer extends docstrip with a few “pseudo-guards” to set “variables” used at installation time. They are used with `<@name@>` at the appropriate places in the source code and defined with either `{(name=value)}`, or with a series of lines between `{(*name)}` and `{(/name)}`. The latter is cumulative (e.g., with *More package options*). That brings a little bit of literate programming. The guards `<-name>` and `<+name>` have been redefined, too. See babel.ins for further details.

2. locale directory

A required component of babel is a set of ini files with basic definitions for about 300 languages. They are distributed as a separate zip file, not packed as dtx. Many of them are essentially finished (except bugs and mistakes, of course). Some of them are still incomplete (but they will be usable), and there are some omissions (e.g., there are no geographic areas in Spanish). Not all include LICR variants.

babel-*.ini files contain the actual data; babel-*.tex files are basically proxies to the corresponding ini files.

See [Keys in ini files](#) in the the babel site.

3. Tools

```
1 <<version=26.13.137466>>
2 <<date=2026/10/05>>
```

Do not use the following macros in ldf files. They may change in the future. This applies mainly to those recently added for replacing, trimming and looping. The older ones, like `\bbl@afterfi`, will not change. We define some basic macros which just make the code cleaner. `\bbl@add` is now used internally instead of `\addto` because of the unpredictable behavior of the latter. Used in babel.def and in babel.sty, which means in $\LaTeX$ is executed twice, but we need them when defining options and babel.def cannot be load until options have been defined. This does not hurt, but should be fixed somehow.

```
3 <<(*Basic macros)>> ≡
4 \bbl@trace{Basic macros}
5 \def\bbl@stripslash{\expandafter\@gobble\string}
6 \def\bbl@add#1#2{%
7   \bbl@ifunset{\bbl@stripslash#1}%
8   {\def#1{#2}}%
9   {\expandafter\def\expandafter#1\expandafter{#1#2}}}
10 \def\bbl@xin@{\@expandtwoargs\in@}
11 \def\bbl@carg#1#2{\expandafter#1\csname#2\endcsname}%
12 \def\bbl@ncarg#1#2#3{\expandafter#1\expandafter#2\csname#3\endcsname}%
13 \def\bbl@ccarg#1#2#3{%
14   \expandafter#1\csname#2\expandafter\endcsname\csname#3\endcsname}%
15 \def\bbl@carg#1#2{\expandafter#1\csname bbl@#2\endcsname}%
16 \def\bbl@carg#1{\csname bbl@#1\endcsname}
17 \def\bbl@carg#1{\csname bbl@#1\language\endcsname}
18 \def\bbl@loop#1#2#3{\bbl@loop#1{#3}#2,\@nnil,}
```

```

19 \def\bbl@loopx#1#2{\expandafter\bbl@loop\expandafter#1\expandafter{#2}}
20 \def\bbl@loop#1#2#3,{%
21   \ifx\@nnil#3\relax\else
22     \def#1{#3}#2\bbl@afterfi\bbl@loop#1{#2}%
23   \fi}
24 \def\bbl@for#1#2#3{\bbl@loopx#1{#2}{\ifx#1\@empty\else#3\fi}}

```

\bbl@add@list This internal macro adds its second argument to a comma separated list in its first argument. When the list is not defined yet (or empty), it will be initiated. It presumes expandable character strings.

```

25 \def\bbl@add@list#1#2{%
26   \edef#1{%
27     \bbl@ifunset{\bbl@stripslash#1}%
28     }%
29     {\ifx#1\@empty\else#1,\fi}%
30   #2}}

```

\bbl@afterelse

\bbl@afterfi Because the code that is used in the handling of active characters may need to look ahead, we take extra care to ‘throw’ it over the \else and \fi parts of an \if-statement¹. These macros will break if another \if... \fi statement appears in one of the arguments and it is not enclosed in braces.

```

31 \long\def\bbl@afterelse#1\else#2\fi{\fi#1}
32 \long\def\bbl@afterfi#1\fi{\fi#1}

```

\bbl@exp Now, just syntactical sugar, but it makes partial expansion of some code a lot more simple and readable. Here \ stands for \noexpand, \<.> for \noexpand applied to a built macro name (which does not define the macro if undefined to \relax, because it is created locally), and \[. .] for one-level expansion (where . . is the macro name without the backslash). The result may be followed by extra arguments, if necessary.

```

33 \def\bbl@exp#1{%
34   \begingroup
35     \let\<\noexpand
36     \let\<\bbl@exp@en
37     \let\[\bbl@exp@ue
38     \edef\bbl@exp@aux{\endgroup#1}%
39     \bbl@exp@aux}
40 \def\bbl@exp@en#1>{\expandafter\noexpand\csname#1\endcsname}%
41 \def\bbl@exp@ue#1]{%
42   \unexpanded\expandafter\expandafter\expandafter{\csname#1\endcsname}}%

```

\bbl@trim The following piece of code is stolen (with some changes) from keyval, by David Carlisle. It defines two macros: \bbl@trim and \bbl@trim@def. The first one strips the leading and trailing spaces from the second argument and then applies the first argument (a macro, \toks@ and the like). The second one, as its name suggests, defines the first argument as the stripped second argument.

```

43 \def\bbl@tempa#1{%
44   \long\def\bbl@trim##1##2{%
45     \futurelet\bbl@trim@a\bbl@trim@c##2\@nil\@nil#1\@nil\relax{##1}}%
46   \def\bbl@trim@c{%
47     \ifx\bbl@trim@a\@sptoken
48       \expandafter\bbl@trim@b
49     \else
50       \expandafter\bbl@trim@b\expandafter#1%
51     \fi}%
52   \long\def\bbl@trim@b#1##1 \@nil{\bbl@trim@i##1}}
53 \bbl@tempa{ }
54 \long\def\bbl@trim@i#1\@nil#2\relax#3{#3{#1}}
55 \long\def\bbl@trim@def#1{\bbl@trim{\def#1}}

```

¹This code is based on code presented in TUGboat vol. 12, no2, June 1991 in “An expansion Power Lemma” by Sonja Maus.

\bbl@ifunset To check if a macro is defined, we create a new macro, which does the same as `\ifundefined`. However, in an ϵ -tex engine, it is based on `\ifcurname`, which is more efficient, and does not waste memory. Defined inside a group, to avoid `\ifcurname` being implicitly set to `\relax` by the `\curname` test.

```

56 \begingroup
57 \gdef\bbl@ifunset#1{%
58   \expandafter\ifx\curname#1\endcurname\relax
59   \expandafter\@firstoftwo
60   \else
61   \expandafter\@secondoftwo
62   \fi}
63 \bbl@ifunset{ifcurname}%
64 {}%
65 {\gdef\bbl@ifunset#1{%
66   \ifcurname#1\endcurname
67   \expandafter\ifx\curname#1\endcurname\relax
68   \bbl@afterelse\expandafter\@firstoftwo
69   \else
70   \bbl@afterfi\expandafter\@secondoftwo
71   \fi
72   \else
73   \expandafter\@firstoftwo
74   \fi}}
75 \endgroup

```

\bbl@ifblank A tool from url, by Donald Arseneau, which tests if a string is empty or space. The companion macros tests if a macro is defined with some ‘real’ value, i.e., not `\relax` and not empty,

```

76 \def\bbl@ifblank#1{%
77   \bbl@ifblank@i#1\@nil\@nil\@secondoftwo\@firstoftwo\@nil}
78 \long\def\bbl@ifblank@i#1#2\@nil#3#4#5\@nil#4#5%
79 \def\bbl@ifset#1#2#3{%
80   \bbl@ifunset{#1}{#3}{\bbl@exp{\bbl@ifblank{\@nameuse{#1}}}{#3}{#2}}}

```

For each element in the comma separated `<key>=<value>` list, execute `<code>` with #1 and #2 as the key and the value of current item (trimmed). In addition, the item is passed verbatim as #3. With the `<key>` alone, it passes `\@empty` as value (i.e., the macro thus named, not an empty argument, which is what you get with `<key>=` and no value).

```

81 \def\bbl@forkv#1#2{%
82   \def\bbl@kvcmd##1##2##3{#2}%
83   \bbl@kvnext#1,\@nil,}
84 \def\bbl@kvnext#1,{%
85   \ifx\@nil#1\relax\else
86   \bbl@ifblank{#1}{\bbl@forkv@eq#1=\@empty=\@nil{#1}}%
87   \expandafter\bbl@kvnext
88   \fi}
89 \def\bbl@forkv@eq#1=#2=#3\@nil#4{%
90   \bbl@trim\def\bbl@forkv@a{#1}%
91   \bbl@trim{\expandafter\bbl@kvcmd\expandafter{\bbl@forkv@a}}{#2}{#4}}

```

A *for* loop. Each item (trimmed) is #1. It cannot be nested (it’s doable, but we don’t need it).

```

92 \def\bbl@vforeach#1#2{%
93   \def\bbl@forcmd##1{#2}%
94   \bbl@fornext#1,\@nil,}
95 \def\bbl@fornext#1,{%
96   \ifx\@nil#1\relax\else
97   \bbl@ifblank{#1}{\bbl@trim\bbl@forcmd{#1}}%
98   \expandafter\bbl@fornext
99   \fi}
100 \def\bbl@foreach#1{\expandafter\bbl@vforeach\expandafter{#1}}

```

Some code should be executed once. The first argument is a flag.

```

101 \global\let\bbl@done\@empty

```

```

102 \def\bbl@once#1#2{%
103   \bbl@xin@{,#1,}{,\bbl@done,}%
104   \ifin@ \else
105     #2%
106   \xdef\bbl@done{\bbl@done,#1,}%
107   \fi}
108 %   \end{macrodef}
109 %
110 % \macro{\bbl@replace}
111 %
112 % Returns implicitly |\toks@| with the modified string.
113 %
114 %   \begin{macrocode}
115 \def\bbl@replace#1#2#3{% in #1 -> repl #2 by #3
116   \toks@{ }%
117   \def\bbl@replace@aux##1#2##2#2{%
118     \ifx\bbl@nil##2%
119       \toks@\expandafter{\the\toks@##1}%
120     \else
121       \toks@\expandafter{\the\toks@##1#3}%
122       \bbl@afterfi
123       \bbl@replace@aux##2#2%
124     \fi}%
125   \expandafter\bbl@replace@aux#1#2\bbl@nil#2%
126   \edef#1{\the\toks@}}

```

An extension to the previous macro. It takes into account the parameters, and it is string based (i.e., if you replace elax by ho, then \relax becomes \rho). No checking is done at all, because it is not a general purpose macro, and it is used by babel only when it works (an example where it does *not* work is in \bbl@TG@@date, and also fails if there are macros with spaces, because they are retokenized). It may change! (or even merged with \bbl@replace; I'm not sure checking the replacement is really necessary or just paranoia).

```

127 \ifx\detokenize\undefined \else % Unused macros if old Plain TeX
128   \bbl@exp{\def\\bbl@parsedef##1\detokenize{macro:}}#2->#3\relax}%
129   \def\bbl@tempa{#1}%
130   \def\bbl@tempb{#2}%
131   \def\bbl@tempe{#3}}
132 \def\bbl@sreplace#1#2#3{%
133   \begingroup
134     \expandafter\bbl@parsedef\meaning#1\relax
135     \def\bbl@tempc{#2}%
136     \edef\bbl@tempc{\expandafter\strip@prefix\meaning\bbl@tempc}%
137     \def\bbl@tempd{#3}%
138     \edef\bbl@tempd{\expandafter\strip@prefix\meaning\bbl@tempd}%
139     \bbl@xin@{\bbl@tempc}{\bbl@tempe}% If not in macro, do nothing
140     \ifin@
141       \bbl@exp{\\bbl@replace\\bbl@tempe{\bbl@tempc}{\bbl@tempd}}%
142       \def\bbl@tempc{% Expanded an executed below as 'uplevel'
143         \\makeatletter % "internal" macros with @ are assumed
144         \\scantokens{%
145           \bbl@tempa\\@namedef{\bbl@stripslash#1}\bbl@tempb{\bbl@tempe}%
146           \noexpand\noexpand}%
147         \catcode64=\the\catcode64\relax}% Restore @
148     \else
149       \let\bbl@tempc\empty % Not \relax
150     \fi
151     \bbl@exp{% For the 'uplevel' assignments
152   \endgroup
153   \bbl@tempc}} % empty or expand to set #1 with changes
154 \fi

```

Two further tools. \bbl@ifsamestring first expand its arguments and then compare their expansion (sanitized, so that the catcodes do not matter). \bbl@engine takes the following values: 0 is pdf_{La}TeX, 1 is luatex, and 2 is xetex. You may use the latter it in your language style if you want.

```

155 \def\bb@ifsamestring#1#2{%
156   \begingroup
157   \protected@edef\bb@tempb{#1}%
158   \edef\bb@tempb{\expandafter\strip@prefix\meaning\bb@tempb}%
159   \protected@edef\bb@tempc{#2}%
160   \edef\bb@tempc{\expandafter\strip@prefix\meaning\bb@tempc}%
161   \ifx\bb@tempb\bb@tempc
162     \aftergroup\@firstoftwo
163   \else
164     \aftergroup\@secondoftwo
165   \fi
166 \endgroup}
167 \chardef\bb@engine=%
168 \ifx\directlua\@undefined
169   \ifx\XeTeXinputencoding\@undefined
170     \z@
171   \else
172     \tw@
173   \fi
174 \else
175   \@ne
176 \fi

```

A somewhat hackish tool (hence its name) to avoid spurious spaces in some contexts.

```

177 \def\bb@bsphack{%
178   \ifhmode
179     \hskip\z@skip
180     \def\bb@esphack{\loop\ifdim\lastskip>\z@\unskip\repeat\unskip}%
181   \else
182     \let\bb@esphack\@empty
183   \fi}

```

Another hackish tool, to apply case changes inside a protected macros. It's based on the internal \let's made by \MakeUppercase and \MakeLowercase between things like \oe and \OE.

```

184 \def\bb@cased{%
185   \ifx\oe\OE
186     \expandafter\in@\expandafter
187     {\expandafter\OE\expandafter}\expandafter{\oe}%
188   \ifin@
189     \bb@afterelse\expandafter\MakeUppercase
190   \else
191     \bb@afterfi\expandafter\MakeLowercase
192   \fi
193 \else
194   \expandafter\@firstofone
195 \fi}

```

Ordered key/value lists (aka associative arrays). The setter either replaces the current value if the key exists, or adds the new key/value at the end. The getter returns either the value if the key exists or \relax. The code is based on arguments delimited by the keys. In the getter, note the value is supposed not to start with \@empty.

```

196 \def\bb@array@set#1#2#3{%
197   \def\bb@array@aux##1\bb@elt#2\@##2##3\bb@array@aux##4{%
198     \ifx\bb@noelt##2%
199       \def#1{##1\bb@elt#2\@{##4}}%
200     \else
201       \def\bb@array@aux####1\bb@elt#2\@####2####3\bb@array@aux####4{%
202         \def#1{####1\bb@elt#2\@{####4}####3}}%
203       \expandafter\bb@array@aux#1\bb@array@aux{#3}%
204     \fi}%
205   \expandafter\bb@array@aux#1\bb@elt#2\@bb@noelt\bb@array@aux{#3}}
206 \def\bb@array@get#1#2#3{%
207   \def\bb@array@aux##1\bb@elt#2\@##2##3\bb@array@aux{%
208     \ifx\@empty##2\let#3\relax\else\def#3{##2}\fi}%

```

```
209 \expandafter\bbbl@array@aux#1\bbbl@elt#2\@empty\bbbl@array@aux}
```

The following adds some code to `\extras...` both before and after, while avoiding doing it twice. It's somewhat convoluted, to deal with #'s. Used to deal with alph, Alph and frenchspacing when there are already changes (with `\babel@save`).

```
210 \def\bbbl@extras@wrap#1#2#3{% 1:in-test, 2:before, 3:after
211 \toks@{\expandafter\expandafter\expandafter{%
212 \cscname extras\language\endcsname}%
213 \bbbl@exp{\in@{#1}{\the\toks@}}%
214 \ifin@else
215 \@temptokena{#2}%
216 \edef\bbbl@tempc{\the\@temptokena\the\toks@}%
217 \toks@\expandafter{\bbbl@tempc#3}%
218 \expandafter\edef\cscname extras\language\endcsname{\the\toks@}%
219 \fi}
220 {/Basic macros}}
```

Some files identify themselves with a $\TeX$ macro. The following code is placed before them to define (and then undefine) if not in $\TeX$.

```
221 {(*Make sure ProvidesFile is defined)} ≡
222 \ifx\ProvidesFile\@undefined
223 \def\ProvidesFile#1[#2 #3 #4]{%
224 \wlog{File: #1 #4 #3 <#2>}%
225 \let\ProvidesFile\@undefined}
226 \fi
227 {/Make sure ProvidesFile is defined}}
```

3.1. A few core definitions

\language Just for compatibility, for not to touch `hyphen.cfg`.

```
228 {(*Define core switching macros)} ≡
229 \ifx\language\@undefined
230 \cscname newcount\endcsname\language
231 \fi
232 {/Define core switching macros}}
```

\last@language Another counter is used to keep track of the allocated languages. $\TeX$ and $\LaTeX$ reserves for this purpose the count 19.

\addlanguage This macro was introduced for $\TeX < 2$. Preserved for compatibility.

```
233 {(*Define core switching macros)} ≡
234 \countdef\last@language=19
235 \def\addlanguage{\cscname newlanguage\endcsname}
236 {/Define core switching macros}}
```

Now we make sure all required files are loaded. When the command `\AtBeginDocument` doesn't exist we assume that we are dealing with a plain-based format. In that case the file `plain.def` is needed (which also defines `\AtBeginDocument`, and therefore it is not loaded twice). We need the first part when the format is created, and `\orig@dump` is used as a flag. Otherwise, we need to use the second part, so `\orig@dump` is not defined (`plain.def` undefines it).

Check if the current version of `switch.def` has been previously loaded (mainly, `hyphen.cfg`). If not, load it now. We cannot load `babel.def` here because we first need to declare and process the package options.

3.2. $\LaTeX$: `babel.sty` (start)

Here starts the style file for $\LaTeX$. It also takes care of a number of compatibility issues with other packages.

```
237 {*package}
238 \NeedsTeXFormat{LaTeX2e}
239 \ProvidesPackage{babel}%
240 [%@date@> v<@version@>
241 The multilingual framework for LuaLaTeX, pdfLaTeX and XeLaTeX]
```

Start with some “private” debugging tools, and then define macros for errors. The global lua ‘space’ Babel is declared here, too (inside the test for debug).

```

242 \@ifpackagewith{babel}{debug}
243 {\providecommand\bbl@trace[1]{\message{^^J[ #1 ]}}%
244 \let\bbl@debug\@firstofone
245 \ifx\directlua\@undefined\else
246 \directlua{
247     Babel = Babel or {}
248     Babel.debug = true }%
249 \input{babel-debug.tex}%
250 \fi}
251 {\providecommand\bbl@trace[1]{}%
252 \let\bbl@debug\@gobble
253 \ifx\directlua\@undefined\else
254 \directlua{
255     Babel = Babel or {}
256     Babel.debug = false }%
257 \fi}

```

Macros to deal with errors, warnings, etc. Errors are stored in a separate file.

```

258 \def\bbl@error#1{% Implicit #2#3#4
259 \begingroup
260 \catcode`\=0 \catcode`\==12 \catcode`\`=12
261 \input errbabel.def
262 \endgroup
263 \bbl@error{#1}}
264 \def\bbl@warning#1{%
265 \begingroup
266 \def\{\MessageBreak}%
267 \PackageWarning{babel}{#1}%
268 \endgroup}
269 \def\bbl@infowarn#1{%
270 \begingroup
271 \def\{\MessageBreak}%
272 \PackageNote{babel}{#1}%
273 \endgroup}
274 \def\bbl@info#1{%
275 \begingroup
276 \def\{\MessageBreak}%
277 \PackageInfo{babel}{#1}%
278 \endgroup}

```

Many of the following options don’t do anything themselves, they are just defined in order to make it possible for babel and language definition files to check if one of them was specified by the user.

But first, include here the *Basic macros* defined above.

```

279 <@Basic macros>
280 \@ifpackagewith{babel}{silent}
281 {\let\bbl@info\@gobble
282 \let\bbl@infowarn\@gobble
283 \let\bbl@warning\@gobble}
284 {}
285 %
286 \def\AfterBabelLanguage#1{%
287 \global\expandafter\bbl@add\csname#1.ldf-h@k\endcsname}%

```

If the format created a list of loaded languages (in \bbl@languages), get the name of the 0-th to show the actual language used. Also available with base, because it just shows info.

```

288 \ifx\bbl@languages\@undefined\else
289 \begingroup
290 \catcode`\^^I=12
291 \@ifpackagewith{babel}{showlanguages}{%
292 \begingroup
293 \def\bbl@elt#1#2#3#4{\wlog{#2^^I#1^^I#3^^I#4}}%

```

```

294      \wlog{<*languages>}%
295      \bbl@languages
296      \wlog{</languages>}%
297      \endgroup{}}
298  \endgroup
299  \def\bbl@elt#1#2#3#4{%
300    \ifnum#2=\z@
301      \gdef\bbl@nulllanguage{#1}%
302      \def\bbl@elt##1##2##3##4{%
303        \fi}%
304    \bbl@languages
305  \fi

```

3.3. base

The first ‘real’ option to be processed is base, which set the hyphenation patterns then resets `ver@babel.sty` so that $\TeX$ forgets about the first loading. After a subset of `babel.def` has been loaded (the old `switch.def`) and `\AfterBabelLanguage` defined, it exits.

Now the base option. With it we can define (and load, with `luatex`) hyphenation patterns, even if we are not interested in the rest of `babel`.

```

306 \bbl@trace{Defining option 'base'}
307 \ifpackagewith{babel}{base}{%
308   \let\bbl@onlyswitch\@empty
309   \let\bbl@provide@locale\relax
310   \input babel.def
311   \let\bbl@onlyswitch\@undefined
312   \ifx\directlua\@undefined
313     \DeclareOption*{\bbl@patterns{\CurrentOption}}%
314   \else
315     \input luababel.def
316     \DeclareOption*{\bbl@patterns@lua{\CurrentOption}}%
317   \fi
318   \DeclareOption{base}{}%
319   \DeclareOption{showlanguages}{}%
320   \ProcessOptions
321   \global\expandafter\let\csname opt@babel.sty\endcsname\relax
322   \global\expandafter\let\csname ver@babel.sty\endcsname\relax
323   \global\let\@ifl@ter@\@ifl@ter
324   \def\@ifl@ter#1#2#3#4#5{\global\let\@ifl@ter\@ifl@ter@}%
325   \endinput}{}%

```

3.4. key=value options and other general option

The following macros extract language modifiers, and only real package options are kept in the option list. Modifiers are saved and assigned to `\BabelModifiers` at `\bbl@load@language`; when no modifiers have been given, the former is `\relax`.

```

326 \bbl@trace{key=value and another general options}
327 \bbl@csarg\let{tempa\expandafter}\csname opt@babel.sty\endcsname
328 \def\bbl@tempb#1.#2{% Removes trailing dot
329   #1\ifx\@empty#2\else,\bbl@afterfi\bbl@tempb#2\fi}%
330 \def\bbl@tempe#1=#2\@{
331   \bbl@csarg\edef{mod@#1}{\bbl@tempb#2}}
332 \def\bbl@tempd#1.#2\@nnil{%
333   \ifx\@empty#2%
334     \edef\bbl@tempc{\ifx\bbl@tempc\@empty\else\bbl@tempc,\fi#1}%
335   \else
336     \in@{,provide=}{, #1}%
337     \ifin@
338       \edef\bbl@tempc{%
339         \ifx\bbl@tempc\@empty\else\bbl@tempc,\fi#1.\bbl@tempb#2}%
340     \else
341       \in@{$modifiers$}{$#1$}%

```

```

342 \ifin@
343 \bbl@tempe#2\@@
344 \else
345 \in@{=}{#1}%
346 \ifin@
347 \edef\bbl@tempc{\ifx\bbl@tempc\@empty\else\bbl@tempc,\fi#1.#2}%
348 \else
349 \edef\bbl@tempc{\ifx\bbl@tempc\@empty\else\bbl@tempc,\fi#1}%
350 \bbl@csarg\edef{mod@#1}{\bbl@tempb#2}%
351 \fi
352 \fi
353 \fi
354 \fi}
355 \let\bbl@tempc\@empty
356 \bbl@foreach\bbl@tempa{\bbl@tempd#1.\@empty\@nnil}
357 \expandafter\let\csname opt@babel.sty\endcsname\bbl@tempc

```

The next option tells babel to leave shorthand characters active at the end of processing the package. This is *not* the default as it can cause problems with other packages, but for those who want to use the shorthand characters in the preamble of their documents this can help.

```

358 \DeclareOption{KeepShorthandsActive}{}
359 \DeclareOption{activeacute}{}
360 \DeclareOption{activegrave}{}
361 \DeclareOption{debug}{}
362 \DeclareOption{debug-german}{} % Temporary
363 \DeclareOption{noconfigs}{}
364 \DeclareOption{showlanguages}{}
365 \DeclareOption{silent}{}
366 \DeclareOption{shorthands=off}{\bbl@tempa shorthands=\bbl@tempa}
367 \chardef\bbl@iniflag\z@
368 \DeclareOption{provide=*}{\chardef\bbl@iniflag\@ne} % main = 1
369 \DeclareOption{provide+=*}{\chardef\bbl@iniflag\tw@} % second = 2
370 \DeclareOption{provide*=*}{\chardef\bbl@iniflag\thr@@} % second + main
371 \chardef\bbl@ldfflag\z@
372 \DeclareOption{provide=!}{\chardef\bbl@ldfflag\@ne} % main = 1
373 \DeclareOption{provide+=!}{\chardef\bbl@ldfflag\tw@} % second = 2
374 \DeclareOption{provide*=!}{\chardef\bbl@ldfflag\thr@@} % second + main
375 \DeclareOption{licr=extended}{}
376 \DeclareOption{licr=unextended}{}
377 \let\bbl@ifmetadata\@gobble
378 \DeclareOption{babel-metadata=on}{%
379 \IfDocumentMetadataT{\AtEndOfPackage{\input{tagbabel.def}}}}
380 \DeclareOption{babel-metadata=tagging}{%
381 \IfDocumentMetadataT{\AtEndOfPackage{\input{tagbabel.def}}}}
382 % Don't use. Experimental.
383 \newif\ifbbl@single
384 \DeclareOption{selectors=off}{\bbl@singletrue}
385 <@More package options@>

```

Handling of package options is done in three passes. (I [JBL] am not very happy with the idea, anyway.) The first one processes options which has been declared above or follow the syntax $\langle key \rangle = \langle value \rangle$, the second one loads the requested languages, except the main one if set with the key main, and the third one loads the latter. First, we “flag” valid keys with a nil value.

```

386 \let\bbl@opt@shorthands\@nnil
387 \let\bbl@opt@config\@nnil
388 \let\bbl@opt@main\@nnil
389 \let\bbl@opt@headfoot\@nnil
390 \let\bbl@opt@layout\@nnil
391 \let\bbl@opt@provide\@nnil

```

The following tool is defined temporarily to store the values of options.

```

392 \def\bbl@tempa#1=#2\bbl@tempa{%
393 \bbl@csarg\ifx{opt@#1}\@nnil
394 \bbl@csarg\edef{opt@#1}{#2}%

```

```

395 \else
396   \bbl@error{bad-package-option}{#1}{#2}{}%
397 \fi}

```

Now the option list is processed, taking into account only currently declared options (including those declared with a =), and $\langle key \rangle = \langle value \rangle$ options (the former take precedence). Unrecognized options are saved in `\bbl@language@opts`, because they are language options.

```

398 \let\bbl@language@opts@empty
399 \DeclareOption*{%
400   \bbl@xin@{\string=}{\CurrentOption}%
401   \ifin@
402     \expandafter\bbl@tempa\CurrentOption\bbl@tempa
403   \else
404     \bbl@add@list\bbl@language@opts{\CurrentOption}%
405   \fi}

```

Now we finish the first pass (and start over).

```

406 \ProcessOptions*

```

3.5. Post-process some options

```

407 \ifx\bbl@opt@provide@nnil
408   \let\bbl@opt@provide@empty %%% MOVE above
409 \else
410   \chardef\bbl@iniflag@ne
411   \bbl@expf{\bbl@forkv{\@nameuse{@raw@opt@babel.sty}}}{%
412     \in@{,provide,}{, #1,}%
413     \ifin@
414       \def\bbl@opt@provide{#2}%
415     \fi}
416 \fi

```

If there is no `shorthands=⟨chars⟩`, the original `babel` macros are left untouched, but if there is, these macros are wrapped (in `babel.def`) to define only those given.

A bit of optimization: if there is no `shorthands=`, then `\bbl@ifshorthand` is always true, and it is always false if `shorthands` is empty. Also, some code makes sense only with `shorthands=...`

```

417 \bbl@trace{Conditional loading of shorthands}
418 \def\bbl@sh@string#1{%
419   \ifx#1@empty\else
420     \ifx#1t\string~%
421     \else\ifx#1c\string,%
422     \else\string#1%
423   \fi\fi
424   \expandafter\bbl@sh@string
425 \fi}
426 \ifx\bbl@opt@shorthands@nnil
427   \def\bbl@ifshorthand#1#2#3{#2}%
428 \else\ifx\bbl@opt@shorthands@empty
429   \def\bbl@ifshorthand#1#2#3{#3}%
430 \else

```

The following macro tests if a shorthand is one of the allowed ones.

```

431 \def\bbl@ifshorthand#1{%
432   \bbl@xin@{\string#1}{\bbl@opt@shorthands}%
433   \ifin@
434     \expandafter\@firstoftwo
435   \else
436     \expandafter\@secondoftwo
437   \fi}

```

We make sure all chars in the string are ‘other’, with the help of an auxiliary macro defined above (which also zaps spaces).

```

438 \edef\bbl@opt@shorthands{%
439   \expandafter\bbl@sh@string\bbl@opt@shorthands@empty}%

```

The following is ignored with `shorthands=off`, since it is intended to take some additional actions for certain chars.

```
440 \bbl@ifshorthand{'}%
441   {\PassOptionsToPackage{activeacute}{babel}}{}
442 \bbl@ifshorthand{'}%
443   {\PassOptionsToPackage{activegrave}{babel}}{}
444 \fi\fi
```

With `headfoot=lang` we can set the language used in heads/feet. For example, in `babel/3796` just add `headfoot=english`. It misuses `\resetactivechars`, but seems to work.

```
445 \ifx\bbl@opt@headfoot\@nnil\else
446   \g@addto@macro\@resetactivechars{%
447     \set@typeset@protect
448     \expandafter\select@language@x\expandafter{\bbl@opt@headfoot}%
449     \let\protect\noexpand}
450 \fi
```

For the option `safe` we use a different approach – `\bbl@opt@safe` says which macros are redefined (B for bibs and R for refs). By default, both are currently set, but in a future release it will be set to `none`.

```
451 \ifx\bbl@opt@safe\@undefined
452   \def\bbl@opt@safe{BR}
453   % \let\bbl@opt@safe\@empty % Pending of \cite
454 \fi
```

For layout an auxiliary macro is provided, available for packages and language styles. Optimization: if there is no layout, just do nothing.

```
455 \bbl@trace{Defining IfBabelLayout}
456 \ifx\bbl@opt@layout\@nnil
457   \newcommand\IfBabelLayout[3]{#3}%
458 \else
459   \bbl@exp{\bbl@forkv{\@nameuse{@raw@opt@babel.sty}}}{%
460     \in@{, layout,}{, #1,}%
461     \ifin@
462       \def\bbl@opt@layout{#2}%
463       \bbl@replace\bbl@opt@layout{ }{.}%
464     \fi}
465   \newcommand\IfBabelLayout[1]{%
466     \@expandtwoargs\in@{.#1.}{.\bbl@opt@layout.}%
467     \ifin@
468       \expandafter\@firstoftwo
469     \else
470       \expandafter\@secondoftwo
471     \fi}
472 \fi
473 (/package)
```

3.6. Plain: `babel.def` (start)

Because of the way `docstrip` works, we need to insert some code for Plain here. However, the tools provided by the `babel` installer for literate programming makes this section a short interlude, because the actual code is below, tagged as *Emulate LaTeX*.

First, exit immediately if previously loaded.

```
474 (*core)
475 \ifx\ldf@quit\@undefined\else
476 \endinput\fi % Same line!
477 <@Make sure ProvidesFile is defined@>
478 \ProvidesFile{babel.def}[<@date@> v<@version@> Babel common definitions]
479 \ifx\AtBeginDocument\@undefined
480   <@Emulate LaTeX@>
481 \fi
482 <@Basic macros@>
483 (/core)
```

That is all for the moment. Now follows some common stuff, for both Plain and $\text{\LaTeX}$. After it, we will resume the $\text{\LaTeX}$ -only stuff.

4. babel.sty and babel.def (common)

```

484 (*package|core)
485 \def\bbl@version{<@version>}
486 \def\bbl@date{<@date>}
487 <@Define core switching macros>

```

\addialect The macro \addialect can be used to add the name of a dialect or variant language, for which an already defined hyphenation table can be used.

```

488 \def\addialect#1#2{%
489   \global\chardef#1#2\relax
490   \bbl@usehooks{addialect}{#1}{#2}}%
491   \begingroup
492     \count@#1\relax
493     \def\bbl@elt##1##2##3##4{%
494       \ifnum\count@=#2\relax
495         \edef\bbl@tempa{\expandafter\@gobbletwo\string#1}%
496         \bbl@info{Hyphen rules for '\expandafter\@gobble\bbl@tempa'
497           set to \expandafter\string\csname l@##1\endcsname\%
498           (\string\language\the\count@). Reported}%
499         \def\bbl@elt####1####2####3####4{%
500           \fi}%
501         \bbl@cs{languages}%
502         \endgroup

```

\bbl@iflanguage executes code only if the language l@ exists. Otherwise raises an error.

The argument of \bbl@fixname has to be a macro name, as it may get “fixed” if casing (lc/uc) is wrong. It’s an attempt to fix a long-standing bug when \foreignlanguage and the like appear in a \MakeXXXcase. However, a lowercase form is not imposed to improve backward compatibility (perhaps you defined a language named MYLANG, but unfortunately mixed case names cannot be trapped). Note l@ is encapsulated, so that its case does not change.

```

503 \def\bbl@fixname#1{%
504   \begingroup
505     \def\bbl@tempe{l@}%
506     \edef\bbl@tempd{\noexpand\@ifundefined{\noexpand\bbl@tempe#1}}%
507     \bbl@tempd
508     {\lowercase\expandafter{\bbl@tempd}%
509      {\uppercase\expandafter{\bbl@tempd}%
510       \@empty
511       {\edef\bbl@tempd{\def\noexpand#1{#1}}%
512        \uppercase\expandafter{\bbl@tempd}}}%
513      {\edef\bbl@tempd{\def\noexpand#1{#1}}%
514       \lowercase\expandafter{\bbl@tempd}}}%
515     \@empty
516     \edef\bbl@tempd{\endgroup\def\noexpand#1{#1}}%
517     \bbl@tempd
518     \bbl@exp{\bbl@usehooks{languagename}{\language}{#1}}%
519 \def\bbl@iflanguage#1{%
520   \@ifundefined{l@#1}{\@nolanerr{#1}\@gobble}\@firstofone}

```

After a name has been ‘fixed’, the selectors will try to load the language. If even the fixed name is not defined, will load it on the fly, either based on its name, or if activated, its BCP 47 code.

We first need a couple of macros for a simple BCP 47 look up. It also makes sure, casing is the usual one, so that sr-latn-ba becomes fr-Latn-BA.

\bbl@bcplookup returns \bbl@bcp with either the found ini tag or \relax. Temporary version, to be refactored with the new \text_bcp_parse:n, which is not currently fully expandable with invalid BCP 47 tags. The argument may be is some cases either a language name or a tag (e.g., \foreignlanguage. This was a huge mistake, because of the potential ambiguities (for example, the name vai-latin is also a syntactically valid tag).

```

521 \def\bbl@cntchars#1{\bbl@cntchars@i#1876543210\@@}
522 \def\bbl@cntchars@i#1#2#3#4#5#6#7#8#9\@{\@car#9\@nil}%
523 \def\bbl@bcplookup#1{%
524   \let\bbl@templ\@empty % language
525   \let\bbl@temps\@empty % script

```

```

526 \let\bbl@tempr\@empty % regions
527 \let\bbl@tempv\@empty % variant
528 \edef\bbl@tempa{#1}%
529 \bbl@replace\bbl@tempa{-}{,}%
530 \bbl@replace\bbl@tempa_{,}%
531 \bbl@foreach\bbl@tempa{%
532   \ifx\bbl@templ\@empty
533     \lowercase{\def\bbl@templ{##1}}%
534   \else\ifnum\bbl@cntchars{##1}=4
535     \bbl@xin@{\@car##1\@nil}{0123456789}%
536     \ifin@
537       \def\bbl@tempv{##1}% eg, 1901
538     \else
539       \uppercase{\edef\bbl@tempb{\@car##1\@nil}}%
540       \lowercase{\edef\bbl@tempb{\bbl@tempb\@gobble##1}}%
541     \fi
542   \else\ifnum\bbl@cntchars{##1}<4
543     \uppercase{\def\bbl@tempr{##1}}%
544   \else\ifnum\bbl@cntchars{##1}>4
545     \lowercase{\def\bbl@tempv{##1}}%
546   \fi\fi\fi\fi}%
547 \bbl@exp{\bbl@bcpllookup@i{\bbl@templ}{\bbl@tempb}{\bbl@tempr}{\bbl@tempv}}
548 \def\bbl@bcpllookup@try#1{%
549   \ifx\bbl@bcp\relax
550     \IfFileExists{babel-#1\bbl@tempe.ini}{\edef\bbl@bcp{#1\bbl@tempe}}{}%
551   \fi}
552 \def\bbl@bcpllookup@i#1#2#3#4{%
553   \let\bbl@bcp\relax
554   \def\bbl@tempe{#4}%
555   \ifx\bbl@tempe\@empty\else
556     \edef\bbl@tempe{-#4}%
557   \fi
558   \edef\bbl@tempa{#2#3\bbl@tempe}% en
559   \ifx\bbl@tempa\@empty
560     \bbl@bcpllookup@try{#1}%
561   \else
562     \edef\bbl@tempa{#3\bbl@tempe}% en-Latn
563     \ifx\bbl@tempa\@empty
564       \bbl@bcpllookup@try{#1-#2}%
565       \bbl@bcpllookup@try{#1}%
566     \else
567       \edef\bbl@tempa{#2\bbl@tempe}% en-US, en-001
568       \ifx\bbl@tempa\@empty
569         \bbl@bcpllookup@try{#1-#3}%
570         \bbl@bcpllookup@try{#1}%
571       \else % en-Latn-US
572         \bbl@bcpllookup@try{#1-#2-#3}%
573         \bbl@bcpllookup@try{#1-#2}%
574         \bbl@bcpllookup@try{#1-#3}%
575         \bbl@bcpllookup@try{#1}%
576       \fi
577     \fi
578   \fi
579   \ifx\bbl@bcp\relax
580     \ifx\bbl@tempe\@empty\else\bbl@bcpllookup@i{#1}{#2}{#3}{#4}\fi%
581   \fi}
582 %
583 \let\bbl@initoload\relax

```

\iflanguage Users might want to test (in a private package for instance) which language is currently active. For this we provide a test macro, `\iflanguage`, that has three arguments. It checks whether the first argument is a known language. If so, it compares the first argument with the value of `\language`. Then, depending on the result of the comparison, it executes either the second or the

third argument.

```
584 \def\iflanguage#1{%
585   \bbl@iflanguage{#1}{%
586     \ifnum\csname l@#1\endcsname=\language
587       \expandafter\@firstoftwo
588     \else
589       \expandafter\@secondoftwo
590     \fi}}
```

4.1. Selecting the language

\selectlanguage It checks whether the language is already defined before it performs its actual task, which is to update `\language` and activate language-specific definitions.

```
591 \let\bbl@select@type\z@
592 \edef\selectlanguage{%
593   \noexpand\protect
594   \expandafter\noexpand\csname selectlanguage \endcsname}
```

Because the command `\selectlanguage` could be used in a moving argument it expands to `\protect\selectlanguage_`. Therefore, we have to make sure that a macro `\protect` exists. If it doesn't it is `\let` to `\relax`.

```
595 \ifx\@undefined\protect\let\protect\relax\fi
```

The following definition is preserved for backwards compatibility (e.g., arabi, koma). It is related to a trick for 2.09, now discarded.

```
596 \let\xstring\string
```

Since version 3.5 babel writes entries to the auxiliary files in order to typeset table of contents etc. in the correct language environment.

\bbl@pop@language But when the language change happens *inside* a group the end of the group doesn't write anything to the auxiliary files. Therefore we need TeX's `aftergroup` mechanism to help us. The command `\aftergroup` stores the token immediately following it to be executed when the current group is closed. So we define a temporary control sequence `\bbl@pop@language` to be executed at the end of the group. It calls `\bbl@set@language` with the name of the current language as its argument.

\bbl@language@stack The previous solution works for one level of nesting groups, but as soon as more levels are used it is no longer adequate. For that case we need to keep track of the nested languages using a stack mechanism. This stack is called `\bbl@language@stack` and initially empty.

```
597 \def\bbl@language@stack{}
```

When using a stack we need a mechanism to push an element on the stack and to retrieve the information afterwards.

\bbl@push@language

\bbl@pop@language The stack is simply a list of languagenames, separated with a '+' sign; the push function can be simple:

```
598 \def\bbl@push@language{%
599   \ifx\language\@undefined\else
600     \ifx\currentgrouplevel\@undefined
601       \xdef\bbl@language@stack{\language+\bbl@language@stack}%
602     \else
603       \ifnum\currentgrouplevel=\z@
604         \xdef\bbl@language@stack{\language+}%
605       \else
606         \xdef\bbl@language@stack{\language+\bbl@language@stack}%
607       \fi
608     \fi
609   \fi}
```

Retrieving information from the stack is a little bit less simple, as we need to remove the element from the stack while storing it in the macro `\language`. For this we first define a helper function.

\bbl@pop@lang This macro stores its first element (which is delimited by the ‘+’-sign) in \language and stores the rest of the string in \bbl@language@stack.

```
610 \def\bbl@pop@lang#1+#2\@@{%
611   \edef\language{#1}%
612   \xdef\bbl@language@stack{#2}}
```

The reason for the somewhat weird arrangement of arguments to the helper function is the fact it is called in the following way. This means that before \bbl@pop@lang is executed T_EX first *expands* the stack, stored in \bbl@language@stack. The result of that is that the argument string of \bbl@pop@lang contains one or more language names, each followed by a ‘+’-sign (zero language names won’t occur as this macro will only be called after something has been pushed on the stack).

```
613 \let\bbl@ifrestoring\@secondoftwo
614 \def\bbl@pop@language{%
615   \expandafter\bbl@pop@lang\bbl@language@stack\@@
616   \let\bbl@ifrestoring\@firstoftwo
617   \expandafter\bbl@set@language\expandafter{\language}%
618   \let\bbl@ifrestoring\@secondoftwo}
```

Once the name of the previous language is retrieved from the stack, it is fed to \bbl@set@language to do the actual work of switching everything that needs switching.

An alternative way to identify languages (in the babel sense) with a numerical value is introduced in 3.30. This is one of the first steps for a new interface based on the concept of locale, which explains the name of \localeid. This means \l@. . . will be reserved for hyphenation patterns (so that two locales can share the same rules).

```
619 \chardef\localeid\z@
620 \gdef\bbl@id@last{0} % No real need for a new counter
621 \def\bbl@id@assign{%
622   \bbl@ifunset\bbl@id@\language}%
623   {\count@\bbl@id@last\relax
624    \advance\count@\@ne
625    \global\bbl@csarg\chardef{id@\language}\count@
626    \xdef\bbl@id@last{\the\count@}%
627    \ifcase\bbl@engine\or
628      \directlua{
629        Babel.locale_props[\bbl@id@last] = {}
630        Babel.locale_props[\bbl@id@last].name = '\language'
631        Babel.locale_props[\bbl@id@last].vars = {}
632      }%
633    \fi}%
634   }%
635   \chardef\localeid\bbl@c{id@}}
```

The unprotected part of \selectlanguage. In case it is used as environment, declare \endselectlanguage, just for safety.

```
636 \let\bbl@select@opts\@empty
637 \expandafter\def\csname selectlanguage \endcsname{%
638   \@ifnextchar[\bbl@select@s{\bbl@select@s[]}}
639 \def\bbl@select@s[#1]#2{%
640   \def\bbl@select@opts{#1}%
641   \ifnum\bbl@hymapsel=\@cclv\let\bbl@hymapsel\tw\fi
642   \bbl@push@language
643   \aftergroup\bbl@pop@language
644   \bbl@set@language{#2}}
645 \let\endselectlanguage\relax
```

\bbl@set@language The macro \bbl@set@language takes care of switching the language environment *and* of writing entries on the auxiliary files. For historical reasons, language names can be either language of \language. To catch either form a trick is used, but unfortunately as a side effect the catcodes of letters in \language are messed up. This is a bug, but preserved for backwards compatibility. The list of auxiliary files can be extended by redefining \BabelContentsFiles, but make sure they are loaded inside a group (as aux, toc, lof, and lot do) or the last language of the document will remain active afterwards.

We also write a command to change the current language in the auxiliary files.

`\bbl@savelastskip` is used to deal with skips before the write whatsit (as suggested by U Fischer). Adapted from `hyperref`, but it might fail, so I'll consider it a temporary hack, while I study other options (the ideal, but very likely unfeasible except perhaps in `luatex`, is to avoid the `\write` altogether when not needed).

```

646 \def\BabelContentsFiles{toc,lof,lot}
647 \def\bbl@set@language#1{% from selectlanguage, pop@
648 % The old buggy way. Preserved for compatibility, but simplified
649 \edef\language{\expandafter\string#1\@empty}%
650 \select@language{\language}%
651 \bbl@xin@{,main,},{, \bbl@select@opts,}%
652 \ifin@
653 \let\bbl@main@language\localename
654 \let\mainlocalename\localename
655 \fi
656 \let\bbl@select@opts\@empty
657 % write to aux files
658 \expandafter\ifx\csname date\language\endcsname\relax\else
659 \if@filesw
660 \bbl@xin@{,nofiles,},{, \bbl@select@opts,}%
661 \ifin@\else
662 \ifx\babel@aux\@gobbletwo\else % Set if single in the first, redundant
663 \bbl@savelastskip
664 \protected@write\@auxout{{}\string\babel@aux{\bbl@auxname}{}}%
665 \bbl@restorelastskip
666 \fi
667 \bbl@usehooks{write}{}%
668 \fi
669 \fi
670 \fi}
671 %
672 \let\bbl@restorelastskip\relax
673 \let\bbl@savelastskip\relax
674 %
675 \def\select@language#1{% from set@, babel@aux, babel@toc
676 \ifx\bbl@selectorname\@empty
677 \def\bbl@selectorname{select}%
678 \fi
679 % set hymap
680 \ifnum\bbl@hymapsel=\@ccclv\chardef\bbl@hymapsel4\relax\fi
681 % set name (when coming from babel@aux)
682 \edef\language{#1}%
683 \bbl@fixname\language
684 % define \localename when coming from set@, with a trick
685 \ifx\scantokens\undefined
686 \def\localename{??}%
687 \else
688 \bbl@exp{\scantokens{\def\localename{\language}\noexpand}\relax}%
689 \fi
690 \bbl@provide@locale
691 \bbl@iflanguage\language{%
692 \let\bbl@select@type\z@
693 \expandafter\bbl@switch\expandafter{\language}}
694 \def\babel@aux#1#2{%
695 \select@language{#1}%
696 \bbl@foreach\BabelContentsFiles{% \relax -> don't assume vertical mode
697 \writefile{##1}{\babel@toc{#1}{#2}\relax}}%
698 \def\babel@toc#1#2{%
699 \select@language{#1}}

```

First, check if the user asks for a known language. If so, update the value of `\language` and call `\originalTeX` to bring `TEX` in a certain pre-defined state.

The name of the language is stored in the control sequence `\language`.

Then we have to *redefine* `\originalTeX` to compensate for the things that have been activated. To

save memory space for the macro definition of `\originalTeX`, we construct the control sequence name for the `\noextras<language>` command at definition time by expanding the `\csname` primitive.

Now activate the language-specific definitions. This is done by constructing the names of three macros by concatenating three words with the argument of `\selectlanguage`, and calling these macros.

The switching of the values of `\lefthyphenmin` and `\righthyphenmin` is somewhat different. First we save their current values, then we check if `\<language>hyphenmins` is defined. If it is not, we set default values (2 and 3), otherwise the values in `\<language>hyphenmins` will be used.

No text is supposed to be added with switching captions and date, so we remove any spurious spaces with `\bbl@bsphack` and `\bbl@esphack`.

```

700 \newif\ifbbl@usedategroup
701 \let\bbl@savextras\@empty
702 \def\bbl@switch#1{% from select@, foreign@
703   % restore
704   \originalTeX
705   \expandafter\def\expandafter\originalTeX\expandafter{%
706     \csname noextras#1\endcsname
707     \let\originalTeX\@empty
708     \babel@beginsave}%
709   \bbl@usehooks{afterreset}{}%
710   \languageshorthands{none}%
711   % set the locale id
712   \bbl@id@assign
713   % switch captions, date
714   \bbl@bsphack
715   \ifcase\bbl@select@type
716     \csname captions#1\endcsname\relax
717     \csname date#1\endcsname\relax
718   \else
719     \bbl@xin@{,captions,}{,\bbl@select@opts,}%
720     \ifin@
721       \csname captions#1\endcsname\relax
722     \fi
723     \bbl@xin@{,date,}{,\bbl@select@opts,}%
724     \ifin@ % if \foreign... within \<language>date
725       \csname date#1\endcsname\relax
726     \fi
727   \fi
728   \bbl@esphack
729   % switch extras
730   \csname bbl@preextras@#1\endcsname
731   \bbl@usehooks{beforeextras}{}%
732   \csname extras#1\endcsname\relax
733   \bbl@usehooks{afterextras}{}%
734   % > babel-ensure
735   % > babel-sh-<short>
736   % > babel-bidi
737   % > babel-fontspec
738   \let\bbl@savextras\@empty
739   % hyphenation - case mapping
740   \ifcase\bbl@opt@hyphenmap\or
741     \def\BabelLower##1##2{\lccode##1=##2\relax}%
742     \ifnum\bbl@hymapsel>4\else
743       \csname\language @bbl@hyphenmap\endcsname
744     \fi
745     \chardef\bbl@opt@hyphenmap\z@
746   \else
747     \ifnum\bbl@hymapsel>\bbl@opt@hyphenmap\else
748       \csname\language @bbl@hyphenmap\endcsname
749     \fi
750   \fi
751   \let\bbl@hymapsel\@cclv
752   % hyphenation - select rules

```

```

753 \ifnum\csname l@\language\endcsname=\l@unhyphenated
754 \edef\bbl@tempa{u}%
755 \else
756 \edef\bbl@tempa{\bbl@cl{\lnbrk}}%
757 \fi
758 % linebreaking - handle u, e, k (v in the future)
759 \bbl@xin@{/u}{/\bbl@tempa}%
760 \ifin@else\bbl@xin@{/e}{/\bbl@tempa}\fi % elongated forms
761 \ifin@else\bbl@xin@{/k}{/\bbl@tempa}\fi % only kashida
762 \ifin@else\bbl@xin@{/p}{/\bbl@tempa}\fi % padding (e.g., Tibetan)
763 \ifin@else\bbl@xin@{/v}{/\bbl@tempa}\fi % variable font
764 % hyphenation - save mins
765 \babel@savevariable\lefthyphenmin
766 \babel@savevariable\righthyphenmin
767 \ifnum\bbl@engine=\@ne
768 \babel@savevariable\hyphenationmin
769 \fi
770 \ifin@
771 % unhyphenated/kashida/elongated/padding = allow stretching
772 \language\l@unhyphenated
773 \babel@savevariable\emergencystretch
774 \emergencystretch\maxdimen
775 \babel@savevariable\hbadness
776 \hbadness\@M
777 \else
778 % other = select patterns
779 \bbl@patterns{#1}%
780 \fi
781 % hyphenation - set mins
782 \expandafter\ifx\csname #1hyphenmins\endcsname\relax
783 \set@hyphenmins\tw@\thr@@\relax
784 \@nameuse{\bbl@hyphenmins@}%
785 \else
786 \expandafter\expandafter\expandafter\set@hyphenmins
787 \csname #1hyphenmins\endcsname\relax
788 \fi
789 \@nameuse{\bbl@hyphenmins@}%
790 \@nameuse{\bbl@hyphenmins@\language}%
791 \@nameuse{\bbl@hyphenatmin@}%
792 \@nameuse{\bbl@hyphenatmin@\language}%
793 \let\bbl@selectorname\@empty

```

otherlanguage It can be used as an alternative to using the `\selectlanguage` declarative command. The `\ignorespaces` command is necessary to hide the environment when it is entered in horizontal mode.

```

794 \edef\otherlanguage{%
795 \noexpand\protect
796 \expandafter\noexpand\csname otherlanguage \endcsname}
797 \expandafter\def\csname otherlanguage \endcsname{%
798 \@ifstar{\@nameuse{otherlanguage*}}{\bbl@otherlanguage}
799 \def\bbl@otherlanguage#1{%
800 \def\bbl@selectorname{other}%
801 \ifnum\bbl@hymapsel=\@cclv\let\bbl@hymapsel\thr@@\fi
802 \csname selectlanguage \endcsname{#1}%
803 \ignorespaces}

```

The `\endotherlanguage` part of the environment tries to hide itself when it is called in horizontal mode.

```

804 \long\def\endotherlanguage{\@ignoretrue\ignorespaces}

```

otherlanguage* It is meant to be used when a large part of text from a different language needs to be typeset, but without changing the translation of words such as ‘figure’. It makes use of

\foreign@language.

```
805 \expandafter\def\csname otherlanguage*\endcsname{%
806   \@ifnextchar[\bbl@otherlanguage@s{\bbl@otherlanguage@s[]}}
807 \def\bbl@otherlanguage@s[#1]#2{%
808   \def\bbl@selectorname{other*}%
809   \ifnum\bbl@hymapset=\@ccclv\chardef\bbl@hymapset4\relax\fi
810   \def\bbl@select@opts{#1}%
811   \foreign@language{#2}}
```

At the end of the environment we need to switch off the extra definitions. The grouping mechanism of the environment will take care of resetting the correct hyphenation rules and “extras”.

```
812 \expandafter\let\csname endotherlanguage*\endcsname\relax
```

\foreignlanguage This command takes two arguments, the first argument is the name of the language to use for typesetting the text specified in the second argument.

Unlike \selectlanguage this command doesn’t switch *everything*, it only switches the hyphenation rules and the extra definitions for the language specified. It does this within a group and assumes the \extras<language> command doesn’t make any \global changes. The coding is very similar to part of \selectlanguage.

\bbl@beforeforeign is a trick to fix a bug in bidi texts. \foreignlanguage is supposed to be a ‘text’ command, and therefore it must emit a \leavevmode, but it does not, and therefore the indent is placed on the opposite margin. For backward compatibility, however, it is done only if a right-to-left script is requested; otherwise, it is no-op.

(3.11) \foreignlanguage* is a temporary, experimental macro for a few lines with a different script direction, while preserving the paragraph format (thank the braces around \par, things like \hangindent are not reset). Do not use it in production, because its semantics and its syntax may change (and very likely will, or even it could be removed altogether). Currently it enters in vmode and then selects the language (which in turn sets the paragraph direction).

(3.11) Also experimental are the hook foreign and foreign*. With them you can redefine \BabelText which by default does nothing. Its behavior is not well defined yet. So, use it in horizontal mode only if you do not want surprises.

In other words, at the beginning of a paragraph \foreignlanguage enters into hmode with the surrounding lang, and with \foreignlanguage* with the new lang.

```
813 \providecommand\bbl@beforeforeign{}
814 \edef\foreignlanguage{%
815   \noexpand\protect
816   \expandafter\noexpand\csname foreignlanguage \endcsname}
817 \expandafter\def\csname foreignlanguage \endcsname{%
818   \@ifstar\bbl@foreign@s\bbl@foreign@x}
819 \providecommand\bbl@foreign@x[3][]{%
820   \begingroup
821     \def\bbl@selectorname{foreign}%
822     \def\bbl@select@opts{#1}%
823     \let\BabelText\@firstofone
824     \bbl@beforeforeign
825     \foreign@language{#2}%
826     \bbl@ifmetadata{\UseTaggingSocket{babel/foreign/begin}}}%
827     \bbl@usehooks{foreign}{}%
828     \BabelText{#3}% Now in horizontal mode!
829     \bbl@ifmetadata{\UseTaggingSocket{babel/foreign/end}}}%
830   \endgroup}
831 \def\bbl@foreign@s#1#2{%
832   \begingroup
833     {\par}%
834     \def\bbl@selectorname{foreign*}%
835     \let\bbl@select@opts\@empty
836     \let\BabelText\@firstofone
837     \foreign@language{#1}%
838     \bbl@usehooks{foreign*}{}%
839     \bbl@dirparastext
840     \BabelText{#2}% Still in vertical mode!
841     {\par}%
```

```

842 \endgroup}
843 \providecommand\BabelWrapText[1]{%
844 \def\bbl@tempa{\def\BabelText####1}%
845 \expandafter\bbl@tempa\expandafter{\BabelText{#1}}}

```

\foreign@language This macro does the work for `\foreignlanguage` and the `otherlanguage*` environment. First we need to store the name of the language and check that it is a known language. Then it just calls `bbl@switch`.

```

846 \def\foreign@language#1{%
847 % set name
848 \edef\language#1%
849 \ifbbl@usedategroup
850 \bbl@add\bbl@select@opts{,date,}%
851 \bbl@usedategroupfalse
852 \fi
853 \bbl@fixname\language
854 \let\localename\language
855 \bbl@provide@locale
856 \bbl@iflanguage\language{%
857 \let\bbl@select@type\@ne
858 \expandafter\bbl@switch\expandafter{\language}}

```

The following macro executes conditionally some code based on the selector being used.

```

859 \def\IfBabelSelectorTF#1{%
860 \bbl@xin@{\bbl@selectorname,}{,\zap@space#1 \@empty,}%
861 \ifin@
862 \expandafter\@firstoftwo
863 \else
864 \expandafter\@secondoftwo
865 \fi}

```

\bbl@patterns This macro selects the hyphenation patterns by changing the `\language` register. If special hyphenation patterns are available specifically for the current font encoding, use them instead of the default.

It also sets hyphenation exceptions, but only once, because they are global (here `\lccode's` has been set, too). `\bbl@hyphenation@` is set to relax until the very first `\babelhyphenation`, so do nothing with this value. If the exceptions for a language (by its number, not its name, so that `:ENC` is taken into account) has been set, then use `\hyphenation` with both global and language exceptions and empty the latter to mark they must not be set again.

```

866 \let\bbl@hyphlist\@empty
867 \let\bbl@hyphenation@\relax
868 \let\bbl@pttnlist\@empty
869 \let\bbl@patterns@\relax
870 \let\bbl@hymapsel=\@cclv
871 \def\bbl@patterns#1{%
872 \language=\expandafter\ifx\csname l@#1:\f@encoding\endcsname\relax
873 \csname l@#1\endcsname
874 \edef\bbl@tempa{#1}%
875 \else
876 \csname l@#1:\f@encoding\endcsname
877 \edef\bbl@tempa{#1:\f@encoding}%
878 \fi
879 \@expandtwoargs\bbl@usehooks{patterns}{#1}{\bbl@tempa}}%
880 % > luatex
881 \ifundefined{bbl@hyphenation@}{% Can be \relax!
882 \begingroup
883 \bbl@xin@{\, \number\language,}{,\bbl@hyphlist}%
884 \ifin@ \else
885 \@expandtwoargs\bbl@usehooks{hyphenation}{#1}{\bbl@tempa}}%
886 \hyphenation{%
887 \bbl@hyphenation@
888 \ifundefined{bbl@hyphenation@#1}%

```

```

889         \@empty
890         {\space\csname bbl@hyphenation@#1\endcsname}}%
891         \xdef\bbl@hyphlist{\bbl@hyphlist\number\language,}%
892         \fi
893     \endgroup}}

```

hyphenrules It can be used to select *just* the hyphenation rules. It does *not* change `\language` and when the hyphenation rules specified were not loaded it has no effect. Note however, `\lccode`'s and font encodings are not set at all, so in most cases you should use `otherlanguage*`.

```

894 \def\hyphenrules#1{%
895     \edef\bbl@tempf{#1}%
896     \bbl@fixname\bbl@tempf
897     \bbl@iflanguage\bbl@tempf{%
898         \expandafter\bbl@patterns\expandafter{\bbl@tempf}%
899         \ifx\languageshorthands\@undefined\else
900             \languageshorthands{none}%
901         \fi
902         \expandafter\ifx\csname\bbl@tempf hyphenmins\endcsname\relax
903             \set@hyphenmins\tw@thr@@\relax
904         \else
905             \expandafter\expandafter\expandafter\set@hyphenmins
906             \csname\bbl@tempf hyphenmins\endcsname\relax
907         \fi}}
908 \let\endhyphenrules\@empty

```

\providehyphenmins The macro `\providehyphenmins` should be used in the language definition files to provide a *default* setting for the hyphenation parameters `\lefthyphenmin` and `\righthyphenmin`. If the macro `\(language)hyphenmins` is already defined this command has no effect.

```

909 \def\providehyphenmins#1#2{%
910     \expandafter\ifx\csname #1hyphenmins\endcsname\relax
911         \@namedef{#1hyphenmins}{#2}%
912     \fi}

```

\set@hyphenmins This macro sets the values of `\lefthyphenmin` and `\righthyphenmin`. It expects two values as its argument.

```

913 \def\set@hyphenmins#1#2{%
914     \lefthyphenmin#1\relax
915     \righthyphenmin#2\relax}

```

\ProvidesLanguage The identification code for each file is something that was introduced in $\text{\LaTeX 2}_{\epsilon}$. When the command `\ProvidesFile` does not exist, a dummy definition is provided temporarily. For use in the language definition file the command `\ProvidesLanguage` is defined by `babel`.

Depending on the format, i.e., or if the former is defined, we use a similar definition or not.

```

916 \ifx\ProvidesFile\@undefined
917     \def\ProvidesLanguage#1[#2 #3 #4]{%
918         \wlog{Language: #1 #4 #3 <#2>}%
919     }
920 \else
921     \def\ProvidesLanguage#1{%
922         \begingroup
923             \catcode`\ 10 %
924             \@makeother\/%
925             \@ifnextchar[%]
926                 {\@provideslanguage{#1}}{\@provideslanguage{#1}[]}
927     \def\@provideslanguage#1[#2]{%
928         \wlog{Language: #1 #2}%
929         \expandafter\xdef\csname ver@#1.ldf\endcsname{#2}%
930     \endgroup}
931 \fi

```

\originalTeX The macro `\originalTeX` should be known to $\TeX$ at this moment. As it has to be expandable we `\let` it to `\@empty` instead of `\relax`.

```
932 \ifx\originalTeX\@undefined\let\originalTeX\@empty\fi
```

Because this part of the code can be included in a format, we make sure that the macro which initializes the save mechanism, `\babel@beginsave`, is not considered to be undefined.

```
933 \ifx\babel@beginsave\@undefined\let\babel@beginsave\relax\fi
```

A few macro names are reserved for future releases of babel, which will use the concept of ‘locale’:

```
934 \providecommand\setlocale{\bbl@error{not-yet-available}}{}{}{}
935 \let\uselocale\setlocale
936 \let\locale\setlocale
937 \let\selectlocale\setlocale
938 \let\textlocale\setlocale
939 \let\textlanguage\setlocale
940 \let\languagegettext\setlocale
```

4.2. Errors

\@nolanerr

\@nopatterns The babel package will signal an error when a documents tries to select a language that hasn’t been defined earlier. When a user selects a language for which no hyphenation patterns were loaded into the format he will be given a warning about that fact. We revert to the patterns for `\language=0` in that case. In most formats that will be (US)english, but it might also be empty.

\@noopterr When the package was loaded without options not everything will work as expected. An error message is issued in that case.

When the format knows about `\PackageError` it must be $\LaTeX 2_{\epsilon}$, so we can safely use its error handling interface. Otherwise we’ll have to ‘keep it simple’.

Infos are not written to the console, but on the other hand many people think warnings are errors, so a further message type is defined: an important info which is sent to the console.

```
941 \edef\bbl@nulllanguage{\string\language=0}
942 \def\bbl@nocaption#1#2{\NoCaseChange{\protect\bbl@nocaption@i{#1}{#2}}}
943 \def\bbl@nocaption@i#1#2{% 1: text to be printed 2: caption macro \langXname
944   \global\@namedef{#2}{\textbf{?#1?}}}%
945   \@nameuse{#2}%
946   \edef\bbl@tempa{#1}%
947   \bbl@sreplace\bbl@tempa{name}}}%
948   \bbl@warning{%
949     \@backslashchar#1 not set for '\language'. Please,\\%
950     define it after the language has been loaded\\%
951     (typically in the preamble) with:\\%
952     \string\setlocalecaption{\language}\bbl@tempa{.}\\%
953     Feel free to contribute on github.com/latex3/babel.\\%
954     Reported}}
955 \def\bbl@tentative{\protect\bbl@tentative@i}
956 \def\bbl@tentative@i#1{%
957   \bbl@warning{%
958     Some functions for '#1' are tentative.\\%
959     They might not work as expected and their behavior\\%
960     could change in the future.\\%
961     Reported}}
962 \def\@nolanerr#1{\bbl@error{undefined-language}{#1}}{}{}
963 \def\@nopatterns#1{%
964   \bbl@warning
965     {No hyphenation patterns were preloaded for\\%
966     the language '#1' into the format.\\%
967     Please, configure your TeX system to add them and\\%
968     rebuild the format. Now I will use the patterns\\%
969     preloaded for \bbl@nulllanguage\space instead}}
970 \let\bbl@usehooks\@gobbletwo
```

Here ended the now discarded switch.def.
 Here also (currently) ends the base option.
 971 \ifx\bbl@onlyswitch\@empty\endinput\fi

4.3. More on selection

\babelensure The user command just parses the optional argument and creates a new macro named `\bbl@e@<language>`. We register a hook at the `afterextras` event which just executes this macro in a “complete” selection (which, if undefined, is `\relax` and does nothing). This part is somewhat involved because we have to make sure things are expanded the correct number of times.

The macro `\bbl@e@<language>` contains `\bbl@ensure{<include>}{<exclude>}{<fontenc>}`, which in turn loops over the macros names in `\bbl@captionslist`, excluding (with the help of `\in@`) those in the exclude list. If the fontenc is given (and not `\relax`), the `\fontencoding` is also added. Then we loop over the include list, but if the macro already contains `\foreignlanguage`, nothing is done. Note this macro (1) is not restricted to the preamble, and (2) changes are local.

```

972 \bbl@trace{Defining babelensure}
973 \newcommand\babelensure[2][]{%
974   \AddBabelHook{babel-ensure}{afterextras}{%
975     \ifcase\bbl@select@type
976       \bbl@c{l}{e}%
977     \fi}%
978   \begingroup
979     \let\bbl@ens@include\@empty
980     \let\bbl@ens@exclude\@empty
981     \def\bbl@ens@fontenc{\relax}%
982     \def\bbl@tempb##1{%
983       \ifx\@empty##1\else\noexpand##1\expandafter\bbl@tempb\fi}%
984     \edef\bbl@tempa{\bbl@tempb#1\@empty}%
985     \def\bbl@tempb##1=##2\@{\@namedef{\bbl@ens@##1}{##2}}%
986     \bbl@foreach\bbl@tempa{\bbl@tempb##1\@}%
987     \def\bbl@tempc{\bbl@ensure}%
988     \expandafter\bbl@add\expandafter\bbl@tempc\expandafter{%
989       \expandafter{\bbl@ens@include}}%
990     \expandafter\bbl@add\expandafter\bbl@tempc\expandafter{%
991       \expandafter{\bbl@ens@exclude}}%
992     \toks@\expandafter{\bbl@tempc}%
993     \bbl@exp{%
994   \endgroup
995   \def<\bbl@e@#2>{\the\toks@{\bbl@ens@fontenc}}}%
996 \def\bbl@ensure#1#2#3{% 1: include 2: exclude 3: fontenc
997   \def\bbl@tempb##1{% elt for (excluding) \bbl@captionslist list
998     \ifx##1\undefined % 3.32 - Don't assume the macro exists
999       \edef##1{\noexpand\bbl@nocaption
1000         {\bbl@stripslash##1}{\language\name\bbl@stripslash##1}}%
1001     \fi
1002     \ifx##1\@empty\else
1003       \in@{##1}{#2}%
1004       \ifin@ \else
1005         \let\bbl@tempc\language\name
1006         \bbl@replace\bbl@tempc{-}{@}%
1007         \bbl@ifunset{\bbl@ensure@\bbl@tempc}%
1008         {\bbl@exp{%
1009           \\DeclareRobustCommand\<\bbl@ensure@\bbl@tempc>[1]{%
1010             \\foreignlanguage{\language\name}%
1011             {\ifx\relax#3\else
1012               \\fontencoding{#3}\\selectfont
1013             \fi
1014             #####1}}}%
1015         {}%
1016         \toks@\expandafter{##1}%
1017         \edef##1{%
1018           \bbl@csarg\noexpand{ensure@\bbl@tempc}%

```

```

1019         {\the\toks@}}}%
1020     \fi
1021     \expandafter\babel@tempb
1022 \fi}%
1023 \expandafter\babel@tempb\babel@captionslist\today\@empty
1024 \def\babel@tempa##1{% elt for include list
1025     \ifx##1\@empty\else
1026         \let\babel@tempc\language\language
1027         \babel@replace\babel@tempc{-}{@}%
1028         \babel@csarg\in@{ensure@\babel@tempc\expandafter}\expandafter{##1}%
1029         \ifin@\else
1030             \babel@tempb##1\@empty
1031         \fi
1032         \expandafter\babel@tempa
1033     \fi}%
1034 \babel@tempa#1\@empty}
1035 \def\babel@captionslist{%
1036     \prefacename\refname\abstractname\bibname\chaptername\appendixname
1037     \contentsname\listfigurename\listtablename\indexname\figurename
1038     \tablename\partname\enclname\ccname\headtoname\pagename\seename
1039     \alsoname\proofname\glossaryname}

```

4.4. Short tags

\babeltags This macro is straightforward. After zapping spaces, we loop over the list and define the macros `\text<tag>` and `\<tag>`. Definitions are first expanded so that they don't contain `\csname` but the actual macro.

```

1040 \babel@trace{Short tags}
1041 \newcommand\babeltags[1]{%
1042     \edef\babel@tempa{\zap@space#1 \@empty}%
1043     \def\babel@tempb##1=##2\@{
1044         \edef\babel@tempc{
1045             \noexpand\newcommand
1046             \expandafter\noexpand\csname ##1\endcsname{
1047                 \noexpand\protect
1048                 \expandafter\noexpand\csname otherlanguage*\endcsname{##2}}
1049             \noexpand\newcommand
1050             \expandafter\noexpand\csname text##1\endcsname{
1051                 \noexpand\foreignlanguage{##2}}}
1052         \babel@tempc}%
1053     \babel@for\babel@tempa\babel@tempa{
1054         \expandafter\babel@tempb\babel@tempa\@{

```

4.5. Compatibility with language.def

Plain e-TeX doesn't rely on `language.dat`, but babel can be made compatible with this format easily.

```

1055 \babel@trace{Compatibility with language.def}
1056 \ifx\directlua\@undefined\else
1057     \ifx\babel@luapatterns\@undefined
1058         \input luababel.def
1059     \fi
1060 \fi
1061 \ifx\babel@languages\@undefined
1062     \ifx\directlua\@undefined
1063         \openin0 = language.def
1064         \ifeof0
1065             \closein0
1066             \message{I couldn't find the file language.def}
1067         \else
1068             \closein0
1069             \begingroup
1070                 \def\addlanguage#1#2#3#4#5{

```

```

1071      \expandafter\ifx\csname lang@#1\endcsname\relax\else
1072      \global\expandafter\let\csname l@#1\expandafter\endcsname
1073      \csname lang@#1\endcsname
1074      \fi}%
1075      \def\uselanguage#1{%
1076      \input language.def
1077      \endgroup
1078      \fi
1079      \fi
1080      \chardef\l@english\z@
1081      \fi

```

\addto It takes two arguments, a *(control sequence)* and TeX-code to be added to the *(control sequence)*.

If the *(control sequence)* has not been defined before it is defined now. The control sequence could also expand to `\relax`, in which case a circular definition results. The net result is a stack overflow. Note there is an inconsistency, because the assignment in the last branch is global.

```

1082 \def\addto#1#2{%
1083   \ifx#1\@undefined
1084     \def#1{#2}%
1085   \else
1086     \ifx#1\relax
1087       \def#1{#2}%
1088     \else
1089       {\toks@\expandafter{#1#2}%
1090        \xdef#1{\the\toks@}}%
1091     \fi
1092   \fi}

```

4.6. Hooks

Admittedly, the current implementation is a somewhat simplistic and does very little to catch errors, but it is meant for developers, after all. `\bbl@usehooks` is the commands used by babel to execute hooks defined for an event.

```

1093 \bbl@trace{Hooks}
1094 \newcommand\AddBabelHook[3][ ]{%
1095   \bbl@iifunset\bbl@hk@#2{\EnableBabelHook{#2}}}%
1096   \def\bbl@tempa##1,##3=##2,##3\@empty{\def\bbl@tempb{##2}}%
1097   \expandafter\bbl@tempa\bbl@evargs,##3=,\@empty
1098   \bbl@iifunset\bbl@ev@#2@#3@#1{%
1099     {\bbl@csarg\bbl@add{ev@#3@#1}{\bbl@elth{#2}}}%
1100     {\bbl@csarg\let{ev@#2@#3@#1}\relax}%
1101   \bbl@csarg\newcommand{ev@#2@#3@#1}{\bbl@tempb}}
1102 \newcommand\EnableBabelHook[1]{\bbl@csarg\let{hk@#1}\@firstofone}
1103 \newcommand\DisableBabelHook[1]{\bbl@csarg\let{hk@#1}\@gobble}
1104 \def\bbl@usehooks{\bbl@usehooks@lang\language}
1105 \def\bbl@usehooks@lang#1#2#3{% Test for Plain
1106   \ifx\UseHook\@undefined\else\UseHook{babel/*/#2}\fi
1107   \def\bbl@elth##1{%
1108     \bbl@cs{hk@##1}{\bbl@cs{ev@##1@#2@#3}}%
1109     \bbl@cs{ev@#2@}%
1110   \ifx\language\@undefined\else % Test required for Plain (?)
1111     \ifx\UseHook\@undefined\else\UseHook{babel/#1/#2}\fi
1112   \def\bbl@elth##1{%
1113     \bbl@cs{hk@##1}{\bbl@cs{ev@##1@#2@#1@#3}}%
1114     \bbl@cs{ev@#2@#1}%
1115   \fi}

```

To ensure forward compatibility, arguments in hooks are set implicitly. So, if a further argument is added in the future, there is no need to change the existing code. Note events intended for `hyphen.cfg` are also loaded (just in case you need them for some reason).

```

1116 \def\bbl@evargs{,% <- don't delete this comma

```

```

1117 everylanguage=1,loadkernel=1,loadpatterns=1,loadexceptions=1,%
1118 adddialect=2,patterns=2,defaultcommands=0,encodedcommands=2,write=0,%
1119 beforeextras=0,afterextras=0,stopcommands=0,stringprocess=0,%
1120 hyphenation=2,initiateactive=3,afterreset=0,foreign=0,foreign*=0,%
1121 beforestart=0,language=2,begindocument=1}
1122 \ifx\NewHook\@undefined\else % Test for Plain (?)
1123 \def\bbl@tempa#1=#2\@{\NewHook{babel/#1}\NewHook{babel/*/#1}}
1124 \bbl@foreach\bbl@evargs{\bbl@tempa#1\@@}
1125 \fi

```

Since the following command is meant for a hook (although a $\LaTeX$ one), it's placed here.

```

1126 \providecommand\PassOptionsToLocale[2]{%
1127 \bbl@csarg\bbl@add@list{passto@#2}{#1}}

```

4.7. Setting up language files

\LdfInit \LdfInit macro takes two arguments. The first argument is the name of the language that will be defined in the language definition file; the second argument is either a control sequence or a string from which a control sequence should be constructed. The existence of the control sequence indicates that the file has been processed before.

At the start of processing a language definition file we always check the category code of the at-sign. We make sure that it is a 'letter' during the processing of the file. We also save its name as the last called option, even if not loaded.

Another character that needs to have the correct category code during processing of language definition files is the equals sign, '=', because it is sometimes used in constructions with the \let primitive. Therefore we store its current catcode and restore it later on.

Now we check whether we should perhaps stop the processing of this file. To do this we first need to check whether the second argument that is passed to \LdfInit is a control sequence. We do that by looking at the first token after passing #2 through string. When it is equal to \@backslashchar we are dealing with a control sequence which we can compare with \@undefined.

If so, we call \ldf@quit to set the main language, restore the category code of the @-sign and call \endinput

When #2 was *not* a control sequence we construct one and compare it with \relax.

Finally we check \originalTeX.

```

1128 \bbl@trace{Macros for setting language files up}
1129 \def\bbl@ldfinit{%
1130 \let\bbl@screset\@empty
1131 \let\BabelStrings\bbl@opt@string
1132 \let\BabelOptions\@empty
1133 \let\BabelLanguages\relax
1134 \ifx\originalTeX\@undefined
1135 \let\originalTeX\@empty
1136 \else
1137 \originalTeX
1138 \fi}
1139 \def\LdfInit#1#2{%
1140 \chardef\atcatcode=\catcode`\@
1141 \catcode`\@=11\relax
1142 \chardef\eqcatcode=\catcode`\=
1143 \catcode`\==12\relax
1144 \ifpackagewith{babel}{ensureinfo=off}{}%
1145 {\ifx\InputIfFileExists\@undefined\else
1146 \bbl@ifunset{\bbl@lname@#1}%
1147 {\let\bbl@ensuring\@empty % Flag used in babel-serbianc.tex
1148 \def\language{#1}%
1149 \bbl@id@assign
1150 \bbl@load@info{#1}}}%
1151 {}%
1152 \fi}%
1153 \expandafter\if\expandafter\@backslashchar
1154 \expandafter\@car\string#2\@nil
1155 \ifx#2\@undefined\else
1156 \ldf@quit{#1}%

```

```

1157 \fi
1158 \else
1159 \expandafter\ifx\csname#2\endcsname\relax\else
1160 \ldf@quit{#1}%
1161 \fi
1162 \fi
1163 \bbl@ldfinit}

```

\ldf@quit This macro interrupts the processing of a language definition file. Remember `\endinput` is not executed immediately, but delayed to the end of the current line in the input file.

```

1164 \def\ldf@quit#1{%
1165 \expandafter\main@language\expandafter{#1}%
1166 \catcode`\@=\atcatcode \let\atcatcode\relax
1167 \catcode`\==\eqcatcode \let\eqcatcode\relax
1168 \endinput}

```

\ldf@finish This macro takes one argument. It is the name of the language that was defined in the language definition file.

We load the local configuration file if one is present, we set the main language (taking into account that the argument might be a control sequence that needs to be expanded) and reset the category code of the `@`-sign.

```

1169 \def\bbl@afterldf{%
1170 \bbl@afterlang
1171 \let\bbl@afterlang\relax
1172 \let\BabelModifiers\relax
1173 \let\bbl@screset\relax}%
1174 \def\ldf@finish#1{%
1175 \loadlocalcfg{#1}%
1176 \bbl@afterldf
1177 \expandafter\main@language\expandafter{#1}%
1178 \catcode`\@=\atcatcode \let\atcatcode\relax
1179 \catcode`\==\eqcatcode \let\eqcatcode\relax}

```

After the preamble of the document the commands `\LdfInit`, `\ldf@quit` and `\ldf@finish` are no longer needed. Therefore they are turned into warning messages in *LaTeX*.

```

1180 \@onlypreamble\LdfInit
1181 \@onlypreamble\ldf@quit
1182 \@onlypreamble\ldf@finish

```

\main@language

\bbl@main@language This command should be used in the various language definition files. It stores its argument in `\bbl@main@language`; to be used to switch to the correct language at the beginning of the document.

```

1183 \def\main@language#1{%
1184 \def\bbl@main@language{#1}%
1185 \let\language\bbl@main@language
1186 \let\localename\bbl@main@language
1187 \let\mainlocalename\bbl@main@language
1188 \bbl@id@assign
1189 \ifcase\bbl@engine\or
1190 \ifx\setattribute\@undefined\else
1191 \setattribute\bbl@attr@locale\localeid
1192 \fi
1193 \fi
1194 \bbl@patterns{\language}

```

We also have to make sure that some code gets executed at the beginning of the document, either when the aux file is read or, if it does not exist, when the `\AtBeginDocument` is executed. Languages do not set `\pagedir`, so we set here for the whole document to the main `\bodydir`.

The code written to the aux file attempts to avoid errors if babel is removed from the document.

```

1195 \def\bbl@beforestart{%

```

```

1196 \def\@nolanerr##1{%
1197   \bbl@carg\chardef{l@##1}\z@
1198   \bbl@warning{Undefined language '##1' in aux.\@Reported}}%
1199 \bbl@usehooks{beforestart}{}%
1200 \global\let\bbl@beforestart\relax
1201 \AtBeginDocument{%
1202   {\@nameuse{bbl@beforestart}}% Group!
1203   \if@filesw
1204     \providecommand\babel@aux[2]{}%
1205     \immediate\write\@mainaux{\unexpanded{%
1206       \providecommand\babel@aux[2]{\global\let\babel@toc\@gobbletwo}}}%
1207     \immediate\write\@mainaux{\string\@nameuse{bbl@beforestart}}}%
1208   \fi
1209   \expandafter\selectlanguage\expandafter{\bbl@main@language}%
1210   \ifbbl@single % must go after the line above.
1211     \renewcommand\selectlanguage[1]{}%
1212     \renewcommand\foreignlanguage[2]{#2}%
1213     \global\let\babel@aux\@gobbletwo % Also as flag
1214   \fi}

A bit of optimization. Select in heads/feet the language only if necessary.

1215 \def\select@language@x#1{%
1216   \ifcase\bbl@select@type
1217     \bbl@ifsamestring\language{#1}{\select@language{#1}}%
1218   \else
1219     \select@language{#1}%
1220   \fi}

```

4.8. Shorthands

The macro `\initiate@active@char` below takes all the necessary actions to make its argument a shorthand character. The real work is performed once for each character. But first we define a little tool.

```

1221 \bbl@trace{Shorhands}
1222 \def\bbl@withactive#1#2{%
1223   \begingroup
1224   \lccode`~=#2\relax
1225   \lowercase{\endgroup#1~}}

```

\bbl@add@special The macro `\bbl@add@special` is used to add a new character (or single character control sequence) to the macro `\dospecials` (and `\@sanitize` if $\TeX$ is used). It is used only at one place, namely when `\initiate@active@char` is called (which is ignored if the char has been made active before). Because `\@sanitize` can be undefined, we put the definition inside a conditional.

Items are added to the lists without checking its existence or the original catcode. It does not hurt, but should be fixed. It's already done with `\nfss@catcodes`, added in 3.10.

```

1226 \def\bbl@add@special#1{% 1:a macro like "\", \?, etc.
1227   \bbl@add\dospecials{\do#1}% test @sanitize = \relax, for back. compat.
1228   \bbl@ifunset{@sanitize}{\bbl@add\@sanitize{\makeother#1}}%
1229   \ifx\nfss@catcodes\undefined\else
1230     \begingroup
1231     \catcode`#1\active
1232     \nfss@catcodes
1233     \ifnum\catcode`#1=\active
1234       \endgroup
1235       \bbl@add\nfss@catcodes{\@makeother#1}%
1236     \else
1237       \endgroup
1238     \fi
1239   \fi}

```

\initiate@active@char A language definition file can call this macro to make a character active. This macro takes one argument, the character that is to be made active. When the character was already active this macro does nothing. Otherwise, this macro defines the control sequence `\normal@char<char>` to expand to the character in its ‘normal state’ and it defines the active character to expand to `\normal@char<char>` by default (`<char>` being the character to be made active). Later its definition can be changed to expand to `\active@char<char>` by calling `\bbl@activate{<char>}`.

For example, to make the double quote character active one could have `\initiate@active@char{"}` in a language definition file. This defines `"` as `\active@prefix "\active@char"` (where the first `"` is the character with its original catcode, when the shorthand is created, and `\active@char` is a single token). In protected contexts, it expands to `\protect "` or `\noexpand "` (i.e., with the original `"`); otherwise `\active@char` is executed. This macro in turn expands to `\normal@char` in “safe” contexts (e.g., `\label`), but `\user@active` in normal “unsafe” ones. The latter search a definition in the user, language and system levels, in this order, but if none is found, `\normal@char` is used. However, a deactivated shorthand (with `\bbl@deactivate` defined as `\active@prefix "\normal@char"`).

The following macro is used to define shorthands in the three levels. It takes 4 arguments: the (string’ed) character, `\<level>@group`, `<level>@active` and `<next-level>@active` (except in system).

```
1240 \def\bbl@active@def#1#2#3#4{%
1241   \namedef{#3#1}{%
1242     \expandafter\ifx\csname#2@sh@#1\endcsname\relax
1243       \bbl@afterelse\bbl@sh@select#2#1{#3@arg#1}{#4#1}%
1244     \else
1245       \bbl@afterfi\csname#2@sh@#1\endcsname
1246     \fi}%
```

When there is also no current-level shorthand with an argument we will check whether there is a next-level defined shorthand for this active character.

```
1247   \long\@namedef{#3@arg#1}##1{%
1248     \expandafter\ifx\csname#2@sh@#1@string##1\endcsname\relax
1249       \bbl@afterelse\csname#4#1\endcsname##1%
1250     \else
1251       \bbl@afterfi\csname#2@sh@#1@string##1\endcsname
1252     \fi}}%
```

`\initiate@active@char` calls `\@initiate@active@char` with 3 arguments. All of them are the same character with different catcodes: active, other (`\string’ed`) and the original one. This trick simplifies the code a lot.

```
1253 \def\@initiate@active@char#1{%
1254   \bbl@ifunset{active@char\string#1}%
1255   {\bbl@withactive
1256     {\expandafter\@initiate@active@char\expandafter}#1\string#1#1}%
1257   {}}
```

The very first thing to do is saving the original catcode and the original definition, even if not active, which is possible (undefined characters require a special treatment to avoid making them `\relax` and preserving some degree of protection).

```
1258 \def\@initiate@active@char#1#2#3{%
1259   \bbl@csarg\edef{oricat@#2}{\catcode`#2=\the\catcode`#2\relax}%
1260   \ifx#1\@undefined
1261     \bbl@csarg\def{oridef@#2}{\def#1{\active@prefix#1\@undefined}}%
1262   \else
1263     \bbl@csarg\let{oridef@#2}#1%
1264     \bbl@csarg\edef{oridef@#2}{%
1265       \let\noexpand#1%
1266       \expandafter\noexpand\csname bbl@oridef@#2\endcsname}%
1267   \fi
```

If the character is already active we provide the default expansion under this shorthand mechanism. Otherwise we write a message in the transcript file, and define `\normal@char<char>` to expand to the character in its default state. If the character is mathematically active when babel is loaded (for example `'`) the normal expansion is somewhat different to avoid an infinite loop (but it does not prevent the loop if the mathcode is set to `"8000 a posteriori`).

```
1268   \ifx#1#3\relax
```

```

1269 \expandafter\let\csname normal@char#2\endcsname#3%
1270 \else
1271 \bbl@info{Making #2 an active character}%
1272 \ifnum\mathcode`#2=\ifodd\bbl@engine"1000000 \else"8000 \fi
1273 \@namedef{normal@char#2}{%
1274 \textormath{#3}{\csname bbl@oridef@#2\endcsname}}%
1275 \else
1276 \@namedef{normal@char#2}{#3}%
1277 \fi

```

To prevent problems with the loading of other packages after babel we reset the catcode of the character to the original one at the end of the package and of each language file (except with KeepShorthandsActive). It is re-activate again at `\begin{document}`. We also need to make sure that the shorthands are active during the processing of the aux file. Otherwise some citations may give unexpected results in the printout when a shorthand was used in the optional argument of `\bibitem` for example. Then we make it active (not strictly necessary, but done for backward compatibility).

```

1278 \bbl@restoreactive{#2}%
1279 \AtBeginDocument{%
1280 \catcode`#2\active
1281 \if@files
1282 \immediate\write\@mainaux{\catcode`\string#2\active}%
1283 \fi}%
1284 \expandafter\bbl@add@special\csname#2\endcsname
1285 \catcode`#2\active
1286 \fi

```

Now we have set `\normal@char{char}`, we must define `\active@char{char}`, to be executed when the character is activated. We define the first level expansion of `\active@char{char}` to check the status of the `@safe@actives` flag. If it is set to true we expand to the ‘normal’ version of this character, otherwise we call `\user@active{char}` to start the search of a definition in the user, language and system levels (or eventually `normal@char{char}`).

```

1287 \let\bbl@tempa\@firstoftwo
1288 \if\string^#2%
1289 \def\bbl@tempa{\noexpand\textormath}%
1290 \else
1291 \ifx\bbl@mathnormal\@undefined\else
1292 \let\bbl@tempa\bbl@mathnormal
1293 \fi
1294 \fi
1295 \expandafter\edef\csname active@char#2\endcsname{%
1296 \bbl@tempa
1297 {\noexpand\if@safe@actives
1298 \noexpand\expandafter
1299 \expandafter\noexpand\csname normal@char#2\endcsname
1300 \noexpand\else
1301 \noexpand\expandafter
1302 \expandafter\noexpand\csname bbl@doactive#2\endcsname
1303 \noexpand\fi}%
1304 {\expandafter\noexpand\csname normal@char#2\endcsname}}%
1305 \bbl@csarg\edef{doactive#2}{%
1306 \expandafter\noexpand\csname user@active#2\endcsname}%

```

We now define the default values which the shorthand is set to when activated or deactivated. It is set to the deactivated form (globally), so that the character expands to

$$\text{active@prefix}\langle char\rangle\text{normal@char}\langle char\rangle$$

(where `\active@char{char}` is *one* control sequence!).

```

1307 \bbl@csarg\edef{active@#2}{%
1308 \noexpand\active@prefix\noexpand#1%
1309 \expandafter\noexpand\csname active@char#2\endcsname}%
1310 \bbl@csarg\edef{normal@#2}{%
1311 \noexpand\active@prefix\noexpand#1%
1312 \expandafter\noexpand\csname normal@char#2\endcsname}%
1313 \bbl@ncarg\let#1\bbl@normal@#2}%

```

The next level of the code checks whether a user has defined a shorthand for himself with this character. First we check for a single character shorthand. If that doesn't exist we check for a shorthand with an argument.

```
1314 \bbl@active@def#2\user@group{user@active}{language@active}%
1315 \bbl@active@def#2\language@group{language@active}{system@active}%
1316 \bbl@active@def#2\system@group{system@active}{normal@char}%
```

In order to do the right thing when a shorthand with an argument is used by itself at the end of the line we provide a definition for the case of an empty argument. For that case we let the shorthand character expand to its non-active self. Also, When a shorthand combination such as ' ' ends up in a heading $\TeX$ would see `\protect'\protect'`. To prevent this from happening a couple of shorthand needs to be defined at user level.

```
1317 \expandafter\edef\csname\user@group @sh#2@@\endcsname
1318 {\expandafter\noexpand\csname normal@char#2\endcsname}%
1319 \expandafter\edef\csname\user@group @sh#2@\string\protect\endcsname
1320 {\expandafter\noexpand\csname user@active#2\endcsname}%
```

Finally, a couple of special cases are taken care of. (1) If we are making the right quote (') active we need to change `\pr@m@s` as well. Also, make sure that a single ' in math mode 'does the right thing'. (2) If we are using the caret (^) as a shorthand character special care should be taken to make sure math still works. Therefore an extra level of expansion is introduced with a check for math mode on the upper level.

```
1321 \if\string'#2%
1322 \let\prim@s\bbl@prim@s
1323 \let\active@math@prime#1%
1324 \fi
1325 \bbl@usehooks{initiateactive}{{#1}{#2}{#3}}}
```

The following package options control the behavior of shorthands in math mode.

```
1326 <(*More package options)> ≡
1327 \DeclareOption{math=active}{}
1328 \DeclareOption{math=normal}{\def\bbl@mathnormal{\noexpand\textormath}}
1329 <(/More package options)>
```

Initiating a shorthand makes active the char. That is not strictly necessary but it is still done for backward compatibility. So we need to restore the original catcode at the end of package *and* the end of the *ldf*.

```
1330 \ifpackagewith{babel}{KeepShorthandsActive}%
1331 {\let\bbl@restoreactive@gobble}%
1332 {\def\bbl@restoreactive#1{%
1333 \bbl@exp{%
1334 \\\AfterBabelLanguage\\CurrentOption
1335 {\catcode`#1=\the\catcode`#1\relax}%
1336 \\\AtEndOfPackage
1337 {\catcode`#1=\the\catcode`#1\relax}}}%
1338 \AtEndOfPackage{\let\bbl@restoreactive@gobble}}
```

\bbl@sh@select This command helps the shorthand supporting macros to select how to proceed.

Note that this macro needs to be expandable as do all the shorthand macros in order for them to work in expansion-only environments such as the argument of `\hyphenation`.

This macro expects the name of a group of shorthands in its first argument and a shorthand character in its second argument. It will expand to either `\bbl@firstcs` or `\bbl@scndcs`. Hence two more arguments need to follow it.

```
1339 \def\bbl@sh@select#1#2{%
1340 \expandafter\ifx\csname#1@sh#2@sel\endcsname\relax
1341 \bbl@afterelse\bbl@scndcs
1342 \else
1343 \bbl@afterfi\csname#1@sh#2@sel\endcsname
1344 \fi}
```

\active@prefix Used in the expansion of active characters has a function similar to \OT1-cmd in that it \protects the active character whenever \protect is *not* \@typeset@protect. The \@gobble is needed to remove a token such as \activechar: (when the double colon was the active character to be dealt with). There are two definitions, depending on \ifincsname is available. If there is, the expansion will be more robust.

```

1345 \begingroup
1346 \bbl@ifunset{ifincsname}
1347 {\gdef\active@prefix#1{%
1348   \ifx\protect\@typeset@protect
1349   \else
1350     \ifx\protect\@unexpandable@protect
1351       \noexpand#1%
1352     \else
1353       \protect#1%
1354     \fi
1355     \expandafter\@gobble
1356   \fi}}
1357 {\gdef\active@prefix#1{%
1358   \ifincsname
1359     \string#1%
1360     \expandafter\@gobble
1361   \else
1362     \ifx\protect\@typeset@protect
1363     \else
1364       \ifx\protect\@unexpandable@protect
1365         \noexpand#1%
1366       \else
1367         \protect#1%
1368       \fi
1369       \expandafter\expandafter\expandafter\@gobble
1370     \fi
1371   \fi}}
1372 \endgroup

```

if@safe@actives In some circumstances it is necessary to be able to reset the shorthand to its ‘normal’ value (usually the character with catcode ‘other’) on the fly. For this purpose the switch @safe@actives is available. The setting of this switch should be checked in the first level expansion of \active@char<char>. When this expansion mode is active (with \@safe@activetrue), something like "13"13 becomes "12"12 in an \edef (in other words, shorthands are \string’ed). This contrasts with \protected@edef, where catcodes are always left unchanged. Once converted, they can be used safely even after this expansion mode is deactivated (with \@safe@activefalse).

```

1373 \newif\if@safe@actives
1374 \@safe@activesfalse

```

\bbl@restore@actives When the output routine kicks in while the active characters were made “safe” this must be undone in the headers to prevent unexpected typeset results. For this situation we define a command to make them “unsafe” again.

```

1375 \def\bbl@restore@actives{\if@safe@actives\@safe@activesfalse\fi}

```

\bbl@activate

\bbl@deactivate Both macros take one argument, like \initiate@active@char. The macro is used to change the definition of an active character to expand to \active@char<char> in the case of \bbl@activate, or \normal@char<char> in the case of \bbl@deactivate.

```

1376 \chardef\bbl@activated\z@
1377 \def\bbl@activate#1{%
1378   \chardef\bbl@activated\@ne
1379   \bbl@withactive{\expandafter\let\expandafter}#1%
1380   \csname bbl@active@\string#1\endcsname}
1381 \def\bbl@deactivate#1{%
1382   \chardef\bbl@activated\tw@
1383   \bbl@withactive{\expandafter\let\expandafter}#1%

```

```
1384 \csname bbl@normal@\string#1\endcsname}
```

\bbl@firstcs

\bbl@scndcs These macros are used only as a trick when declaring shorthands.

```
1385 \def\bbl@firstcs#1#2{\csname#1\endcsname}
1386 \def\bbl@scndcs#1#2{\csname#2\endcsname}
```

\declare@shorthand Used to declare a shorthand on a certain level. It takes three arguments:

1. a name for the collection of shorthands, i.e., ‘system’, or ‘dutch’;
2. the character (sequence) that makes up the shorthand, i.e., ~ or "a;
3. the code to be executed when the shorthand is encountered.

The auxiliary macro `\babel@texpdf` improves the interoperativity with `hyperref` and takes 4 arguments: (1) The $\TeX$ code in text mode, (2) the string for `hyperref`, (3) the $\TeX$ code in math mode, and (4), which is currently ignored, but it’s meant for a string in math mode, like a minus sign instead of an hyphen (currently `hyperref` doesn’t discriminate the mode). This macro may be used in `ldf` files.

```
1387 \def\babel@texpdf#1#2#3#4{%
1388 \ifx\texorpdfstring\undefined
1389 \textormath{#1}{#3}%
1390 \else
1391 \texorpdfstring{\textormath{#1}{#3}}{#2}%
1392 % \texorpdfstring{\textormath{#1}{#3}}{\textormath{#2}{#4}}%
1393 \fi}
1394 %
1395 \def\declare@shorthand#1#2{\@decl@short{#1}#2@nil}
1396 \def\@decl@short#1#2#3@nil#4{%
1397 \def\bbl@tempa{#3}%
1398 \ifx\bbl@tempa\empty
1399 \expandafter\let\csname #1@sh@\string#2@sel\endcsname\bbl@scndcs
1400 \bbl@ifunset{#1@sh@\string#2@}{}%
1401 {\def\bbl@tempa{#4}%
1402 \expandafter\ifx\csname#1@sh@\string#2@\endcsname\bbl@tempa
1403 \else
1404 \bbl@info
1405 {Redefining #1 shorthand \string#2\}%
1406 in language \CurrentOption}%
1407 \fi}%
1408 \@namedef{#1@sh@\string#2@}{#4}%
1409 \else
1410 \expandafter\let\csname #1@sh@\string#2@sel\endcsname\bbl@firstcs
1411 \bbl@ifunset{#1@sh@\string#2@\string#3@}{}%
1412 {\def\bbl@tempa{#4}%
1413 \expandafter\ifx\csname#1@sh@\string#2@\string#3@\endcsname\bbl@tempa
1414 \else
1415 \bbl@info
1416 {Redefining #1 shorthand \string#2\string#3\}%
1417 in language \CurrentOption}%
1418 \fi}%
1419 \@namedef{#1@sh@\string#2@\string#3@}{#4}%
1420 \fi}
```

\textormath Some of the shorthands that will be declared by the language definition files have to be usable in both text and mathmode. To achieve this the helper macro `\textormath` is provided.

```
1421 \def\textormath{%
1422 \ifmmode
1423 \expandafter\@secondoftwo
1424 \else
1425 \expandafter\@firstoftwo
1426 \fi}
```

\user@group

\language@group

\system@group The current concept of ‘shorthands’ supports three levels or groups of shorthands. For each level the name of the level or group is stored in a macro. The default is to have a user group; use language group ‘english’ and have a system group called ‘system’.

```
1427 \def\user@group{user}
1428 \def\language@group{english}
1429 \def\system@group{system}
```

\usesshorthands This is the user level macro. It initializes and activates the character for use as a shorthand character (i.e., it’s active in the preamble). Languages can deactivate shorthands, so a starred version is also provided which activates them always after the language has been switched.

```
1430 \def\usesshorthands{%
1431   \ifstar\bbl@usesesh@s{\bbl@usesesh@x{}}
1432 \def\bbl@usesesh@s#1{%
1433   \bbl@usesesh@x
1434   {\AddBabelHook{babel-sh-\string#1}{afterextras}{\bbl@activate{#1}}}%
1435   {#1}}
1436 \def\bbl@usesesh@x#1#2{%
1437   \bbl@ifshorthand{#2}%
1438   {\def\user@group{user}%
1439    \initiate@active@char{#2}%
1440    #1%
1441    \bbl@activate{#2}}%
1442   {\bbl@error{shorthand-is-off}{#2}}}
```

\defineshorthand Currently we only support two groups of user level shorthands, named internally user and user@(\language) (language-dependent user shorthands). By default, only the first one is taken into account, but if the former is also used (in the optional argument of \defineshorthand) a new level is inserted for it (user@generic, done by \bbl@set@user@generic); we make also sure {} and \protect are taken into account in this new top level.

```
1443 \def\user@language@group{user@\language@group}
1444 \def\bbl@set@user@generic#1#2{%
1445   \bbl@ifunset{user@generic@active#1}%
1446   {\bbl@active@def#1\user@language@group{user@active}{user@generic@active}%
1447    \bbl@active@def#1\user@group{user@generic@active}{language@active}%
1448    \expandafter\edef\csname#2@sh@#1@\endcsname{%
1449     \expandafter\noexpand\csname normal@char#1\endcsname}%
1450    \expandafter\edef\csname#2@sh@#1@\string\protect\endcsname{%
1451     \expandafter\noexpand\csname user@active#1\endcsname}}%
1452   \@empty}
1453 \newcommand\defineshorthand[3][user]{%
1454   \edef\bbl@tempa{\zap@space#1 \@empty}%
1455   \bbl@for\bbl@tempb\bbl@tempa{%
1456     \if*\expandafter\@car\bbl@tempb\@nil
1457     \edef\bbl@tempb{user@\expandafter\@gobble\bbl@tempb}%
1458     \@expandtwoargs
1459     \bbl@set@user@generic{\expandafter\string\@car#2\@nil}\bbl@tempb
1460   \fi
1461   \declare@shorthand{\bbl@tempb}{#2}{#3}}
```

\languageshorthands A user level command to change the language from which shorthands are used. Unfortunately, babel currently does not keep track of defined groups, and therefore there is no way to catch a possible change in casing to fix it in the same way languages names are fixed.

```
1462 \def\languageshorthands#1{%
1463   \bbl@ifsamestring{none}{#1}{}%
1464   \bbl@once{short-\localename-#1}{%
1465     \bbl@info{'\localename' activates '#1' shorthands.\@Reported}}%
1466   \def\language@group{#1}}
```

\aliasshorthand *Deprecated.* First the new shorthand needs to be initialized. Then, we define the new shorthand in terms of the original one, but note with `\aliasshandands{"}{/}` is `\active@prefix / \active@char/`, so we still need to let the latter to `\active@char`".

```

1467 \def\aliasshorthand#1#2{%
1468   \bbl@ifshorthand{#2}%
1469   {\expandafter\ifx\csname active@char\string#2\endcsname\relax
1470     \ifx\document\@notprerr
1471       \@notshorthand{#2}%
1472     \else
1473       \initiate@active@char{#2}%
1474       \bbl@ccarg\let{active@char\string#2}{active@char\string#1}%
1475       \bbl@ccarg\let{normal@char\string#2}{normal@char\string#1}%
1476       \bbl@activate{#2}%
1477     \fi
1478   \fi}%
1479   {\bbl@error{shorthand-is-off}{#2}{}}}
```

\@notshorthand

```

1480 \def\@notshorthand#1{\bbl@error{not-a-shorthand}{#1}{}}
```

\shorthandon

\shorthandoff The first level definition of these macros just passes the argument on to `\bbl@switch@sh`, adding `\@nil` at the end to denote the end of the list of characters.

```

1481 \newcommand*\shorthandon[1]{\bbl@switch@sh\@ne#1\@nnil}
1482 \DeclareRobustCommand*\shorthandoff{%
1483   \@ifstar{\bbl@shorthandoff\tw@}{\bbl@shorthandoff\z@}}
1484 \def\bbl@shorthandoff#1#2{\bbl@switch@sh#1#2\@nnil}
```

\bbl@switch@sh The macro `\bbl@switch@sh` takes the list of characters apart one by one and subsequently switches the category code of the shorthand character according to the first argument of `\bbl@switch@sh`.

But before any of this switching takes place we make sure that the character we are dealing with is known as a shorthand character. If it is, a macro such as `\active@char`" should exist.

Switching off and on is easy – we just set the category code to ‘other’ (12) and `\active`. With the starred version, the original catcode and the original definition, saved in `@initiate@active@char`, are restored.

```

1485 \def\bbl@switch@sh#1#2{%
1486   \ifx#2\@nnil\else
1487     \bbl@ifunset{bbl@active@\string#2}%
1488     {\bbl@error{not-a-shorthand-b}{#2}{}}%
1489     {\ifcase#1 off, on, off*
1490       \catcode`#2\relax
1491     \or
1492       \catcode`#2\active
1493       \bbl@ifunset{bbl@shdef@\string#2}%
1494       {}%
1495       {\bbl@withactive{\expandafter\let\expandafter}#2%
1496         \csname bbl@shdef@\string#2\endcsname
1497         \bbl@csarg\let{shdef@\string#2}\relax}%
1498       \ifcase\bbl@activated\or
1499         \bbl@activate{#2}%
1500       \else
1501         \bbl@deactivate{#2}%
1502       \fi
1503     \or
1504       \bbl@ifunset{bbl@shdef@\string#2}%
1505       {\bbl@withactive{\bbl@csarg\let{shdef@\string#2}}#2}%
1506       {}%
1507       \csname bbl@oricat@\string#2\endcsname
1508       \csname bbl@oridef@\string#2\endcsname
1509     \fi}%

```

```

1510 \bbl@afterfi\bbl@switch@sh#1%
1511 \fi}

```

Note the value is that at the expansion time; e.g., in the preamble shorthands are usually deactivated.

```

1512 \def\babelshorthand{\active@prefix\babelshorthand\bbl@putsh}
1513 \def\bbl@putsh#1{%
1514 \bbl@ifunset{\bbl@active@\string#1}%
1515 {\bbl@putsh@i#1\@empty\@nnil}%
1516 {\csname bbl@active@\string#1\endcsname}}
1517 \def\bbl@putsh@i#1#2\@nnil{%
1518 \csname\language@group @sh@\string#1@%
1519 \ifx\@empty#2\else\string#2@\fi\endcsname}
1520 %
1521 \ifx\bbl@opt@shorthands\@nnil\else
1522 \let\bbl@s@initiate@active@char@initiate@active@char
1523 \def\initiate@active@char#1{%
1524 \bbl@ifshorthand{#1}{\bbl@s@initiate@active@char{#1}}{}}
1525 \let\bbl@s@switch@sh\bbl@switch@sh
1526 \def\bbl@switch@sh#1#2{%
1527 \ifx#2\@nnil\else
1528 \bbl@afterfi
1529 \bbl@ifshorthand{#2}{\bbl@s@switch@sh#1{#2}}{\bbl@switch@sh#1}%
1530 \fi}
1531 \let\bbl@s@activate\bbl@activate
1532 \def\bbl@activate#1{%
1533 \bbl@ifshorthand{#1}{\bbl@s@activate{#1}}{}}
1534 \let\bbl@s@deactivate\bbl@deactivate
1535 \def\bbl@deactivate#1{%
1536 \bbl@ifshorthand{#1}{\bbl@s@deactivate{#1}}{}}
1537 \fi

```

You may want to test if a character is a shorthand. Note it does not test whether the shorthand is on or off.

```

1538 \newcommand\ifbabelshorthand[3]{\bbl@ifunset{\bbl@active@\string#1}{#3}{#2}}

```

\bbl@prim@s

\bbl@pr@m@s One of the internal macros that are involved in substituting `\prime` for each right quote in mathmode is `\prim@s`. This checks if the next character is a right quote. When the right quote is active, the definition of this macro needs to be adapted to look also for an active right quote; the hat could be active, too.

```

1539 \def\bbl@prim@s{%
1540 \prime\futurelet\@let@token\bbl@pr@m@s}
1541 \def\bbl@if@primes#1#2{%
1542 \ifx#1\@let@token
1543 \expandafter\@firstoftwo
1544 \else\ifx#2\@let@token
1545 \bbl@afterelse\expandafter\@firstoftwo
1546 \else
1547 \bbl@afterfi\expandafter\@secondoftwo
1548 \fi\fi}
1549 \begingroup
1550 \catcode`\^=7 \catcode`\*=\active \lccode`\*='\^
1551 \catcode`\'=12 \catcode`\\"=\active \lccode`\\"='\`
1552 \lowercase{%
1553 \gdef\bbl@pr@m@s{%
1554 \bbl@if@primes" '%
1555 \pr@@@s
1556 {\bbl@if@primes*\^{\pr@@@t\egroup}}}
1557 \endgroup

```

Usually the `~` is active and expands to `\penalty\@M\_\_`. When it is written to the aux file it is written expanded. To prevent that and to be able to use the character `~` as a start character for a shorthand, it

is redefined here as a one character shorthand on system level. The system declaration is in most cases redundant (when ~ is still a non-break space), and in some cases is inconvenient (if ~ has been redefined); however, for backward compatibility it is maintained (some existing documents may rely on the babel value).

```
1558 \initiate@active@char{~}
1559 \declare@shorthand{system}{~}{\leavevmode\nobreak\ }
1560 \bbl@activate{~}
```

\OT1dqpos

\T1dqpos The position of the double quote character is different for the OT1 and T1 encodings. It will later be selected using the \f@encoding macro. Therefore we define two macros here to store the position of the character in these encodings.

```
1561 \expandafter\def\csname OT1dqpos\endcsname{127}
1562 \expandafter\def\csname T1dqpos\endcsname{4}
```

When the macro \f@encoding is undefined (as it is in plain T_EX) we define it here to expand to OT1

```
1563 \ifx\f@encoding\undefined
1564 \def\f@encoding{OT1}
1565 \fi
```

4.9. Language attributes

Language attributes provide a means to give the user control over which features of the language definition files he wants to enable.

\languageattribute The macro \languageattribute checks whether its arguments are valid and then activates the selected language attribute. First check whether the language is known, and then process each attribute in the list.

To make sure each attribute is selected only once, we store the already selected attributes in \bbl@known@attrs. When that control sequence is not yet defined this attribute is certainly not selected before.

When we end up here the attribute is not selected before. So, we add it to the list of selected attributes and execute the associated T_EX code.

```
1566 \bbl@trace{Language attributes}
1567 \newcommand\languageattribute[2]{%
1568 \def\bbl@tempc{#1}%
1569 \bbl@fixname\bbl@tempc
1570 \bbl@iflanguage\bbl@tempc{%
1571 \bbl@vforeach{#2}{%
1572 \ifx\bbl@known@attrs\undefined
1573 \in@false
1574 \else
1575 \bbl@xin@{,\bbl@tempc-##1,}{,\bbl@known@attrs,}%
1576 \fi
1577 \ifin@
1578 \bbl@warning{%
1579 You have more than once selected the attribute '##1'\%
1580 for language #1. Reported}%
1581 \else
1582 \bbl@info{Activated '##1' attribute for\\%
1583 '\bbl@tempc'. Reported}%
1584 \bbl@exp{%
1585 \\ \bbl@add@list\\ \bbl@known@attrs{\bbl@tempc-##1}}%
1586 \edef\bbl@tempa{\bbl@tempc-##1}%
1587 \expandafter\bbl@ifknown@ttrib\expandafter{\bbl@tempa}\bbl@attributes
1588 {\csname\bbl@tempc @attr##1\endcsname}%
1589 {\bbl@error{unknown-attribute}{\bbl@tempc}{##1}}}%
1590 \fi}}
1591 \@onlypreamble\languageattribute
```

\bbl@declare@ttribute This command adds the new language/attribute combination to the list of known attributes.

Then it defines a control sequence to be executed when the attribute is used in a document. The result of this should be that the macro `\extras...` for the current language is extended, otherwise the attribute will not work as its code is removed from memory at `\begin{document}`.

```
1592 \def\bbl@declare@ttribute#1#2#3{%
1593   \bbl@xin@{,#2,}{,\BabelModifiers,}%
1594   \ifin@
1595     \AfterBabelLanguage{#1}{\languageattribute{#1}{#2}}%
1596   \fi
1597   \bbl@add@list\bbl@attributes{#1-#2}%
1598   \expandafter\def\csname#1@attr@#2\endcsname{#3}}
```

\bbl@ifattributeset This internal macro has 4 arguments. It can be used to interpret $\TeX$ code based on whether a certain attribute was set. This command should appear inside the argument to `\AtBeginDocument` because the attributes are set in the document preamble, *after* babel is loaded.

The first argument is the language, the second argument the attribute being checked, and the third and fourth arguments are the true and false clauses.

```
1599 \def\bbl@ifattributeset#1#2#3#4{%
1600   \ifx\bbl@known@attribs\@undefined
1601     \in@false
1602   \else
1603     \bbl@xin@{,#1-#2,}{,\bbl@known@attribs,}%
1604   \fi
1605   \ifin@
1606     \bbl@afterelse#3%
1607   \else
1608     \bbl@afterfi#4%
1609   \fi}
```

\bbl@ifknown@ttrib An internal macro to check whether a given language/attribute is known. The macro takes 4 arguments, the language/attribute, the attribute list, the $\TeX$ -code to be executed when the attribute is known and the $\TeX$ -code to be executed otherwise.

We first assume the attribute is unknown. Then we loop over the list of known attributes, trying to find a match.

```
1610 \def\bbl@ifknown@ttrib#1#2{%
1611   \let\bbl@tempa\@secondoftwo
1612   \bbl@loopx\bbl@tempb{#2}{%
1613     \expandafter\in@\expandafter{\expandafter,\bbl@tempb,}{,#1,}%
1614   \ifin@
1615     \let\bbl@tempa\@firstoftwo
1616   \else
1617     \fi}%
1618   \bbl@tempa}
```

\bbl@clear@ttribs This macro removes all the attribute code from $\TeX$'s memory at `\begin{document}` time (if any is present).

```
1619 \def\bbl@clear@ttribs{%
1620   \ifx\bbl@attributes\@undefined\else
1621     \bbl@loopx\bbl@tempa{\bbl@attributes}{%
1622       \expandafter\bbl@clear@ttrib\bbl@tempa.}%
1623     \let\bbl@attributes\@undefined
1624   \fi}
1625 \def\bbl@clear@ttrib#1-#2.{%
1626   \expandafter\let\csname#1@attr@#2\endcsname\@undefined}
1627 \AtBeginDocument{\bbl@clear@ttribs}
```

4.10. Support for saving and redefining macros

To save the meaning of control sequences using `\babel@save`, we use temporary control sequences. To save hash table entries for these control sequences, we don't use the name of the control sequence

to be saved to construct the temporary name. Instead we simply use the value of a counter, which is reset to zero each time we begin to save new values. This works well because we release the saved meanings before we begin to save a new set of control sequence meanings (see `\selectlanguage` and `\originalTeX`). Note undefined macros are not undefined any more when saved – they are `\relax`’ed.

`\babel@savecnt`

`\babel@beginsave` The initialization of a new save cycle: reset the counter to zero.

```
1628 \bbl@trace{Macros for saving definitions}
1629 \def\babel@beginsave{\babel@savecnt\z@}
```

Before it’s forgotten, allocate the counter and initialize all.

```
1630 \newcount\babel@savecnt
1631 \babel@beginsave
```

`\babel@save`

`\babel@savevariable` The macro `\babel@save{csname}` saves the current meaning of the control sequence `(csname)` to `\originalTeX` (which has to be expandable, i.e., you shouldn’t let it to `\relax`). To do this, we let the current meaning to a temporary control sequence, the restore commands are appended to `\originalTeX` and the counter is incremented. The macro `\babel@savevariable{variable}` saves the value of the variable. `(variable)` can be anything allowed after the `\the` primitive. To avoid messing saved definitions up, they are saved only the very first time.

```
1632 \def\babel@save#1{%
1633   \def\bbl@tempa{,{#1,}}% Clumsy, for Plain
1634   \expandafter\bbl@add\expandafter\bbl@tempa\expandafter{%
1635     \expandafter{\expandafter,\bbl@savextras,}}%
1636   \expandafter\in@\bbl@tempa
1637   \ifin@ \else
1638     \bbl@add\bbl@savextras{,{#1,}}%
1639     \bbl@carg\let\babel@number\babel@savecnt\#1\relax
1640     \toks@ \expandafter{\originalTeX\let#1=}
1641     \bbl@exp{%
1642       \def\\originalTeX{\the\toks@<\babel@number\babel@savecnt>\relax}}%
1643     \advance\babel@savecnt@ne
1644   \fi}
1645 \def\babel@savevariable#1{%
1646   \toks@ \expandafter{\originalTeX #1=}
1647   \bbl@exp{\def\\originalTeX{\the\toks@ \the#1\relax}}}
```

`\bbl@redefine` To redefine a command, we save the old meaning of the macro. Then we redefine it to call the original macro with the ‘sanitized’ argument. The reason why we do it this way is that we don’t want to redefine the $\TeX$ macros completely in case their definitions change (they have changed in the past). A macro named `\macro` will be saved new control sequences named `\org@macro`.

```
1648 \def\bbl@redefine#1{%
1649   \edef\bbl@tempa{\bbl@stripslash#1}%
1650   \expandafter\let\csname org@\bbl@tempa\endcsname#1%
1651   \expandafter\def\csname\bbl@tempa\endcsname}
1652 \onlypreamble\bbl@redefine
```

`\bbl@redefine@long` This version of `\babel@redefine` can be used to redefine `\long` commands such as `\ifthenelse`.

```
1653 \def\bbl@redefine@long#1{%
1654   \edef\bbl@tempa{\bbl@stripslash#1}%
1655   \expandafter\let\csname org@\bbl@tempa\endcsname#1%
1656   \long\expandafter\def\csname\bbl@tempa\endcsname}
1657 \onlypreamble\bbl@redefine@long
```

\bbl@redefineroobust For commands that are redefined, but which *might* be robust we need a slightly more intelligent macro. A robust command `foo` is defined to expand to `\protect\foo`. So it is necessary to check whether `\foo` exists. The result is that the command that is being redefined is always robust afterwards. Therefore all we need to do now is define `\foo`.

```

1658 \def\bbl@redefineroobust#1{%
1659   \edef\bbl@tempa{\bbl@stripslash#1}%
1660   \bbl@iifunset{\bbl@tempa\space}%
1661   {\expandafter\let\csname org@\bbl@tempa\endcsname#1%
1662    \bbl@exp{\def\#1{\protect\<\bbl@tempa\space>}}}%
1663   {\bbl@exp{\let\<org@\bbl@tempa>\<\bbl@tempa\space>}}}%
1664   \@namedef{\bbl@tempa\space}}
1665 \@onlypreamble\bbl@redefineroobust

```

4.11. French spacing

\bbl@frenchspacing

\bbl@nonfrenchspacing Some languages need to have `\frenchspacing` in effect. Others don't want that. The command `\bbl@frenchspacing` switches it on when it isn't already in effect and `\bbl@nonfrenchspacing` switches it off if necessary.

```

1666 \def\bbl@frenchspacing{%
1667   \ifnum\the\sfcode`\.\=\@m
1668     \let\bbl@nonfrenchspacing\relax
1669   \else
1670     \frenchspacing
1671     \let\bbl@nonfrenchspacing\nonfrenchspacing
1672   \fi}
1673 \let\bbl@nonfrenchspacing\nonfrenchspacing

```

A more refined way to switch the catcodes is done with `ini` files. Here an auxiliary macro is defined, but the main part is in `\babelprovide`. This new method should be ideally the default one.

```

1674 \let\bbl@elt\relax
1675 \edef\bbl@fs@chars{%
1676   \bbl@elt{\string.}\@m{3000}\bbl@elt{\string?}\@m{3000}%
1677   \bbl@elt{\string!}\@m{3000}\bbl@elt{\string:}\@m{2000}%
1678   \bbl@elt{\string;}\@m{1500}\bbl@elt{\string,}\@m{1250}}
1679 \def\bbl@pre@fs{%
1680   \def\bbl@elt##1##2##3{\sfcode`##1=\the\sfcode`##1\relax}%
1681   \edef\bbl@save@sfcodes{\bbl@fs@chars}%
1682 \def\bbl@post@fs{%
1683   \bbl@save@sfcodes
1684   \edef\bbl@tempa{\bbl@cl{frspc}}%
1685   \edef\bbl@tempa{\expandafter\car\bbl@tempa\@nil}%
1686   \if u\bbl@tempa      % do nothing
1687   \else\if n\bbl@tempa % non french
1688     \def\bbl@elt##1##2##3{%
1689       \ifnum\sfcode`##1=##2\relax
1690         \babel@savevariable{\sfcode`##1}%
1691         \sfcode`##1=##3\relax
1692       \fi}%
1693     \bbl@fs@chars
1694   \else\if y\bbl@tempa % french
1695     \def\bbl@elt##1##2##3{%
1696       \ifnum\sfcode`##1=##3\relax
1697         \babel@savevariable{\sfcode`##1}%
1698         \sfcode`##1=##2\relax
1699       \fi}%
1700     \bbl@fs@chars
1701   \fi\fi\fi}

```

4.12. Hyphens

\babelhyphenation This macro saves hyphenation exceptions. Two macros are used to store them: `\bbl@hyphenation@` for the global ones and `\bbl@hyphenation@(<language>)` for language ones. See

\bbl@patterns above for further details. We make sure there is a space between words when multiple commands are used.

```

1702 \bbl@trace{Hyphens}
1703 \@onlypreamble\babelhyphenation
1704 \AtEndOfPackage{%
1705   \newcommand\babelhyphenation[2][\@empty]{%
1706     \ifx\bbl@hyphenation@\relax
1707       \let\bbl@hyphenation@\@empty
1708     \fi
1709     \ifx\bbl@hyphlist@\@empty\else
1710       \bbl@warning{%
1711         You must not intermingle \string\selectlanguage\space and\\%
1712         \string\babelhyphenation\space or some exceptions will not\\%
1713         be taken into account. Reported}%
1714       \fi
1715       \ifx\@empty#1%
1716         \protected@edef\bbl@hyphenation@{\bbl@hyphenation@\space#2}%
1717       \else
1718         \bbl@vforeach{#1}{%
1719           \def\bbl@tempa{##1}%
1720           \bbl@fixname\bbl@tempa
1721           \bbl@iflanguage\bbl@tempa{%
1722             \bbl@csarg\protected@edef{hyphenation@\bbl@tempa}{%
1723               \bbl@ifunset{bbl@hyphenation@\bbl@tempa}%
1724               {}%
1725               {\csname bbl@hyphenation@\bbl@tempa\endcsname\space}%
1726               #2}}}%
1727         \fi}}

```

\babelhyphenmins Only L^AT_EX (basically because it's defined with a L^AT_EX tool).

```

1728 \ifx\NewDocumentCommand\@undefined\else
1729   \NewDocumentCommand\babelhyphenmins{sommo}{%
1730     \IfNoValueTF{#2}%
1731       {\protected@edef\bbl@hyphenmins@{\set@hyphenmins{#3}{#4}}}%
1732       \IfValueT{#5}{%
1733         \protected@edef\bbl@hyphenatmin@{\hyphenationmin=#5\relax}}%
1734       \IfBooleanT{#1}{%
1735         \lefthyphenmin=#3\relax
1736         \righthyphenmin=#4\relax
1737         \IfValueT{#5}{\hyphenationmin=#5\relax}}}%
1738     {\edef\bbl@tempb{\zap@space#2 \@empty}%
1739      \bbl@for\bbl@tempa\bbl@tempb{%
1740        \@namedef{bbl@hyphenmins@\bbl@tempa}{\set@hyphenmins{#3}{#4}}}%
1741        \IfValueT{#5}{%
1742          \@namedef{bbl@hyphenatmin@\bbl@tempa}{\hyphenationmin=#5\relax}}}%
1743        \IfBooleanT{#1}{\bbl@error{hyphenmins-args}{}}}%
1744   \fi

```

\bbl@allowhyphens This macro makes hyphenation possible. Basically its definition is nothing more than \nobreak \hskip 0pt plus 0pt. T_EX begins and ends a word for hyphenation at a glue node. The penalty prevents a linebreak at this glue node.

```

1745 \def\bbl@allowhyphens{\ifvmode\else\nobreak\hskip\z@skip\fi}
1746 \def\bbl@t@one{Tl}
1747 \def\allowhyphens{\ifx\cf@encoding\bbl@t@one\else\bbl@allowhyphens\fi}

```

\babelhyphen Macros to insert common hyphens. Note the space before @ in \babelhyphen. Instead of protecting it with \DeclareRobustCommand, which could insert a \relax, we use the same procedure as shorthands, with \active@prefix.

```

1748 \newcommand\babelnullhyphen{\char\hyphenchar\font}
1749 \def\babelhyphen{\active@prefix\babelhyphen\bbl@hyphen}
1750 \def\bbl@hyphen{%

```

```

1751 \ifstar{\bbl@hyphen@i @}{\bbl@hyphen@i\@empty}}
1752 \def\bbl@hyphen@i#1#2{%
1753 \lowercase{\bbl@ifunset{\bbl@hy#1#2\@empty}}}%
1754 {\csname bbl@lusehyphen\endcsname{\discretionary{#2}{}{#2}}}%
1755 {\lowercase{\csname bbl@hy#1#2\@empty\endcsname}}}

```

The following two commands are used to wrap the “hyphen” and set the behavior of the rest of the word – the version with a single @ is used when further hyphenation is allowed, while that with @@ if no more hyphens are allowed. In both cases, if the hyphen is preceded by a positive space, breaking after the hyphen is disallowed.

There should not be a discretionary after a hyphen at the beginning of a word, so it is prevented if preceded by a skip. Unfortunately, this does handle cases like “(-suffix)”. \nobreak is always preceded by \leavevmode, in case the shorthand starts a paragraph.

```

1756 \def\bbl@usehyphen#1{%
1757 \leavevmode
1758 \ifdim\lastskip>\z@\mbox{#1}\else\nobreak#1\fi
1759 \nobreak\hskip\z@skip}
1760 \def\bbl@usehyphen#1{%
1761 \leavevmode\ifdim\lastskip>\z@\mbox{#1}\else#1\fi}

```

The following macro inserts the hyphen char.

```

1762 \def\bbl@hyphenchar{%
1763 \ifnum\hyphenchar\font=\m@ne
1764 \babelnullhyphen
1765 \else
1766 \char\hyphenchar\font
1767 \fi}

```

Finally, we define the hyphen “types”. Their names will not change, so you may use them in ldf’s. After a space, the \mbox in \bbl@hy@nobreak is redundant.

```

1768 \def\bbl@hy@soft{\bbl@usehyphen{\discretionary{\bbl@hyphenchar}{}}{}}
1769 \def\bbl@hy@soft{\bbl@usehyphen{\discretionary{\bbl@hyphenchar}{}}{}}
1770 \def\bbl@hy@hard{\bbl@usehyphen\bbl@hyphenchar}
1771 \def\bbl@hy@hard{\bbl@usehyphen\bbl@hyphenchar}
1772 \def\bbl@hy@nobreak{\bbl@usehyphen{\mbox{\bbl@hyphenchar}}}
1773 \def\bbl@hy@nobreak{\mbox{\bbl@hyphenchar}}
1774 \def\bbl@hy@repeat{%
1775 \bbl@usehyphen{%
1776 \discretionary{\bbl@hyphenchar}{\bbl@hyphenchar}{\bbl@hyphenchar}}}
1777 \def\bbl@hy@repeat{%
1778 \bbl@usehyphen{%
1779 \discretionary{\bbl@hyphenchar}{\bbl@hyphenchar}{\bbl@hyphenchar}}}
1780 \def\bbl@hy@empty{\hskip\z@skip}
1781 \def\bbl@hy@empty{\discretionary{}{}{}}

```

\bbl@disc For some languages the macro \bbl@disc is used to ease the insertion of discretionaries for letters that behave ‘abnormally’ at a breakpoint.

```

1782 \def\bbl@disc#1#2{\nobreak\discretionary{#2-}{}{#1}\bbl@allowhyphens}

```

4.13. Multiencoding strings

The aim following commands is to provide a common interface for strings in several encodings. They also contains several hooks which can be used by luatex and xetex. The code is organized here with pseudo-guards, so we start with the basic commands.

Tools But first, a tool. It makes global a local variable. This is not the best solution, but it works.

```

1783 \bbl@trace{Multiencoding strings}
1784 \def\bbl@tglobal#1{\global\let#1#1}

```

The following option is currently no-op. It was meant for the deprecated \SetCase.

```

1785 <(<*More package options>>) ≡
1786 \DeclareOption{nocase}{}
1787 <(</More package options>>)

```

The following package options control the behavior of `\SetString`.

```

1788 ({*More package options}) ≡
1789 \let\bbl@opt@strings\@nnil % accept strings=value
1790 \DeclareOption{strings}{\def\bbl@opt@strings{\BabelStringsDefault}}
1791 \DeclareOption{strings=encoded}{\let\bbl@opt@strings\relax}
1792 \def\BabelStringsDefault{generic}
1793 ({/More package options})

```

Main command This is the main command. With the first use it is redefined to omit the basic setup in subsequent blocks. We make sure strings contain actual letters in the range 128-255, not active characters.

```

1794 \@onlypreamble\StartBabelCommands
1795 \def\StartBabelCommands{%
1796   \begingroup
1797   \@tempcnta="7F
1798   \def\bbl@tempa{%
1799     \ifnum\@tempcnta>"FF\else
1800       \catcode\@tempcnta=11
1801       \advance\@tempcnta\@ne
1802       \expandafter\bbl@tempa
1803     \fi}%
1804   \bbl@tempa
1805   <@Macros local to BabelCommands@>
1806   \def\bbl@provstring##1##2{%
1807     \providecommand##1{##2}%
1808     \bbl@tglobal##1}%
1809   \global\let\bbl@scafter\@empty
1810   \let\StartBabelCommands\bbl@startcmds
1811   \ifx\BabelLanguages\relax
1812     \let\BabelLanguages\CurrentOption
1813   \fi
1814   \begingroup
1815   \let\bbl@screset\@nnil % local flag - disable 1st stopcommands
1816   \StartBabelCommands}
1817 \def\bbl@startcmds{%
1818   \ifx\bbl@screset\@nnil\else
1819     \bbl@usehooks{stopcommands}{}%
1820   \fi
1821   \endgroup
1822   \begingroup
1823   \@ifstar
1824     {\ifx\bbl@opt@strings\@nnil
1825       \let\bbl@opt@strings\BabelStringsDefault
1826       \fi
1827       \bbl@startcmds@i}%
1828     \bbl@startcmds@i}
1829 \def\bbl@startcmds@i#1#2{%
1830   \edef\bbl@L{\zap@space#1 \@empty}%
1831   \edef\bbl@G{\zap@space#2 \@empty}%
1832   \bbl@startcmds@ii}
1833 \let\bbl@startcmds\StartBabelCommands

```

Parse the encoding info to get the label, input, and font parts.

Select the behavior of `\SetString`. There are two main cases, depending of if there is an optional argument: without it and `strings=encoded`, strings are defined always; otherwise, they are set only if they are still undefined (i.e., fallback values). With labelled blocks and `strings=encoded`, define the strings, but with another value, define strings only if the current label or font encoding is the value of strings; otherwise (i.e., no strings or a block whose label is not in `strings=`) do nothing.

We presume the current block is not loaded, and therefore set (above) a couple of default values to gobble the arguments. Then, these macros are redefined if necessary according to several parameters.

```

1834 \newcommand\bbl@startcmds@ii[1][\@empty]{%

```

```

1835 \let\SetString\@gobbletwo
1836 \let\bbl@stringdef\@gobbletwo
1837 \let\AfterBabelCommands\@gobble
1838 \ifx\@empty#1%
1839   \def\bbl@sc@label{generic}%
1840   \def\bbl@encstring##1##2{%
1841     \ProvideTextCommandDefault##1{##2}%
1842     \bbl@tglobal##1%
1843     \expandafter\bbl@tglobal\csname\string?\string##1\endcsname}%
1844   \let\bbl@sctest\in@true
1845 \else
1846   \let\bbl@sc@charset\space % <- zapped below
1847   \let\bbl@sc@fontenc\space % <- " "
1848   \def\bbl@tempa##1=##2\@nil{%
1849     \bbl@csarg\edef{sc@zap@space##1 \@empty}{##2 }}%
1850   \bbl@vforeach{label=#1}{\bbl@tempa##1\@nil}%
1851   \def\bbl@tempa##1 ##2{% space -> comma
1852     ##1%
1853     \ifx\@empty##2\else\ifx,##1,\else,\fi\bbl@afterfi\bbl@tempa##2\fi}%
1854   \edef\bbl@sc@fontenc{\expandafter\bbl@tempa\bbl@sc@fontenc\@empty}%
1855   \edef\bbl@sc@label{\expandafter\zap@space\bbl@sc@label\@empty}%
1856   \edef\bbl@sc@charset{\expandafter\zap@space\bbl@sc@charset\@empty}%
1857   \def\bbl@encstring##1##2{%
1858     \bbl@foreach\bbl@sc@fontenc{%
1859       \bbl@ifunset{T@###1}%
1860       {}%
1861       {\ProvideTextCommand##1{###1}{##2}%
1862         \bbl@tglobal##1%
1863         \expandafter
1864         \bbl@tglobal\csname###1\string##1\endcsname}}}%
1865   \def\bbl@sctest{%
1866     \bbl@xin@{\bbl@opt@strings,}{,\bbl@sc@label,\bbl@sc@fontenc,}%
1867   \fi
1868   \ifx\bbl@opt@strings\@nnil % i.e., no strings key -> defaults
1869   \else\ifx\bbl@opt@strings\relax % i.e., strings=encoded
1870     \let\AfterBabelCommands\bbl@aftercmds
1871     \let\SetString\bbl@setstring
1872     \let\bbl@stringdef\bbl@encstring
1873   \else % i.e., strings=value
1874     \bbl@sctest
1875   \fin@
1876   \let\AfterBabelCommands\bbl@aftercmds
1877   \let\SetString\bbl@setstring
1878   \let\bbl@stringdef\bbl@provstring
1879 \fi\fi\fi
1880 \bbl@scswitch
1881 \ifx\bbl@G\@empty
1882   \def\SetString##1##2{%
1883     \bbl@error{missing-group}{##1}{}}%
1884 \fi
1885 \ifx\@empty#1%
1886   \bbl@usehooks{defaultcommands}{}%
1887 \else
1888   \@expandtwoargs
1889   \bbl@usehooks{encodedcommands}{\bbl@sc@charset}\bbl@sc@fontenc}%
1890 \fi}

```

There are two versions of `\bbl@scswitch`. The first version is used when `ldfs` are read, and it makes sure `\langle group \rangle \langle language \rangle` is reset, but only once (`\bbl@screset` is used to keep track of this). The second version is used in the preamble and packages loaded after `babel` and does nothing.

The macro `\bbl@forlang` loops `\bbl@L` but its body is executed only if the value is in `\BabelLanguages` (inside `babel`) or `\date\langle language \rangle` is defined (after `babel` has been loaded). There are also two version of `\bbl@forlang`. The first one skips the current iteration if the language is not in `\BabelLanguages` (used in `ldfs`), and the second one skips undefined languages (after `babel` has

been loaded).

```

1891 \def\bbl@forlang#1#2{%
1892   \bbl@for#1\bbl@L{%
1893     \bbl@xin@{,#1,},{,\BabelLanguages,}%
1894     \ifin@#2\relax\fi}}
1895 \def\bbl@scswitch{%
1896   \bbl@forlang\bbl@tempa{%
1897     \ifx\bbl@G\@empty\else
1898       \ifx\SetString@gobbletwo\else
1899         \edef\bbl@GL{\bbl@G\bbl@tempa}%
1900         \bbl@xin@{,\bbl@GL,}{,\bbl@screset,}%
1901         \ifin@\else
1902           \global\expandafter\let\csname\bbl@GL\endcsname\@undefined
1903           \xdef\bbl@screset{\bbl@screset,\bbl@GL}%
1904         \fi
1905       \fi
1906     \fi}}
1907 \AtEndOfPackage{%
1908   \def\bbl@forlang#1#2{\bbl@for#1\bbl@L{\bbl@ifunset{date#1}{\#2}}}%
1909   \let\bbl@scswitch\relax}
1910 \@onlypreamble\EndBabelCommands
1911 \def\EndBabelCommands{%
1912   \bbl@usehooks{stopcommands}{}%
1913   \endgroup
1914   \endgroup
1915   \bbl@scafter}
1916 \let\bbl@endcommands\EndBabelCommands

```

Now we define commands to be used inside \StartBabelCommands.

Strings The following macro is the actual definition of \SetString when it is “active”

First save the “switcher”. Create it if undefined. Strings are defined only if undefined (i.e., like \providescommand). With the event stringprocess you can preprocess the string by manipulating the value of \BabelString. If there are several hooks assigned to this event, preprocessing is done in the same order as defined. Finally, the string is set.

```

1917 \def\bbl@setstring#1#2{% e.g., \prefacename{<string>}
1918   \bbl@forlang\bbl@tempa{%
1919     \edef\bbl@LC{\bbl@tempa\bbl@stripslash#1}%
1920     \bbl@ifunset{\bbl@LC}% e.g., \germanchaptername
1921     {\bbl@exp{%
1922       \global\bbbl@add\<\bbl@G\bbl@tempa>{\bbbl@scset\#1\<\bbl@LC>}}}%
1923     }%
1924     \def\BabelString{#2}%
1925     \bbl@usehooks{stringprocess}{}%
1926     \expandafter\bbl@stringdef
1927     \csname\bbl@LC\expandafter\endcsname\expandafter{\BabelString}}

```

A little auxiliary command sets the string. Formerly used with casing. Very likely no longer necessary, although it’s used in \setlocalecaption.

```

1928 \def\bbl@scset#1#2{\def#1{#2}}

```

Define \SetStringLoop, which is actually set inside \StartBabelCommands. The current definition is somewhat complicated because we need a count, but \count@ is not under our control (remember \SetString may call hooks). Instead of defining a dedicated count, we just “pre-expand” its value.

```

1929 <(*Macros local to BabelCommands)> ≡
1930 \def\SetStringLoop###1#2{%
1931   \def\bbl@templ#####1{\expandafter\noexpand\csname##1\endcsname}%
1932   \count@\z@
1933   \bbl@loop\bbl@tempa{##2}{% empty items and spaces are ok
1934     \advance\count@\@ne
1935     \toks@\expandafter{\bbl@tempa}%
1936     \bbl@exp{%
1937       \SetString\bbl@templ{\romannumeral\count@}{\the\toks@}%

```

```

1938 \count@=\the\count@\relax}}}%
1939 <</Macros local to BabelCommands>>

```

Delaying code Now the definition of \AfterBabelCommands when it is activated.

```

1940 \def\bbl@aftercmds#1{%
1941 \toks@\expandafter{\bbl@scafter#1}%
1942 \xdef\bbl@scafter{\the\toks@}}

```

Case mapping The command \SetCase is deprecated. Currently it consists in a definition with a hack just for backward compatibility in the macro mapping.

```

1943 <<*Macros local to BabelCommands>> ≡
1944 \newcommand\SetCase[3][]{%
1945 \def\bbl@tempa####1####2{%
1946 \ifx####1\@empty\else
1947 \bbl@carg\bbl@add{extras\CurrentOption}{%
1948 \bbl@carg\babel@save{c__text_uppercase_\string####1_tl}%
1949 \bbl@carg\def{c__text_uppercase_\string####1_tl}{####2}%
1950 \bbl@carg\babel@save{c__text_lowercase_\string####2_tl}%
1951 \bbl@carg\def{c__text_lowercase_\string####2_tl}{####1}}%
1952 \expandafter\bbl@tempa
1953 \fi}%
1954 \bbl@tempa##1\@empty\@empty
1955 \bbl@carg\bbl@tglobal{extras\CurrentOption}}%
1956 <</Macros local to BabelCommands>>

```

Macros to deal with case mapping for hyphenation. To decide if the document is monolingual or multilingual, we make a rough guess – just see if there is a comma in the languages list, built in the first pass of the package options.

```

1957 <<*Macros local to BabelCommands>> ≡
1958 \newcommand\SetHyphenMap[1]{%
1959 \bbl@forlang\bbl@tempa{%
1960 \expandafter\bbl@stringdef
1961 \csname\bbl@tempa @bbl@hyphenmap\endcsname{##1}}}%
1962 <</Macros local to BabelCommands>>

```

There are 3 helper macros which do most of the work for you.

```

1963 \newcommand\BabelLower[2]{% one to one.
1964 \ifnum\lccode#1=#2\else
1965 \babel@savevariable{\lccode#1}%
1966 \lccode#1=#2\relax
1967 \fi}
1968 \newcommand\BabelLowerMM[4]{% many-to-many
1969 \@tempcnta=#1\relax
1970 \@tempcntb=#4\relax
1971 \def\bbl@tempa{%
1972 \ifnum\@tempcnta>#2\else
1973 \@expandtwoargs\BabelLower{\the\@tempcnta}{\the\@tempcntb}%
1974 \advance\@tempcnta#3\relax
1975 \advance\@tempcntb#3\relax
1976 \expandafter\bbl@tempa
1977 \fi}%
1978 \bbl@tempa}
1979 \newcommand\BabelLowerM0[4]{% many-to-one
1980 \@tempcnta=#1\relax
1981 \def\bbl@tempa{%
1982 \ifnum\@tempcnta>#2\else
1983 \@expandtwoargs\BabelLower{\the\@tempcnta}{#4}%
1984 \advance\@tempcnta#3
1985 \expandafter\bbl@tempa
1986 \fi}%
1987 \bbl@tempa}

```

The following package options control the behavior of hyphenation mapping.

```

1988 ({*More package options})  ≡
1989 \DeclareOption{hyphenmap=off}{\chardef\bbl@opt@hyphenmap\z@}
1990 \DeclareOption{hyphenmap=first}{\chardef\bbl@opt@hyphenmap\@ne}
1991 \DeclareOption{hyphenmap=select}{\chardef\bbl@opt@hyphenmap\tw@}
1992 \DeclareOption{hyphenmap=other}{\chardef\bbl@opt@hyphenmap\thr@@}
1993 \DeclareOption{hyphenmap=other*}{\chardef\bbl@opt@hyphenmap4\relax}
1994 (/More package options)

```

Initial setup to provide a default behavior if hyphenmap is not set.

```

1995 \AtEndOfPackage{%
1996   \ifx\bbl@opt@hyphenmap\undefined
1997     \bbl@xin@{,}{\bbl@language@opts}%
1998     \chardef\bbl@opt@hyphenmap\ifin@4\else\@ne\fi
1999   \fi}

```

4.14. Tailor captions

A general tool for resetting the caption names with a unique interface. With the old way, which mixes the switcher and the string, we convert it to the new one, which separates these two steps.

```

2000 \newcommand\setlocalecaption{%
2001   \ifstar\bbl@setcaption@s\bbl@setcaption@x}
2002 \def\bbl@setcaption@x#1#2#3{% language caption-name string
2003   \bbl@trim@def\bbl@tempa{#2}%
2004   \bbl@xin@{.template}{\bbl@tempa}%
2005   \ifin@
2006     \bbl@ini@captions@template{#3}{#1}%
2007   \else
2008     \edef\bbl@tempd{%
2009       \expandafter\expandafter\expandafter
2010       \strip@prefix\expandafter\meaning\csname captions#1\endcsname}%
2011     \bbl@xin@
2012       {\expandafter\string\csname #2name\endcsname}%
2013       {\bbl@tempd}%
2014     \ifin@ % Renew caption
2015       \bbl@xin@{\string\bbl@scset}{\bbl@tempd}%
2016     \ifin@
2017       \bbl@exp{%
2018         \\bbl@ifsamestring{\bbl@tempa}{\language name}%
2019         {\bbl@scset\<#2name>\<#1#2name>}%
2020         {}}%
2021       \else % Old way converts to new way
2022         \bbl@ifunset{#1#2name}%
2023         {\bbl@exp{%
2024           \\bbl@add\<captions#1>\def\<#2name>\<#1#2name>}}%
2025           \\bbl@ifsamestring{\bbl@tempa}{\language name}%
2026           {\def\<#2name>\<#1#2name>}}%
2027           {}}}%
2028       {}%
2029     \fi
2030   \else
2031     \bbl@xin@{\string\bbl@scset}{\bbl@tempd}% New
2032     \ifin@ % New way
2033     \bbl@exp{%
2034       \\bbl@add\<captions#1>\bbl@scset\<#2name>\<#1#2name>}%
2035       \\bbl@ifsamestring{\bbl@tempa}{\language name}%
2036       {\bbl@scset\<#2name>\<#1#2name>}%
2037       {}}%
2038     \else % Old way, but defined in the new way
2039     \bbl@exp{%
2040       \\bbl@add\<captions#1>\def\<#2name>\<#1#2name>}}%
2041       \\bbl@ifsamestring{\bbl@tempa}{\language name}%

```

```

2042      {\def\<#2name>{\<#1#2name>}}}%
2043      {}}}%
2044      \fi%
2045      \fi
2046      \@namedef{#1#2name}{#3}%
2047      \toks@{\expandafter{\bbl@captionslist}%
2048      \bbl@exp{\in{\<#2name>}{\the\toks@}}}%
2049      \ifin@else
2050      \bbl@exp{\bbl@add{\bbl@captionslist{\<#2name>}}}%
2051      \bbl@tglobal\bbl@captionslist
2052      \fi
2053      \fi}

```

4.15. Making glyphs available

This section makes a number of glyphs available that either do not exist in the OT1 encoding and have to be ‘faked’, or that are not accessible through Tlenc.def.

\set@low@box The following macro is used to lower quotes to the same level as the comma. It prepares its argument in box register 0.

```

2054 \bbl@trace{Macros related to glyphs}
2055 \def\set@low@box#1{\setbox\tw@{\hbox{,}}\setbox\z@{\hbox{#1}}%
2056   \dimen\z@{\ht\z@ \advance\dimen\z@ -\ht\tw@}%
2057   \setbox\z@{\hbox{\lower\dimen\z@ \box\z@}\ht\z@{\ht\tw@ \dp\z@\dp\tw@}}

```

\save@sf@q The macro \save@sf@q is used to save and reset the current space factor.

```

2058 \def\save@sf@q#1{\leavevmode
2059   \begingroup
2060   \edef\@SF{\spacefactor\the\spacefactor}#1\@SF
2061   \endgroup}

```

4.15.1. Quotation marks

\quotedblbase In the T1 encoding the opening double quote at the baseline is available as a separate character, accessible via \quotedblbase. In the OT1 encoding it is not available, therefore we make it available by lowering the normal open quote character to the baseline.

```

2062 \ProvideTextCommand{\quotedblbase}{OT1}{%
2063   \save@sf@q{\set@low@box{\textquotedblright/}}%
2064   \box\z@\kern-.04em\bbl@allowhyphens}}

```

Make sure that when an encoding other than OT1 or T1 is used this glyph can still be typeset.

```

2065 \ProvideTextCommandDefault{\quotedblbase}{%
2066   \UseTextSymbol{OT1}{\quotedblbase}}

```

\quotesinglbase We also need the single quote character at the baseline.

```

2067 \ProvideTextCommand{\quotesinglbase}{OT1}{%
2068   \save@sf@q{\set@low@box{\textquoteright/}}%
2069   \box\z@\kern-.04em\bbl@allowhyphens}}

```

Make sure that when an encoding other than OT1 or T1 is used this glyph can still be typeset.

```

2070 \ProvideTextCommandDefault{\quotesinglbase}{%
2071   \UseTextSymbol{OT1}{\quotesinglbase}}

```

\guillemetleft

\guillemetright The guillemet characters are not available in OT1 encoding. They are faked. (Wrong names with o preserved for compatibility.)

```
2072 \ProvideTextCommand{\guillemetleft}{OT1}{%
2073   \ifmmode
2074     \ll
2075   \else
2076     \save@sf@q{\nobreak
2077       \raise.2ex\hbox{$\scriptscriptstyle\ll$}\bbl@allowhyphens}%
2078   \fi}
2079 \ProvideTextCommand{\guillemetright}{OT1}{%
2080   \ifmmode
2081     \gg
2082   \else
2083     \save@sf@q{\nobreak
2084       \raise.2ex\hbox{$\scriptscriptstyle\gg$}\bbl@allowhyphens}%
2085   \fi}
2086 \ProvideTextCommand{\guillemotleft}{OT1}{%
2087   \ifmmode
2088     \ll
2089   \else
2090     \save@sf@q{\nobreak
2091       \raise.2ex\hbox{$\scriptscriptstyle\ll$}\bbl@allowhyphens}%
2092   \fi}
2093 \ProvideTextCommand{\guillemotright}{OT1}{%
2094   \ifmmode
2095     \gg
2096   \else
2097     \save@sf@q{\nobreak
2098       \raise.2ex\hbox{$\scriptscriptstyle\gg$}\bbl@allowhyphens}%
2099   \fi}
```

Make sure that when an encoding other than OT1 or T1 is used these glyphs can still be typeset.

```
2100 \ProvideTextCommandDefault{\guillemetleft}{%
2101   \UseTextSymbol{OT1}{\guillemetleft}}
2102 \ProvideTextCommandDefault{\guillemetright}{%
2103   \UseTextSymbol{OT1}{\guillemetright}}
2104 \ProvideTextCommandDefault{\guillemotleft}{%
2105   \UseTextSymbol{OT1}{\guillemotleft}}
2106 \ProvideTextCommandDefault{\guillemotright}{%
2107   \UseTextSymbol{OT1}{\guillemotright}}
```

\guilsinglleft

\guilsinglright The single guillemets are not available in OT1 encoding. They are faked.

```
2108 \ProvideTextCommand{\guilsinglleft}{OT1}{%
2109   \ifmmode
2110     <%
2111   \else
2112     \save@sf@q{\nobreak
2113       \raise.2ex\hbox{$\scriptscriptstyle<$}\bbl@allowhyphens}%
2114   \fi}
2115 \ProvideTextCommand{\guilsinglright}{OT1}{%
2116   \ifmmode
2117     >%
2118   \else
2119     \save@sf@q{\nobreak
2120       \raise.2ex\hbox{$\scriptscriptstyle>$}\bbl@allowhyphens}%
2121   \fi}
```

Make sure that when an encoding other than OT1 or T1 is used these glyphs can still be typeset.

```
2122 \ProvideTextCommandDefault{\guilsinglleft}{%
2123   \UseTextSymbol{OT1}{\guilsinglleft}}
2124 \ProvideTextCommandDefault{\guilsinglright}{%
2125   \UseTextSymbol{OT1}{\guilsinglright}}
```

4.15.2. Letters

\ij

\IJ The dutch language uses the letter 'ij'. It is available in T1 encoded fonts, but not in the OT1 encoded fonts. Therefore we fake it for the OT1 encoding.

```
2126 \DeclareTextCommand{\ij}{OT1}{%
2127   i\kern-0.02em\bbl@allowhyphens j}
2128 \DeclareTextCommand{\IJ}{OT1}{%
2129   I\kern-0.02em\bbl@allowhyphens J}
2130 \DeclareTextCommand{\ij}{T1}{\char188}
2131 \DeclareTextCommand{\IJ}{T1}{\char156}
```

Make sure that when an encoding other than OT1 or T1 is used these glyphs can still be typeset.

```
2132 \ProvideTextCommandDefault{\ij}{%
2133   \UseTextSymbol{OT1}{\ij}}
2134 \ProvideTextCommandDefault{\IJ}{%
2135   \UseTextSymbol{OT1}{\IJ}}
```

\dj

\DJ The croatian language needs the letters \dj and \DJ; they are available in the T1 encoding, but not in the OT1 encoding by default.

Some code to construct these glyphs for the OT1 encoding was made available to me by Stipčević Mario, (stipcevic@olimp.irb.hr).

```
2136 \def\crrtic@{\hrule height0.1ex width0.3em}
2137 \def\crttic@{\hrule height0.1ex width0.33em}
2138 \def\ddj@{%
2139   \setbox0\hbox{d}\dimen@=\ht0
2140   \advance\dimen@lex
2141   \dimen@.45\dimen@
2142   \dimen@ii\expandafter\rem@pt\the\fontdimen\@ne\font\dimen@
2143   \advance\dimen@ii.5ex
2144   \leavevmode\rlap{\raise\dimen@\hbox{\kern\dimen@ii\vbox{\crrtic@}}}}
2145 \def\DDJ@{%
2146   \setbox0\hbox{D}\dimen@=.55\ht0
2147   \dimen@ii\expandafter\rem@pt\the\fontdimen\@ne\font\dimen@
2148   \advance\dimen@ii.15ex % correction for the dash position
2149   \advance\dimen@ii-.15\fontdimen7\font % correction for cmtt font
2150   \dimen\thr@@\expandafter\rem@pt\the\fontdimen7\font\dimen@
2151   \leavevmode\rlap{\raise\dimen@\hbox{\kern\dimen@ii\vbox{\crttic@}}}}
2152 %
2153 \DeclareTextCommand{\dj}{OT1}{\ddj@ d}
2154 \DeclareTextCommand{\DJ}{OT1}{\DDJ@ D}
```

Make sure that when an encoding other than OT1 or T1 is used these glyphs can still be typeset.

```
2155 \ProvideTextCommandDefault{\dj}{%
2156   \UseTextSymbol{OT1}{\dj}}
2157 \ProvideTextCommandDefault{\DJ}{%
2158   \UseTextSymbol{OT1}{\DJ}}
```

\SS For the T1 encoding \SS is defined and selects a specific glyph from the font, but for other encodings it is not available. Therefore we make it available here.

```
2159 \DeclareTextCommand{\SS}{OT1}{SS}
2160 \ProvideTextCommandDefault{\SS}{\UseTextSymbol{OT1}{\SS}}
```

4.15.3. Shorthands for quotation marks

Shorthands are provided for a number of different quotation marks, which make them usable both outside and inside mathmode. They are defined with \ProvideTextCommandDefault, but this is very likely not required because their definitions are based on encoding-dependent macros.

\glq

\grq The ‘german’ single quotes.

```
2161 \ProvideTextCommandDefault{\glq}{%  
2162 \textormath{\quotesinglbase}{\mbox{\quotesinglbase}}}
```

The definition of \grq depends on the fontencoding. With T1 encoding no extra kerning is needed.

```
2163 \ProvideTextCommand{\grq}{T1}{%  
2164 \textormath{\kern\z@\textquoteleft}{\mbox{\textquoteleft}}}  
2165 \ProvideTextCommand{\grq}{TU}{%  
2166 \textormath{\textquoteleft}{\mbox{\textquoteleft}}}  
2167 \ProvideTextCommand{\grq}{OT1}{%  
2168 \save@sf@q{\kern-.0125em  
2169 \textormath{\textquoteleft}{\mbox{\textquoteleft}}}%  
2170 \kern.07em\relax}}  
2171 \ProvideTextCommandDefault{\grq}{\UseTextSymbol{OT1}\grq}
```

\glqq

\grqq The ‘german’ double quotes.

```
2172 \ProvideTextCommandDefault{\glqq}{%  
2173 \textormath{\quotedblbase}{\mbox{\quotedblbase}}}
```

The definition of \grqq depends on the fontencoding. With T1 encoding no extra kerning is needed.

```
2174 \ProvideTextCommand{\grqq}{T1}{%  
2175 \textormath{\textquotedblleft}{\mbox{\textquotedblleft}}}  
2176 \ProvideTextCommand{\grqq}{TU}{%  
2177 \textormath{\textquotedblleft}{\mbox{\textquotedblleft}}}  
2178 \ProvideTextCommand{\grqq}{OT1}{%  
2179 \save@sf@q{\kern-.07em  
2180 \textormath{\textquotedblleft}{\mbox{\textquotedblleft}}}%  
2181 \kern.07em\relax}}  
2182 \ProvideTextCommandDefault{\grqq}{\UseTextSymbol{OT1}\grqq}
```

\flq

\frq The ‘french’ single guillemets.

```
2183 \ProvideTextCommandDefault{\flq}{%  
2184 \textormath{\guilsinglleft}{\mbox{\guilsinglleft}}}  
2185 \ProvideTextCommandDefault{\frq}{%  
2186 \textormath{\guilsinglright}{\mbox{\guilsinglright}}}
```

\flqq

\frqq The ‘french’ double guillemets.

```
2187 \ProvideTextCommandDefault{\flqq}{%  
2188 \textormath{\guillemetleft}{\mbox{\guillemetleft}}}  
2189 \ProvideTextCommandDefault{\frqq}{%  
2190 \textormath{\guillemetright}{\mbox{\guillemetright}}}
```

4.15.4. Umlauts and tremas

The command \ " needs to have a different effect for different languages. For German for instance, the ‘umlaut’ should be positioned lower than the default position for placing it over the letters a, o, u, A, O and U. When placed over an e, i, E or I it can retain its normal position. For Dutch the same glyph is always placed in the lower position.

\umlauthigh

\umlautlow To be able to provide both positions of `\` we provide two commands to switch the positioning, the default will be `\umlauthigh` (the normal positioning).

```

2191 \def\umlauthigh{%
2192   \def\bbbl@umlauta##1{\leavevmode\bgroup%
2193     \accent\csname\fontencoding dqpos\endcsname
2194     ##1\bbbl@allowhyphens\egroup}%
2195   \let\bbbl@umlaute\bbbl@umlauta}
2196 \def\umlautlow{%
2197   \def\bbbl@umlauta{\protect\lower@umlaut}}
2198 \def\umlautelow{%
2199   \def\bbbl@umlaute{\protect\lower@umlaut}}
2200 \umlauthigh

```

\lower@umlaut Used to position the `\` closer to the letter. We want the umlaut character lowered, nearer to the letter. To do this we need an extra (*dimen*) register.

```

2201 \expandafter\ifx\csname U@D\endcsname\relax
2202   \csname newdimen\endcsname\U@D
2203 \fi

```

The following code fools $\TeX$'s `make\_accent` procedure about the current x-height of the font to force another placement of the umlaut character. First we have to save the current x-height of the font, because we'll change this font dimension and this is always done globally.

Then we compute the new x-height in such a way that the umlaut character is lowered to the base character. The value of `.45ex` depends on the METAFONT parameters with which the fonts were built. (Just try out, which value will look best.) If the new x-height is too low, it is not changed. Finally we call the `\accent` primitive, reset the old x-height and insert the base character in the argument.

```

2204 \def\lower@umlaut#1{%
2205   \leavevmode\bgroup
2206     \U@D lex%
2207     {\setbox\z@\hbox{%
2208       \char\csname\fontencoding dqpos\endcsname}%
2209       \dimen@ -.45ex\advance\dimen@ht\z@
2210       \ifdim lex<\dimen@ \fontdimen5\font\dimen@ \fi}%
2211     \accent\csname\fontencoding dqpos\endcsname
2212     \fontdimen5\font\U@D #1%
2213   \egroup}

```

For all vowels we declare `\` to be a composite command which uses `\bbbl@umlauta` or `\bbbl@umlaute` to position the umlaut character. We need to be sure that these definitions override the ones that are provided when the package `fontenc` with option `OT1` is used. Therefore these declarations are postponed until the beginning of the document. Note these definitions only apply to some languages, but `babel` sets them for *all* languages – you may want to redefine `\bbbl@umlauta` and/or `\bbbl@umlaute` for a language in the corresponding `ldf` (using the `babel` switching mechanism, of course).

```

2214 \AtBeginDocument{%
2215   \DeclareTextCompositeCommand{\}{OT1}{a}{\bbbl@umlauta{a}}%
2216   \DeclareTextCompositeCommand{\}{OT1}{e}{\bbbl@umlaute{e}}%
2217   \DeclareTextCompositeCommand{\}{OT1}{i}{\bbbl@umlaute{i}}%
2218   \DeclareTextCompositeCommand{\}{OT1}{\i}{\bbbl@umlaute{i}}%
2219   \DeclareTextCompositeCommand{\}{OT1}{o}{\bbbl@umlauta{o}}%
2220   \DeclareTextCompositeCommand{\}{OT1}{u}{\bbbl@umlauta{u}}%
2221   \DeclareTextCompositeCommand{\}{OT1}{A}{\bbbl@umlauta{A}}%
2222   \DeclareTextCompositeCommand{\}{OT1}{E}{\bbbl@umlaute{E}}%
2223   \DeclareTextCompositeCommand{\}{OT1}{I}{\bbbl@umlaute{I}}%
2224   \DeclareTextCompositeCommand{\}{OT1}{O}{\bbbl@umlauta{O}}%
2225   \DeclareTextCompositeCommand{\}{OT1}{U}{\bbbl@umlauta{U}}%

```

Finally, make sure the default hyphenrules are defined (even if empty). For internal use, another empty `\language` is defined. Currently used in `Amharic`.

```

2226 \ifx\l@english\undefined
2227   \chardef\l@english\z@
2228 \fi

```

```

2229 % The following is used to cancel rules in ini files (see Amharic).
2230 \ifx\l@unhyphenated\@undefined
2231   \newlanguage\l@unhyphenated
2232 \fi

```

4.16. Layout

Layout is mainly intended to set bidi documents, but there is at least a tool useful in general.

```

2233 \bbl@trace{Bidi layout}
2234 \providecommand\IfBabelLayout[3]{#3}%

```

4.17. Load engine specific macros

Some macros are not defined in all engines, so, after loading the files define them if necessary to raise an error.

```

2235 \bbl@trace{Input engine specific macros}
2236 \ifcase\bbl@engine
2237   \input txtbabel.def
2238 \or
2239   \input luababel.def
2240 \or
2241   \input xebabel.def
2242 \fi
2243 \providecommand\babelfont{\bbl@error{only-lua-xe}{}}{}{}
2244 \providecommand\babelprehyphenation{\bbl@error{only-lua}{}}{}{}
2245 \ifx\babelposthyphenation\@undefined
2246   \let\babelposthyphenation\babelprehyphenation
2247   \let\babelpatterns\babelprehyphenation
2248   \let\babelcharproperty\babelprehyphenation
2249 \fi
2250 (/package|core)

```

4.18. Creating and modifying languages

Continue with $\TeX$ only.

`\babelprovide` is a general purpose tool for creating and modifying languages. It creates the language infrastructure, and loads, if requested, an ini file. It may be used in conjunction to previously loaded `ldf` files.

```

2251 (*package)
2252 \bbl@trace{Creating languages and reading ini files}
2253 \let\bbl@extend@ini\@gobble
2254 \newcommand\babelprovide[2][]{%
2255   \let\bbl@savelangname\languagename
2256   \edef\bbl@savelocaleid{\the\localeid}%
2257   \global\let\bbl@afterload\@empty
2258   % Set name and locale id
2259   \edef\languagename{#2}%
2260   \bbl@id@assign
2261   % Initialize keys
2262   \bbl@vforeach{captions,date,import,main,script,language,%
2263     hyphenrules,linebreaking,justification,mapfont,maparabic,%
2264     mapdigits,intraspaces,intrapenalty,onchar,transforms,alph,%
2265     Alph,labels,labels*,mapdot,calendar,date,casing,interchar,%
2266     @import}%
2267     {\bbl@csarg\let{KVP@##1}\@nnil}%
2268   \global\let\bbl@release@transforms\@empty
2269   \global\let\bbl@release@casing\@empty
2270   \let\bbl@calendars\@empty
2271   \global\let\bbl@inidata\@empty
2272   \global\let\bbl@extend@ini\@gobble
2273   \global\let\bbl@included@inis\@empty
2274   \gdef\bbl@key@list{;}%

```

```

2275 \bbl@ifunset{bbl@passto@#2}%
2276 {\def\bbl@tempa{#1}}%
2277 {\bbl@exp{\def\\bbl@tempa{[bbl@passto@#2],\unexpanded{#1}}}%
2278 \expandafter\bbl@forkv\expandafter{\bbl@tempa}{%
2279 \in@{/}{##1}% With /, (re)sets a value in the ini
2280 \ifin@
2281 \bbl@renewinikey##1\@{##2}%
2282 \else
2283 \bbl@csarg\ifx{KVP@##1}\@nnil\else
2284 \bbl@error{unknown-provide-key}{##1}{}}%
2285 \fi
2286 \bbl@csarg\def{KVP@##1}{##2}%
2287 \fi}%
2288 \chardef\bbl@howloaded=0:none; 1:ldf without ini; 2:ini
2289 \bbl@ifunset{date#2}\z@{\bbl@ifunset{bbl@llevel@#2}\@ne\tw@}%
2290 % == init ==
2291 \ifx\bbl@screset\undefined
2292 \bbl@ldfinit
2293 \fi
2294 % ==
2295 % If there is no import (last wins), use @import (internal, there
2296 % must be just one). To consider any order (because
2297 % \PassOptionsToLocale).
2298 \ifx\bbl@KVP@import\@nnil
2299 \let\bbl@KVP@import\bbl@KVP@@import
2300 \fi
2301 % == date (as option) ==
2302 % \ifx\bbl@KVP@date\@nnil\else
2303 % \fi
2304 % ==
2305 \let\bbl@lbkflag\relax % \@empty = do setup linebreak, only in 3 cases:
2306 \ifcase\bbl@howloaded
2307 \let\bbl@lbkflag\@empty % new
2308 \else
2309 \ifx\bbl@KVP@hyphenrules\@nnil\else
2310 \let\bbl@lbkflag\@empty
2311 \fi
2312 \ifx\bbl@KVP@import\@nnil\else
2313 \let\bbl@lbkflag\@empty
2314 \fi
2315 \fi
2316 % == import, captions ==
2317 \ifx\bbl@KVP@import\@nnil\else
2318 \bbl@exp{\\bbl@ifblank{\bbl@KVP@import}}%
2319 {\ifx\bbl@initoload\relax
2320 \begingroup
2321 \def\BabelBeforeIni##1##2{\gdef\bbl@KVP@import{##1}\endinput}%
2322 \bbl@input@texini{##2}%
2323 \endgroup
2324 \else
2325 \xdef\bbl@KVP@import{\bbl@initoload}%
2326 \fi}%
2327 {}%
2328 \let\bbl@KVP@date\@empty
2329 \fi
2330 \let\bbl@KVP@captions@@\bbl@KVP@captions
2331 \ifx\bbl@KVP@captions\@nnil
2332 \let\bbl@KVP@captions\bbl@KVP@import
2333 \fi
2334 % ==
2335 \ifx\bbl@KVP@transforms\@nnil\else
2336 \bbl@replace\bbl@KVP@transforms{ }{,}%
2337 \fi

```

```

2338 % Load ini
2339 % -----
2340 \ifcase\bb@howloaded
2341 \bb@provide@new{#2}%
2342 \else
2343 \bb@ifblank{#1}%
2344 {}% With \bb@load@basic below
2345 {\bb@provide@renew{#2}}%
2346 \fi
2347 % Post tasks
2348 % -----
2349 % == subsequent calls after the first provide for a locale ==
2350 \ifx\bb@inidata\@empty\else
2351 \bb@extend@ini{#2}%
2352 \fi
2353 % ==
2354 \ifx\bb@KVP@mapdot\@nnil\else
2355 \def\bb@tempa{\@empty}%
2356 \ifx\bb@KVP@mapdot\bb@tempa\else
2357 \bb@exp{\gdef\<bb@map@. @\language>{\bb@KVP@mapdot}}}%
2358 \fi
2359 \fi
2360 % == ensure captions ==
2361 \ifx\bb@KVP@captions\@nnil\else
2362 \bb@ifunset{bb@extracaps@#2}%
2363 {\bb@exp{\bb@babelensure[exclude=\\today]{#2}}}%
2364 {\bb@exp{\bb@babelensure[exclude=\\today,
2365 include=\bb@extracaps@#2]{#2}}}%
2366 \let\bb@tempc\language
2367 \bb@replace\bb@tempc{-}{@}%
2368 \bb@ifunset{bb@ensure@\bb@tempc}%
2369 {\bb@exp{%
2370 \\DeclareRobustCommand\<bb@ensure@\bb@tempc>[1]{%
2371 \\foreignlanguage{\language}%
2372 {###1}}}%
2373 }%
2374 \bb@exp{%
2375 \\bb@tglobal\<bb@ensure@\bb@tempc>%
2376 \\bb@tglobal\<bb@ensure@\bb@tempc\space>%
2377 \fi

```

At this point all parameters are defined if 'import'. Now we execute some code depending on them. But what about if nothing was imported? We just set the basic parameters, but still loading the whole ini file.

```

2378 \bb@load@basic{#2}%
2379 % == script, language ==
2380 % Override the values from ini or defines them
2381 \ifx\bb@KVP@script\@nnil\else
2382 \bb@csarg\edef{sname@#2}{\bb@KVP@script}%
2383 \fi
2384 \ifx\bb@KVP@language\@nnil\else
2385 \bb@csarg\edef{lname@#2}{\bb@KVP@language}%
2386 \fi
2387 \ifcase\bb@engine\or
2388 \bb@ifunset{bb@chrng@\language}%
2389 {\directlua{
2390 Babel.set_chranges_b('\bb@cl{sbc}', '\bb@cl{chrng}') }}%
2391 \fi
2392 % == Line breaking: intraspace, intrapenalty ==
2393 % For CJK, East Asian, Southeast Asian, if interspace in ini
2394 \ifx\bb@KVP@intraspace\@nnil\else % We can override the ini or set
2395 \bb@csarg\edef{intsp@#2}{\bb@KVP@intraspace}%
2396 \fi

```

```

2397 \bbl@provide@intraspace
2398 % == Line breaking: justification ==
2399 \ifx\bbl@KVP@justification\@nnil\else
2400 \let\bbl@KVP@linebreaking\bbl@KVP@justification
2401 \fi
2402 \ifx\bbl@KVP@linebreaking\@nnil\else
2403 \bbl@xin@{\,\bbl@KVP@linebreaking,}%
2404 {,elongated,kashida,cjk,padding,unhyphenated,}%
2405 \ifin@
2406 \bbl@csarg\xdef
2407 {lnbrk@{\language\name}}{\expandafter\@car\bbl@KVP@linebreaking\@nil}%
2408 \fi
2409 \fi
2410 \bbl@xin@{/e}{/\bbl@cl{lnbrk}}%
2411 \ifin@ \else \bbl@xin@{/k}{/\bbl@cl{lnbrk}} \fi
2412 \ifin@ \bbl@arabicjust \fi
2413 \bbl@xin@{/p}{/\bbl@cl{lnbrk}}%
2414 \ifin@ \AtBeginDocument{\@nameuse{bbl@tibetanjust}} \fi
2415 % == Line breaking: hyphenate.other.(locale|script) ==
2416 \ifx\bbl@lbfkflag\@empty
2417 \bbl@ifunset{bbl@hyotl@{\language\name}}{%
2418 {\bbl@csarg\bbl@replace{hyotl@{\language\name}}{ }{,}%
2419 \bbl@startcommands*{\language\name}}%
2420 \bbl@csarg\bbl@foreach{hyotl@{\language\name}}{%
2421 \ifcase\bbl@engine
2422 \ifnum##1<257
2423 \SetHyphenMap{\BabelLower{##1}{##1}}%
2424 \fi
2425 \else
2426 \SetHyphenMap{\BabelLower{##1}{##1}}%
2427 \fi}%
2428 \bbl@endcommands}%
2429 \bbl@ifunset{bbl@hyots@{\language\name}}{%
2430 {\bbl@csarg\bbl@replace{hyots@{\language\name}}{ }{,}%
2431 \bbl@csarg\bbl@foreach{hyots@{\language\name}}{%
2432 \ifcase\bbl@engine
2433 \ifnum##1<257
2434 \global\lccode##1=##1\relax
2435 \fi
2436 \else
2437 \global\lccode##1=##1\relax
2438 \fi}}%
2439 \fi
2440 % == Counters: maparabic ==
2441 % Native digits, if provided in ini (TeX level, xe and lua)
2442 \ifcase\bbl@engine\else
2443 \bbl@ifunset{bbl@dgnat@{\language\name}}{%
2444 {\expandafter\ifx\csname bbl@dgnat@{\language\name}\endcsname\@empty\else
2445 \expandafter\expandafter\expandafter
2446 \bbl@setdigits\csname bbl@dgnat@{\language\name}\endcsname
2447 \ifx\bbl@KVP@maparabic\@nnil\else
2448 \ifx\bbl@latinarabic\@undefined
2449 \expandafter\let\expandafter\@arabic
2450 \csname bbl@counter@{\language\name}\endcsname
2451 \else % i.e., if layout=counters, which redefines \@arabic
2452 \expandafter\let\expandafter\bbl@latinarabic
2453 \csname bbl@counter@{\language\name}\endcsname
2454 \fi
2455 \fi
2456 \fi}%
2457 \fi
2458 % == Counters: mapdigits ==
2459 % > luababel.def

```

```

2460 % == Counters: alph, Alph ==
2461 \ifx\bbbl@KVP@alph\@nnil\else
2462   \bbbl@exp{%
2463     \\bbbl@add\<bbbl@preextras@\language\name>{%
2464       \\babel@save\\@alph
2465       \let\\@alph\<bbbl@cntr@\bbbl@KVP@alph @\language\name>}}%
2466 \fi
2467 \ifx\bbbl@KVP@Alph\@nnil\else
2468   \bbbl@exp{%
2469     \\bbbl@add\<bbbl@preextras@\language\name>{%
2470       \\babel@save\\@Alph
2471       \let\\@Alph\<bbbl@cntr@\bbbl@KVP@Alph @\language\name>}}%
2472 \fi
2473 % == Counters: mapdot ==
2474 \ifx\bbbl@KVP@mapdot\@nnil\else
2475   \bbbl@foreach\bbbl@list@the{%
2476     \bbbl@ifunset{the##1}{}%
2477     {{\bbbl@ncarg\let\bbbl@tempd{the##1}%
2478       \bbbl@carg\bbbl@sreplace{the##1}{.}{\bbbl@map@lbl{.}}%
2479       \expandafter\ifx\csname the##1\endcsname\bbbl@tempd\else
2480         \bbbl@exp{\gdef\<the##1>{{\the##1}}}%
2481         \fi}}%
2482   \edef\bbbl@tempb{enumi,enumii,enumiii,enumiv}%
2483   \bbbl@foreach\bbbl@tempb{%
2484     \bbbl@ifunset{label##1}{}%
2485     {{\bbbl@ncarg\let\bbbl@tempd{label##1}%
2486       \bbbl@carg\bbbl@sreplace{label##1}{.}{\bbbl@map@lbl{.}}%
2487       \expandafter\ifx\csname label##1\endcsname\bbbl@tempd\else
2488         \bbbl@exp{\gdef\<label##1>{{\label##1}}}%
2489         \fi}}%
2490 \fi
2491 % == Casing ==
2492 \bbbl@release@casing
2493 \ifx\bbbl@KVP@casing\@nnil\else
2494   \bbbl@csarg\xdef{casing@\language\name}%
2495   {\@nameuse{bbbl@casing@\language\name}\bbbl@maybextx\bbbl@KVP@casing}%
2496 \fi
2497 % == Calendars ==
2498 \ifx\bbbl@KVP@calendar\@nnil
2499   \edef\bbbl@KVP@calendar{\bbbl@cl{calpr}}%
2500 \fi
2501 \def\bbbl@tempe##1 ##2\@{
2502   \def\bbbl@tempe{##1}%
2503   \bbbl@exp{\bbbl@tempe\bbbl@KVP@calendar\space\@}%
2504   \def\bbbl@tempe##1.##2.##3.##4\@{
2505     \def\bbbl@tempf{##3}% First
2506     \let\bbbl@tempc\@empty \let\bbbl@tempb\@empty
2507     \bbbl@cal@getconvert\bbbl@tempc{##1}%
2508     \bbbl@cal@getconvert\bbbl@tempb{##2}%
2509     \expandafter\bbbl@tempe\bbbl@tempe...\@
2510     \bbbl@csarg\edef{calpr@\language\name}{%
2511       \ifx\bbbl@tempc\@empty\else
2512         calendar=\bbbl@tempc
2513       \fi
2514       \ifx\bbbl@tempb\@empty\else
2515         ,variant=\bbbl@tempb
2516       \fi
2517       \ifx\bbbl@tempf\@empty\else
2518         ,convert=\bbbl@tempf
2519       \fi}%
2520 % == engine specific extensions ==
2521 % Defined in XXXbabel.def
2522 \bbbl@provide@extra{##2}%

```

```

2523 % == require.babel in ini ==
2524 % To load or reload the babel-*.tex, if require.babel in ini
2525 \ifx\bbl@beforestart\relax\else % But not in doc aux or body
2526   \bbl@ifunset{bbl@rqtex@\language\name}{}%
2527   {\expandafter\ifx\csname bbl@rqtex@\language\name\endcsname\@empty\else
2528     \let\BabelBeforeIni@gobbletwo
2529     \chardef\atcatcode=\catcode\@
2530     \catcode\@=11\relax
2531     \def\CurrentOption{#2}%
2532     \bbl@input@texini{\bbl@cs{rqtex@\language\name}}%
2533     \catcode\@=\atcatcode
2534     \let\atcatcode\relax
2535     \global\bbl@csarg\let{rqtex@\language\name}\relax
2536   \fi}%
2537 \bbl@foreach\bbl@calendars{%
2538   \bbl@ifunset{bbl@ca##1}{%
2539     \chardef\atcatcode=\catcode\@
2540     \catcode\@=11\relax
2541     \InputIfFileExists{babel-ca-##1.tex}{\fi}%
2542     \catcode\@=\atcatcode
2543     \let\atcatcode\relax}%
2544   }%
2545 \fi
2546 % == frenchspacing ==
2547 \ifcase\bbl@howloaded\in@true\else\in@false\fi
2548 \ifin@else\bbl@xin@{typography/frenchspacing}{\bbl@key@list}\fi
2549 \ifin@
2550   \bbl@extras@wrap{\bbl@pre@fs}%
2551   {\bbl@pre@fs}%
2552   {\bbl@post@fs}%
2553 \fi
2554 % == transforms ==
2555 % > luababel.def
2556 \def\CurrentOption{#2}%
2557 \@nameuse{bbl@icsave#2}%
2558 % == main ==
2559 \ifx\bbl@KVP@main\@nnil % Restore only if not 'main'
2560   \let\language\bbl@savelangname
2561   \chardef\localeid\bbl@savelocaleid\relax
2562 \fi
2563 % == ==
2564 \ifx\ldf@finish\@onlypreamble\else
2565   \bbl@afterload
2566 \fi
2567 % == hyphenrules (apply if current) ==
2568 \ifx\bbl@KVP@hyphenrules\@nnil\else
2569   \ifnum\bbl@savelocaleid=\localeid
2570     \language\@nameuse{l@\language\name}%
2571   \fi
2572 \fi}

```

Depending on whether or not the language exists (based on `\date<language>`), we define two macros. Remember `\bbl@startcommands` opens a group.

```

2573 \def\bbl@provide@new#1{%
2574   \@namedef{date#1}{}% marks lang exists - required by \StartBabelCommands
2575   \@namedef{extras#1}{}%
2576   \@namedef{noextras#1}{}%
2577   \bbl@startcommands*{#1}{captions}%
2578   \ifx\bbl@KVP@captions\@nnil % and also if import, implicit
2579     \def\bbl@tempb##1{%      elt for \bbl@captionslist
2580       \ifx##1\@nnil\else
2581         \bbl@exp{%
2582           \SetString\##1{%

```

```

2583         \\bbl@nocaption{\bbl@stripslash##1}{#1\bbl@stripslash##1}}}%
2584     \expandafter\bbl@tempb
2585     \fi}%
2586     \expandafter\bbl@tempb\bbl@captionslist\@nnil
2587 \else
2588     \ifx\bbl@initoload\relax
2589         \bbl@read@ini{\bbl@KVP@captions}2%    % Here letters cat = 11
2590     \else
2591         \bbl@read@ini{\bbl@initoload}2%        % Same
2592     \fi
2593 \fi
2594 \StartBabelCommands*{#1}{date}%
2595 \ifx\bbl@KVP@date\@nnil
2596     \bbl@exp{%
2597         \\SetString\\today{\\bbl@nocaption{today}{#1today}}}%
2598     \else
2599         \bbl@savetoday
2600         \bbl@savedate
2601     \fi
2602 \bbl@endcommands
2603 \bbl@load@basic{#1}%
2604 % == hyphenmins == (only if new)
2605 \bbl@exp{%
2606     \gdef<#1hyphenmins>{%
2607         {\bbl@ifunset{\bbl@lfthm@#1}{2}{\bbl@cs{lfthm@#1}}}%
2608         {\bbl@ifunset{\bbl@rgthm@#1}{3}{\bbl@cs{rgthm@#1}}}}}%
2609 % == hyphenrules (also in renew) ==
2610 \bbl@provide@hyphens{#1}%
2611 % == main ==
2612 \ifx\bbl@KVP@main\@nnil\else
2613     \expandafter\main@language\expandafter{#1}%
2614 \fi}
2615 %
2616 \def\bbl@provide@renew#1{%
2617     \ifx\bbl@KVP@captions\@nnil\else
2618         \StartBabelCommands*{#1}{captions}%
2619         \bbl@read@ini{\bbl@KVP@captions}2%    % Here all letters cat = 11
2620         \EndBabelCommands
2621     \fi
2622     \ifx\bbl@KVP@date\@nnil\else
2623         \StartBabelCommands*{#1}{date}%
2624         \bbl@savetoday
2625         \bbl@savedate
2626         \EndBabelCommands
2627     \fi
2628 % == hyphenrules (also in new) ==
2629 \ifx\bbl@lbkflag\@empty
2630     \bbl@provide@hyphens{#1}%
2631 \fi
2632 % == main ==
2633 \ifx\bbl@KVP@main\@nnil\else
2634     \expandafter\main@language\expandafter{#1}%
2635 \fi}

```

Load the basic parameters (ids, typography, counters, and a few more), while captions and dates are left out. But it may happen some data has been loaded before automatically, so we first discard the saved values.

```

2636 \def\bbl@load@basic#1{%
2637     \ifcase\bbl@howloaded\or\or
2638         \ifcase\csname bbl@llevel@\language\endcsname
2639             \bbl@csarg\let{lname@\language}\relax
2640         \fi
2641     \fi

```

```

2642 \bbl@ifunset{bbl@lname@#1}%
2643 {\def\BabelBeforeIni##1##2{%
2644   \begingroup
2645     \let\bbl@ini@captions@aux\@gobbletwo
2646     \def\bbl@inidate ####1.####2.####3.####4\relax ####5####6{%
2647       \bbl@read@ini{##1}l%
2648       \ifx\bbl@initoload\relax\endinput\fi
2649     \endgroup}%
2650   \begingroup      % boxed, to avoid extra spaces:
2651   \ifx\bbl@initoload\relax
2652     \bbl@input@texini{#1}%
2653   \else
2654     \setbox\z@\hbox{\BabelBeforeIni{\bbl@initoload}{}}%
2655   \fi
2656   \endgroup}%
2657   {}%

```

The following ini reader ignores everything but the identification section. It is called when a font is defined (i.e., when the language is first selected) to know which script/language must be enabled. This means we must make sure a few characters are not active. The ini is not read directly, but with a proxy tex file named as the language (which means any code in it must be skipped, too).

```

2658 \def\bbl@load@info#1{%
2659   \def\BabelBeforeIni##1##2{%
2660     \begingroup
2661       \bbl@read@ini{##1}0%
2662       \endinput      % babel- .tex may contain onlypreamble's
2663     \endgroup}%
2664     {\bbl@input@texini{#1}}%

```

The hyphenrules option is handled with an auxiliary macro. This macro is called in three cases: when a language is first declared with \babelprovide, with hyphenrules and with import.

```

2665 \def\bbl@provide@hyphens#1{%
2666   \@tempcnta@m@ne % a flag
2667   \ifx\bbl@KVP@hyphenrules\@nnil\else
2668     \bbl@replace\bbl@KVP@hyphenrules{ }{,}%
2669     \bbl@foreach\bbl@KVP@hyphenrules{%
2670       \ifnum\@tempcnta=\m@ne % if not yet found
2671         \bbl@ifsamestring{##1}{+}%
2672         {\bbl@carg\addlanguage{l@#1}}%
2673       }%
2674       \bbl@ifunset{l@#1}% After a possible +
2675       {}%
2676       {\@tempcnta\@nameuse{l@#1}}%
2677     \fi}%
2678   \ifnum\@tempcnta=\m@ne
2679     \bbl@warning{%
2680       Requested 'hyphenrules' for '\language' not found:\\%
2681       \bbl@KVP@hyphenrules.\\%
2682       Using the default value. Reported}%
2683   \fi
2684   \fi
2685   \ifnum\@tempcnta=\m@ne % if no opt or no language in opt found
2686     \ifx\bbl@KVP@captions@\@nnil
2687       \bbl@ifunset{bbl@hyphr@#1}{}% use value in ini, if exists
2688       {\bbl@exp{\bbl@ifblank{\bbl@cs{hyphr@#1}}}%
2689       }%
2690       {\bbl@ifunset{l@\bbl@cl{hyphr}}}%
2691       {}%
2692       {\@tempcnta\@nameuse{l@\bbl@cl{hyphr}}}%
2693     \fi
2694     \fi
2695     \bbl@ifunset{l@#1}%
2696     {\ifnum\@tempcnta=\m@ne
2697       \bbl@carg\adddialect{l@#1}\language

```

```

2698 \else
2699 \bbl@carg\adddialect{l@#1}\@tempcnta
2700 \fi}%
2701 {\ifnum\@tempcnta=\m@ne\else
2702 \global\bbl@carg\chardef{l@#1}\@tempcnta
2703 \fi}}

```

The reader of babel-...tex files. We reset temporarily some catcodes (and make sure no space is accidentally inserted).

```

2704 \def\bbl@input@texini#1{%
2705 \bbl@bsphack
2706 \bbl@exp{%
2707 \catcode`\\=\14 \catcode`\\=\0
2708 \catcode`\\=\1 \catcode`\\=\2
2709 \lowercase{\InputIfFileExists{babel-#1.tex}}{}}%
2710 \catcode`\\=\the\catcode`\relax
2711 \catcode`\\=\the\catcode`\relax
2712 \catcode`\\=\the\catcode`\relax
2713 \catcode`\\=\the\catcode`\relax}%
2714 \bbl@esphack}

```

The following macros read and store ini files (but don't process them). For each line, there are 3 possible actions: ignore if starts with ;, switch section if starts with [, and store otherwise. There are used in the first step of \bbl@read@ini.

```

2715 \def\bbl@iniline#1\bbl@iniline{%
2716 \@ifnextchar[\bbl@iniset{\@ifnextchar;\bbl@iniskip\bbl@inistore}#1\@@}% ]
2717 \def\bbl@iniset[#1]#2\@@{\def\bbl@section{#1}}
2718 \def\bbl@iniskip#1\@@{% if starts with ;
2719 \def\bbl@inistore#1=#2\@@{% full (default)
2720 \bbl@trim@def\bbl@tempa{#1}%
2721 \bbl@trim\toks@{#2}%
2722 \bbl@ifsamestring{\bbl@tempa}{\include}%
2723 {\bbl@read@subini{\the\toks@}}%
2724 {\bbl@xin@;\bbl@section/\bbl@tempa;}{\bbl@key@list}%
2725 \ifin@ \else
2726 \bbl@xin@{,identification/include.}%
2727 {,\bbl@section/\bbl@tempa}%
2728 \ifin@\xdef\bbl@included@inis{\the\toks@}\fi
2729 \bbl@exp{%
2730 \\g@addto@macro\\bbl@inidata{%
2731 \\bbl@elt{\bbl@section}{\bbl@tempa}{\the\toks@}}}%
2732 \fi}}
2733 \def\bbl@inistore@min#1=#2\@@{% minimal (maybe set in \bbl@read@ini)
2734 \bbl@trim@def\bbl@tempa{#1}%
2735 \bbl@trim\toks@{#2}%
2736 \bbl@xin@{.identification.}{.\bbl@section.}%
2737 \ifin@
2738 \bbl@exp{\\g@addto@macro\\bbl@inidata{%
2739 \\bbl@elt{identification}{\bbl@tempa}{\the\toks@}}}%
2740 \fi}

```

4.19. Main loop in 'provide'

Now, the 'main loop', \bbl@read@ini, which **must be executed inside a group**. At this point, \bbl@inidata may contain data declared in \babelprovide, with 'slashed' keys. There are 3 steps: first read the ini file and store it; then traverse the stored values, and process some groups if required (date, captions, labels, counters); finally, 'export' some values by defining global macros (identification, typography, characters, numbers). The second argument is 0 when called to read the minimal data for fonts; with \babelprovide it's either 1 (without import) or 2 (which import). The value -1 is used with \DocumentMetadata.

\bbl@loop@ini is the reader, line by line (1: stream), and calls \bbl@iniline to save the key/value pairs. If \bbl@inistore finds the @include directive, the input stream is switched temporarily and \bbl@read@subini is called.

When the language is being set based on the document metadata (#2 in \bbl@read@ini is -1), there is an interlude to get the name, after the data have been collected, and before it's processed.

```

2741 \def\bbl@loop@ini#1{%
2742   \loop
2743     \if T\ifeof#1 F\fi T\relax % Trick, because inside \loop
2744     \endlinechar\m@ne
2745     \read#1 to \bbl@line
2746     \endlinechar\^^M
2747     \ifx\bbl@line\empty\else
2748       \expandafter\bbl@iniline\bbl@line\bbl@iniline
2749     \fi
2750   \repeat}
2751 %
2752 \def\bbl@read@subini#1{%
2753   \ifx\bbl@readsubstream\undefined
2754     \csname newread\endcsname\bbl@readsubstream
2755   \fi
2756   \openin\bbl@readsubstream=babel-#1.ini
2757   \ifeof\bbl@readsubstream
2758     \bbl@error{no-ini-file}{#1}{}{}%
2759   \else
2760     {\bbl@loop@ini\bbl@readsubstream}%
2761   \fi
2762   \closein\bbl@readsubstream}
2763 %
2764 \ifx\bbl@readstream\undefined
2765   \csname newread\endcsname\bbl@readstream
2766 \fi
2767 \def\bbl@read@ini#1#2{%
2768   \global\let\bbl@extend@ini@gobble
2769   \openin\bbl@readstream=babel-#1.ini
2770   \ifeof\bbl@readstream
2771     \bbl@error{no-ini-file}{#1}{}{}%
2772   \else
2773     % == Store ini data in \bbl@inidata ==
2774     \catcode\ =10 \catcode\ =12
2775     \catcode\[ =12 \catcode\] =12 \catcode\ =12 \catcode\& =12
2776     \catcode\; =12 \catcode\| =12 \catcode\% =14 \catcode\ - =12
2777     \ifnum#2=\m@ne % Just for the info
2778       \edef\language{tag \bbl@metalang}%
2779     \fi
2780     \ifnum#2=\m@ne
2781       \bbl@info{Metadata: '\bbl@metalang' requested, '#1' provided.\\%
2782         Fetching the locale name from babel-#1.ini\@gobble}%
2783     \else
2784       \bbl@info{Importing
2785         \ifcase#2font and identification \or basic \fi
2786         data for '\language'\%
2787         from babel-#1.ini. Reported}%
2788     \fi
2789     \ifnum#2<\@ne
2790       \global\let\bbl@inidata\empty
2791       \let\bbl@inistore\bbl@inistore@min % Remember it's local
2792     \fi
2793     \def\bbl@section{identification}%
2794     \bbl@exp{%
2795       \\ \bbl@inistore tag.ini=#1\\ \@
2796       \\ \bbl@inistore load.level=\ifnum#2<\@ne 0\else #2\fi\\ \@}%
2797     \bbl@loop@ini\bbl@readstream
2798     % == Process stored data ==
2799     \ifnum#2=\m@ne
2800       \def\bbl@tempa##1 ##2\@{##1}% Get first name
2801       \def\bbl@elt##1##2##3{%

```

```

2802     \bbl@ifsamestring{identification/name.babel}{##1/##2}%
2803     {\edef\language\name{\bbl@tempa##3 \@@}%
2804     \let\localename\language
2805     \bbl@id@assign
2806     \def\bbl@elt###1###2###3{}}%
2807     {}}%
2808     \bbl@inidata
2809 \fi
2810 \bbl@csarg\xdef\l@ini{\language}{#1}%
2811 \bbl@read@ini@aux
2812 % == 'Export' data ==
2813 \bbl@ini@exports{#2}%
2814 \global\bbl@csarg\let\inidata@\language\bbl@inidata
2815 \global\let\bbl@inidata\empty
2816 \bbl@xin@{,\language,},{,\bbl@ini@loaded,}%
2817 \ifin@else
2818     \bbl@exp{\bbl@add@list\bbl@ini@loaded{\language}}%
2819     \bbl@tglobal\bbl@ini@loaded
2820 \fi
2821 \@ifpackagewith{babel}{licr=unextended}{}%
2822     {\ifcase\bbl@engine\else % Find a better place
2823     \bbl@xin@{,\bbl@cl{sbc},},{,Cyr,}%
2824     \ifin@
2825     \bbl@once{licr-cyr}{\g@addto@macro\bbl@afterload{\input{cyr2uni.def}}}%
2826     \fi
2827     \fi}%
2828 \fi
2829 \closein\bbl@readstream}
2830 \def\bbl@read@ini@aux{%
2831     \let\bbl@savestrings\empty
2832     \let\bbl@savetoday\empty
2833     \let\bbl@savestate\empty
2834     \def\bbl@elt###1###2###3{%
2835     \def\bbl@section{##1}%
2836     \in@{=date.}{=##1}% Find a better place
2837     \ifin@
2838     \bbl@ifunset{bbl@inikv@##1}%
2839     {\bbl@ini@calendar{##1}}%
2840     {}}%
2841     \fi
2842     \bbl@ifunset{bbl@inikv@##1}{}%
2843     {\csname bbl@inikv@##1\endcsname{##2}{##3}}}%
2844     \bbl@inidata}

```

A variant to be used when the ini file has been already loaded, because it's not the first \babelprovide for this language.

```

2845 \def\bbl@extend@ini@aux#1{%
2846     \bbl@startcommands*{#1}{captions}%
2847     % Activate captions/... and modify exports
2848     \bbl@csarg\def\inikv@captions.licr{##1##2}%
2849     \setlocalecaption{#1}{##1}{##2}%
2850     \def\bbl@inikv@captions##1##2{%
2851     \bbl@ini@captions@aux{##1}{##2}}%
2852     \def\bbl@stringdef##1##2{\gdef##1{##2}}%
2853     \def\bbl@exportkey##1##2##3{%
2854     \bbl@ifunset{bbl@kv@##2}{}%
2855     {\expandafter\ifx\csname bbl@kv@##2\endcsname\empty\else
2856     \bbl@exp{\global\let<bbl@##1\language>\<bbl@kv@##2>}}%
2857     \fi}}%
2858     % As with \bbl@read@ini, but with some changes
2859     \bbl@read@ini@aux
2860     \bbl@ini@exports\tw@
2861     % Update inidata@lang by pretending the ini is read.

```

```

2862 \def\bbl@elt##1##2##3{%
2863 \def\bbl@section{##1}%
2864 \bbl@iniline##2=##3\bbl@iniline}%
2865 \csname bbl@inidata@#1\endcsname
2866 \global\bbl@csarg\let{inidata@#1}\bbl@inidata
2867 \StartBabelCommands*{#1}{date}% And from the import stuff
2868 \def\bbl@stringdef##1##2{\gdef##1{##2}}%
2869 \bbl@savetoday
2870 \bbl@savestate
2871 \bbl@endcommands}

```

A somewhat hackish tool to handle calendar sections.

```

2872 \def\bbl@ini@calendar#1{%
2873 \lowercase{\def\bbl@tempa{=#1=}}%
2874 \bbl@replace\bbl@tempa{=date.gregorian}{}%
2875 \bbl@replace\bbl@tempa{=date.}{}%
2876 \in@{.licr=}{#1=}%
2877 \ifin@
2878 \ifcase\bbl@engine
2879 \bbl@replace\bbl@tempa{.licr=}{}%
2880 \else
2881 \let\bbl@tempa\relax
2882 \fi
2883 \fi
2884 \ifx\bbl@tempa\relax\else
2885 \bbl@replace\bbl@tempa{=}{}%
2886 \ifx\bbl@tempa@empty\else
2887 \xdef\bbl@calendars{\bbl@calendars,\bbl@tempa}%
2888 \fi
2889 \bbl@exp{%
2890 \def\<bbl@inikv@#1>####1####2{%
2891 \\\bbl@inidate####1...\relax{####2}{\bbl@tempa}}}%
2892 \fi}

```

A key with a slash in \babelprovide replaces the value in the ini file (which is ignored altogether). The mechanism is simple (but suboptimal): add the data to the ini one (at this point the ini file has not yet been read), and define a dummy macro. When the ini file is read, just skip the corresponding key and reset the macro (in \bbl@inistore above).

```

2893 \def\bbl@renewinikey#1/#2\@#3{%
2894 \global\let\bbl@extend@ini\bbl@extend@ini@aux
2895 \edef\bbl@tempa{\zap@space #1 \@empty}% section
2896 \edef\bbl@tempb{\zap@space #2 \@empty}% key
2897 \bbl@trim\toks@{#3}% value
2898 \bbl@exp{%
2899 \edef\\bbl@key@list{\bbl@key@list \bbl@tempa/\bbl@tempb;}%
2900 \\\g@addto@macro\\bbl@inidata{%
2901 \\\bbl@elt{\bbl@tempa}{\bbl@tempb}{\the\toks@}}}%

```

The previous assignments are local, so we need to export them. If the value is empty, we can provide a default value.

```

2902 \def\bbl@exportkey#1#2#3{%
2903 \bbl@ifunset{bbl@kv@#2}%
2904 {\bbl@csarg\gdef{#1@language}{#3}}%
2905 {\expandafter\ifx\csname bbl@kv@#2\endcsname\@empty
2906 \bbl@csarg\gdef{#1@language}{#3}%
2907 \else
2908 \bbl@exp{\global\let\<bbl@#1@language>\<bbl@kv@#2>}%
2909 \fi}}

```

Key-value pairs are treated differently depending on the section in the ini file. The following macros are the readers for identification and typography. Note \bbl@ini@exports is called always (via \bbl@inisec), while \bbl@after@ini must be called explicitly after \bbl@read@ini if necessary.

Although BCP 47 doesn't treat '-x-' as an extension, the CLDR and many other sources do (as a *private use extension*). For consistency with other single-letter subtags or 'singletons', here is considered an extension, too.

The identification section is used internally by babel in the following places [to be completed]: BCP 47 script tag in the Unicode ranges, which is in turn used by onchar; the language system is set with the names, and then fontspec maps them to the opentype tags, but if the latter package doesn't define them, then babel does it; encodings are used in pdfTeX to select a font encoding valid (and preloaded) for a language loaded on the fly.

```
2910 \def\bbl@iniwarning#1{%
2911   \bbl@ifunset{\bbl@kv@identification.warning#1}{}%
2912   {\bbl@warning{%
2913     From babel-\bbl@cs{lini@language}.ini:\\%
2914     \bbl@cs{@kv@identification.warning#1}\\%
2915     Reported}}}%
2916 %
2917 \let\bbl@release@transforms\@empty
2918 \let\bbl@release@casing\@empty
```

Relevant keys are 'exported', i.e., global macros with short names are created with values taken from the corresponding keys. The number of exported keys depends on the loading level (#1): -1 and 0 only info (the identification section), 1 also basic (like linebreaking or character ranges), 2 also (re)new (with date and captions).

```
2919 \def\bbl@ini@exports#1{%
2920   % Identification always exported
2921   \bbl@iniwarning{}%
2922   \ifcase\bbl@engine
2923     \bbl@iniwarning{.pdfLaTeX}%
2924   \or
2925     \bbl@iniwarning{.luaLaTeX}%
2926   \or
2927     \bbl@iniwarning{.XeLaTeX}%
2928   \fi%
2929   \bbl@exportkey{lllevel}{identification.load.level}{}%
2930   \bbl@exportkey{elname}{identification.name.english}{}%
2931   \bbl@expf{\bbl@exportkey{lname}{identification.name.opentype}%
2932     {\csname bbl@elname@language\endcsname}}%
2933   \bbl@exportkey{tbcpr}{identification.tag.bcp47}{}%
2934   \bbl@exportkey{casing}{identification.tag.bcp47}{}%
2935   \bbl@exportkey{lbcpr}{identification.language.tag.bcp47}{}%
2936   \bbl@exportkey{lotf}{identification.tag.opentype}{dflt}%
2937   \bbl@exportkey{esname}{identification.script.name}{}%
2938   \bbl@expf{\bbl@exportkey{sname}{identification.script.name.opentype}%
2939     {\csname bbl@esname@language\endcsname}}%
2940   \bbl@exportkey{sbcpr}{identification.script.tag.bcp47}{}%
2941   \bbl@exportkey{sotf}{identification.script.tag.opentype}{DFLT}%
2942   \bbl@exportkey{rbcp}{identification.region.tag.bcp47}{}%
2943   \bbl@exportkey{vbcp}{identification.variant.tag.bcp47}{}%
2944   \bbl@exportkey{extt}{identification.extension.t.tag.bcp47}{}%
2945   \bbl@exportkey{extu}{identification.extension.u.tag.bcp47}{}%
2946   \bbl@exportkey{extx}{identification.extension.x.tag.bcp47}{}%
2947   % Also maps bcp47 -> language
2948   \bbl@csarg\def{bcp@map@bbl@cl{tbcpr}}{\language}%
2949   \ifcase\bbl@engine\or
2950     \directlua{%
2951       Babel.locale_props[\the\bbl@cs{id@language}].script
2952       = '\bbl@cl{sbcpr}}%
2953   \fi
2954   % Conditional
2955   \ifnum#1>\z@ % -1 or 0 = only info, 1 = basic, 2 = (re)new
2956     \bbl@exportkey{calpr}{date.calendar.preferred}{}%
2957     \bbl@exportkey{lnbrk}{typography.linebreaking}{h}%
2958     \bbl@exportkey{hyphr}{typography.hyphenrules}{}%
2959     \bbl@exportkey{lftm}{typography.lefthyphenmin}{2}%

```

```

2960 \bbl@exportkey{rgthm}{typography.righthyphenmin}{3}%
2961 \bbl@exportkey{prehc}{typography.prehyphenchar}{}%
2962 \bbl@exportkey{hyotl}{typography.hyphenate.other.locale}{}%
2963 \bbl@exportkey{hyots}{typography.hyphenate.other.script}{}%
2964 \bbl@exportkey{intsp}{typography.intraspace}{}%
2965 \bbl@exportkey{frspc}{typography.frenchspacing}{u}%
2966 \bbl@exportkey{chrng}{characters.ranges}{}%
2967 \bbl@exportkey{quote}{characters.delimiters.quotes}{}%
2968 \bbl@exportkey{dgnat}{numbers.digits.native}{}%
2969 \ifnum#1=\tw@ % only (re)new
2970 \bbl@exportkey{rqtex}{identification.require.babel}{}%
2971 \bbl@tglobal\bbl@savetoday
2972 \bbl@tglobal\bbl@savedate
2973 \bbl@savestrings
2974 \fi
2975 \fi}

```

4.20. Processing keys in ini

A shared handler for key=val lines to be stored in \bbl@kv@<section>.<key>.

```

2976 \def\bbl@inikv#1#2{%      key=value
2977 \toks@{#2}%              This hides #'s from ini values
2978 \bbl@csarg\edef{@kv@\bbl@section.#1}{\the\toks@}}

```

By default, the following sections are just read. Actions are taken later.

```

2979 \let\bbl@inikv@identification\bbl@inikv
2980 \let\bbl@inikv@date\bbl@inikv
2981 \let\bbl@inikv@typography\bbl@inikv
2982 \let\bbl@inikv@numbers\bbl@inikv

```

The characters section also stores the values, but casing is treated in a different fashion. Much like transforms, a set of commands calling the parser are stored in \bbl@release@casing, which is executed in \babelprovide.

```

2983 \def\bbl@maybextx{-\bbl@csarg\ifx{extx@\language}\@empty x-\fi}
2984 \def\bbl@inikv@characters#1#2{%
2985 \bbl@ifsamestring{#1}{casing}% e.g., casing = uV
2986 {\bbl@exp{%
2987 \\\g@addto@macro{\bbl@release@casing{%
2988 \\\bbl@casemapping}{\language}\unexpanded{#2}}}%
2989 {\in@{casing.}{#1}% e.g., casing.Uv = uV
2990 \ifin@
2991 \lowercase{\def\bbl@tempb{#1}%
2992 \bbl@replace\bbl@tempb{casing.}}}%
2993 \bbl@exp{\\\g@addto@macro{\bbl@release@casing{%
2994 \\\bbl@casemapping
2995 {\\\bbl@maybextx\bbl@tempb}{\language}\unexpanded{#2}}}%
2996 \else
2997 \bbl@inikv{#1}{#2}%
2998 \fi}}

```

Additive numerals require an additional definition. When .1 is found, two macros are defined – the basic one, without .1 called by \localenumeral, and another one preserving the trailing .1 for the ‘units’. The asterisk is a ‘terminator’.

```

2999 \def\bbl@inikv@counters#1#2{%
3000 \bbl@ifsamestring{#1}{digits}%
3001 {\bbl@error{digits-is-reserved}{}}}%
3002 {}%
3003 \def\bbl@tempc{#1}%
3004 \bbl@trim@def{\bbl@tempb*}{#2}%
3005 \in@{.1$}{#1$}%
3006 \ifin@
3007 \bbl@replace\bbl@tempc{.1}{}%
3008 \bbl@csarg\protected@xdef{cntr@\bbl@tempc @\language}{%

```

```

3009 \noexpand\bbl@alphanumeric{\bbl@tempc}}%
3010 \fi
3011 \in@{.F.}{#1}%
3012 \ifin@else\in@{.S.}{#1}\fi
3013 \ifin@
3014 \bbl@csarg\protected@xdef{cnt@#1@language}{\bbl@tempb*}%
3015 \else
3016 \toks@{}% Required by \bbl@buildifcase, which returns \bbl@tempa
3017 \expandafter\bbl@buildifcase\bbl@tempb* \ \ % Space after \
3018 \bbl@csarg\global\expandafter\let{cnt@#1@language}\bbl@tempa
3019 \fi
3020 \in@{.D$}{#1$}%
3021 \ifin@else\in@{.C$}{#1$}\fi
3022 \ifin@
3023 \bbl@replace\bbl@tempc{.D}{}%
3024 \bbl@replace\bbl@tempc{.C}{}%
3025 \bbl@exp\gdef\<bbl@cnt@bbl@tempc @language>####1{%
3026 \expandafter\\bbl@zhnumeral\\expandafter{\\number####1}{\bbl@tempc}}}%
3027 \fi}

```

Now captions and captions.licr, depending on the engine. And below also for dates. They rely on a few auxiliary macros. It is expected the ini file provides the complete set in Unicode and LICR, in that order.

```

3028 \ifcase\bbl@engine
3029 \bbl@csarg\def{inikv@captions.licr}#1#2{%
3030 \bbl@ini@captions@aux{#1}{#2}}
3031 \else
3032 \def\bbl@inikv@captions#1#2{%
3033 \bbl@ini@captions@aux{#1}{#2}}
3034 \fi

```

The auxiliary macro for captions define \<caption>name.

```

3035 \def\bbl@ini@captions@template#1#2{% string language tempa=capt-name
3036 \bbl@replace\bbl@tempa{.template}{}%
3037 \def\bbl@toreplace{#1}{}%
3038 \bbl@replace\bbl@toreplace{[ ]}{\nobreakspace}}%
3039 \bbl@replace\bbl@toreplace{[ ]}{\csname}%
3040 \bbl@replace\bbl@toreplace{[ ]}{\csname the}%
3041 \bbl@replace\bbl@toreplace{[ ]}{\name\endcsname}}%
3042 \bbl@replace\bbl@toreplace{[ ]}{\endcsname}}%
3043 \bbl@xin@{,\bbl@tempa,}{,chapter,appendix,part,}%
3044 \ifin@
3045 \@nameuse{\bbl@patch\bbl@tempa}%
3046 \global\bbl@csarg\let{\bbl@tempa fmt@#2}\bbl@toreplace
3047 \fi
3048 \bbl@xin@{,\bbl@tempa,}{,figure,table,}%
3049 \ifin@
3050 \global\bbl@csarg\let{\bbl@tempa fmt@#2}\bbl@toreplace
3051 \bbl@exp\gdef\<fnum@bbl@tempa>{%
3052 \\bbl@ifunset{\bbl@bbl@tempa fmt@\\language}%
3053 {[fnum@bbl@tempa]}%
3054 {\\@nameuse{\bbl@bbl@tempa fmt@\\language}}}%
3055 \fi}
3056 %
3057 \def\bbl@ini@captions@aux#1#2{%
3058 \bbl@trim\def\bbl@tempa{#1}%
3059 \bbl@xin@{.template}{\bbl@tempa}%
3060 \ifin@
3061 \bbl@ini@captions@template{#2}\language
3062 \else
3063 \bbl@ifblank{#2}%
3064 {\bbl@exp{%
3065 \toks@{\\bbl@nocaption{\bbl@tempa name}{\language\bbl@tempa name}}}%
3066 {\bbl@trim\toks@{#2}}}%

```

```

3067 \bbl@exp{%
3068   \\\bbl@add\\bbl@savestrings{%
3069     \\\SetString<\bbl@tempa name>{\the\toks@}}}%
3070 \toks@ \expandafter{\bbl@captionslist}%
3071 \bbl@exp{\\\in@{\<\bbl@tempa name>}{\the\toks@}}%
3072 \ifin@ \else
3073   \bbl@exp{%
3074     \\\bbl@add<\bbl@extracaps@ \language name>{\<\bbl@tempa name>}}%
3075     \\\bbl@to global \<\bbl@extracaps@ \language name>}%
3076 \fi
3077 \fi}

```

Labels. Captions must contain just strings, no format at all, so there is new group in ini files.

```

3078 \def\bbl@list@the{%
3079   part,chapter,section,subsection,subsubsection,paragraph,%
3080   subparagraph,enumi,enumii,enumiii,enumiv,equation,figure,%
3081   table,page,footnote,mpfootnote,mpfn}
3082 %
3083 \def\bbl@map@cnt#1{% #1:roman,etc, // #2:enumi,etc
3084   \bbl@ifunset{\bbl@map@#1@ \language name}%
3085     {\@nameuse{#1}}%
3086     {\@nameuse{\bbl@map@#1@ \language name}}}
3087 %
3088 \def\bbl@map@lbl#1{% #1:a sign, eg, .
3089   \ifin@csname#1\else
3090     \bbl@ifunset{\bbl@map@#1@ \language name}%
3091       {#1}%
3092       {\@nameuse{\bbl@map@#1@ \language name}}}
3093 \fi}
3094 %
3095 \def\bbl@inikv@labels#1#2{%
3096   \in@{.map}{#1}%
3097   \ifin@
3098     \in@{,dot.map,}{, #1,}%
3099     \ifin@
3100       \global\@namedef{\bbl@map@. @ \language name}{#2}%
3101       \fi
3102     \ifx\bbl@KVP@labels\@nnil\else
3103       \bbl@xin@{ map }{ \bbl@KVP@labels \space}%
3104       \ifin@
3105         \def\bbl@tempc{#1}%
3106         \bbl@replace\bbl@tempc{.map}{}%
3107         \in@{, #2,}{,arabic,roman,Roman,alph,Alph,fnsymbol,}%
3108         \bbl@exp{%
3109           \gdef\<\bbl@map@\bbl@tempc @ \language name>%
3110             {\ifin@ \<#2> \else \\\localecounter{#2} \fi}}%
3111         \bbl@foreach\bbl@list@the{%
3112           \bbl@ifunset{the##1}{}%
3113           {\bbl@ncarg\let\bbl@tempd{the##1}%
3114             \bbl@exp{%
3115               \\\bbl@sreplace\<the##1>%
3116                 {\<\bbl@tempc>{##1}}%
3117                 {\\\bbl@map@cnt{\bbl@tempc}{##1}}%
3118               \\\bbl@sreplace\<the##1>%
3119                 {\<\@empty @\bbl@tempc>\<c@##1>%
3120                   {\\\bbl@map@cnt{\bbl@tempc}{##1}}%
3121                 \\\bbl@sreplace\<the##1>%
3122                   {\\\csname @\bbl@tempc\\endcsname\<c@##1>%
3123                     {\\\bbl@map@cnt{\bbl@tempc}{##1}}}%
3124                 \expandafter\ifx\csname the##1\endcsname\bbl@tempd\else
3125                   \bbl@exp{\gdef\<the##1>{\[the##1]}}%
3126                 \fi}}%
3127         \fi

```

```

3128 \fi
3129 %
3130 \else
3131 % The following code is still under study. You can test it and make
3132 % suggestions. E.g., enumerate.2 = ([enumi]).([enumii]). It's
3133 % language dependent.
3134 \in@{enumerate.}{#1}%
3135 \ifin@
3136 \def\bbl@tempa{#1}%
3137 \bbl@replace\bbl@tempa{enumerate.}{}%
3138 \def\bbl@toreplace{#2}%
3139 \bbl@replace\bbl@toreplace{[ ]}{\nobreakspace}%
3140 \bbl@replace\bbl@toreplace{[ ]}{\csname the}%
3141 \bbl@replace\bbl@toreplace{[ ]}{\endcsname}}%
3142 \toks@{\expandafter\bbl@toreplace}%
3143 \bbl@exp{%
3144 \\\bbl@add<extras\language>{%
3145 \\\babel@save<labelenum\romannumeral\bbl@tempa>%
3146 \def<labelenum\romannumeral\bbl@tempa>{\the\toks@}%
3147 \\\bbl@tglobal<extras\language>}%
3148 \fi
3149 \fi}

```

To show correctly some captions in a few languages, we need to patch some internal macros, because the order is hardcoded. For example, in Japanese the chapter number is surrounded by two string, while in Hungarian is placed after. These replacement works in many classes, but not all. Actually, the following lines are somewhat tentative.

```

3150 \def\bbl@chapttype{chapter}
3151 \ifx\@makechapterhead\@undefined
3152 \let\bbl@patchchapter\relax
3153 \else\ifx\thechapter\@undefined
3154 \let\bbl@patchchapter\relax
3155 \else\ifx\ps@headings\@undefined
3156 \let\bbl@patchchapter\relax
3157 \else
3158 \def\bbl@patchchapter{%
3159 \global\let\bbl@patchchapter\relax
3160 \gdef\bbl@chfmt{%
3161 \bbl@ifunset\bbl@\bbl@chapttype fmt@\language}%
3162 {\@chapapp\space\thechapter}%
3163 {\@nameuse\bbl@\bbl@chapttype fmt@\language}}}%
3164 \bbl@add\appendix{\def\bbl@chapttype{appendix}}% Not harmful, I hope
3165 \bbl@sreplace\ps@headings{\@chapapp\ \thechapter}{\bbl@chfmt}%
3166 \bbl@sreplace\chaptermark{\@chapapp\ \thechapter}{\bbl@chfmt}%
3167 \bbl@sreplace\@makechapterhead{\@chapapp\space\thechapter}{\bbl@chfmt}%
3168 \bbl@tglobal\appendix
3169 \bbl@tglobal\ps@headings
3170 \bbl@tglobal\chaptermark
3171 \bbl@tglobal\@makechapterhead}
3172 \let\bbl@patchappendix\bbl@patchchapter
3173 \fi\fi\fi
3174 \ifx\@part\@undefined
3175 \let\bbl@patchpart\relax
3176 \else
3177 \def\bbl@patchpart{%
3178 \global\let\bbl@patchpart\relax
3179 \gdef\bbl@partformat{%
3180 \bbl@ifunset\bbl@partfmt@\language}%
3181 {\@partname\nobreakspace\thepart}%
3182 {\@nameuse\bbl@partfmt@\language}}}%
3183 \bbl@sreplace\@part{\@partname\nobreakspace\thepart}{\bbl@partformat}%
3184 \bbl@tglobal\@part}
3185 \fi

```

Date. Arguments (year, month, day) are *not* protected, on purpose. In \today, arguments are always gregorian, and therefore always converted with other calendars.

```

3186 \def\bbl@cal@getconvert#1#2{% Used in provide
3187   \in{:}{#2}%
3188   \ifin@def\bbl@tempf{#2}\else\def#1{#2}\fi
3189 \let\bbl@calendar@empty
3190 \DeclareRobustCommand\localedate[1][\bbl@localedate{#1}]
3191 \def\bbl@localedate#1#2#3#4{%
3192   \begingroup
3193     \edef\bbl@they{#2}%
3194     \edef\bbl@them{#3}%
3195     \edef\bbl@thed{#4}%
3196     \edef\bbl@tempe{%
3197       \bbl@ifunset\bbl@calpr{\language\language}\bbl@cl{calpr}},%
3198       #1}%
3199     \bbl@exp{\lowercase{\edef\\bbl@tempe{\bbl@tempe}}}%
3200     \let\bbl@ld@calendar@empty
3201     \let\bbl@ld@variant@empty
3202     \let\bbl@ld@convert@relax
3203     \let\bbl@ld@@convert@relax
3204     \expandafter\bbl@forkv\expandafter{\bbl@tempe}{\bbl@csarg\edef{ld##1}{##2}}%
3205     \bbl@replace\bbl@ld@calendar{gregorian}%
3206     \ifx\bbl@ld@convert@relax
3207       \let\bbl@ld@convert\bbl@ld@@convert
3208     \fi
3209     \ifx\bbl@ld@convert@empty
3210       \ifx\bbl@ld@calendar@empty
3211         \let\bbl@ld@convert@relax
3212       \else
3213         \let\bbl@ld@convert\bbl@ld@calendar
3214       \fi
3215     \fi
3216     \ifx\bbl@ld@convert@relax\else
3217       \babelcalendar[\bbl@they-\bbl@them-\bbl@thed]%
3218       {\bbl@ld@convert}\bbl@they\bbl@them\bbl@thed
3219     \fi
3220     \@nameuse{\bbl@precalendar}% Remove, e.g., +, -civil (-ca-islamic)
3221     \edef\bbl@calendar{% Used in \month..., too
3222       \bbl@ld@calendar
3223       \ifx\bbl@ld@variant@empty\else
3224         .\bbl@ld@variant
3225       \fi}%
3226     \bbl@cased
3227     {\@nameuse{\bbl@date@\language @\bbl@calendar}%
3228      \bbl@they\bbl@them\bbl@thed}%
3229   \endgroup}
3230 %
3231 \def\bbl@printdate#1{%
3232   \ifnextchar[\bbl@printdate@i{#1}]{\bbl@printdate@i{#1}[]}}
3233 \def\bbl@printdate@i#1[#2]#3#4#5{%
3234   \bbl@usedategrouptrue
3235   \def\bbl@tempc{#1}%
3236   \bbl@replace\bbl@tempc{-}{@}%
3237   \@nameuse{\bbl@ensure@\bbl@tempc}{\localedate[#2][#3][#4][#5]}
3238 %
3239 % e.g.: 1=months, 2=wide, 3=1, 4=dummy, 5=value, 6=calendar
3240 \def\bbl@inidate#1.#2.#3.#4\relax#5#6{%
3241   \bbl@trim@def\bbl@tempa{#1.#2}%
3242   \bbl@ifsamestring{\bbl@tempa}{months.wide}%      to savedate
3243   {\bbl@trim@def\bbl@tempa{#3}%
3244    \bbl@trim\toks@{#5}%
3245    \@temptokena\expandafter{\bbl@savedate}%
3246    \bbl@exp{% Reverse order - in ini last wins

```

```

3247 \def\\bbl@savestate{%
3248   \\SetString\<month\romannumeral\bbl@tempa#6name>{\the\toks@}%
3249   \the\@temptokena}}}%
3250 {\bbl@ifsamestring{\bbl@tempa}{date.long}% defined now
3251 {\lowercase{\def\bbl@tempb{#6}}}%
3252 \bbl@trim@def\bbl@toreplace{#5}%
3253 \bbl@TG@date
3254 \global\bbl@csarg\let{date@\language name @\bbl@tempb}\bbl@toreplace
3255 \ifx\bbl@savestate\empty
3256 \bbl@exp{%
3257   \\AfterBabelCommands{%
3258     \gdef\<\language name date>{\\protect\<\language name date >}%
3259     \gdef\<\language name date >{\\bbl@printdate{\language name}}}%
3260   \def\\bbl@savestate{%
3261     \\SetString\\today{%
3262       \<\language name date>[@convert]%
3263       {\the\year}{\the\month}{\the\day}}}%
3264   \fi}%
3265   }}%

```

Dates will require some macros for the basic formatting. They may be redefined by language, so “semi-public” names (camel case) are used. Oddly enough, the CLDR places particles like “de” inconsistently in either in the date or in the month name. Note after `\bbl@replace \toks@` contains the resulting string, which is used by `\bbl@replace@finish@iii` (this implicit behavior doesn’t seem a good idea, but it’s efficient).

```

3266 \let\bbl@calendar\empty
3267 \newcommand\babelcalendar[2][\the\year-\the\month-\the\day]{%
3268   \bbl@xin@{\$calendrica:}{\$#2}%
3269   \ifin@
3270     \directlua{Babel.calendrica = Babel.calendrica or require("calendrica")}%
3271     \edef\bbl@tempa{\$#2}%
3272     \bbl@replace\bbl@tempa{\$calendrica:}{}%
3273     \bbl@replace\bbl@tempa{-}{_}%
3274     \bbl@afterelse
3275     \bbl@exp{\\bbl@calendrica#1\\@@{\bbl@tempa}}%
3276   \else
3277     \bbl@afterfi
3278     \@nameuse{\bbl@ca#2}#1\@@
3279   \fi}
3280 \def\bbl@calendrica#1-#2-#3\@@#4#5#6#7{%
3281   \directlua{
3282     local d = Babel.calendrica.new_date(
3283       {year=\number#1, month=\number#2, day=\number#3},
3284       Babel.calendrica.cal_gregorian)
3285     local r = Babel.calendrica.as_date(d, Babel.calendrica.cal_#4)
3286     token.set_macro('\bbl@stripslash#5', r.year)
3287     token.set_macro('\bbl@stripslash#6', r.month)
3288     token.set_macro('\bbl@stripslash#7', r.day) }}
3289 \newcommand\BabelDateSpace{\nobreakspace}
3290 \newcommand\BabelDateDot{.\@}
3291 \newcommand\BabelDated[1]{\number#1}
3292 \newcommand\BabelDatedd[1]{\ifnum#1<10 0\fi\number#1}
3293 \newcommand\BabelDateM[1]{\number#1}
3294 \newcommand\BabelDateMM[1]{\ifnum#1<10 0\fi\number#1}
3295 \newcommand\BabelDateMMM[1]{%
3296   \csname month\romannumeral#1\bbl@calendar name\endcsname}%
3297 \newcommand\BabelDatey[1]{\number#1}%
3298 \newcommand\BabelDateyy[1]{%
3299   \ifnum#1<10 0\number#1 %
3300   \else\ifnum#1<100 \number#1 %
3301   \else\ifnum#1<1000 \expandafter\@gobble\number#1 %
3302   \else\ifnum#1<10000 \expandafter\@gobbletwo\number#1 %
3303   \else

```

```

3304 \bbl@error{limit-two-digits}{\fi}{\fi}%
3305 \fi\fi\fi\fi}}
3306 \newcommand\BabelDateyyy[1]{\number#1}}
3307 \newcommand\BabelDateU[1]{\number#1}}%
3308 \def\bbl@replace@finish@iii#1{%
3309 \bbl@exp{\def\#1###1###2###3{\the\toks@}}
3310 \def\bbl@TG@date{%
3311 \bbl@replace\bbl@toreplace{[ ]}{\BabelDateSpace{}}%
3312 \bbl@replace\bbl@toreplace{[. ]}{\BabelDateDot{}}%
3313 \bbl@replace\bbl@toreplace{[y]}{\BabelDatey{###1}}%
3314 \bbl@replace\bbl@toreplace{[y|]}{\bbl@datecctr{###1|}}%
3315 \bbl@replace\bbl@toreplace{[yy]}{\BabelDateyy{###1}}%
3316 \bbl@replace\bbl@toreplace{[yyyy]}{\BabelDateyyyy{###1}}%
3317 \bbl@replace\bbl@toreplace{[M]}{\BabelDateM{###2}}%
3318 \bbl@replace\bbl@toreplace{[M|]}{\bbl@datecctr{###2|}}%
3319 \bbl@replace\bbl@toreplace{[MM]}{\BabelDateMM{###2}}%
3320 \bbl@replace\bbl@toreplace{[MMM]}{\BabelDateMMM{###2}}%
3321 \bbl@replace\bbl@toreplace{[d]}{\BabelDated{###3}}%
3322 \bbl@replace\bbl@toreplace{[d|]}{\bbl@datecctr{###3|}}%
3323 \bbl@replace\bbl@toreplace{[dd]}{\BabelDatedd{###3}}%
3324 \bbl@replace\bbl@toreplace{[U]}{\BabelDateU{###1}}%
3325 \bbl@replace\bbl@toreplace{[U|]}{\bbl@datecctr{###1|}}%
3326 \bbl@replace@finish@iii\bbl@toreplace}
3327 \def\bbl@datecctr{\expandafter\bbl@xdatecctr\expandafter}
3328 \def\bbl@xdatecctr[#1|#2]{\localenumeral{#2}{#1}}

```

4.21. French spacing (again)

For the following declarations, see issue #240. `\nonfrenchspacing` is set by document too early, so it's a hack.

```

3329 \AddToHook{begindocument/before}{%
3330 \let\bbl@normalsf\normalsfcodes
3331 \let\normalsfcodes\relax}
3332 \AtBeginDocument{%
3333 \ifx\bbl@normalsf\empty
3334 \ifnum\sfcodes\@m
3335 \let\normalsfcodes\frenchspacing
3336 \else
3337 \let\normalsfcodes\nonfrenchspacing
3338 \fi
3339 \else
3340 \let\normalsfcodes\bbl@normalsf
3341 \fi}

```

Transforms.

Process the transforms read from ini files, converts them to a form close to the user interface (with `\babelprehyphenation` and `\babelposthyphenation`), wrapped with `\bbl@transforms@aux` ...`\relax`, and stores them in `\bbl@release@transforms`. However, since building a list enclosed in braces isn't trivial, the replacements are added after a comma, and then `\bbl@transforms@aux` adds the braces.

```

3342 \bbl@csarg\let{inikv@transforms.prehyphenation}\bbl@inikv
3343 \bbl@csarg\let{inikv@transforms.posthyphenation}\bbl@inikv
3344 \def\bbl@transforms@aux#1#2#3#4,#5\relax{%
3345 #1[#2]{#3}{#4}{#5}}
3346 \begingroup
3347 \catcode\%=12
3348 \catcode\&=14
3349 \gdef\bbl@transforms#1#2#3{\&%
3350 \directlua{
3351 local str = [=[#2]=]
3352 str = str:gsub('%.%d+%.%d+$', '')
3353 token.set_macro('babeltempa', str)
3354 }&%

```

```

3355 \def\babeltempc{}&%
3356 \bbl@xin@{\babeltempa,}{,\bbl@KVP@transforms,}&%
3357 \ifin@else
3358 \bbl@xin@{\babeltempa,}{,\bbl@KVP@transforms,}&%
3359 \fi
3360 \ifin@
3361 \bbl@foreach\bbl@KVP@transforms{&%
3362 \bbl@xin@{\babeltempa,}{,##1,}&%
3363 \ifin@ &% font:font:transform syntax
3364 \directlua{
3365 local t = {}
3366 for m in string.gmatch('##1'..'':', '(.):') do
3367 table.insert(t, m)
3368 end
3369 table.remove(t)
3370 token.set_macro('babeltempc', ', fonts=' .. table.concat(t, ' '))
3371 }&%
3372 \fi}&%
3373 \in@{.0$}{#2$}&%
3374 \ifin@
3375 \directlua{&% (\attribute) syntax
3376 local str = string.match([[ \bbl@KVP@transforms]],
3377 '%([^(%[]-)%)[^%)]-\babeltempa')
3378 if str == nil then
3379 token.set_macro('babeltempb', '')
3380 else
3381 token.set_macro('babeltempb', ', attribute=' .. str)
3382 end
3383 }&%
3384 \toks@{#3}&%
3385 \bbl@exp{&%
3386 \\g@addto@macro\\bbl@release@transforms{&%
3387 \relax &% Closes previous \bbl@transforms@aux
3388 \\bbl@transforms@aux
3389 \#1{label=\babeltempa\babeltempb\babeltempc}&%
3390 {\language\the\toks@}}&%
3391 \else
3392 \g@addto@macro\bbl@release@transforms{, {#3}}&%
3393 \fi
3394 \fi}
3395 \endgroup

```

4.22. Handle language system

The language system (i.e., Language and Script) to be used when defining a font or setting the direction are set with the following macros. It also deals with unhyphenated line breaking in xetex (e.g., Thai and traditional Sanskrit), which is done with a hack at the font level because this engine doesn't support it.

```

3396 \def\bbl@provide@lsys#1{%
3397 \bbl@ifunset\bbl@lname@#1{%
3398 {\bbl@load@info{#1}}%
3399 }%
3400 \bbl@csarg\let{lsys@#1}\@empty
3401 \bbl@ifunset\bbl@sname@#1{\bbl@csarg\gdef{sname@#1}{Default}}{%
3402 \bbl@ifunset\bbl@sotf@#1{\bbl@csarg\gdef{sotf@#1}{DFLT}}{%
3403 \bbl@csarg\bbl@add@list{lsys@#1}{Script=\bbl@cs{sname@#1}}%
3404 \bbl@ifunset\bbl@lname@#1{%
3405 {\bbl@csarg\bbl@add@list{lsys@#1}{Language=\bbl@cs{lname@#1}}}%
3406 \ifcase\bbl@engine\or\or
3407 \bbl@ifunset\bbl@prehc@#1{%
3408 {\bbl@exp{\\bbl@ifblank{\bbl@cs{prehc@#1}}}%
3409 }%
3410 {\ifx\bbl@xenohyph\undefined

```

```

3411      \global\let\bbl@xenoxyph\bbl@xenoxyph@d
3412      \ifx\AtBeginDocument\@notprerr
3413        \expandafter\@secondoftwo % to execute right now
3414      \fi
3415      \AtBeginDocument{%
3416        \bbl@patchfont{\bbl@xenoxyph}%
3417        {\expandafter\select@language\expandafter{\language}}}%
3418    \fi}%
3419  \fi
3420  \bbl@csarg\bbl@tglobal{\sys@#1}}

```

4.23. Numerals

A tool to define the macros for native digits from the list provided in the `ini` file. Somewhat convoluted because there are 10 digits, but only 9 arguments in $\text{T}_{\text{E}}\text{X}$. Non-digits characters are kept. The first macro is the generic “localized” command.

[illegible]

Alphabetic counters must be converted from a space separated list to an \ifcase structure.

```

3452 \def\bbl@buildifcase#1 {% Returns \bbl@tempa, requires \toks@={}%
3453   \ifx\\#1%
3454     \bbl@exp{%
3455       \def\\bbl@tempa####1{%
3456         \<ifcase>####1\space\the\toks@\<else>\\@ctrerr\<fi>}}%
3457   \else
3458     \toks@\expandafter{\the\toks@\or #1}%
3459     \expandafter\bbl@buildifcase
3460 \fi}

```

The code for additive counters is somewhat tricky and it's based on the fact the arguments just before `@@` collects digits which have been left 'unused' in previous arguments, the first of them being the number of digits in the number to be converted. This explains the reverse set 76543210.

Digits above 10000 are not handled yet. When the key contains the subkey .F., the number after is treated as an special case, for a fixed form (see babel-he.ini, for example).

```

3461 \newcommand\localenumberal[2]{%
3462   \bbl@ifunset{\bbl@cntr@#1@\language\language}%
3463   {#2}%
3464   {\bbl@cs{cntr@#1@\language}{#2}}}
3465 \def\bbl@localecntr#1#2{\localenumberal{#2}{#1}}
3466 \newcommand\localecounter[2]{%
3467   \expandafter\bbl@localecntr
3468   \expandafter{\number\csname c@#2\endcsname}{#1}}
3469 \def\bbl@alphnumer#1#2{%
3470   \expandafter\bbl@alphnumer@i\number#2 76543210\@@{#1}}
3471 \def\bbl@alphnumer@i#1#2#3#4#5#6#7#8\@@#9{%
3472   \ifcase\@car#8\@nil\or    % Currently <10000, but prepared for bigger
3473   \bbl@alphnumer@ii{#9}00000#1\or
3474   \bbl@alphnumer@ii{#9}00000#1#2\or
3475   \bbl@alphnumer@ii{#9}00000#1#2#3\or
3476   \bbl@alphnumer@ii{#9}00000#1#2#3#4\else
3477   \bbl@error{counter-too-large}{>9999}{}}%
3478   \fi}
3479 \def\bbl@alphnumer@ii#1#2#3#4#5#6#7#8{%
3480   \bbl@ifunset{\bbl@cntr@#1.F.\number#5#6#7#8@\language}%
3481   {\bbl@cs{cntr@#1.4@\language}{#5}%
3482    \bbl@cs{cntr@#1.3@\language}{#6}%
3483    \bbl@cs{cntr@#1.2@\language}{#7}%
3484    \bbl@cs{cntr@#1.1@\language}{#8}%
3485    \ifnum#6#7#8>\z@
3486      \bbl@ifunset{\bbl@cntr@#1.S.321@\language}{}}%
3487      {\bbl@cs{cntr@#1.S.321@\language}{}}%
3488      \fi}%
3489   {\bbl@cs{cntr@#1.F.\number#5#6#7#8@\language}}}
```

Some Chinese counters follow special rules.

```

3490 \def\bbl@zhnumer#1#2{%
3491   \ifnum#1<10 \bbl@zhnum000#1{#2}%
3492   \else\ifnum#1<100 \bbl@zhnum00#1{#2}%
3493   \else\ifnum#1<1000 \bbl@zhnum0#1{#2}%
3494   \else\ifnum#1<10000 \bbl@zhnum#1{#2}%
3495   \else\bbl@error{counter-too-large}{#1}{}}%
3496   \fi\fi\fi\fi}
3497 \def\bbl@zhnum#1#2#3#4#5{%
3498   \ifnum\numexpr#1#2#3#4\relax=\z@
3499   \localenumberal{#5.C}\@ne
3500   \else
3501     \ifnum#1>\z@\localenumberal{#5.D}{#1}\localenumberal{#5.C}5\fi % Thousands
3502     \ifnum#2>\z@\localenumberal{#5.D}{#2}\localenumberal{#5.C}4\fi % Hundreds
3503     \else\ifnum#1>\z@\ifnum#3#4>\z@\localenumberal{#5.C}\@ne\fi
3504     \fi\fi
3505     \ifnum#3>\z@ % Tens
3506       \ifnum#1#2=\z@\ifnum#3=\@ne\else\localenumberal{#5.D}{#3}\fi % No  for 10-19:
3507       \else\localenumberal{#5.D}{#3}%
3508       \fi
3509       \localenumberal{#5.C}\thr@@
3510     \else
3511       \ifnum#2>\z@\ifnum#4>\z@\localenumberal{#5.C}\@ne\fi\fi
3512       \fi
3513       \ifnum#4>\z@\localenumberal{#5.D}{#4}\fi % Units
3514     \fi}
```

4.24. Casing

```

3515 \newcommand\BabelUppercaseMapping[3]{%
3516   \DeclareUppercaseMapping[\@nameuse{\bbl@casing@#1}]{#2}{#3}}
```

```

3517 \newcommand\BabelTitlecaseMapping[3]{%
3518   \DeclareTitlecaseMapping[\@nameuse{bbl@casing@#1}]{#2}{#3}}
3519 \newcommand\BabelLowercaseMapping[3]{%
3520   \DeclareLowercaseMapping[\@nameuse{bbl@casing@#1}]{#2}{#3}}

  The parser for casing and casing. (variant).
3521 \ifcase\bbl@engine % Converts utf8 to its code (expandable)
3522   \def\bbl@uftocode#1{\the\numexpr\decode@UTFviii#1\relax}
3523 \else
3524   \def\bbl@uftocode#1{\expandafter`\string#1}
3525 \fi
3526 \def\bbl@casemapping#1#2#3{% 1:variant
3527   \def\bbl@tempa##1 ##2{% Loop
3528     \bbl@casemapping@i{##1}%
3529     \ifx\@empty##2\else\bbl@afterfi\bbl@tempa##2\fi}%
3530   \edef\bbl@templ{\@nameuse{bbl@casing@#2}#1}% Language code
3531   \def\bbl@tempe{0}% Mode (upper/lower...)
3532   \def\bbl@tempc{#3}% Casing list
3533   \expandafter\bbl@tempa\bbl@tempc\@empty}
3534 \def\bbl@casemapping@i#1{%
3535   \def\bbl@tempb{#1}%
3536   \ifcase\bbl@engine % Handle utf8 in pdftex, by surrounding chars with {}
3537     \@nameuse{regex_replace_all:nnN}%
3538     {[{\x{c0}-\x{ff}}][\x{80}-\x{bf}]}*}{\@empty}\bbl@tempb
3539   \else
3540     \@nameuse{regex_replace_all:nnN}{.}{\@empty}\bbl@tempb
3541   \fi
3542   \expandafter\bbl@casemapping@ii\bbl@tempb\@empty}
3543 \def\bbl@casemapping@ii#1#2#3\@empty{%
3544   \in@{#1#3}{<>}% i.e., if <u>, <l>, <t>
3545   \ifin@
3546     \edef\bbl@tempe{%
3547       \if#2u1 \else\if#2l2 \else\if#2t3 \fi\fi\fi}%
3548   \else
3549     \ifcase\bbl@tempe\relax
3550       \DeclareUppercaseMapping[\bbl@templ]{\bbl@uftocode{#1}}{#2}%
3551       \DeclareLowercaseMapping[\bbl@templ]{\bbl@uftocode{#2}}{#1}%
3552     \or
3553       \DeclareUppercaseMapping[\bbl@templ]{\bbl@uftocode{#1}}{#2}%
3554     \or
3555       \DeclareLowercaseMapping[\bbl@templ]{\bbl@uftocode{#1}}{#2}%
3556     \or
3557       \DeclareTitlecaseMapping[\bbl@templ]{\bbl@uftocode{#1}}{#2}%
3558     \fi
3559   \fi}

```

4.25. Getting info

The information in the identification section can be useful, so the following macro just exposes it with a user command.

```

3560 \def\bbl@localeinfo#1#2{%
3561   \bbl@ifunset{bbl@info@#2}{#1}%
3562   {\bbl@ifunset{bbl@csname bbl@info@#2\endcsname @\languagename}{#1}%
3563    {\bbl@cs{csname bbl@info@#2\endcsname @\languagename}}}%
3564 \newcommand\localeinfo[1]{%
3565   \ifx*#1\@empty
3566     \bbl@afterelse\bbl@localeinfo{}%
3567   \else
3568     \bbl@localeinfo
3569     {\bbl@error{no-ini-info}}{}}}%
3570   {#1}%
3571 \fi}
3572 % \@namedef{bbl@info@name.locale}{lcname}
3573 \@namedef{bbl@info@tag.ini}{lini}

```

```

3574 \@namedef{bbl@info@name.english}{elname}
3575 \@namedef{bbl@info@name.opentype}{lname}
3576 \@namedef{bbl@info@tag.bcp47}{tbc}
3577 \@namedef{bbl@info@language.tag.bcp47}{lbc}
3578 \@namedef{bbl@info@tag.opentype}{lotf}
3579 \@namedef{bbl@info@script.name}{esname}
3580 \@namedef{bbl@info@script.name.opentype}{sname}
3581 \@namedef{bbl@info@script.tag.bcp47}{sbcp}
3582 \@namedef{bbl@info@script.tag.opentype}{sotf}
3583 \@namedef{bbl@info@region.tag.bcp47}{rbcp}
3584 \@namedef{bbl@info@variant.tag.bcp47}{vbcp}
3585 \@namedef{bbl@info@extension.t.tag.bcp47}{extt}
3586 \@namedef{bbl@info@extension.u.tag.bcp47}{extu}
3587 \@namedef{bbl@info@extension.x.tag.bcp47}{extx}

```

With version 3.75 \BabelEnsureInfo is executed always, but there is an option to disable it. Since the info in ini files are always loaded, it has been made no-op in version 25.8.

```

3588 (<*More package options>) ≡
3589 \DeclareOption{ensureinfo=off}{}
3590 (</More package options>)
3591 \let\BabelEnsureInfo\relax

```

More general, but non-expandable, is \getlocaleproperty.

```

3592 \newcommand\getlocaleproperty{%
3593   \@ifstar\bbl@getproperty@s\bbl@getproperty@x}
3594 \def\bbl@getproperty@s#1#2#3{%
3595   \let#1\relax
3596   \def\bbl@elt##1##2##3{%
3597     \bbl@ifsamestring{##1/##2}{#3}%
3598     {\providecommand#1{##3}%
3599     \def\bbl@elt####1####2####3{}}}%
3600   {}}%
3601   \bbl@cs{inidata@#2}}%
3602 \def\bbl@getproperty@x#1#2#3{%
3603   \bbl@getproperty@s{#1}{#2}{#3}%
3604   \ifx#1\relax
3605     \bbl@error{unknown-locale-key}{#1}{#2}{#3}%
3606   \fi}

```

To inspect every possible loaded ini, we define \LocaleForEach, where \bbl@ini@loaded is a comma-separated list of locales, built by \bbl@read@ini.

```

3607 \let\bbl@ini@loaded\@empty
3608 \newcommand\LocaleForEach{\bbl@foreach\bbl@ini@loaded}
3609 \def\ShowLocaleProperties#1{%
3610   \typeout{}}%
3611   \typeout{*** Properties for language '#1' ***}%
3612   \def\bbl@elt##1##2##3{\typeout{##1/##2 = \unexpanded{##3}}}%
3613   \@nameuse{bbl@inidata@#1}%
3614   \typeout{*****}}

```

4.26. BCP 47 related commands

This macro is called by language selectors when the language isn't recognized. So, it's the core for (1) mapping from a BCP 47 tag to the actual language, if bcp47.toname is enabled (i.e., if bbl@bcptoname is true), and (2) lazy loading. With autoload.bcp47 enabled *and* lazy loading, we must first build a name for the language, with the help of autoload.bcp47.prefix. Then we use \provideprovide passing the options set with autoload.bcp47.options (by default import). Finally, and if the locale has not been loaded before, we use \provideprovide with the language name as passed to the selector.

```

3615 \newif\ifbbl@bcppallowed
3616 \bbl@bcppallowedfalse
3617 \def\bbl@autoload@options{@import}
3618 \def\bbl@provide@locale{%

```

```

3619 \ifx\babelprovide\undefined
3620   \bbl@error{base-on-the-fly}{\}\}%
3621 \fi
3622 \let\bbl@auxname\language
3623 \ifbbl@bcptoname
3624   \bbl@ifunset{bbl@bcp@map@\language}{\}% Move uplevel??
3625   {\edef\language{\@nameuse{bbl@bcp@map@\language}}\%
3626     \let\localname\language}%
3627 \fi
3628 \ifbbl@bcpallowed
3629   \expandafter\ifx\csname date\language\endcsname\relax
3630     \expandafter
3631     \bbl@bcplookup{\language}%
3632     \ifx\bbl@bcp\relax\else % Returned by \bbl@bcplookup
3633       \edef\language{\bbl@bcp@prefix\bbl@bcp}%
3634       \let\localname\language
3635       \expandafter\ifx\csname date\language\endcsname\relax
3636         \let\bbl@initoload\bbl@bcp
3637         \bbl@exp{\babelprovide[\bbl@autoload@bcptoptions]{\language}}%
3638         \let\bbl@initoload\relax
3639       \fi
3640       \bbl@csarg\xdef{bcp@map@\bbl@bcp}{\localname}%
3641     \fi
3642   \fi
3643 \fi
3644 \expandafter\ifx\csname date\language\endcsname\relax
3645   \IfFileExists{babel-\language.tex}%
3646   {\bbl@exp{\babelprovide[\bbl@autoload@options]{\language}}}%
3647   {\}%
3648 \fi}

```

$\text{\LaTeX}$ needs to know the BCP 47 codes for some features. For that, it expects `\BCPdata` to be defined. While language, region, script, and variant are recognized, extension `.<s>` for singletons may change.

Still somewhat hackish. Note `\str_if_eq:nnTF` is fully expandable (`\bbl@ifsamestring` isn't). The argument is the prefix to tag.bcp47.

```

3649 \providecommand\BCPdata{}
3650 \ifx\renewcommand\undefined\else
3651   \renewcommand\BCPdata[1]{\bbl@bcpdata@i#1\@empty\@empty\@empty}
3652   \def\bbl@bcpdata@i#1#2#3#4#5#6\@empty{%
3653     \@nameuse{str_if_eq:nnTF}{#1#2#3#4#5}{main.}%
3654     {\bbl@bcpdata@ii{#6}\bbl@main@language}%
3655     {\bbl@bcpdata@ii{#1#2#3#4#5#6}\language}}%
3656   \def\bbl@bcpdata@ii#1#2{%
3657     \bbl@ifunset{bbl@info@#1.tag.bcp47}%
3658     {\bbl@error{unknown-ini-field}{#1}{\}\}%
3659     {\bbl@ifunset{bbl@\csname bbl@info@#1.tag.bcp47\endcsname @#2}{\}%
3660      {\bbl@cs{\csname bbl@info@#1.tag.bcp47\endcsname @#2}}}}
3661 \fi
3662 \@namedef{bbl@info@casing.tag.bcp47}{casing}
3663 \@namedef{bbl@info@tag.tag.bcp47}{tbc} % For \BCPdata

```

5. Adjusting the Babel behavior

A generic high level interface is provided to adjust some global and general settings.

```

3664 \newcommand\babeladjust[1]{%
3665   \bbl@forkv{#1}{%
3666     \bbl@ifunset{bbl@ADJ@##1@##2}%
3667     {\bbl@cs{ADJ@##1}{##2}}%
3668     {\bbl@cs{ADJ@##1@##2}}}%
3669 %
3670 \def\bbl@adjust@lua#1#2{%

```

```

3671 \ifvmode
3672   \ifnum\currentgrouplevel=\z@
3673     \directlua{ Babel.#2 }%
3674     \expandafter\expandafter\expandafter\@gobble
3675   \fi
3676 \fi
3677 {\bbl@error{adjust-only-vertical}{#1}{}}}% Gobbled if everything went ok.
3678 \@namedef{bbl@ADJ@bidi.mirroring@on}{%
3679   \bbl@adjust@lua{bidi}{mirroring_enabled=true}}
3680 \@namedef{bbl@ADJ@bidi.mirroring@off}{%
3681   \bbl@adjust@lua{bidi}{mirroring_enabled=false}}
3682 \@namedef{bbl@ADJ@bidi.text@on}{%
3683   \bbl@adjust@lua{bidi}{bidi_enabled=true}}
3684 \@namedef{bbl@ADJ@bidi.text@off}{%
3685   \bbl@adjust@lua{bidi}{bidi_enabled=false}}
3686 \@namedef{bbl@ADJ@bidi.math@on}{%
3687   \let\bbl@noamsmath\@empty}
3688 \@namedef{bbl@ADJ@bidi.math@off}{%
3689   \let\bbl@noamsmath\relax}
3690 %
3691 \@namedef{bbl@ADJ@bidi.mapdigits@on}{%
3692   \bbl@adjust@lua{bidi}{digits_mapped=true}}
3693 \@namedef{bbl@ADJ@bidi.mapdigits@off}{%
3694   \bbl@adjust@lua{bidi}{digits_mapped=false}}
3695 %
3696 \@namedef{bbl@ADJ@linebreak.sea@on}{%
3697   \bbl@adjust@lua{linebreak}{sea_enabled=true}}
3698 \@namedef{bbl@ADJ@linebreak.sea@off}{%
3699   \bbl@adjust@lua{linebreak}{sea_enabled=false}}
3700 \@namedef{bbl@ADJ@linebreak.cjk@on}{%
3701   \bbl@adjust@lua{linebreak}{cjk_enabled=true}}
3702 \@namedef{bbl@ADJ@linebreak.cjk@off}{%
3703   \bbl@adjust@lua{linebreak}{cjk_enabled=false}}
3704 \@namedef{bbl@ADJ@justify.arabic@on}{%
3705   \bbl@adjust@lua{linebreak}{arabic.justify_enabled=true}}
3706 \@namedef{bbl@ADJ@justify.arabic@off}{%
3707   \bbl@adjust@lua{linebreak}{arabic.justify_enabled=false}}
3708 %
3709 \def\bbl@adjust@layout#1{%
3710   \ifvmode
3711     #1%
3712     \expandafter\@gobble
3713   \fi
3714   {\bbl@error{layout-only-vertical}{}}}% Gobbled if everything went ok.
3715 \@namedef{bbl@ADJ@layout.tabular@on}{%
3716   \ifnum\bbl@tabular@mode=\tw@
3717     \bbl@adjust@layout{\let\@tabular\bbl@NL@@tabular}%
3718   \else
3719     \chardef\bbl@tabular@mode\@ne
3720   \fi}
3721 \@namedef{bbl@ADJ@layout.tabular@off}{%
3722   \ifnum\bbl@tabular@mode=\tw@
3723     \bbl@adjust@layout{\let\@tabular\bbl@OL@@tabular}%
3724   \else
3725     \chardef\bbl@tabular@mode\z@
3726   \fi}
3727 \@namedef{bbl@ADJ@layout.lists@on}{%
3728   \bbl@adjust@layout{\let\list\bbl@NL@list}}
3729 \@namedef{bbl@ADJ@layout.lists@off}{%
3730   \bbl@adjust@layout{\let\list\bbl@OL@list}}
3731 %
3732 \@namedef{bbl@ADJ@autoload.bcp47@on}{%
3733   \bbl@bcpallowedtrue}

```

```

3734 \@namedef{bbl@ADJ@autoload.bcp47@off}{%
3735   \bbl@bcppallowedfalse}
3736 \@namedef{bbl@ADJ@autoload.bcp47.prefix}#1{%
3737   \def\bbl@bcp@prefix{#1}}
3738 \def\bbl@bcp@prefix{bcp47-}
3739 \@namedef{bbl@ADJ@autoload.options}#1{%
3740   \def\bbl@autoload@options{#1}}
3741 \def\bbl@autoload@bcptoptions{import}
3742 \@namedef{bbl@ADJ@autoload.bcp47.options}#1{%
3743   \def\bbl@autoload@bcptoptions{#1}}
3744 \newif\ifbbl@bcptname
3745 %
3746 \@namedef{bbl@ADJ@bcp47.toname@on}{%
3747   \bbl@bcptnametrue}
3748 \@namedef{bbl@ADJ@bcp47.toname@off}{%
3749   \bbl@bcptnamefalse}
3750 %
3751 \@namedef{bbl@ADJ@prehyphenation.disable@nohyphenation}{%
3752   \directlua{ Babel.ignore_pre_char = function(node)
3753     return (node.lang == \the\csname \l@nohyphenation\endcsname)
3754   end }}
3755 \@namedef{bbl@ADJ@prehyphenation.disable@off}{%
3756   \directlua{ Babel.ignore_pre_char = function(node)
3757     return false
3758   end }}
3759 %
3760 \@namedef{bbl@ADJ@interchar.disable@nohyphenation}{%
3761   \def\bbl@ignoreinterchar{%
3762     \ifnum\language=\l@nohyphenation
3763       \expandafter\@gobble
3764     \else
3765       \expandafter\@firstofone
3766     \fi}}
3767 \@namedef{bbl@ADJ@interchar.disable@off}{%
3768   \let\bbl@ignoreinterchar\@firstofone}
3769 %
3770 \@namedef{bbl@ADJ@select.write@shift}{%
3771   \let\bbl@restorelastskip\relax
3772   \def\bbl@savelastskip{%
3773     \let\bbl@restorelastskip\relax
3774     \ifvmode
3775       \ifdim\lastskip=\z@
3776         \let\bbl@restorelastskip\nobreak
3777       \else
3778         \bbl@exp{%
3779           \def\\bbl@restorelastskip{%
3780             \skip@=\the\lastskip
3781             \\nobreak \vskip-\skip@ \vskip\skip@}}%
3782         \fi
3783       \fi}}
3784 \@namedef{bbl@ADJ@select.write@keep}{%
3785   \let\bbl@restorelastskip\relax
3786   \let\bbl@savelastskip\relax}
3787 \@namedef{bbl@ADJ@select.write@omit}{%
3788   \AddBabelHook{babel-select}{beforestart}{%
3789     \expandafter\babel@aux\expandafter{\bbl@main@language}}}%
3790   \let\bbl@restorelastskip\relax
3791   \def\bbl@savelastskip##1\bbl@restorelastskip{}}
3792 \@namedef{bbl@ADJ@select.encoding@off}{%
3793   \let\bbl@encoding@select@off\@empty}

```

5.1. Cross referencing macros

The $\LaTeX$ book states:

The *key* argument is any sequence of letters, digits, and punctuation symbols; upper- and lowercase letters are regarded as different.

When the above quote should still be true when a document is typeset in a language that has active characters, special care has to be taken of the category codes of these characters when they appear in an argument of the cross referencing macros.

When a cross referencing command processes its argument, all tokens in this argument should be character tokens with category ‘letter’ or ‘other’.

The following package options control which macros are to be redefined.

```
3794 ({*More package options}) ≡
3795 \DeclareOption{safe=none}{\let\bbl@opt@safe\empty}
3796 \DeclareOption{safe=bib}{\def\bbl@opt@safe{B}}
3797 \DeclareOption{safe=ref}{\def\bbl@opt@safe{R}}
3798 \DeclareOption{safe=refbib}{\def\bbl@opt@safe{BR}}
3799 \DeclareOption{safe=bibref}{\def\bbl@opt@safe{BR}}
3800 (/More package options)
```

\@newl@bel First we open a new group to keep the changed setting of `\protect local` and then we set the `@safe@actives` switch to true to make sure that any shorthand that appears in any of the arguments immediately expands to its non-active self.

```
3801 \bbl@trace{Cross referencing macros}
3802 \ifx\bbl@opt@safe\empty\else % i.e., if 'ref' and/or 'bib'
3803   \def\@newl@bel#1#2#3{%
3804     {\@safe@activestrue
3805       \bbl@ifunset{#1@#2}%
3806         \relax
3807         {\gdef\@multiplelabels{%
3808           \@latex@warning@no@line{There were multiply-defined labels}}%
3809           \@latex@warning@no@line{Label `#2' multiply defined}}%
3810       \global\@namedef{#1@#2}{#3}}}
```

\@testdef An internal $\LaTeX$ macro used to test if the labels that have been written on the aux file have changed. It is called by the `\enddocument` macro.

```
3811 \CheckCommand*\@testdef[3]{%
3812   \def\reserved@a{#3}%
3813   \expandafter\ifx\csname#1@#2\endcsname\reserved@a
3814   \else
3815     \@tempswatrue
3816   \fi}
```

Now that we made sure that `\@testdef` still has the same definition we can rewrite it. First we make the shorthands ‘safe’. Then we use `\bbl@tempa` as an ‘alias’ for the macro that contains the label which is being checked. Then we define `\bbl@tempb` just as `\@newl@bel` does it. When the label is defined we replace the definition of `\bbl@tempa` by its meaning. If the label didn’t change, `\bbl@tempa` and `\bbl@tempb` should be identical macros.

```
3817 \def\@testdef#1#2#3{%
3818   \@safe@activestrue
3819   \expandafter\let\expandafter\bbl@tempa\csname #1@#2\endcsname
3820   \def\bbl@tempb{#3}%
3821   \@safe@activesfalse
3822   \ifx\bbl@tempa\relax
3823   \else
3824     \edef\bbl@tempa{\expandafter\strip@prefix\meaning\bbl@tempa}%
3825   \fi
3826   \edef\bbl@tempb{\expandafter\strip@prefix\meaning\bbl@tempb}%
3827   \ifx\bbl@tempa\bbl@tempb
3828   \else
3829     \@tempswatrue
3830   \fi}
3831 \fi
```

\ref

\pageref The same holds for the macro `\ref` that references a label and `\pageref` to reference a page. We make them robust as well (if they weren't already) to prevent problems if they should become expanded at the wrong moment.

```
3832 \bbl@xin@{R}\bbl@opt@safe
3833 \ifin@
3834 \edef\bbl@tempc{\expandafter\string\csname ref_code\endcsname}%
3835 \bbl@xin@{\expandafter\strip@prefix\meaning\bbl@tempc}%
3836 {\expandafter\strip@prefix\meaning\ref}%
3837 \ifin@
3838 \bbl@redefine\@kernel@ref#1{%
3839   \@safe@activetrue\org@@kernel@ref{#1}\@safe@activfalse}
3840 \bbl@redefine\@kernel@pageref#1{%
3841   \@safe@activetrue\org@@kernel@pageref{#1}\@safe@activfalse}
3842 \bbl@redefine\@kernel@sref#1{%
3843   \@safe@activetrue\org@@kernel@sref{#1}\@safe@activfalse}
3844 \bbl@redefine\@kernel@spageref#1{%
3845   \@safe@activetrue\org@@kernel@spageref{#1}\@safe@activfalse}
3846 \else
3847 \bbl@redefinero bust\ref#1{%
3848   \@safe@activetrue\org@ref{#1}\@safe@activfalse}
3849 \bbl@redefinero bust\pageref#1{%
3850   \@safe@activetrue\org@pageref{#1}\@safe@activfalse}
3851 \fi
3852 \else
3853 \let\org@ref\ref
3854 \let\org@pageref\pageref
3855 \fi
```

\@citex The macro used to cite from a bibliography, `\cite`, uses an internal macro, `\@citex`. It is this internal macro that picks up the argument(s), so we redefine this internal macro and leave `\cite` alone. The first argument is used for typesetting, so the shorthands need only be deactivated in the second argument.

```
3856 \bbl@xin@{B}\bbl@opt@safe
3857 \ifin@
3858 \bbl@redefine\@citex[#1]#2{%
3859   \@safe@activetrue\edef\bbl@tempa{#2}\@safe@activfalse
3860   \org@@citex[#1]{\bbl@tempa}}
```

Unfortunately, the packages `natbib` and `cite` need a different definition of `\@citex`... To begin with, `natbib` has a definition for `\@citex` with *three* arguments... We only know that a package is loaded when `\begin{document}` is executed, so we need to postpone the different redefinition.

Notice that we use `\def` here instead of `\bbl@redefine` because `\org@@citex` is already defined and we don't want to overwrite that definition (it would result in parameter stack overflow because of a circular definition).

(Recent versions of `natbib` change dynamically `\@citex`, so PR4087 doesn't seem fixable in a simple way. Just load `natbib` before.)

```
3861 \AtBeginDocument{%
3862   \@ifpackageloaded{natbib}{%
3863     \def\@citex[#1][#2]#3{%
3864       \@safe@activetrue\edef\bbl@tempa{#3}\@safe@activfalse
3865       \org@@citex[#1][#2]{\bbl@tempa}}%
3866   }}
```

The package `cite` has a definition of `\@citex` where the shorthands need to be turned off in both arguments.

```
3867 \AtBeginDocument{%
3868   \@ifpackageloaded{cite}{%
3869     \def\@citex[#1]#2{%
3870       \@safe@activetrue\org@@citex[#1][#2]\@safe@activfalse}%
3871   }}
```

\nocite The macro \nocite which is used to instruct BiBTeX to extract uncited references from the database.

```
3872 \bbl@redefine\nocite#1{%
3873   \@safe@activetrue\org@nocite{#1}\@safe@activesfalse}
```

\bibcite The macro that is used in the aux file to define citation labels. When packages such as natbib or cite are not loaded its second argument is used to typeset the citation label. In that case, this second argument can contain active characters but is used in an environment where \@safe@activetrue is in effect. This switch needs to be reset inside the \hbox which contains the citation label. In order to determine during aux file processing which definition of \bibcite is needed we define \bibcite in such a way that it redefines itself with the proper definition. We call \bbl@cite@choice to select the proper definition for \bibcite. This new definition is then activated.

```
3874 \bbl@redefine\bibcite{%
3875   \bbl@cite@choice
3876   \bibcite}
```

\bbl@bibcite The macro \bbl@bibcite holds the definition of \bibcite needed when neither natbib nor cite is loaded.

```
3877 \def\bbl@bibcite#1#2{%
3878   \org@bibcite{#1}{\@safe@activesfalse#2}}
```

\bbl@cite@choice The macro \bbl@cite@choice determines which definition of \bibcite is needed. First we give \bibcite its default definition.

```
3879 \def\bbl@cite@choice{%
3880   \global\let\bibcite\bbl@bibcite
3881   \@ifpackageloaded{natbib}{\global\let\bibcite\org@bibcite}{}%
3882   \@ifpackageloaded{cite}{\global\let\bibcite\org@bibcite}{}%
3883   \global\let\bbl@cite@choice\relax}
```

When a document is run for the first time, no aux file is available, and \bibcite will not yet be properly defined. In this case, this has to happen before the document starts.

```
3884 \AtBeginDocument{\bbl@cite@choice}
```

\@bibitem One of the two internal L^AT_EX macros called by \bibitem that write the citation label on the aux file.

```
3885 \bbl@redefine\@bibitem#1{%
3886   \@safe@activetrue\org@@bibitem{#1}\@safe@activesfalse}
3887 \else
3888   \let\org@nocite\nocite
3889   \let\org@@citex\@citex
3890   \let\org@bibcite\bibcite
3891   \let\org@@bibitem\@bibitem
3892 \fi
```

5.2. Layout

```
3893 \newcommand\BabelPatchSection[1]{%
3894   \@ifundefined{#1}{}{%
3895     \bbl@exp{\let\<bbl@ss@#1>\<#1>}%
3896     \@namedef{#1}{%
3897       \@ifstar{\bbl@presec@s{#1}}%
3898       {\@dblarg{\bbl@presec@x{#1}}}}}%
3899
3900 \def\bbl@presec@x#1[#2]#3{%
3901   \bbl@exp{%
3902     \\select@language@x{\bbl@main@language}%
3903     \\bbl@cs{sspre@#1}%
3904     \\bbl@cs{ss@#1}%
3905     [\\foreignlanguage{\language@name}{\unexpanded{#2}}]%
3906     {\\foreignlanguage{\language@name}{\unexpanded{#3}}}%
3907   }
```

```

3907     \ifcase\bbl@bidimode\else\\babelsublr{}\fi}%
3908     \\select@language@x{\languagename}}
3909 \def\bbl@presec#1#2{%
3910   \bbl@exp{%
3911     \\select@language@x{\bbl@main@language}%
3912     \\bbl@cs{sspre@#1}%
3913     \\bbl@cs{ss@#1}*%
3914     {\\foreignlanguage{\languagename}{\unexpanded{#2}}}%
3915     \ifcase\bbl@bidimode\else\\babelsublr{}\fi}%
3916     \\select@language@x{\languagename}}
3917 %
3918 \IfBabelLayout{sectioning}%
3919   {\BabelPatchSection{part}%
3920    \BabelPatchSection{chapter}%
3921    \BabelPatchSection{section}%
3922    \BabelPatchSection{subsection}%
3923    \BabelPatchSection{subsubsection}%
3924    \BabelPatchSection{paragraph}%
3925    \BabelPatchSection{subparagraph}%
3926    \def\babel@toc#1{%
3927      \select@language@x{\bbl@main@language}}\}}
3928 \IfBabelLayout{captions}%
3929   {\BabelPatchSection{caption}}\}}

```

\BabelFootnote Footnotes.

```

3930 \bbl@trace{Footnotes}
3931 \def\bbl@footnote#1#2#3{%
3932   \@ifnextchar[%
3933     {\bbl@footnote@o{#1}{#2}{#3}}%
3934     {\bbl@footnote@x{#1}{#2}{#3}}}
3935 \long\def\bbl@footnote@x#1#2#3#4{%
3936   \bgroup
3937   \select@language@x{\bbl@main@language}%
3938   \bbl@fn@footnote{#2#1{\ignorespaces#4}#3}%
3939   \egroup}
3940 \long\def\bbl@footnote@o#1#2#3[#4]#5{%
3941   \bgroup
3942   \select@language@x{\bbl@main@language}%
3943   \bbl@fn@footnote[#4]{#2#1{\ignorespaces#5}#3}%
3944   \egroup}
3945 \def\bbl@footnotetext#1#2#3{%
3946   \@ifnextchar[%
3947     {\bbl@footnotetext@o{#1}{#2}{#3}}%
3948     {\bbl@footnotetext@x{#1}{#2}{#3}}}
3949 \long\def\bbl@footnotetext@x#1#2#3#4{%
3950   \bgroup
3951   \select@language@x{\bbl@main@language}%
3952   \bbl@fn@footnotetext{#2#1{\ignorespaces#4}#3}%
3953   \egroup}
3954 \long\def\bbl@footnotetext@o#1#2#3[#4]#5{%
3955   \bgroup
3956   \select@language@x{\bbl@main@language}%
3957   \bbl@fn@footnotetext[#4]{#2#1{\ignorespaces#5}#3}%
3958   \egroup}
3959 \def\BabelFootnote#1#2#3#4{%
3960   \ifx\bbl@fn@footnote\undefined
3961     \let\bbl@fn@footnote\footnote
3962   \fi
3963   \ifx\bbl@fn@footnotetext\undefined
3964     \let\bbl@fn@footnotetext\footnotetext
3965   \fi
3966   \bbl@ifblank{#2}%
3967   {\def#1{\bbl@footnote{\@firstofone}{#3}{#4}}}

```

```

3968 \namedef{\bbl@stripslash#1text}%
3969 {\bbl@footnotetext{\@firstofone}{#3}{#4}}}%
3970 {\def#1{\bbl@exp{\bbl@footnote{\bbl@foreignlanguage{#2}}}{#3}{#4}}}%
3971 \namedef{\bbl@stripslash#1text}%
3972 {\bbl@exp{\bbl@footnotetext{\bbl@foreignlanguage{#2}}}{#3}{#4}}}%
3973 \IfBabelLayout{footnotes}%
3974 {\let\bbl@0L@footnote\footnote
3975 \BabelFootnote\footnote\language\language}%
3976 \BabelFootnote\localfootnote\language\language}%
3977 \BabelFootnote\mainfootnote{}{}%
3978 {}

```

5.3. Marks

\markright Because the output routine is asynchronous, we must pass the current language attribute to the head lines. To achieve this we need to adapt the definition of `\markright` and `\markboth` somewhat. However, headlines and footlines can contain text outside marks; for that we must take some actions in the output routine if the 'headfoot' options is used.

We need to make some redefinitions to the output routine to avoid an endless loop and to correctly handle the page number in bidi documents.

```

3979 \bbl@trace{Marks}
3980 \IfBabelLayout{sectioning}
3981 {\ifx\bbl@opt@headfoot\@nnil
3982 \g@addto@macro\@resetactivechars{%
3983 \set@typeset@protect
3984 \expandafter\select@language@x\expandafter{\bbl@main@language}%
3985 \let\protect\noexpand
3986 \ifcase\bbl@bidimode\else % Only with bidi. See also above
3987 \edef\thepage{%
3988 \noexpand\babelsublr{\unexpanded\expandafter{\thepage}}}%
3989 \fi}%
3990 \fi}
3991 {\ifbbl@single\else
3992 \bbl@ifunset{markright } \bbl@redefine\bbl@redefineroobust
3993 \markright#1{%
3994 \bbl@ifblank{#1}%
3995 {\org@markright{}}%
3996 {\toks@{#1}%
3997 \bbl@exp{%
3998 \org@markright{\bbl@protect\bbl@foreignlanguage{\language}%
3999 \bbl@protect\bbl@restore@actives\the\toks@}}}%

```

\markboth

\@mkboth The definition of `\markboth` is equivalent to that of `\markright`, except that we need two token registers. The documentclasses report and book define and set the headings for the page. While doing so they also store a copy of `\markboth` in `\@mkboth`. Therefore we need to check whether `\@mkboth` has already been set. If so we need to do that again with the new definition of `\markboth`. (As of Oct 2019, $\LaTeX$ stores the definition in an intermediate macro, so it's not necessary anymore, but it's preserved for older versions.)

```

4000 \ifx\@mkboth\markboth
4001 \def\bbl@tempc{\let\@mkboth\markboth}%
4002 \else
4003 \def\bbl@tempc{%
4004 \fi
4005 \bbl@ifunset{markboth } \bbl@redefine\bbl@redefineroobust
4006 \markboth#1#2{%
4007 \protected@edef\bbl@tempb#1{%
4008 \protect\foreignlanguage
4009 {\language\language}{\protect\bbl@restore@actives#1}}%
4010 \bbl@ifblank{#1}%
4011 {\toks@{}}%
4012 {\toks@\expandafter{\bbl@tempb{#1}}}%

```

```

4013 \bbl@ifblank{#2}%
4014 {\@temptokena{}}%
4015 {\@temptokena\expandafter{\bbl@tempb{#2}}}%
4016 \bbl@exp{\@org@markboth{\the\toks@}{\the\@temptokena}}}%
4017 \bbl@tempc
4018 \fi} % end ifbbl@single, end \IfBabelLayout

```

5.4. Other packages

5.4.1. ifthen

\ifthenelse Sometimes a document writer wants to create a special effect depending on the page a certain fragment of text appears on. This can be achieved by the following piece of code:

```

% \ifthenelse{\isodd{\pageref{some-label}}}
% {code for odd pages}
% {code for even pages}
%

```

In order for this to work the argument of `\isodd` needs to be fully expandable. With the above redefinition of `\pageref` it is not in the case of this example. To overcome that, we add some code to the definition of `\ifthenelse` to make things work.

We want to revert the definition of `\pageref` and `\ref` to their original definition for the first argument of `\ifthenelse`, so we first need to store their current meanings.

Then we can set the `\@safe@actives` switch and call the original `\ifthenelse`. In order to be able to use shorthands in the second and third arguments of `\ifthenelse` the resetting of the switch *and* the definition of `\pageref` happens inside those arguments.

```

4019 \bbl@trace{Preventing clashes with other packages}
4020 \ifx\org@ref\undefined\else
4021 \bbl@xin@{R}\bbl@opt@safe
4022 \ifin@
4023 \AtBeginDocument{%
4024 \@ifpackageloaded{ifthen}{%
4025 \bbl@redefine@long\ifthenelse#1#2#3{%
4026 \let\bbl@temp@pref\pageref
4027 \let\pageref\org@pageref
4028 \let\bbl@temp@ref\ref
4029 \let\ref\org@ref
4030 \@safe@activestrue
4031 \org@ifthenelse{#1}%
4032 {\let\pageref\bbl@temp@pref
4033 \let\ref\bbl@temp@ref
4034 \@safe@activesfalse
4035 #2}%
4036 {\let\pageref\bbl@temp@pref
4037 \let\ref\bbl@temp@ref
4038 \@safe@activesfalse
4039 #3}%
4040 }%
4041 }}}%
4042 }
4043 \fi

```

5.4.2. varioref

\@vpageref

\vrefpagemum

\Ref When the package `varioref` is in use we need to modify its internal command `\@vpageref` in order to prevent problems when an active character ends up in the argument of `\vref`. The same needs to happen for `\vrefpagemum`.

```

4044 \AtBeginDocument{%
4045 \@ifpackageloaded{varioref}{%
4046 \bbl@redefine\@vpageref#1[#2]#3{%

```

```

4047      \@safe@activetrue
4048      \org@@vpageref{#1}{#2}{#3}%
4049      \@safe@activesfalse}%
4050      \bbl@redefine\vrefpagenum#1#2{%
4051      \@safe@activetrue
4052      \org@vrefpagenum{#1}{#2}%
4053      \@safe@activesfalse}%

```

The package `varioref` defines `\Ref` to be a robust command which uppercases the first character of the reference text. In order to be able to do that it needs to access the expandable form of `\ref`. So we employ a little trick here. We redefine the (internal) command `\Ref_` to call `\org@ref` instead of `\ref`. The disadvantage of this solution is that whenever the definition of `\Ref` changes, this definition needs to be updated as well.

```

4054      \expandafter\def\csname Ref \endcsname#1{%
4055      \protected@edef\@tempa{\org@ref{#1}}\expandafter\MakeUppercase\@tempa}
4056      }{}%
4057  }
4058 \fi

```

5.4.3. `hhline`

`\hhline` Delaying the activation of the shorthand characters has introduced a problem with the `hhline` package. The reason is that it uses the ‘:’ character which is made active by the french support in `babel`. Therefore we need to *reload* the package when the ‘:’ is an active character. Note that this happens *after* the category code of the @-sign has been changed to other, so we need to temporarily change it to letter again.

```

4059 \AtEndOfPackage{%
4060   \AtBeginDocument{%
4061     \@ifpackageloaded{hhline}%
4062     {\expandafter\ifx\csname normal@char\string\endcsname\relax
4063       \else
4064         \makeatletter
4065         \def\@currname{hhline}\input{hhline.sty}\makeatother
4066         \fi}%
4067     {}}}

```

`\substitutefontfamily` *Deprecated.* It creates an `fd` file on the fly. The first argument is an encoding mnemonic, the second and third arguments are font family names. Use the tools provided by $\TeX$ (`\DeclareFontFamilySubstitution`).

```

4068 \def\substitutefontfamily#1#2#3{%
4069   \lowercase{\immediate\openout15=#1#2.fd\relax}%
4070   \immediate\writel5{%
4071     \string\ProvidesFile{#1#2.fd}%
4072     [\the\year/\two@digits{\the\month}/\two@digits{\the\day}
4073     \space generated font description file]^J
4074     \string\DeclareFontFamily{#1}{#2}{}}^J
4075     \string\DeclareFontShape{#1}{#2}{m}{n}{<->ssub * #3/m/n}{}}^J
4076     \string\DeclareFontShape{#1}{#2}{m}{it}{<->ssub * #3/m/it}{}}^J
4077     \string\DeclareFontShape{#1}{#2}{m}{sl}{<->ssub * #3/m/sl}{}}^J
4078     \string\DeclareFontShape{#1}{#2}{m}{sc}{<->ssub * #3/m/sc}{}}^J
4079     \string\DeclareFontShape{#1}{#2}{b}{n}{<->ssub * #3/bx/n}{}}^J
4080     \string\DeclareFontShape{#1}{#2}{b}{it}{<->ssub * #3/bx/it}{}}^J
4081     \string\DeclareFontShape{#1}{#2}{b}{sl}{<->ssub * #3/bx/sl}{}}^J
4082     \string\DeclareFontShape{#1}{#2}{b}{sc}{<->ssub * #3/bx/sc}{}}^J
4083   }%
4084   \closeout15
4085 }
4086 \@onlypreamble\substitutefontfamily

```

5.5. Encoding and fonts

Because documents may use non-ASCII font encodings, we make sure that the logos of $\TeX$ and $\LaTeX$ always come out in the right encoding. There is a list of non-ASCII encodings. Requested encodings

are currently stored in \@fontenc@load@list. If a non-ASCII has been loaded, we define versions of \TeX and \LaTeX for them using \ensureascii. The default ASCII encoding is set, too (in reverse order): the “main” encoding (when the document begins), the last loaded, or OT1.

\ensureascii

```

4087 \bbl@trace{Encoding and fonts}
4088 \newcommand\BabelNonASCII{LGR,LGI,X2,OT2,OT3,OT6,LHE,LWN,LMA,LMC,LMS,LMU}
4089 \newcommand\BabelNonText{TS1,T3,TS3}
4090 \let\org@TeX\TeX
4091 \let\org@LaTeX\LaTeX
4092 \let\ensureascii@firstofone
4093 \let\asciientcoding\@empty
4094 \AtBeginDocument{%
4095   \def\@elt#1{,#1,}%
4096   \edef\bbl@tempa{\expandafter\@gobbletwo\@fontenc@load@list}%
4097   \let\@elt\relax
4098   \let\bbl@tempb\@empty
4099   \def\bbl@tempc{OT1}%
4100   \bbl@foreach\BabelNonASCII{% LGR loaded in a non-standard way
4101     \bbl@ifunset{T@#1}{\def\bbl@tempb{#1}}}%
4102   \bbl@foreach\bbl@tempa{%
4103     \bbl@xin@{,#1,}{,\BabelNonASCII,}%
4104     \ifin@
4105       \def\bbl@tempb{#1}% Store last non-ascii
4106     \else\bbl@xin@{,#1,}{,\BabelNonText,}% Pass
4107       \ifin@else
4108         \def\bbl@tempc{#1}% Store last ascii
4109       \fi
4110     \fi}%
4111   \ifx\bbl@tempb\@empty\else
4112     \bbl@xin@{,\cf@encoding,}{,\BabelNonASCII,\BabelNonText,}%
4113     \ifin@else
4114       \edef\bbl@tempc{\cf@encoding}% The default if ascii wins
4115     \fi
4116     \let\asciientcoding\bbl@tempc
4117     \renewcommand\ensureascii[1]{%
4118       {\fontencoding{\asciientcoding}\selectfont#1}}%
4119     \DeclareTextCommandDefault{\TeX}{\ensureascii{\org@TeX}}%
4120     \DeclareTextCommandDefault{\LaTeX}{\ensureascii{\org@LaTeX}}%
4121   \fi}

```

Now comes the old deprecated stuff (with a little change in 3.9l, for fontspec). The first thing we need to do is to determine, at \begin{document}, which latin fontencoding to use.

\latinencoding When text is being typeset in an encoding other than ‘latin’ (OT1 or T1), it would be nice to still have Roman numerals come out in the Latin encoding. So we first assume that the current encoding at the end of processing the package is the Latin encoding.

```

4122 \AtEndOfPackage{\edef\latinencoding{\cf@encoding}}

```

But this might be overruled with a later loading of the package fontenc. Therefore we check at the execution of \begin{document} whether it was loaded with the T1 option. The normal way to do this (using \ifpackageloaded) is disabled for this package. Now we have to revert to parsing the internal macro \@filelist which contains all the filenames loaded.

```

4123 \AtBeginDocument{%
4124   \ifpackageloaded{fontspec}%
4125     {\xdef\latinencoding{%
4126       \ifx\UTFencname\undefined
4127         EU\ifcase\bbl@engine\or2\or1\fi
4128       \else
4129         \UTFencname
4130       \fi}}%
4131     {\gdef\latinencoding{OT1}%
4132       \ifx\cf@encoding\bbl@t@one

```

```

4133     \xdef\latinencoding{\bbl@t@one}%
4134   \else
4135     \def\@elt#1{, #1,}%
4136     \edef\bbl@tempa{\expandafter\@gobbletwo\@fontenc@load@list}%
4137     \let\@elt\relax
4138     \bbl@xin@{, T1,}\bbl@tempa
4139     \ifin@
4140       \xdef\latinencoding{\bbl@t@one}%
4141     \fi
4142   \fi}}

```

\latintext Then we can define the command `\latintext` which is a declarative switch to a latin font-encoding. Usage of this macro is deprecated.

```

4143 \DeclareRobustCommand{\latintext}{%
4144   \fontencoding{\latinencoding}\selectfont
4145   \def\encodingdefault{\latinencoding}}

```

\textlatin This command takes an argument which is then typeset using the requested font encoding. In order to avoid many encoding switches it operates in a local scope.

```

4146 \ifx\@undefined\DeclareTextFontCommand
4147   \DeclareRobustCommand{\textlatin}[1]{\leavevmode{\latintext #1}}
4148 \else
4149   \DeclareTextFontCommand{\textlatin}{\latintext}
4150 \fi

```

For several functions, we need to execute some code with `\selectfont`. With $\TeX$ 2021-06-01, there is a hook for this purpose.

```

4151 \def\bbl@patchfont#1{\AddToHook{selectfont}{#1}}

```

5.6. Basic bidi support

This code is currently placed here for practical reasons. It will be moved to the correct place soon, I hope.

It is loosely based on `rlbabel.def`, but most of it has been developed from scratch. This babel module (by Johannes Braams and Boris Lavva) has served the purpose of typesetting R documents for two decades, and despite its flaws I think it is still a good starting point (some parts have been copied here almost verbatim), partly thanks to its simplicity. I’ve also looked at ARABI (by Youssef Jabri), which is compatible with babel.

There are two ways of modifying macros to make them “bidi”, namely, by patching the internal low-level macros (which is what I have done with lists, columns, counters, tocs, much like `rlbabel` did), and by introducing a “middle layer” just below the user interface (sectioning, footnotes).

- `pdftex` provides a minimal support for bidi text, and it must be done by hand. Vertical typesetting is not possible.
- `xetex` is somewhat better, thanks to its font engine (even if not always reliable) and a few additional tools. However, very little is done at the paragraph level. Another challenging problem is text direction does not honour $\TeX$ grouping.
- `luatex` can provide the most complete solution, as we can manipulate almost freely the node list, the generated lines, and so on, but bidi text does not work out of the box and some development is necessary. It also provides tools to properly set left-to-right and right-to-left page layouts. As `Lua $\TeX$ -ja` shows, vertical typesetting is possible, too.

```

4152 \bbl@trace{Loading basic (internal) bidi support}
4153 \ifodd\bbl@engine
4154 \else % Any xe+lua bidi
4155   \ifnum\bbl@bidimode>100 \ifnum\bbl@bidimode<200
4156     \bbl@error{bidi-only-lua}{\}\}%
4157     \let\bbl@beforeforeign\leavevmode
4158     \AtEndOfPackage{%
4159       \EnableBabelHook{babel-bidi}%
4160       \bbl@xebidipar}
4161   \fi\fi

```

```

4162 \def\bbl@loadxebidi#1{%
4163   \ifx\RTLfootnotetext\@undefined
4164     \AtEndOfPackage{%
4165       \EnableBabelHook{babel-bidi}%
4166       \ifx\fontspec\@undefined
4167         \usepackage{fontspec}% bidi needs fontspec
4168       \fi
4169       \usepackage#1{bidi}%
4170       \let\bbl@digitsdotdash\DigitsDotDashInterCharToks
4171       \def\DigitsDotDashInterCharToks{% See the 'bidi' package
4172         \ifnum\@nameuse{bbl@wdir\@languagename}=\tw@ % 'AL' bidi
4173           \bbl@digitsdotdash % So ignore in 'R' bidi
4174         \fi}%
4175     \fi}
4176 \ifnum\bbl@bidimode>200 % Any xe bidi=
4177   \ifcase\expandafter\@gobbletwo\the\bbl@bidimode\or
4178     \bbl@tentative{bidi=bidi}
4179     \bbl@loadxebidi{}
4180   \or
4181     \bbl@loadxebidi{[rldocument]}
4182   \or
4183     \bbl@loadxebidi{}
4184   \fi
4185 \fi
4186 \fi
4187 \ifnum\bbl@bidimode=\@ne % bidi=default
4188   \let\bbl@beforeforeign\leavevmode
4189   \ifodd\bbl@engine % lua
4190     \newattribute\bbl@attr@dir
4191     \directlua{ Babel.attr_dir = luatexbase.registernumber'bbl@attr@dir' }
4192     \bbl@exp{\output{\bodydir\pagedir\the\output}}
4193   \fi
4194   \AtEndOfPackage{%
4195     \EnableBabelHook{babel-bidi}% pdf/lua/xe
4196     \ifodd\bbl@engine\else % pdf/xe
4197       \bbl@xebidipar
4198     \fi}
4199 \fi

```

Now come the macros used to set the direction when a language is switched. Testing are based on script names, because it's the user interface (including language and script in \babelprovide. First the (mostly) common macros.

```

4200 \bbl@trace{Macros to switch the text direction}
4201 \def\bbl@alscripts{%
4202   ,Arabic,Syriac,Thaana,Hanifi Rohingya,Hanifi,Sogdian,}
4203 \def\bbl@rscripts{%
4204   Adlam,Avestan,Chorasmian,Cypriot,Elymaic,Garay,%
4205   Hatran,Hebrew,Imperial Aramaic,Inscriptional Pahlavi,%
4206   Inscriptional Parthian,Kharoshthi,Lydian,Mandaic,Manichaeen,%
4207   Mende Kikakui,Meroitic Cursive,Meroitic Hieroglyphs,Nabataean,%
4208   Nko,Old Hungarian,Old North Arabian,Old Sogdian,%
4209   Old South Arabian,Old Turkic,Old Uyghur,Palmyrene,Phoenician,%
4210   Psalter Pahlavi,Samaritan,Yezidi,Mandaean,%
4211   Meroitic,N'Ko,Orkhon,Todhri}
4212 %
4213 \def\bbl@provide@dirs#1{%
4214   \bbl@xin{\csname bbl@sname@#1\endcsname}{\bbl@alscripts\bbl@rscripts}%
4215   \ifin@
4216     \global\bbl@csarg\chardef{wdir@#1}\@ne
4217     \bbl@xin{\csname bbl@sname@#1\endcsname}{\bbl@alscripts}%
4218   \ifin@
4219     \global\bbl@csarg\chardef{wdir@#1}\tw@
4220   \fi

```

```

4221 \else
4222   \global\bbbl@csarg\chardef{wdir@#1}\z@
4223 \fi
4224 \ifodd\bbbl@engine
4225   \bbbl@csarg\ifcase{wdir@#1}%
4226   \directlua{ Babel.locale_props[\the\localeid].texdir = 'l' }%
4227   \or
4228   \directlua{ Babel.locale_props[\the\localeid].texdir = 'r' }%
4229   \or
4230   \directlua{ Babel.locale_props[\the\localeid].texdir = 'al' }%
4231 \fi
4232 \fi}
4233 %
4234 \def\bbbl@switchdir{%
4235   \bbbl@ifunset{bbbl@sys@\language name}{\bbbl@provide@sys@\language name}}{}%
4236   \bbbl@ifunset{bbbl@wdir@\language name}{\bbbl@provide@dirs@\language name}}{}%
4237   \bbbl@exp{\bbbl@setdirs\bbbl@cl{wdir}}%
4238 \def\bbbl@setdirs#1{%
4239   \ifcase\bbbl@select@type
4240     \bbbl@bodydir{#1}%
4241     \bbbl@pardir{#1}% <- Must precede \bbbl@texdir
4242   \fi
4243   \bbbl@texdir{#1}}
4244 \ifnum\bbbl@bidimode>\z@
4245   \AddBabelHook{babel-bidi}{afterextras}{\bbbl@switchdir}
4246   \DisableBabelHook{babel-bidi}
4247 \fi

```

Now the engine-dependent macros.

```

4248 \ifodd\bbbl@engine % luatex=1
4249 \else % pdftex=0, xetex=2
4250   \newcount\bbbl@dirlevel
4251   \chardef\bbbl@thetexdir\z@
4252   \chardef\bbbl@thepardir\z@
4253   \def\bbbl@texdir#1{%
4254     \ifcase#1\relax
4255       \chardef\bbbl@thetexdir\z@
4256       \@nameuse{setlatin}%
4257       \bbbl@texdir@i\beginL\endL
4258     \else
4259       \chardef\bbbl@thetexdir\@ne
4260       \@nameuse{setnonlatin}%
4261       \bbbl@texdir@i\beginR\endR
4262     \fi}
4263   \def\bbbl@texdir@i#1#2{%
4264     \ifhmode
4265       \ifnum\currentgrouplevel>\z@
4266         \ifnum\currentgrouplevel=\bbbl@dirlevel
4267           \bbbl@error{multiple-bidi}{}{}%
4268           \bgroup\aftergroup#2\aftergroup\egroup
4269         \else
4270           \ifcase\currentgrouptype\or % 0 bottom
4271             \aftergroup#2% 1 simple {}
4272           \or
4273             \bgroup\aftergroup#2\aftergroup\egroup % 2 hbox
4274           \or
4275             \bgroup\aftergroup#2\aftergroup\egroup % 3 adj hbox
4276           \or\or\or % vbox vtop align
4277           \or
4278             \bgroup\aftergroup#2\aftergroup\egroup % 7 noalign
4279           \or\or\or\or\or\or % output math disc insert vcent mathchoice
4280           \or
4281             \aftergroup#2% 14 \begingroup

```

```

4282         \else
4283         \bgroup\aftergroup#2\aftergroup\egroup % 15 adj
4284         \fi
4285         \fi
4286         \bbl@dirlevel\currentgrouplevel
4287         \fi
4288         #1%
4289         \fi}
4290 \def\bbl@pardir#1{\chardef\bbl@thepardir#1\relax}
4291 \let\bbl@bodydir\@gobble
4292 \def\bbl@dirparastext{\chardef\bbl@thepardir\bbl@thetextdir}

```

The following command is executed only if there is a right-to-left script (once). It activates the `\everypar` hack for xetex, to properly handle the par direction. Note text and par dirs are decoupled to some extent (although not completely).

```

4293 \def\bbl@xebidipar{%
4294   \let\bbl@xebidipar\relax
4295   \TeXeTstate\@ne
4296   \def\bbl@xeeverypar{%
4297     \ifcase\bbl@thepardir
4298     \ifcase\bbl@thetextdir\else\beginR\fi
4299     \else
4300       {\setbox\z@\lastbox\beginR\box\z@}%
4301       \fi}%
4302   \AddToHook{para/begin}{\bbl@xeeverypar}}
4303 \ifnum\bbl@bidimode>200 % Any xe bidi=
4304   \let\bbl@textdir\i\@gobbletwo
4305   \let\bbl@xebidipar\@empty
4306   \AddBabelHook{bidi}{foreign}{%
4307     \ifcase\bbl@thetextdir
4308     \BabelWrapText{\LR{##1}}%
4309     \else
4310     \BabelWrapText{\RL{##1}}%
4311     \fi}
4312   \def\bbl@pardir#1{\ifcase#1\relax\setLR\else\setRL\fi}
4313   \fi
4314 \fi

```

A tool for weak L (mainly digits). We also disable warnings with hyperref. There is a variant for `bidi=unibidi`, but using formatting characters is discouraged by Unicode UAX9, so it must be revised.

```

4315 \ifnum\bbl@bidimode>300
4316   \DeclareRobustCommand\babelsublr[1]{\leavevmode{^^^200f#1}}
4317 \else
4318   \DeclareRobustCommand\babelsublr[1]{\leavevmode{\bbl@textdir\z@#1}}
4319 \fi
4320 \AtBeginDocument{%
4321   \ifx\pdfstringdefDisableCommands\@undefined\else
4322     \ifx\pdfstringdefDisableCommands\relax\else
4323       \pdfstringdefDisableCommands{\let\babelsublr\@firstofone}%
4324     \fi
4325   \fi}

```

5.7. Local Language Configuration

\loadlocalcfg At some sites it may be necessary to add site-specific actions to a language definition file. This can be done by creating a file with the same name as the language definition file, but with the extension `.cfg`. For instance the file `norsk.cfg` will be loaded when the language definition file `norsk.ldf` is loaded.

For plain-based formats we don't want to override the definition of `\loadlocalcfg` from `plain.def`.

```

4326 \bbl@trace{Local Language Configuration}
4327 \ifx\loadlocalcfg\@undefined

```

```

4328 \ifpackagewith{babel}{noconfigs}%
4329 {\let\loadlocalcfg@gobble}%
4330 {\def\loadlocalcfg#1{%
4331 \InputIfFileExists{#1.cfg}%
4332 {\typeout{*****^J%
4333 * Local config file #1.cfg used^^J%
4334 *}}}%
4335 \@empty}}
4336 \fi

```

5.8. Language options

Languages are loaded when processing the corresponding option *except* if a main language has been set. In such a case, it is not loaded until all options has been processed. The following macro inputs the ldf file and does some additional checks (\input works, too, but possible errors are not caught).

```

4337 \bbl@trace{Language options}
4338 \def\BabelDefinitionFile#1#2#3{}
4339 \let\bbl@afterlang\relax
4340 \let\BabelModifiers\relax
4341 \let\bbl@loaded\@empty
4342 \def\bbl@load@language#1{%
4343 \InputIfFileExists{#1.ldf}%
4344 {\edef\bbl@loaded{\CurrentOption
4345 \ifx\bbl@loaded\@empty\else,\bbl@loaded\fi}%
4346 \expandafter\let\expandafter\bbl@afterlang
4347 \csname\CurrentOption.ldf-h@k\endcsname
4348 \expandafter\let\expandafter\BabelModifiers
4349 \csname bbl@mod@\CurrentOption\endcsname
4350 \bbl@exp{\AtBeginDocument{%
4351 \bbl@usehooks@lang{\CurrentOption}{\begin{document}}{\CurrentOption}}}%
4352 {\bbl@error{unknown-package-option}}}}

```

Another way to extend the list of ‘known’ options for babel was to create the file `bblopts.cfg` in which one can add option declarations. However, this mechanism is deprecated – if you want an alternative name for a language, just create a new ldf file loading the actual one. You can also set the name of the file with the package option `config=<name>`, which will load `<name>.cfg` instead.

If the language has been set as metadata, read the info from the corresponding ini file and extract the babel name. Then added it as a package option at the end, so that it becomes the main language. The behavior of a metatag with a global language option is not well defined, so if there is not a main option we set here explicitly.

Tagging PDF Span elements requires horizontal mode. With DocumentMetadata we also force it with `\foreignlanguage` (this is also done in bidi texts).

```

4353 \ifx\GetDocumentProperties\@undefined\else
4354 \let\bbl@beforeforeign\leavevmode
4355 \edef\bbl@tempa{\GetDocumentProperties{document/other-languages}}%
4356 \let\bbl@collect@other\@empty
4357 \bbl@foreach\bbl@tempa{%
4358 \edef\bbl@metalang{#1}%
4359 \ifx\bbl@metalang\@empty\else
4360 \begin{group}
4361 \expandafter
4362 \bbl@bcpllookup{\bbl@metalang}%
4363 \ifx\bbl@bcp\relax
4364 \ifx\bbl@opt@main\@nnil
4365 \bbl@error{no-locale-for-meta}{\bbl@metalang}}}%
4366 \fi
4367 \else
4368 \bbl@read@ini{\bbl@bcp}\m@ne
4369 \xdef\bbl@collect@other{\bbl@collect@other,\language}%
4370 \GenericInfo{(babel) \spaces\@spaces\@spaces
4371 Passing '\language' to babel@gobble}%
4372 \fi
4373 \end{group}

```

```

4374 \fi}
4375 \ifx\babel@collect@other\@empty\else
4376 \xdef\babel@language@opts{\babel@collect@other,\babel@language@opts}%
4377 \fi
4378 \edef\babel@metalang{\GetDocumentProperties{document/language}}%
4379 \ifx\babel@metalang\@empty\else
4380 \begingroup
4381 \expandafter
4382 \babel@bcpllookup{\babel@metalang}%
4383 \ifx\babel@bcp\relax
4384 \ifx\babel@opt@main\@nnil
4385 \babel@error{no-locale-for-meta}{\babel@metalang}{}}%
4386 \fi
4387 \else
4388 \babel@read@ini{\babel@bcp}\m@ne
4389 \xdef\babel@language@opts{\babel@language@opts,\language}%
4390 \ifx\babel@opt@main\@nnil
4391 \global\let\babel@opt@main\language
4392 \fi
4393 \GenericInfo{{(babel) \@spaces\@spaces\@spaces
4394 Passing '\language' to babel\@gobble}}%
4395 \fi
4396 \endgroup
4397 \fi
4398 \fi
4399 \ifx\babel@opt@config\@nnil
4400 \@ifpackagewith{babel}{noconfigs}}%
4401 {\InputIfFileExists{bblopts.cfg}%
4402 {\babel@info{Configuration files are deprecated, as\\%
4403 they can break document portability.\\%
4404 Reported}%
4405 \typeout{*****^J%
4406 * Local config file bblopts.cfg used^J%
4407 *}}%
4408 {}}%
4409 \else
4410 \InputIfFileExists{\babel@opt@config.cfg}%
4411 {\babel@info{Configuration files are deprecated, as\\%
4412 they can break document portability.\\%
4413 Reported}%
4414 \typeout{*****^J%
4415 * Local config file \babel@opt@config.cfg used^J%
4416 *}}%
4417 {\babel@error{config-not-found}{}}}%
4418 \fi

```

Recognizing global options in packages not having a closed set of them is not trivial, as for them to be processed they must be defined explicitly. So, package options not yet taken into account and stored in `\babel@language@opts` are assumed to be languages. If not declared above, the names of the option and the file are the same. We first pre-process the class and package options to determine the available locales, and which version (ldf or ini) will be loaded. This is done by first loading the corresponding `\babel-⟨name⟩.tex` file.

The second argument of `\BabelBeforeIni` may contain a `\BabelDefinitionFile` which defines `\babel@tempa` and `\babel@tempb` and saves the third argument for the moment of the actual loading. If there is no `\BabelDefinitionFile` the last element is usually empty, and the ini file is loaded. The values are used to build a list in the form ‘main-or-not’ / ‘ldf-or-ldfini-flag’ // ‘option-name’ // ‘bcp-tag’ / ‘ldf-name-or-none’. The ‘main-or-not’ element is 0 by default and set to 10 later if necessary (by prepending 1). The ‘bcp-tag’ is stored here so that the corresponding ini file can be loaded directly (with `@import`).

```

4419 \def\BabelBeforeIni#1#2{%
4420 \def\babel@tempa{\@m}% <- Default if no \BDefFile
4421 \let\babel@tempb\@empty
4422 #2%

```

```

4423 \edef\bbl@toload{%
4424   \ifx\bbl@toload\@empty\else\bbl@toload,\fi
4425   \bbl@toload@last}%
4426 \edef\bbl@toload@last{0/\bbl@tempa//\CurrentOption//\#1/\bbl@tempb}}
4427 \def\BabelDefinitionFile#1#2#3{%
4428   \def\bbl@tempa{#1}\def\bbl@tempb{#2}%
4429   \@namedef{bbl@preldf@CurrentOption}{#3}%
4430   \endinput}%

```

For efficiency, first preprocess the class options to remove those with =, which are becoming increasingly frequent (no language should contain this character). Here we use the more robust macro to traverse a clist from the $\TeX$ 3 layer.

```

4431 \def\bbl@tempf{,}
4432 \@nameuse{clist_map_inline:Nn}\@raw@classoptionslist{%
4433   \in@{=}{#1}%
4434   \ifin@else
4435     \edef\bbl@tempf{\bbl@tempf\zap@space#1 \@empty,}%
4436   \fi}

```

Store the class/package options in a list. If there is an explicit main, it's placed as the last option. Then loop it to read the tex files, which can have a \BabelDefinitionFile. If there is no tex file, we attempt loading the ldf for the option name; if it fails, an error is raised. Note the option name is surrounded by //...//. Class and package options are separated with @@, because errors and info are dealt with in different ways. Consecutive identical languages count as one.

```

4437 \let\bbl@toload\@empty
4438 \let\bbl@toload@last\@empty
4439 \let\bbl@unkopt\@gobble %% <- Ugly
4440 \edef\bbl@tempc{%
4441   \bbl@tempf,@@,\bbl@language@opts
4442   \ifx\bbl@opt@main\@nnil\else,\bbl@opt@main\fi}
4443 \let\BabelLocalesTentative\bbl@tempc
4444 %
4445 \bbl@foreach\bbl@tempc{%
4446   \in@{@@}{#1}% <- Ugly
4447   \ifin@
4448     \def\bbl@unkopt##1{%
4449       \DeclareOption{##1}{\bbl@error{unknown-package-option}{}}}%
4450   \else
4451     \def\CurrentOption{#1}%
4452     \bbl@xin@{/#1//}{\bbl@toload@last}% Collapse consecutive
4453     \ifin@else
4454       \lowercase{\InputIfFileExists{babel-#1.tex}}{%
4455         \IfFileExists{#1.ldf}%
4456         {\edef\bbl@toload{%
4457           \ifx\bbl@toload\@empty\else\bbl@toload,\fi
4458           \bbl@toload@last}%
4459           \edef\bbl@toload@last{0/0//\CurrentOption//und/#1}%
4460           {\bbl@unkopt{#1}}}%
4461         \fi
4462       \fi}

```

We have to determine (1) if no language has been loaded (in which case we fallback to 'nil', with a special tag), and (2) the main language. With an explicit 'main' language, remove repeated elements. The number 1 flags it as the main language (relevant in *ini* locales), because with 0 becomes 10.

```

4463 \ifx\bbl@opt@main\@nnil
4464   \ifx\bbl@toload@last\@empty
4465     \def\bbl@toload@last{0/0//nil//und-x-nil/nil}
4466     \bbl@info{%
4467       You haven't specified a language as a class or package\%
4468       option. I'll load 'nil'. Reported}
4469   \fi
4470 \else
4471   \let\bbl@tempc\@empty
4472   \bbl@foreach\bbl@toload{%

```

```

4473 \bbl@xin@{//\bbl@opt@main//}{#1}%
4474 \ifin@else
4475 \bbl@add@list\bbl@tempc{#1}%
4476 \fi}
4477 \let\bbl@toload\bbl@tempc
4478 \fi
4479 \edef\bbl@toload{\bbl@toload,1\bbl@toload@last}

```

Finally, load the ‘ini’ file or the pair ‘ini’/‘ldf’ file. Babel resorts to its own mechanism, not the default one based on \ProcessOptions (which is still present to make some internal clean-up). First, handle provide= and friends (with a recursive call if they are present), and then provide=* and friend. \count@ is used as flag: 0 if ‘ini’, 1 if ‘ldf’.

```

4480 \def\AfterBabelLanguage#1{%
4481 \bbl@ifsamestring\CurrentOption{#1}{\global\bbl@add\bbl@afterlang{}}
4482 \NewHook{babel/presets}
4483 \UseHook{babel/presets}
4484 %
4485 \let\bbl@tempb\@empty
4486 \def\bbl@tempc#1/#2//#3//#4/#5\@{%
4487 \count@z@
4488 \ifnum#2=\@m % if no \BabelDefinitionFile
4489 \ifnum#1=\z@ % not main. -- % if provide+=!, provide*=!
4490 \ifnum\bbl@ldfflag>\@ne\bbl@tempc 0/0//#3//#4/#3\@
4491 \else\bbl@tempd{#1}{#2}{#3}{#4}{#5}%
4492 \fi
4493 \else % 10 = main -- % if provide+=!, provide*=!
4494 \ifodd\bbl@ldfflag\bbl@tempc 10/0//#3//#4/#3\@
4495 \else\bbl@tempd{#1}{#2}{#3}{#4}{#5}%
4496 \fi
4497 \fi
4498 \else
4499 \ifnum#1=\z@ % not main
4500 \ifnum\bbl@iniflag>\@ne\else % if ø, provide
4501 \ifcase#2\count@\@ne\else\ifcase\bbl@engine\count@\@ne\fi\fi
4502 \fi
4503 \else % 10 = main
4504 \ifodd\bbl@iniflag\else % if provide+, provide*
4505 \ifcase#2\count@\@ne\else\ifcase\bbl@engine\count@\@ne\fi\fi
4506 \fi
4507 \fi
4508 \bbl@tempd{#1}{#2}{#3}{#4}{#5}%
4509 \fi}

```

Based on the value of \count@, do the actual loading. If ‘ldf’, we load the basic info from the ‘ini’ file before.

```

4510 \def\bbl@tempd#1#2#3#4#5{%
4511 \DeclareOption{#3}{%
4512 \ifcase\count@
4513 \bbl@exp{\bbl@add{\bbl@tempb{
4514 \global\let\bbl@afterload\@empty
4515 \\\@nameuse{bbl@preini@#3}%
4516 \\\bbl@ldfinit
4517 \def{\CurrentOption{#3}%
4518 \\\babelprovide[import=#4,\ifnum#1=\z@else\bbl@opt@provide,main\fi]{#3}%
4519 \\\bbl@afterldf
4520 \\\bbl@afterload}}}%
4521 \else
4522 \bbl@add\bbl@tempb{%
4523 \global\let\bbl@afterload\@empty
4524 \def\CurrentOption{#3}%
4525 \let\localename\CurrentOption
4526 \let\languagename\localename
4527 \def\BabelIniTag{#4}%
4528 \\\@nameuse{bbl@preldf@#3}%

```

```

4529 \begingroup
4530 \bbl@id@assign
4531 \bbl@read@ini{\BabelIniTag}0%
4532 \endgroup
4533 \bbl@load@language{#5}%
4534 \bbl@afterload}%
4535 \fi}
4536 %
4537 \bbl@foreach\bbl@toload{\bbl@tempc#1\@@}
4538 \bbl@tempb
4539 \DeclareOption*{}
4540 \ProcessOptions
4541 %
4542 \bbl@exp{%
4543 \\AtBeginDocument{\bbl@usehooks@lang/{\beginDocument}{\}}}%
4544 \def\AfterBabelLanguage{\bbl@error{late-after-babel}{\}}
4545 \AddToHook{beginDocument/end}{%
4546 \bbl@info{Main language set to '\mainlocalename'\@gobble}}
4547 (/package)

```

6. Tagging

Preliminary and experimental. Only `\foreignlanguage`, for the moment. Requires the package option `babel-metadata=tagging`. Also changes for `\DocumentMetadata`. Option `babel-metadata=on` makes these changes without activating the changes for tagging.

```

4548 (*tagging)
4549 \ifx\bbl@chfmt\@undefined\else
4550 \EditInstance{heading}{chapter}{%
4551   prefix = ,
4552   number-format = \unskip\bbl@chfmt
4553 }
4554 \fi
4555 \ifx\bbl@partformat\@undefined\else
4556 \EditInstance{heading}{part}{%
4557   prefix = ,
4558   number-format = \unskip\bbl@partformat
4559 }
4560 \fi
4561 \ifpackagewith{babel}{babel-metadata=tagging}{\endinput}
4562 \let\bbl@ifmetadata\IfDocumentMetadataT
4563 \ExplSyntaxOn
4564 \NewTaggingSocket{babel/foreign/begin}{0}
4565 \NewTaggingSocket{babel/foreign/end}{0}
4566 \NewTaggingSocketPlug{babel/foreign/begin}{default}{
4567   \tag_mc_end_push:
4568   \tagstructbegin{tag=Span, lang=\BCPdata{tag}}
4569   \tagmcbegin {} }
4570 \NewTaggingSocketPlug{babel/foreign/end}{default}{
4571   \tagmcend
4572   \tagstructend
4573   \tag_mc_begin_pop:n {}}
4574 \AssignTaggingSocketPlug{babel/foreign/begin}{default}
4575 \AssignTaggingSocketPlug{babel/foreign/end}{default}
4576 \ExplSyntaxOff
4577 (/tagging)

```

7. The kernel of Babel

The kernel of the babel system is currently stored in `babel.def`. The file `babel.def` contains most of the code. The file `hyphen.cfg` is a file that can be loaded into the format, which is necessary when you want to be able to switch hyphenation patterns.

Because plain $\TeX$ users might want to use some of the features of the babel system too, care has to be taken that plain $\TeX$ can process the files. For this reason the current format will have to be checked in a number of places. Some of the code below is common to plain $\TeX$ and $\LaTeX$, some of it is for the $\LaTeX$ case only.

Plain formats based on etex (etex, xetex, luatex) don't load hyphen.cfg but etex.src, which follows a different naming convention, so we need to define the babel names. It presumes language.def exists and it is the same file used when formats were created.

A proxy file for switch.def

```
4578 (*kernel)
4579 \let\bbl@onlyswitch\@empty
4580 \input babel.def
4581 \let\bbl@onlyswitch\@undefined
4582 (/kernel)
```

8. Error messages

They are loaded when \bbl@error is first called. To save space, the main code just identifies them with a tag, and messages are stored in a separate file. Since it can be loaded anywhere, you make sure some catcodes have the right value, although those for \, `, ^M, % and = are reset before loading the file.

```
4583 (*errors)
4584 \catcode\{=1 \catcode\}=2 \catcode\#=6
4585 \catcode\:=12 \catcode\.,=12 \catcode\.=12 \catcode\-=12
4586 \catcode\'=12 \catcode\'(=12 \catcode\')=12
4587 \catcode\@=11 \catcode\^=7
4588 %
4589 \ifx\MessageBreak\@undefined
4590 \gdef\bbl@error@i#1#2{%
4591 \begingroup
4592 \newlinechar=^^J
4593 \def\{^^J(babel) }%
4594 \errhelp{#2}\errmessage{\{#1}%
4595 \endgroup}
4596 \else
4597 \gdef\bbl@error@i#1#2{%
4598 \begingroup
4599 \def\{\MessageBreak}%
4600 \PackageError{babel}{#1}{#2}%
4601 \endgroup}
4602 \fi
4603 \def\bbl@errmessage#1#2#3{%
4604 \expandafter\gdef\csname bbl@err@#1\endcsname##1##2##3{%
4605 \bbl@error@i{#2}{#3}}
4606 % Implicit #2#3#4:
4607 \gdef\bbl@error#1{\csname bbl@err@#1\endcsname}
4608 %
4609 \bbl@errmessage{not-yet-available}
4610 {Not yet available}%
4611 {Find an armchair, sit down and wait}
4612 \bbl@errmessage{bad-package-option}%
4613 {Bad option '#1=#2'. Either you have misspelled the\\%
4614 key or there is a previous setting of '#1'. Valid\\%
4615 keys are, among others, 'shorthands', 'main', 'bidi',\\%
4616 'strings', 'config', 'headfoot', 'safe', 'math'.}%
4617 {See the manual for further details.}
4618 \bbl@errmessage{base-on-the-fly}
4619 {For a language to be defined on the fly 'base'\\%
4620 is not enough, and the whole package must be\\%
4621 loaded. Either delete the 'base' option or\\%
4622 request the languages explicitly}%
4623 {See the manual for further details.}
```

```

4624 \bbl@errmessage{undefined-language}
4625 {You haven't defined the language '#1' yet.\\%
4626 Perhaps you misspelled it or your installation\\%
4627 is not complete}%
4628 {Your command will be ignored, type <return> to proceed}
4629 \bbl@errmessage{invalid-ini-name}
4630 {'#1' not valid with the 'ini' mechanism.\\%
4631 I think you want '#2' instead. You may continue,\\%
4632 but you should fix the name. See the babel manual\\%
4633 for the available locales with 'provide'}%
4634 {See the manual for further details.}
4635 \bbl@errmessage{shorthand-is-off}
4636 {I can't declare a shorthand turned off (\string#2)}
4637 {Sorry, but you can't use shorthands which have been\\%
4638 turned off in the package options}
4639 \bbl@errmessage{not-a-shorthand}
4640 {The character '\string #1' should be made a shorthand character;\\%
4641 add the command \string\usesshorthands\string{#1\string} to
4642 the preamble.\\%
4643 I will ignore your instruction}%
4644 {You may proceed, but expect unexpected results}
4645 \bbl@errmessage{not-a-shorthand-b}
4646 {I can't switch '\string#2' on or off--not a shorthand\\%
4647 This character is not a shorthand. Maybe you made\\%
4648 a typing mistake?}%
4649 {I will ignore your instruction.}
4650 \bbl@errmessage{unknown-attribute}
4651 {The attribute #2 is unknown for language #1.}%
4652 {Your command will be ignored, type <return> to proceed}
4653 \bbl@errmessage{missing-group}
4654 {Missing group for string \string#1}%
4655 {You must assign strings to some category, typically\\%
4656 captions or extras, but you set none}
4657 \bbl@errmessage{only-lua-xe}
4658 {This macro is available only in LuaLaTeX and XeLaTeX.}%
4659 {Consider switching to these engines.}
4660 \bbl@errmessage{only-lua}
4661 {This macro is available only in LuaLaTeX}%
4662 {Consider switching to that engine.}
4663 \bbl@errmessage{unknown-provide-key}
4664 {Unknown key '#1' in \string\babelprovide}%
4665 {See the manual for valid keys}%
4666 \bbl@errmessage{unknown-mapfont}
4667 {Option '\bbl@KVP@mapfont' unknown for\\%
4668 mapfont. Use 'direction'}%
4669 {See the manual for details.}
4670 \bbl@errmessage{no-ini-file}
4671 {There is no ini file for the requested language\\%
4672 (#1: \language#1). Perhaps you misspelled it or your\\%
4673 installation is not complete}%
4674 {Fix the name or reinstall babel.}
4675 \bbl@errmessage{digits-is-reserved}
4676 {The counter name 'digits' is reserved for mapping\\%
4677 decimal digits}%
4678 {Use another name.}
4679 \bbl@errmessage{limit-two-digits}
4680 {Currently two-digit years are restricted to the\\5
4681 range 0-9999}%
4682 {There is little you can do. Sorry.}
4683 \bbl@errmessage{counter-too-large}
4684 {Counter too large (#1)}%
4685 {Currently this is the limit.}
4686 \bbl@errmessage{no-ini-info}

```

```

4687 {I've found no info for the current locale.\\%
4688 The corresponding ini file has not been loaded\\%
4689 Perhaps it doesn't exist}%
4690 {See the manual for details.}
4691 \bbl@errmessage{unknown-ini-field}
4692 {Unknown field '#1' in \string\BCPdata.\\%
4693 Perhaps you misspelled it}%
4694 {See the manual for details.}
4695 \bbl@errmessage{unknown-locale-key}
4696 {Unknown key for locale '#2':\\%
4697 #3\\%
4698 \string#1 will be set to \string\relax}%
4699 {Perhaps you misspelled it.}%
4700 \bbl@errmessage{adjust-only-vertical}
4701 {Currently, #1 related features can be adjusted only\\%
4702 in the main vertical list}%
4703 {Maybe things change in the future, but this is what it is.}
4704 \bbl@errmessage{layout-only-vertical}
4705 {Currently, layout related features can be adjusted only\\%
4706 in vertical mode}%
4707 {Maybe things change in the future, but this is what it is.}
4708 \bbl@errmessage{bidi-only-lua}
4709 {The bidi method 'basic' is available only in\\%
4710 luatex. I'll continue with 'bidi=default', so\\%
4711 expect wrong results.\\%
4712 Suggested actions:\\%
4713 * If possible, switch to luatex, as xetex is not\\%
4714 recommend anymore.\\
4715 * If you can't, try 'bidi=bidi' with xetex.\\%
4716 * With pdftex, only 'bidi=default' is available.}%
4717 {See the manual for further details.}
4718 \bbl@errmessage{multiple-bidi}
4719 {Multiple bidi settings inside a group\\%
4720 I'll insert a new group, but expect wrong results.\\%
4721 Suggested action:\\%
4722 * Add a new group where appropriate.}
4723 {See the manual for further details.}
4724 \bbl@errmessage{unknown-package-option}
4725 {Unknown option '\CurrentOption'.\\%
4726 Suggested actions:\\%
4727 * Make sure you haven't misspelled it\\%
4728 * Check in the babel manual that it's supported\\%
4729 * If supported and it's a language, you may\\%
4730 \space\space need in some distributions a separate\\%
4731 \space\space installation\\%
4732 * If installed, check there isn't an old\\%
4733 \space\space version of the required files in your system\\%
4734 * If it's an unsupported language, create it with\\%
4735 \space\space \string\babelprovide\space (see the manual)}
4736 {Valid options are, among others: shorthands=, KeepShorthandsActive,\\%
4737 activeacute, activegrave, noconfigs, safe=, main=, math=\\%
4738 headfoot=, strings=, config=, hyphenmap=, or a language name.}
4739 \bbl@errmessage{config-not-found}
4740 {Local config file '\bbl@opt@config.cfg' not found.\\%
4741 Suggested actions:\\%
4742 * Make sure you haven't misspelled it in config=\\%
4743 * Check it exists and it's in the correct path}%
4744 {Perhaps you misspelled it.}
4745 \bbl@errmessage{late-after-babel}
4746 {Too late for \string\AfterBabelLanguage}%
4747 {Languages have been loaded, so I can do nothing}
4748 \bbl@errmessage{double-hyphens-class}
4749 {Double hyphens aren't allowed in \string\babelcharclass\\%

```

```

4750     because it's potentially ambiguous}%
4751     {See the manual for further info}
4752 \bbl@errmessage{unknown-interchar}
4753     {'#1' for '\language' cannot be enabled.\\%
4754     Maybe there is a typo}%
4755     {See the manual for further details.}
4756 \bbl@errmessage{unknown-interchar-b}
4757     {'#1' for '\language' cannot be disabled.\\%
4758     Maybe there is a typo}%
4759     {See the manual for further details.}
4760 \bbl@errmessage{charproperty-only-vertical}
4761     {\string\babelcharproperty\space can be used only in\\%
4762     vertical mode (preamble or between paragraphs)}%
4763     {See the manual for further info}
4764 \bbl@errmessage{unknown-char-property}
4765     {No property named '#2'. Allowed values are\\%
4766     direction (bc), mirror (bmg), and linebreak (lb)}%
4767     {See the manual for further info}
4768 \bbl@errmessage{bad-transform-option}
4769     {Bad option '#1' in a transform.\\%
4770     I'll ignore it but expect more errors}%
4771     {See the manual for further info.}
4772 \bbl@errmessage{font-conflict-transforms}
4773     {Transforms cannot be re-assigned to different\\%
4774     fonts. The conflict is in '\bbl@kv@label'.\\%
4775     Apply the same fonts or use a different label}%
4776     {See the manual for further details.}
4777 \bbl@errmessage{transform-not-available}
4778     {'#1' for '\language' cannot be enabled.\\%
4779     Maybe there is a typo or it's a font-dependent transform}%
4780     {See the manual for further details.}
4781 \bbl@errmessage{transform-not-available-b}
4782     {'#1' for '\language' cannot be disabled.\\%
4783     Maybe there is a typo or it's a font-dependent transform}%
4784     {See the manual for further details.}
4785 \bbl@errmessage{year-out-range}
4786     {Year out of range.\\%
4787     The allowed range is #1}%
4788     {See the manual for further details.}
4789 \bbl@errmessage{only-pdftex-lang}
4790     {The '#1' ldf style doesn't work with #2,\\%
4791     but you can use the ini locale instead.\\%
4792     Try adding 'provide=*' to the option list. You may\\%
4793     also want to set 'bidi=' to some value}%
4794     {See the manual for further details.}
4795 \bbl@errmessage{hyphenmins-args}
4796     {\string\babelhyphenmins\ accepts either the optional\\%
4797     argument or the star, but not both at the same time}%
4798     {See the manual for further details.}
4799 \bbl@errmessage{no-locale-for-meta}
4800     {There isn't currently a locale for the 'lang' requested\\%
4801     in the PDF metadata ('#1'). To fix it, you can\\%
4802     set explicitly a similar language (using the same\\%
4803     script) with the key main= when loading babel. If you\\%
4804     continue, I'll fallback to the 'nil' language, with\\%
4805     tag 'und' and script 'Latn', but expect a bad font\\%
4806     rendering with other scripts. You may also need set\\%
4807     explicitly captions and date, too}%
4808     {See the manual for further details.}
4809 (/errors)
4810 (*patterns)

```

9. Loading hyphenation patterns

The following code is meant to be read by $\text{\texttt{iniT\TeX}}$ because it should instruct $\text{\texttt{T\TeX}}$ to read hyphenation patterns. To this end the `docstrip` option `patterns` is used to include this code in the file `hyphen.cfg`. Code is written with lower level macros.

```
4811 <@Make sure ProvidesFile is defined>
4812 \ProvidesFile{hyphen.cfg}[<@date@> v<@version@> Babel hyphens]
4813 \xdef\bbl@format{\jobname}
4814 \def\bbl@version{<@version@>}
4815 \def\bbl@date{<@date@>}
4816 \ifx\AtBeginDocument\undefined
4817   \def\@empty{}
4818 \fi
4819 <@Define core switching macros@>
```

\process@line Each line in the file `language.dat` is processed by `\process@line` after it is read. The first thing this macro does is to check whether the line starts with `=`. When the first token of a line is an `=`, the macro `\process@synonym` is called; otherwise the macro `\process@language` will continue.

```
4820 \def\process@line#1#2 #3 #4 {%
4821   \ifx=#1%
4822     \process@synonym{#2}%
4823   \else
4824     \process@language{#1#2}{#3}{#4}%
4825   \fi
4826   \ignorespaces}
```

\process@synonym This macro takes care of the lines which start with an `=`. It needs an empty token register to begin with. `\bbl@languages` is also set to empty.

```
4827 \toks@{}
4828 \def\bbl@languages{}
```

When no languages have been loaded yet, the name following the `=` will be a synonym for hyphenation register 0. So, it is stored in a token register and executed when the first pattern file has been processed. (The `\relax` just helps to the `\if` below catching synonyms without a language.)

Otherwise the name will be a synonym for the language loaded last.

We also need to copy the `hyphenmin` parameters for the synonym.

```
4829 \def\process@synonym#1{%
4830   \ifnum\last@language=\m@ne
4831     \toks@\expandafter{\the\toks@\relax\process@synonym{#1}}%
4832   \else
4833     \expandafter\chardef\csname l@#1\endcsname\last@language
4834     \wlog{\string\l@#1=\string\language\the\last@language}%
4835     \expandafter\let\csname #1hyphenmins\expandafter\endcsname
4836       \csname\language\hyphenmins\endcsname
4837     \let\bbl@elt\relax
4838     \edef\bbl@languages{\bbl@languages\bbl@elt{#1}{\the\last@language}}}%
4839   \fi}
```

\process@language The macro `\process@language` is used to process a non-empty line from the ‘configuration file’. It has three arguments, each delimited by white space. The first argument is the ‘name’ of a language; the second is the name of the file that contains the patterns. The optional third argument is the name of a file containing hyphenation exceptions.

The first thing to do is call `\addlanguage` to allocate a pattern register and to make that register ‘active’. Then the pattern file is read.

For some hyphenation patterns it is needed to load them with a specific font encoding selected. This can be specified in the file `language.dat` by adding for instance ‘:T1’ to the name of the language. The macro `\bbl@get@enc` extracts the font encoding from the language name and stores it in `\bbl@hyph@enc`. The latter can be used in hyphenation files if you need to set a behavior depending on the given encoding (it is set to empty if no encoding is given).

Pattern files may contain assignments to `\lefthyphenmin` and `\righthyphenmin`. $\text{\texttt{T\TeX}}$ does not keep track of these assignments. Therefore we try to detect such assignments and store them in the `\<language>hyphenmins` macro. When no assignments were made we provide a default setting.

Some pattern files contain changes to the `\lccode` en `\uccode` arrays. Such changes should remain local to the language; therefore we process the pattern file in a group; the `\patterns` command acts globally so its effect will be remembered.

Then we globally store the settings of `\lefthyphenmin` and `\righthyphenmin` and close the group.

When the hyphenation patterns have been processed we need to see if a file with hyphenation exceptions needs to be read. This is the case when the third argument is not empty and when it does not contain a space token. (Note however there is no need to save hyphenation exceptions into the format.)

`\bbl@languages` saves a snapshot of the loaded languages in the form `\bbl@elt{<language-name>}{<number>}{<patterns-file>}{<exceptions-file>}`. Note the last 2 arguments are empty in ‘dialects’ defined in `language.dat` with `=`. Note also the language name can have encoding info.

Finally, if the counter `\language` is equal to zero we execute the synonyms stored.

```

4840 \def\process@language#1#2#3{%
4841   \expandafter\addlanguage\csname l@#1\endcsname
4842   \expandafter\language\csname l@#1\endcsname
4843   \edef\language#1{%
4844     \bbl@hook@everylanguage{#1}%
4845     % > luatex
4846     \bbl@get@enc#1::\@@@
4847   \begingroup
4848     \lefthyphenmin\m@ne
4849     \bbl@hook@loadpatterns{#2}%
4850     % > luatex
4851     \ifnum\lefthyphenmin=\m@ne
4852     \else
4853       \expandafter\xdef\csname #1hyphenmins\endcsname{%
4854         \the\lefthyphenmin\the\righthyphenmin}%
4855     \fi
4856   \endgroup
4857   \def\bbl@tempa{#3}%
4858   \ifx\bbl@tempa\@empty\else
4859     \bbl@hook@loadexceptions{#3}%
4860     % > luatex
4861   \fi
4862   \let\bbl@elt\relax
4863   \edef\bbl@languages{%
4864     \bbl@languages\bbl@elt{#1}{\the\language}{#2}{\bbl@tempa}}%
4865   \ifnum\the\language=\z@
4866     \expandafter\ifx\csname #1hyphenmins\endcsname\relax
4867       \set@hyphenmins\tw@\thr@@\relax
4868     \else
4869       \expandafter\expandafter\expandafter\set@hyphenmins
4870       \csname #1hyphenmins\endcsname
4871     \fi
4872     \the\toks@
4873     \toks@{}%
4874   \fi}

```

`\bbl@get@enc`

`\bbl@hyph@enc` The macro `\bbl@get@enc` extracts the font encoding from the language name and stores it in `\bbl@hyph@enc`. It uses delimited arguments to achieve this.

```

4875 \def\bbl@get@enc#1:#2:#3\@@@{\def\bbl@hyph@enc{#2}}

```

Now, hooks are defined. For efficiency reasons, they are dealt here in a special way. Besides `luatex`, format-specific configuration files are taken into account. `loadkernel` currently loads nothing, but define some basic macros instead.

```

4876 \def\bbl@hook@everylanguage#1{}
4877 \def\bbl@hook@loadpatterns#1{\input #1\relax}
4878 \let\bbl@hook@loadexceptions\bbl@hook@loadpatterns
4879 \def\bbl@hook@loadkernel#1{%
4880   \def\addlanguage{\csname newlanguage\endcsname}%

```

```

4881 \def\adddialect##1##2{%
4882   \global\chardef##1##2\relax
4883   \wlog{\string##1 = a dialect from \string\language##2}}%
4884 \def\iflanguage##1{%
4885   \expandafter\ifx\csname l@##1\endcsname\relax
4886     \@nolanner{##1}%
4887   \else
4888     \ifnum\csname l@##1\endcsname=\language
4889       \expandafter\expandafter\expandafter\@firstoftwo
4890     \else
4891       \expandafter\expandafter\expandafter\@secondoftwo
4892     \fi
4893   \fi}%
4894 \def\providehyphenmins##1##2{%
4895   \expandafter\ifx\csname ##1hyphenmins\endcsname\relax
4896     \@namedef{##1hyphenmins}{##2}%
4897   \fi}%
4898 \def\set@hyphenmins##1##2{%
4899   \lefthyphenmin##1\relax
4900   \righthyphenmin##2\relax}%
4901 \def\selectlanguage{%
4902   \errhelp{Selecting a language requires a package supporting it}%
4903   \errmessage{No multilingual package has been loaded}}%
4904 \let\foreignlanguage\selectlanguage
4905 \let\otherlanguage\selectlanguage
4906 \expandafter\let\csname otherlanguage*\endcsname\selectlanguage
4907 \def\bbl@usehooks##1##2{%
4908 \def\setlocale{%
4909   \errhelp{Find an armchair, sit down and wait}%
4910   \errmessage{(babel) Not yet available}}%
4911 \let\uselocale\setlocale
4912 \let\locale\setlocale
4913 \let\selectlocale\setlocale
4914 \let\localename\setlocale
4915 \let\textlocale\setlocale
4916 \let\textlanguage\setlocale
4917 \let\languagetext\setlocale}
4918 \begingroup
4919 \def\AddBabelHook#1#2{%
4920   \expandafter\ifx\csname bbl@hook@#2\endcsname\relax
4921     \def\next{\toks1}%
4922   \else
4923     \def\next{\expandafter\gdef\csname bbl@hook@#2\endcsname####1}%
4924   \fi
4925   \next}
4926 \ifx\directlua\@undefined
4927   \ifx\XeTeXinputencoding\@undefined\else
4928     \input xebabel.def
4929   \fi
4930 \else
4931   \input luababel.def
4932 \fi
4933 \openin1 = babel-\bbl@format.cfg
4934 \ifeof1
4935 \else
4936   \input babel-\bbl@format.cfg\relax
4937 \fi
4938 \closein1
4939 \endgroup
4940 \bbl@hook@loadkernel{switch.def}

```

\readconfigfile The configuration file can now be opened for reading.

```

4941 \openin1 = language.dat

```

See if the file exists, if not, use the default hyphenation file `hyphen.tex`. The user will be informed about this.

```
4942 \def\language\language{english}%
4943 \ifeof1
4944   \message{I couldn't find the file language.dat,\space
4945           I will try the file hyphen.tex}
4946   \input hyphen.tex\relax
4947   \chardef\l@english\z@
4948 \else
```

Pattern registers are allocated using count register `\last@language`. Its initial value is 0. The definition of the macro `\newlanguage` is such that it first increments the count register and then defines the language. In order to have the first patterns loaded in pattern register number 0 we initialize `\last@language` with the value `-1`.

```
4949 \last@language\m@ne
```

We now read lines from the file until the end is found. While reading from the input, it is useful to switch off recognition of the end-of-line character. This saves us stripping off spaces from the contents of the control sequence.

```
4950 \loop
4951   \endlinechar\m@ne
4952   \read1 to \bbl@line
4953   \endlinechar\^^M
```

If the file has reached its end, exit from the loop here. If not, empty lines are skipped. Add 3 space characters to the end of `\bbl@line`. This is needed to be able to recognize the arguments of `\process@line` later on. The default language should be the very first one.

```
4954   \if T\ifeof1F\fi T\relax
4955   \ifx\bbl@line\empty\else
4956     \edef\bbl@line{\bbl@line\space\space\space}%
4957     \expandafter\process@line\bbl@line\relax
4958   \fi
4959 \repeat
```

Check for the end of the file. We must reverse the test for `\ifeof` without `\else`. Then reactivate the default patterns, and close the configuration file.

```
4960 \begingroup
4961   \def\bbl@elt#1#2#3#4{%
4962     \global\language=#2\relax
4963     \gdef\language\language{#1}%
4964     \def\bbl@elt##1##2##3##4{}}%
4965   \bbl@languages
4966 \endgroup
4967 \fi
4968 \closein1
```

We add a message about the fact that `babel` is loaded in the format and with which language patterns to the `\everyjob` register.

```
4969 \if\the\toks@\else
4970   \errhelp{language.dat loads no language, only synonyms}
4971   \errmessage{Orphan language synonym}
4972 \fi
```

Also remove some macros from memory and raise an error if `\toks@` is not empty. Finally load `switch.def`, but the latter is not required and the line inputting it may be commented out.

```
4973 \let\bbl@line\undefined
4974 \let\process@line\undefined
4975 \let\process@synonym\undefined
4976 \let\process@language\undefined
4977 \let\bbl@get@enc\undefined
4978 \let\bbl@hyph@enc\undefined
4979 \let\bbl@tempa\undefined
4980 \let\bbl@hook@loadkernel\undefined
4981 \let\bbl@hook@everylanguage\undefined
```

```

4982 \let\bbl@hook@loadpatterns\@undefined
4983 \let\bbl@hook@loadexceptions\@undefined
4984 (/patterns)

```

Here the code for `iniTeX` ends.

10. luatex + xetex: common stuff

Add the bidi handler just before `luaotfload`, which is loaded by default by LaTeX. Just in case, consider the possibility it has not been loaded. First, a couple of definitions related to bidi (although default also applies to `pdfTeX`).

```

4985 (<More package options>) ≡
4986 \chardef\bbl@bidimode\z@
4987 \DeclareOption{bidi=default}{\chardef\bbl@bidimode=\@ne}
4988 \DeclareOption{bidi=basic}{\chardef\bbl@bidimode=101 }
4989 \DeclareOption{bidi=basic-r}{\chardef\bbl@bidimode=102 }
4990 \DeclareOption{bidi=bidi}{\chardef\bbl@bidimode=201 }
4991 \DeclareOption{bidi=bidi-r}{\chardef\bbl@bidimode=202 }
4992 \DeclareOption{bidi=bidi-l}{\chardef\bbl@bidimode=203 }
4993 \DeclareOption{bidi=unibidi}{\chardef\bbl@bidimode=301 }
4994 (/More package options)

```

\babelfont With explicit languages, we could define the font at once, but we don't. Just wait and see if the language is actually activated. `bbl@font` replaces hardcoded font names inside `\. . family` by the corresponding macro `\. . default`.

```

4995 (<Font selection>) ≡
4996 \bbl@trace{Font handling with fontspec}
4997 \AddBabelHook{babel-fontspec}{afterextras}{\bbl@switchfont}
4998 \AddBabelHook{babel-fontspec}{beforestart}{\bbl@ckeckstdfonts}
4999 \DisableBabelHook{babel-fontspec}
5000 \onlypreamble\babelfont
5001 \ifx\NewDocumentCommand\@undefined\else % Not plain
5002   \NewDocumentCommand\babelfont{0}{m0}{m0}{m0}{m0}{%
5003     \bbl@bblfont@o[#1]{#2}{#3,#5}{#4}}
5004 \fi
5005 \newcommand\bbl@bblfont@o[2][l]{% 1=langs/scripts 2=fam
5006   \ifx\fontspec\@undefined
5007     \usepackage{fontspec}%
5008   \fi
5009   \EnableBabelHook{babel-fontspec}%
5010   \edef\bbl@tempa{#1}%
5011   \def\bbl@tempb{#2}% Used by \bbl@bblfont
5012   \bbl@bblfont}
5013 \newcommand\bbl@bblfont[2][l]{% 1=features 2=fontname, @font=rm|sf|tt
5014   \bbl@ifunset{\bbl@tempb family}%
5015     {\bbl@providefam{\bbl@tempb}}%
5016     {}%
5017   % For the default font, just in case:
5018   \bbl@ifunset{\bbl@lsys\language}\bbl@provide@lsys{\language}}{%
5019     \expandafter\bbl@ifblank\expandafter{\bbl@tempa}%
5020     {\bbl@csarg\edef{\bbl@tempb dflt@}{<#1>{#2}}}% save bbl@rmdflt@
5021     \bbl@exp{%
5022       \let<\bbl@tempb dflt@\language><\bbl@tempb dflt@>%
5023       \bbl@font@set<\bbl@tempb dflt@\language>%
5024       \<\bbl@tempb default><\bbl@tempb family>}}%
5025     {\bbl@foreach\bbl@tempa{% i.e., bbl@rmdflt@lang / *scrt
5026       \bbl@csarg\def{\bbl@tempb dflt@##1}{<#1>{#2}}}}%

```

If the family in the previous command does not exist, it must be defined. Here is how:

```

5027 \def\bbl@providefam#1{%
5028   \bbl@exp{%
5029     \newcommand<#1default>{}% Just define it

```

```

5030 \\bbl@add@list\\bbl@font@fams{#1}%
5031 \\NewHook{#1family}%
5032 \\DeclareRobustCommand\<#1family>{%
5033   \\not@math@alphabet\<#1family>\relax
5034   % \\prepare@family@series@update{#1}\<#1default>% TODO. Fails
5035   \\fontfamily\<#1default>%
5036   \\UseHook{#1family}%
5037   \\selectfont}%
5038 \\DeclareTextFontCommand{\<text#1>}\<#1family>}}

```

The following macro is activated when the hook babel-fontspec is enabled. But before, we define a macro for a warning, which sets a flag to avoid duplicate them.

```

5039 \def\bbl@nostdfont#1{%
5040   \bbl@once{nostdfam-\f@family}%
5041   {\bbl@infowarn{The current font is not a babel standard family:\%
5042     #1%
5043     \fontname\font\\%
5044     There is nothing intrinsically wrong, and you can\\%,
5045     ignore this message altogether if you do not need\\%
5046     this font. If they are used in the document, be aware\\%
5047     'babel' will not set Script and Language for it, so\\%
5048     you may consider defining a new family with \string\babelfont.\\%
5049     See the manual for further details about \string\babelfont.
5050     Reported}}}%
5051   {}}%
5052 \gdef\bbl@switchfont{%
5053   \bbl@ifunset{bbl@lsys{\language}\bbl@provide@lsys{\language}}{}}%
5054   \bbl@exp{% e.g., Arabic -> arabic
5055     \lowercase{\edef\\bbl@tempa{\bbl@cl{sname}}}}%
5056   \bbl@foreach\bbl@font@fams{%
5057     \bbl@ifunset{bbl@##1dflt@\language}% (1) language?
5058     {\bbl@ifunset{bbl@##1dflt*\bbl@tempa}% (2) from script?
5059       {\bbl@ifunset{bbl@##1dflt@}% 2=F - (3) from generic?
5060         {}% 123=F - nothing!
5061         {\bbl@exp{% 3=T - from generic
5062           \global\let\<bbl@##1dflt@\language>%
5063           \<bbl@##1dflt@>}}}%
5064         {\bbl@exp{% 2=T - from script
5065           \global\let\<bbl@##1dflt@\language>%
5066           \<bbl@##1dflt*\bbl@tempa>}}}%
5067       {}}% 1=T - language, already defined
5068   \def\bbl@tempa{\bbl@nostdfont{}}}%
5069   \bbl@foreach\bbl@font@fams{% don't gather with prev for
5070     \bbl@ifunset{bbl@##1dflt@\language}%
5071     {\bbl@cs{famrst@##1}%
5072     \global\bbl@csarg\let{famrst@##1}\relax}%
5073     {\bbl@exp{% order is relevant.
5074       \\bbl@add\\originalTeX{%
5075         \\bbl@font@rst{\bbl@cl{##1dflt}}%
5076         \<##1default>\<##1family>{##1}}%
5077       \\bbl@font@set\<bbl@##1dflt@\language>% the main part!
5078       \<##1default>\<##1family>}}}%
5079   \bbl@ifrestoring{ }\bbl@tempa}}%

```

The following is executed at the beginning of the aux file or the document to warn about fonts not defined with \babelfont.

```

5080 \ifx\f@family\undefined\else % if latex
5081   \ifcase\bbl@engine % if pdftex
5082     \let\bbl@ckeckstdfonts\relax
5083   \else
5084     \def\bbl@ckeckstdfonts{%
5085       \begingroup
5086       \global\let\bbl@ckeckstdfonts\relax
5087       \let\bbl@tempa\empty

```

```

5088 \bbl@foreach\bbl@font@fams{%
5089 \bbl@ifunset{\bbl@##1dflt@}%
5090 {\@nameuse{##1family}}%
5091 \bbl@csarg\gdef{WFF@{\f@family}}{% Flag
5092 \bbl@exp{\bbl@add{\bbl@tempa{* \<##1family>= \f@family\\}%
5093 \space\space\fontname\font\\}%
5094 \bbl@csarg\xdef{##1dflt@}{\f@family}%
5095 \expandafter\xdef\csname ##1default\endcsname{\f@family}}%
5096 {}}%
5097 \ifx\bbl@tempa\@empty\else
5098 \bbl@infowarn{The following font families will use the default\\%
5099 settings for all or some languages:\\%
5100 \bbl@tempa
5101 There is nothing intrinsically wrong with it, but\\%
5102 'babel' will no set Script and Language, which could\\%
5103 be relevant in some languages. If your document uses\\%
5104 these families, consider redefining them with \string\babelfont.\\%
5105 Reported}%
5106 \fi
5107 \endgroup}
5108 \fi
5109 \fi

```

Now the macros defining the font with fontspec.

When there are repeated keys in fontspec, the last value wins. So, we just place the ini settings at the beginning, and user settings will take precedence. We must deactivate temporarily `\bbl@mapselect` because `\selectfont` is called internally when a font is defined.

For historical reasons, $\TeX$ can select two different series (bx and b), for what is conceptually a single one. This can lead to problems when a single family requires several fonts, depending on the language, mainly because ‘substitutions’ with some combinations are not done consistently – sometimes bx/sc is the correct font, but sometimes points to b/n, even if b/sc exists. So, some substitutions are redefined (in a somewhat hackish way, by inspecting if the variant declaration contains `>ssub*`).

```

5110 \def\bbl@font@set#1#2#3{% e.g., \bbl@rmdflt@lang \rmdefault \rmfamily
5111 \bbl@xin@{<>}{#1}%
5112 \ifin@
5113 \bbl@exp{\bbl@fontspec@set\#1\expandafter\@gobbletwo#1\#3}%
5114 \fi
5115 \bbl@exp{%
5116 \def\#2{#1}% e.g., \rmdefault{\bbl@rmdflt@lang}
5117 \bbl@ifsamestring{#2}{\f@family}%
5118 {\#3%
5119 \bbl@ifsamestring{\f@series}{\bfdefault}{\bfseries}}%
5120 \let\bbl@tempa\relax}%
5121 {}}}

```

Loaded locally, which does its job, but very must be global. The problem is how. This actually defines a font predeclared with `\babelfont`, making sure Script and Language names are defined. If they are not, the corresponding data in the ini file is used. The font is actually set temporarily to get the family name (`\f@family`). There is also a hack because by default some replacements related to the bold series are sometimes assigned to the wrong font (see issue #92).

```

5122 \def\bbl@fontspec@set#1#2#3#4{% eg \bbl@rmdflt@lang fnt-opt fnt-nme \xxfamily
5123 \let\bbl@tempe\bbl@mapselect
5124 \edef\bbl@tempb{\bbl@stripslash#4}% Catcodes hack (better pass it).
5125 \bbl@exp{\bbl@replace\bbl@tempb{\bbl@stripslash\family/}}%
5126 \let\bbl@mapselect\relax
5127 \let\bbl@temp@fam#4% e.g., '\rmfamily', to be restored below
5128 \let#4@empty % Make sure \renewfontfamily is valid
5129 \bbl@set@renderer
5130 \bbl@exp{%
5131 \let\bbl@temp@pfam<\bbl@stripslash#4\space>% e.g., '\rmfamily '
5132 \<keys_if_exist:nnF>{fontspec-opentype}{Script/\bbl@cl{sname}}%
5133 {\bbl@newfontscript{\bbl@cl{sname}}{\bbl@cl{sotf}}}%
5134 \<keys_if_exist:nnF>{fontspec-opentype}{Language/\bbl@cl{lname}}%

```

```

5135      {\newfontlanguage{\bbl@cl{lname}}{\bbl@cl{lotf}}}%
5136      \renewfontfamily\#4%
5137      [\bbl@cl{lsys},% xetex removes unknown features :-(
5138      \ifcase\bbl@engine\or RawFeature={family=\bbl@tempb},\fi
5139      #2}}{#3}% i.e., \bbl@exp{.}{#3}
5140      \bbl@unset@renderer
5141      \begingroup
5142      #4%
5143      \xdef#1{\f@family}%      e.g., \bbl@rmdflt@lang{FreeSerif(0)}
5144      \endgroup
5145      \bbl@xin@{\detokenize{>ssub*}}%
5146      {\expandafter\meaning\csname TU/#1/bx/sc\endcsname}%
5147      \ifin@
5148      \global\bbl@ccarg\let{TU/#1/bx/sc}{TU/#1/b/sc}%
5149      \fi
5150      \bbl@xin@{\detokenize{>ssub*}}%
5151      {\expandafter\meaning\csname TU/#1/bx/scit\endcsname}%
5152      \ifin@
5153      \global\bbl@ccarg\let{TU/#1/bx/scit}{TU/#1/b/scit}%
5154      \fi
5155      \let#4\bbl@temp@fam
5156      \bbl@exp{\let<\bbl@stripslash#4\space>}\bbl@temp@pfam
5157      \let\bbl@mapselect\bbl@tempe}%

```

font@rst and famrst are only used when there is no global settings, to save and restore de previous families. Not really necessary, but done for optimization.

```

5158 \def\bbl@font@rst#1#2#3#4{%
5159   \bbl@csarg\def{famrst@#4}{\bbl@font@set{#1}#2#3}}

```

The default font families. They are eurocentric, but the list can be expanded easily with \babelfont.

```

5160 \def\bbl@font@fams{rm,sf,tt}
5161 (</Font selection>)

```

11. Hooks for XeTeX and LuaTeX

11.1. XeTeX

Unfortunately, the current encoding cannot be retrieved and therefore it is reset always to utf8, which seems a sensible default.

Now, the code.

```

5162 (<*xetex>)
5163 \def\BabelStringsDefault{unicode}
5164 \let\xebbl@stop\relax
5165 \AddBabelHook{xetex}{encodedcommands}{%
5166   \def\bbl@tempa{#1}%
5167   \ifx\bbl@tempa@empty
5168     \XeTeXinputencoding"bytes"%
5169   \else
5170     \XeTeXinputencoding"#1"%
5171   \fi
5172   \def\xebbl@stop{\XeTeXinputencoding"utf8"}}
5173 \AddBabelHook{xetex}{stopcommands}{%
5174   \xebbl@stop
5175   \let\xebbl@stop\relax}
5176 \def\bbl@input@classes{% Used in CJK intraspaces
5177   \input{load-unicode-xetex-classes.tex}%
5178   \let\bbl@input@classes\relax}
5179 \def\bbl@intraspace#1 #2 #3\@@{%
5180   \bbl@csarg\gdef{xeisp@\languagename}%
5181   {\XeTeXlinebreakskip #1em plus #2em minus #3em\relax}}
5182 \def\bbl@intrapenalty#1\@@{%

```

```

5183 \bbl@csarg\gdef{xeipn@\language}%
5184 {\XeTeXlinebreakpenalty #1\relax}
5185 \def\bbl@provide@intraspace{%
5186 \bbl@xin@{/s}{/\bbl@cl{\lnbrk}}%
5187 \ifin@else\bbl@xin@{/c}{/\bbl@cl{\lnbrk}}\fi
5188 \ifin@
5189 \bbl@ifunset{\bbl@intsp@\language}{}%
5190 {\expandafter\ifx\csname bbl@intsp@\language\endcsname\@empty\else
5191 \ifx\bbl@KVP@intraspace\@nnil
5192 \bbl@exp{%
5193 \\\bbl@intraspace\bbl@cl{intsp}\@@}%
5194 \fi
5195 \ifx\bbl@KVP@intrapenalty\@nnil
5196 \bbl@intrapenalty0\@@
5197 \fi
5198 \fi
5199 \ifx\bbl@KVP@intraspace\@nnil\else % We may override the ini
5200 \expandafter\bbl@intraspace\bbl@KVP@intraspace\@@
5201 \fi
5202 \ifx\bbl@KVP@intrapenalty\@nnil\else
5203 \expandafter\bbl@intrapenalty\bbl@KVP@intrapenalty\@@
5204 \fi
5205 \bbl@exp{%
5206 \\\bbl@add\<extras\language>{%
5207 \XeTeXlinebreaklocale "\bbl@cl{tbc}"%
5208 \<bbl@xeisp@\language>%
5209 \<bbl@xeipn@\language>%
5210 \\\bbl@toglobal\<extras\language>%
5211 \\\bbl@add\<noextras\language>{%
5212 \XeTeXlinebreaklocale ""}%
5213 \\\bbl@toglobal\<noextras\language>%
5214 \ifx\bbl@ispace\@undefined
5215 \gdef\bbl@ispace{\bbl@cl{xeisp}}%
5216 \ifx\AtBeginDocument\@notprerr
5217 \expandafter\@secondoftwo % to execute right now
5218 \fi
5219 \AtBeginDocument{\bbl@patchfont{\bbl@ispace}}%
5220 \fi}%
5221 \fi}
5222 \ifx\DisableBabelHook\@undefined\endinput\fi
5223 \let\bbl@set@renderer\relax
5224 \let\bbl@unset@renderer\relax
5225 <@Font selection@>
5226 \def\bbl@provide@extra#1{}

```

Hack for unhyphenated line breaking. See \bbl@provide@lsys in the common code.

```

5227 \def\bbl@xenohyph@{%
5228 \bbl@ifset{\bbl@prehc@\language}%
5229 {\ifnum\hyphenchar\font=\defaultthyphenchar
5230 \iffontchar\font\bbl@cl{prehc}\relax
5231 \hyphenchar\font\bbl@cl{prehc}\relax
5232 \else\iffontchar\font"200B
5233 \hyphenchar\font"200B
5234 \else
5235 \bbl@warning
5236 {Neither 0 nor ZERO WIDTH SPACE are available\\%
5237 in the current font, and therefore the hyphen\\%
5238 will be printed. Try changing the fontspec's\\%
5239 'HyphenChar' to another value, but be aware\\%
5240 this setting is not safe (see the manual).\\%
5241 Reported}%
5242 \hyphenchar\font\defaultthyphenchar
5243 \fi\fi

```

```

5244 \fi}%
5245 {\hyphenchar\font\defaultthyphenchar}}

```

11.2. Support for interchar

xetex reserves some values for CJK (although they are not set in XELATEX), so we make sure they are skipped. Define some user names for the global classes, too.

```

5246 \ifnum\xe@alloc@intercharclass<\thr@@
5247 \xe@alloc@intercharclass\thr@@
5248 \fi
5249 \chardef\bbl@xe@class@default@=\z@
5250 \chardef\bbl@xe@class@cjkideogram@=\@ne
5251 \chardef\bbl@xe@class@cjkleftpunctuation@=\tw@
5252 \chardef\bbl@xe@class@cjkrightpunctuation@=\thr@@
5253 \chardef\bbl@xe@class@boundary@=4095
5254 \chardef\bbl@xe@class@ignore@=4096

```

The machinery is activated with a hook (enabled only if actually used). Here \bbl@tempc is pre-set with \bbl@usingxe@class, defined below. The standard mechanism based on \originalTeX to save, set and restore values is used. \count@ stores the previous char to be set, except at the beginning (0) and after \bbl@upto, which is the previous char negated, as a flag to mark a range.

```

5255 \AddBabelHook{babel-interchar}{beforeextras}{%
5256 \@nameuse{bbl@xechars@\language@}}
5257 \DisableBabelHook{babel-interchar}
5258 \protected\def\bbl@charclass#1{%
5259 \ifnum\count@<\z@
5260 \count@-\count@
5261 \loop
5262 \bbl@exp{%
5263 \\\babel@savevariable{\XeTeXcharclass`\Uchar\count@}}%
5264 \XeTeXcharclass\count@ \bbl@tempc
5265 \ifnum\count@<`#1\relax
5266 \advance\count@ \@ne
5267 \repeat
5268 \else
5269 \babel@savevariable{\XeTeXcharclass`#1}%
5270 \XeTeXcharclass`#1 \bbl@tempc
5271 \fi
5272 \count@`#1\relax}

```

Now the two user macros. Char classes are declared implicitly, and then the macro to be executed at the babel-interchar hook is created. The list of chars to be handled by the hook defined above has internally the form \bbl@usingxe@class\bbl@xe@class@punct@english\bbl@charclass{.} \bbl@charclass{,} (etc.), where \bbl@usingxe@class stores the class to be applied to the subsequent characters. The \ifcat part deals with the alternative way to enter characters as macros (e.g., \j). As a special case, hyphens are stored as \bbl@upto, to deal with ranges.

```

5273 \newcommand\bbl@ifinterchar[1]{%
5274 \let\bbl@tempa\@gobble % Assume to ignore
5275 \edef\bbl@tempb{\zap@space#1 \@empty}%
5276 \ifx\bbl@KVP@interchar\@nnil\else
5277 \bbl@replace\bbl@KVP@interchar{ }{,}%
5278 \bbl@foreach\bbl@tempb{%
5279 \bbl@xin@{,##1,}{, \bbl@KVP@interchar,}%
5280 \ifin@
5281 \let\bbl@tempa\@firstofone
5282 \fi}%
5283 \fi
5284 \bbl@tempa}
5285 \newcommand\IfBabelIntercharT[2]{%
5286 \bbl@carg\bbl@add{\bbl@icsave@\CurrentOption}{\bbl@ifinterchar{#1}{#2}}}%
5287 \newcommand\babelcharclass[3]{%
5288 \EnableBabelHook{babel-interchar}%
5289 \bbl@carg\newXeTeXintercharclass{xe@class@#2@#1}%

```

```

5290 \def\bbl@tempb##1{%
5291   \ifx##1\@empty\else
5292     \ifx##1-%
5293       \bbl@upto
5294     \else
5295       \bbl@charclass{%
5296         \ifcat\noexpand##1\relax\bbl@stripslash##1\else\string##1\fi}%
5297       \fi
5298       \expandafter\bbl@tempb
5299     \fi}%
5300 \bbl@ifunset\bbl@xechars@#1{%
5301   {\toks@{%
5302     \babel@savevariable\XeTeXinterchartokenstate
5303     \XeTeXinterchartokenstate\@ne
5304   }}%
5305   {\toks@\expandafter\expandafter\expandafter{%
5306     \csname bbl@xechars@#1\endcsname}}%
5307 \bbl@csarg\edef\xechars@#1{%
5308   \the\toks@
5309   \bbl@usingxeclass\csname bbl@xeclass@#2@#1\endcsname
5310   \bbl@tempb#3\@empty}}
5311 \protected\def\bbl@usingxeclass#1{\count@\z@ \let\bbl@tempc#1}
5312 \protected\def\bbl@upto{%
5313   \ifnum\count@>\z@
5314     \advance\count@\@ne
5315     \count@-\count@
5316   \else\ifnum\count@=\z@
5317     \bbl@charclass{-}%
5318   \else
5319     \bbl@error{double-hyphens-class}{\}\{\}%
5320   \fi\fi}

```

And finally, the command with the code to be inserted. If the language doesn't define a class, then use the global one, as defined above. For the definition there is a intermediate macro, which can be 'disabled' with `\bbl@ic@<label>@<language>`.

```

5321 \def\bbl@ignoreinterchar{%
5322   \ifnum\language=\l@nohyphenation
5323     \expandafter\@gobble
5324   \else
5325     \expandafter\@firstofone
5326   \fi}
5327 \newcommand\babelinterchar[5][{}]{%
5328   \let\bbl@kv@label\@empty
5329   \bbl@forkv{#1}{\bbl@csarg\edef\kv@##1{##2}}%
5330   \@namedef{\zap@space bbl@xeinter@\bbl@kv@label @#3@#4@#2 \@empty}%
5331   {\bbl@ignoreinterchar{#5}}%
5332   \bbl@csarg\let\ic@\bbl@kv@label @#2\@firstofone
5333   \bbl@exp{\bbl@for{\bbl@tempa{\zap@space#3 \@empty}}{%
5334     \bbl@exp{\bbl@for{\bbl@tempb{\zap@space#4 \@empty}}{%
5335       \XeTeXinterchartoks
5336       \@nameuse\bbl@xeclass@\bbl@tempa @%
5337       \bbl@ifunset\bbl@xeclass@\bbl@tempa @#2\}\{#2\} %
5338       \@nameuse\bbl@xeclass@\bbl@tempb @%
5339       \bbl@ifunset\bbl@xeclass@\bbl@tempb @#2\}\{#2\} %
5340     = \expandafter{%
5341       \csname bbl@ic@\bbl@kv@label @#2\expandafter\endcsname
5342       \csname\zap@space bbl@xeinter@\bbl@kv@label
5343         @#3@#4@#2 \@empty\endcsname}}}}
5344 \DeclareRobustCommand\enablelocaleinterchar[1]{%
5345   \bbl@ifunset\bbl@ic@#1\@languagename}%
5346   {\bbl@error{unknown-interchar}{#1}{\}\{\}%
5347   {\bbl@csarg\let\ic@#1\@languagename\@firstofone}}
5348 \DeclareRobustCommand\disablelocaleinterchar[1]{%

```

```

5349 \bbl@ifunset{bbl@ic@#1@\languagename}%
5350 {\bbl@error{unknown-interchar-b}{#1}{}}}%
5351 {\bbl@csarg\let{ic@#1@\languagename}\@gobble}}
5352 (/xetex)

```

11.3. Layout

Note elements like headlines and margins can be modified easily with packages like fancyhdr, typearea or titles, and geometry.

\bbl@startskip and \bbl@endskip are available to package authors. Thanks to the $\TeX$ expansion mechanism the following constructs are valid: \advance\bbl@startskip\adim, \bbl@startskip\adim.

Consider txtbabel as a shorthand for *tex-xet babel*, which is the bidi model in both pdftex and xetex.

```

5353 (*xetex|texxet)
5354 \providecommand\bbl@provide@intraspace{}
5355 \bbl@trace{Redefinitions for bidi layout}

Finish here if there is no layout.

5356 \ifx\bbl@opt@layout\@nnil\else % if layout=..
5357 \ifx\bbl@opt@layout\@undefined\else % Plain
5358 \IfBabelLayout{nopars}
5359 {}
5360 {\edef\bbl@opt@layout{\bbl@opt@layout.pars.}}%
5361 \fi
5362 \def\bbl@startskip{\ifcase\bbl@thepardir\leftskip\else\rightskip\fi}
5363 \def\bbl@endskip{\ifcase\bbl@thepardir\rightskip\else\leftskip\fi}
5364 \ifnum\bbl@bidimode>\z@
5365 \IfBabelLayout{pars}
5366 {\def\@hangfrom#1{%
5367 \setbox\@tempboxa\hbox{#1}}%
5368 \hangindent\ifcase\bbl@thepardir\wd\@tempboxa\else-\wd\@tempboxa\fi
5369 \noindent\box\@tempboxa}
5370 \def\raggedright{%
5371 \let\\\@centercr
5372 \bbl@startskip\z@skip
5373 \@rightskip\@flushglue
5374 \bbl@endskip\@rightskip
5375 \parindent\z@
5376 \parfillskip\bbl@startskip}
5377 \def\raggedleft{%
5378 \let\\\@centercr
5379 \bbl@startskip\@flushglue
5380 \bbl@endskip\z@skip
5381 \parindent\z@
5382 \parfillskip\bbl@endskip}}
5383 {}
5384 \fi
5385 \IfBabelLayout{lists}
5386 {\bbl@sreplace\list
5387 {\@totalleftmargin\leftmargin}{\@totalleftmargin\bbl@listleftmargin}%
5388 \def\bbl@listleftmargin{%
5389 \ifcase\bbl@thepardir\leftmargin\else\rightmargin\fi}%
5390 \ifcase\bbl@engine
5391 \def\labelenumii{}\theenumii{}% pdftex doesn't reverse ()
5392 \def\p@enumiii{\p@enumii}\theenumii{}%
5393 \fi
5394 \bbl@sreplace\@verbatim
5395 {\leftskip\@totalleftmargin}%
5396 {\bbl@startskip\textwidth
5397 \advance\bbl@startskip-\linewidth}%
5398 \bbl@sreplace\@verbatim
5399 {\rightskip\z@skip}%

```

```

5400     {\bbl@endskip\z@skip}}%
5401   {}
5402 \IfBabelLayout{contents}
5403   {\bbl@sreplace\@dottedtocline{\leftskip}{\bbl@startskip}%
5404    \bbl@sreplace\@dottedtocline{\rightskip}{\bbl@endskip}}
5405   {}
5406 \IfBabelLayout{columns}
5407   {\bbl@sreplace\@outputdblcol{\hb@xt@\textwidth}{\bbl@outputbox}%
5408    \def\bbl@outputbox#1{%
5409      \hb@xt@\textwidth{%
5410        \hskip\columnwidth
5411        \hfil
5412        {\normalcolor\vrule \@width\columnseprule}%
5413        \hfil
5414        \hb@xt@\columnwidth{\box\@leftcolumn \hss}%
5415        \hskip-\textwidth
5416        \hb@xt@\columnwidth{\box\@outputbox \hss}%
5417        \hskip\columnsep
5418        \hskip\columnwidth}}}%
5419   {}

```

Implicitly reverses sectioning labels in bidi=basic, because the full stop is not in contact with L numbers any more. I think there must be a better way.

```

5420 \IfBabelLayout{counters*}%
5421   {\bbl@add\bbl@opt@layout{.counters.}%
5422    \AddToHook{shipout/before}{%
5423      \let\bbl@tempa\babelsublr
5424      \let\babelsublr\@firstofone
5425      \let\bbl@save@thepage\thepage
5426      \protected@edef\thepage{\thepage}%
5427      \let\babelsublr\bbl@tempa}%
5428    \AddToHook{shipout/after}{%
5429      \let\thepage\bbl@save@thepage}}{}
5430 \IfBabelLayout{counters}%
5431   {\let\bbl@latinarabic=\@arabic
5432    \def\@arabic#1{\babelsublr{\bbl@latinarabic#1}}%
5433    \let\bbl@asciroman=\@roman
5434    \def\@roman#1{\babelsublr{\ensureascii{\bbl@asciroman#1}}}%
5435    \let\bbl@asciiRoman=\@Roman
5436    \def\@Roman#1{\babelsublr{\ensureascii{\bbl@asciiRoman#1}}}}{}
5437 \fi % end if layout
5438 (/xetex|texet)

```

11.4. 8-bit TeX

Which start just above, because some code is shared with xetex. Now, 8-bit specific stuff. If just one encoding has been declared, then assume no switching is necessary (1).

```

5439 (*texet)
5440 \def\bbl@provide@extra#1{%
5441   % == auto-select encoding ==
5442   \ifx\bbl@encoding@select@off\@empty\else
5443     \bbl@ifunset{\bbl@encoding@#1}%
5444     {\def\@elt##1{,##1,}%
5445      \edef\bbl@tempe{\expandafter\@gobbletwo\@fontenc@load@list}%
5446      \count@\z@
5447      \bbl@foreach\bbl@tempe{%
5448        \def\bbl@tempd{##1}% Save last declared
5449        \advance\count@\@ne}%
5450      \ifnum\count@>\@ne % (1)
5451        \getlocaleproperty*\bbl@tempa{#1}{identification/encodings}%
5452        \ifx\bbl@tempa\relax \let\bbl@tempa\@empty \fi
5453        \bbl@replace\bbl@tempa{ },,%
5454        \global\bbl@csarg\let{encoding@#1}\@empty

```

```

5455      \bbl@xin@{,\bbl@tempd,}{,\bbl@tempa,}%
5456      \ifin@{\else % if main encoding included in ini, do nothing
5457        \let\bbl@tempb\relax
5458        \bbl@foreach\bbl@tempa{%
5459          \ifx\bbl@tempb\relax
5460            \bbl@xin@{,##1,}{,\bbl@tempe,}%
5461            \ifin@{\def\bbl@tempb{##1}\fi
5462              \fi}%
5463          \ifx\bbl@tempb\relax\else
5464            \bbl@exp{%
5465              \global\<bbl@add>\<bbl@preextras@#1>\<bbl@encoding@#1>}%
5466            \gdef\<bbl@encoding@#1>{%
5467              \\babel@save\\f@encoding
5468              \\bbl@add\\originalTeX{\\selectfont}%
5469              \\fontencoding{\bbl@tempb}%
5470              \\selectfont}}%
5471          \fi
5472        \fi
5473      \fi}%
5474    }%
5475  \fi}
5476 (/texet)

```

11.5. LuaTeX

The loader for luatex is based solely on `language.dat`, which is read on the fly. The code shouldn't be executed when the format is build, so we check if `\AddBabelHook` is defined. Then comes a modified version of the loader in `hyphen.cfg` (without the `hyphenmins` stuff, which is under the direct control of `babel`).

The names `\l@<language>` are defined and take some value from the beginning because all `ldf` files assume this for the corresponding language to be considered valid, but patterns are not loaded (except the first one). This is done later, when the language is first selected (which usually means when the `ldf` finishes). If a language has been loaded, `\bbl@hyphendata@<num>` exists (with the names of the files read).

The default setup preloads the first language into the format. This is intended mainly for 'english', so that it's available without further intervention from the user. To avoid duplicating it, the following rule applies: if the "0th" language and the first language in `language.dat` have the same name then just ignore the latter. If there are new synonymous, they are added, but note if the language patterns have not been preloaded they won't at run time.

Other preloaded languages could be read twice, if they have been preloaded into the format. This is not optimal, but it shouldn't happen very often – with luatex patterns are best loaded when the document is typeset, and the "0th" language is preloaded just for backwards compatibility.

As of 1.1b, `lua(e)tex` is taken into account. Formerly, loading of patterns on the fly didn't work in this format, but with the new loader it does. Unfortunately, the format is not based on `babel`, and data could be duplicated, because languages are reassigned above those in the format (nothing serious, anyway). Note even with this format `language.dat` is used (under the principle of a single source), instead of `language.def`.

Of course, there is room for improvements, like tools to read and reassign languages, which would require modifying the language list, and better error handling.

We need catcode tables, but no format (targeted by `babel`) provide a command to allocate them (although there are packages like `ctablestack`). FIX - This isn't true anymore. For the moment, a dangerous approach is used - just allocate a high random number and cross the fingers. To complicate things, `etex.sty` changes the way languages are allocated.

This files is read at three places: (1) when `plain.def`, `babel.sty` starts, to read the list of available languages from `language.dat` (for the base option); (2) at `hyphen.cfg`, to modify some macros; (3) in the middle of `plain.def` and `babel.sty`, by `babel.def`, with the commands and other definitions for luatex (e.g., `\babelpatterns`).

```

5477 (*luatex)
5478 \directlua{ Babel = Babel or {} } % DL2
5479 \ifx\AddBabelHook\undefined % When plain.def, babel.sty starts
5480 \bbl@trace{Read language.dat}
5481 \ifx\bbl@readstream\undefined
5482   \csname newread\endcsname\bbl@readstream

```

```

5483 \fi
5484 \beginingroup
5485 \toks@{}
5486 \count@\z@ % 0=start, 1=0th, 2=normal
5487 \def\bbl@process@line#1#2 #3 #4 {%
5488 \ifx=#1%
5489 \bbl@process@synonym{#2}%
5490 \else
5491 \bbl@process@language{#1#2}{#3}{#4}%
5492 \fi
5493 \ignorespaces}
5494 \def\bbl@manylang{%
5495 \ifnum\bbl@last>\@ne
5496 \bbl@info{Non-standard hyphenation setup}%
5497 \fi
5498 \let\bbl@manylang\relax}
5499 \def\bbl@process@language#1#2#3{%
5500 \ifcase\count@
5501 \@ifundefined{zth@#1}{\count@\tw@}{\count@\@ne}%
5502 \or
5503 \count@\tw@
5504 \fi
5505 \ifnum\count@=\tw@
5506 \expandafter\addlanguage\csname l@#1\endcsname
5507 \language\allocationnumber
5508 \chardef\bbl@last\allocationnumber
5509 \bbl@manylang
5510 \let\bbl@elt\relax
5511 \xdef\bbl@languages{%
5512 \bbl@languages\bbl@elt{#1}{\the\language}{#2}{#3}}%
5513 \fi
5514 \the\toks@
5515 \toks@{}}
5516 \def\bbl@process@synonym@aux#1#2{%
5517 \global\expandafter\chardef\csname l@#1\endcsname#2\relax
5518 \let\bbl@elt\relax
5519 \xdef\bbl@languages{%
5520 \bbl@languages\bbl@elt{#1}{#2}{}}}%
5521 \def\bbl@process@synonym#1{%
5522 \ifcase\count@
5523 \toks@\expandafter{\the\toks@\relax\bbl@process@synonym{#1}}%
5524 \or
5525 \@ifundefined{zth@#1}{\bbl@process@synonym@aux{#1}{0}}{}%
5526 \else
5527 \bbl@process@synonym@aux{#1}{\the\bbl@last}%
5528 \fi}
5529 \ifx\bbl@languages\@undefined % Just a (sensible?) guess
5530 \chardef\l@english\z@
5531 \chardef\l@USenglish\z@
5532 \chardef\bbl@last\z@
5533 \global\@namedef{bbl@hyphendata@0}{{hyphen.tex}}
5534 \gdef\bbl@languages{%
5535 \bbl@elt{english}{0}{hyphen.tex}}%
5536 \bbl@elt{USenglish}{0}{}%
5537 \else
5538 \global\let\bbl@languages@format\bbl@languages
5539 \def\bbl@elt#1#2#3#4{% Remove all except language 0
5540 \ifnum#2>\z@ \else
5541 \noexpand\bbl@elt{#1}{#2}{#3}{#4}%
5542 \fi}%
5543 \xdef\bbl@languages{\bbl@languages}%
5544 \fi
5545 \def\bbl@elt#1#2#3#4{\@namedef{zth@#1}} % Define flags

```

```

5546 \bbl@languages
5547 \openin\bbl@readstream=language.dat
5548 \ifeof\bbl@readstream
5549 \bbl@warning{I couldn't find language.dat. No additional\\%
5550 patterns loaded. Reported}%
5551 \else
5552 \loop
5553 \endlinechar\m@ne
5554 \read\bbl@readstream to \bbl@line
5555 \endlinechar`\^^M
5556 \if T\ifeof\bbl@readstream F\fi T\relax
5557 \ifx\bbl@line\empty\else
5558 \edef\bbl@line{\bbl@line\space\space\space}%
5559 \expandafter\bbl@process@line\bbl@line\relax
5560 \fi
5561 \repeat
5562 \fi
5563 \closein\bbl@readstream
5564 \endgroup
5565 \bbl@trace{Macros for reading patterns files}
5566 \def\bbl@get@enc#1:#2:#3\@@{\def\bbl@hyph@enc{#2}}
5567 \ifx\babelcatcodetablenum\undefined
5568 \ifx\newcatcodetable\undefined
5569 \def\babelcatcodetablenum{5211}
5570 \def\bbl@pattcodes{\numexpr\babelcatcodetablenum+1\relax}
5571 \else
5572 \newcatcodetable\babelcatcodetablenum
5573 \newcatcodetable\bbl@pattcodes
5574 \fi
5575 \else
5576 \def\bbl@pattcodes{\numexpr\babelcatcodetablenum+1\relax}
5577 \fi
5578 \def\bbl@luapatterns#1#2{%
5579 \bbl@get@enc#1:.\@@@
5580 \setbox\z@\hbox\bgroup
5581 \begingroup
5582 \savecatcodetable\babelcatcodetablenum\relax
5583 \initcatcodetable\bbl@pattcodes\relax
5584 \catcodetable\bbl@pattcodes\relax
5585 \catcode\#=6 \catcode`\$=3 \catcode`\&=4 \catcode`\^=7
5586 \catcode`\_ =8 \catcode`\{=1 \catcode`\}=2 \catcode`\~=13
5587 \catcode`\@=11 \catcode`\^^I=10 \catcode`\^^J=12
5588 \catcode`\<=12 \catcode`\>=12 \catcode`\*=12 \catcode`\.=12
5589 \catcode`\-=12 \catcode`\/=12 \catcode`\[=12 \catcode`\]=12
5590 \catcode`\`=12 \catcode`\'=12 \catcode`\`=12
5591 \input #1\relax
5592 \catcodetable\babelcatcodetablenum\relax
5593 \endgroup
5594 \def\bbl@tempa{#2}%
5595 \ifx\bbl@tempa\empty\else
5596 \input #2\relax
5597 \fi
5598 \egroup}%
5599 \def\bbl@patterns@lua#1{%
5600 \language=\expandafter\ifx\csname l@#1:\f@encoding\endcsname\relax
5601 \csname l@#1\endcsname
5602 \edef\bbl@tempa{#1}%
5603 \else
5604 \csname l@#1:\f@encoding\endcsname
5605 \edef\bbl@tempa{#1:\f@encoding}%
5606 \fi\relax
5607 \@namedef{lu@texhyphen@loaded@the\language}{}% Temp
5608 \@ifundefined{bbl@hyphendata@the\language}%

```

```

5609 {\def\bbl@elt##1##2##3##4{%
5610   \ifnum##2=\csname l@\bbl@tempa\endcsname % #2=spanish, dutch:OT1...
5611   \def\bbl@tempb{##3}%
5612   \ifx\bbl@tempb\empty\else % if not a synonymous
5613     \def\bbl@tempc{##3}{##4}%
5614   \fi
5615   \bbl@csarg\xdef{hyphendata@##2}{\bbl@tempc}%
5616   \fi}%
5617 \bbl@languages
5618 \@ifundefined{bbl@hyphendata@the\language}%
5619   {\bbl@info{No hyphenation patterns were set for\%
5620     language '\bbl@tempa'. Reported}}%
5621   {\expandafter\expandafter\expandafter\bbl@luapatterns
5622     \csname bbl@hyphendata@the\language\endcsname}}}%
5623 \endinput\fi

```

Here ends \ifx\AddBabelHook\@undefined. A few lines are only read by HYPHEN.CFG.

```

5624 \ifx\DisableBabelHook\@undefined
5625   \AddBabelHook{luatex}{everylanguage}{%
5626     \def\process@language##1##2##3{%
5627       \def\process@line###1###2 ###3 ###4 {}}}
5628   \AddBabelHook{luatex}{loadpatterns}{%
5629     \input #1\relax
5630     \expandafter\gdef\csname bbl@hyphendata@the\language\endcsname
5631       {##1}{}}
5632   \AddBabelHook{luatex}{loadexceptions}{%
5633     \input #1\relax
5634     \def\bbl@tempb##1##2{##1}{##1}%
5635     \expandafter\xdef\csname bbl@hyphendata@the\language\endcsname
5636       {\expandafter\expandafter\expandafter\bbl@tempb
5637         \csname bbl@hyphendata@the\language\endcsname}}
5638   \endinput\fi

```

Here stops reading code for HYPHEN.CFG. The following is read the 2nd time it's loaded. First, global declarations for lua.

```

5639 \begingroup
5640 \catcode`\%=12
5641 \catcode`\'=12
5642 \catcode`\|=12
5643 \catcode`\:=12
5644 \directlua{
5645   Babel.locale_props = Babel.locale_props or {}
5646
5647   function Babel.lua_error(e, a)
5648     tex.print([[noexpand\csname bbl@error\endcsname{]] ..
5649       e .. '}' .. (a or '') .. '}{}}')
5650   end
5651
5652   function Babel.bytes(line)
5653     return line:gsub(".",
5654       function (chr) return unicode.utf8.char(string.byte(chr)) end)
5655   end
5656
5657   function Babel.priority_in_callback(name,description)
5658     for i,v in ipairs(luatexbase.callback_descriptions(name)) do
5659       if v == description then return i end
5660     end
5661     return false
5662   end
5663
5664   function Babel.begin_process_input()
5665     if luatexbase and luatexbase.add_to_callback then
5666       luatexbase.add_to_callback('process_input_buffer',
5667         Babel.bytes, 'Babel.bytes')

```

```

5668     else
5669         Babel.callback = callback.find('process_input_buffer')
5670         callback.register('process_input_buffer',Babel.bytes)
5671     end
5672 end
5673 function Babel.end_process_input ()
5674     if luatexbase and luatexbase.remove_from_callback then
5675         luatexbase.remove_from_callback('process_input_buffer','Babel.bytes')
5676     else
5677         callback.register('process_input_buffer',Babel.callback)
5678     end
5679 end
5680
5681 function Babel.str_to_nodes(fn, matches, base)
5682     local n, head, last
5683     if fn == nil then return nil end
5684     for s in string.utfvalues(fn(matches)) do
5685         if base.id == 7 then
5686             base = base.replace
5687         end
5688         n = node.copy(base)
5689         n.char = s
5690         if not head then
5691             head = n
5692         else
5693             last.next = n
5694         end
5695         last = n
5696     end
5697     return head
5698 end
5699
5700 Babel.linebreaking = Babel.linebreaking or {}
5701 Babel.linebreaking.before = {}
5702 Babel.linebreaking.after = {}
5703 Babel.locale = {}
5704 function Babel.linebreaking.add_before(func, pos)
5705     tex.print([[\\noexpand\\csname bbl@luahyphenate\\endcsname]])
5706     if pos == nil then
5707         table.insert(Babel.linebreaking.before, func)
5708     else
5709         table.insert(Babel.linebreaking.before, pos, func)
5710     end
5711 end
5712 function Babel.linebreaking.add_after(func)
5713     tex.print([[\\noexpand\\csname bbl@luahyphenate\\endcsname]])
5714     table.insert(Babel.linebreaking.after, func)
5715 end
5716
5717 function Babel.addpatterns(pp, lg)
5718     local lg = lang.new(lg)
5719     local pats = lang.patterns(lg) or ''
5720     lang.clear_patterns(lg)
5721     for p in pp:gmatch('[^%s]+') do
5722         ss = ''
5723         for i in string.utfcharacters(p:gsub('%d', '')) do
5724             ss = ss .. '%d?' .. i
5725         end
5726         ss = ss:gsub('^%d%?%.','%.') .. '%d?'
5727         ss = ss:gsub('%.%d%?$', '%%.')
5728         pats, n = pats:gsub('%s' .. ss .. '%s', ' ' .. p .. ' ')
5729         if n == 0 then
5730             tex.sprint(

```

```

5731         [[\string\csname\space bbl@info\endcsname{New pattern: }]
5732         .. p .. [[]]]
5733         pats = pats .. ' ' .. p
5734     else
5735         tex.sprint(
5736             [[\string\csname\space bbl@info\endcsname{Renew pattern: }]
5737             .. p .. [[]]]
5738         end
5739     end
5740     lang.patterns(lg, pats)
5741 end
5742
5743 Babel.characters = Babel.characters or {}
5744 Babel.ranges = Babel.ranges or {}
5745 function Babel.hlist_has_bidi(head)
5746     local has_bidi = false
5747     local ranges = Babel.ranges
5748     for item in node.traverse(head) do
5749         if item.id == node.id'glyph' then
5750             local itemchar = item.char
5751             local chardata = Babel.characters[itemchar]
5752             local dir = chardata and chardata.d or nil
5753             if not dir then
5754                 for nn, et in ipairs(ranges) do
5755                     if itemchar < et[1] then
5756                         break
5757                     elseif itemchar <= et[2] then
5758                         dir = et[3]
5759                         break
5760                     end
5761                 end
5762             end
5763             if dir and (dir == 'al' or dir == 'r') then
5764                 has_bidi = true
5765             end
5766         end
5767     end
5768     return has_bidi
5769 end
5770 function Babel.set_chranges_b (script, chrng)
5771     if chrng == '' then return end
5772     texio.write('Package babel Info: Replacing ' .. script .. ' script ranges')
5773     Babel.script_blocks[script] = {}
5774     for s, e in string.gmatch(chrng..' ', '(.)%.%.(-)%s') do
5775         table.insert(
5776             Babel.script_blocks[script], {tonumber(s,16), tonumber(e,16)})
5777     end
5778 end
5779
5780 function Babel.discard_sublr(str)
5781     if str:find( [[\string\indexentry]] ) and
5782        str:find( [[\string\babelsublr]] ) then
5783         str = str:gsub( [[\string\babelsublr%*{%b{}}]],
5784             function(m) return m:sub(2,-2) end )
5785     end
5786     return str
5787 end
5788 }
5789 \endgroup
5790 \ifx\newattribute\undefined\else % Test for plain
5791     \newattribute\bbl@attr@locale % DL4
5792     \directlua{ Babel.attr_locale = luatexbase.registernumber'bbl@attr@locale' }
5793     \AddBabelHook{luatex}{beforeextras}{%

```

```

5794 \setattribute\bbbl@attr@locale\localeid}
5795 \fi
5796 %
5797 \def\BabelStringsDefault{unicode}
5798 \let\luabbl@stop\relax
5799 \AddBabelHook{luatex}{encodedcommands}{%
5800 \def\bbbl@tempa{utf8}\def\bbbl@tempb{#1}%
5801 \ifx\bbbl@tempa\bbbl@tempb\else
5802 \directlua{Babel.begin_process_input()}%
5803 \def\luabbl@stop{%
5804 \directlua{Babel.end_process_input()}}%
5805 \fi}%
5806 \AddBabelHook{luatex}{stopcommands}{%
5807 \luabbl@stop
5808 \let\luabbl@stop\relax}
5809 %
5810 \AddBabelHook{luatex}{patterns}{%
5811 \@ifundefined{bbbl@hyphendata@the\language}%
5812 {\def\bbbl@elt##1##2##3##4{%
5813 \ifnum##2=\csname l@##2\endcsname % #2=spanish, dutch:0T1...
5814 \def\bbbl@tempb{##3}%
5815 \ifx\bbbl@tempb\@empty\else % if not a synonymous
5816 \def\bbbl@tempc{{##3}{##4}}%
5817 \fi
5818 \bbbl@csarg\xdef{hyphendata@##2}{\bbbl@tempc}%
5819 \fi}%
5820 \bbbl@languages
5821 \@ifundefined{bbbl@hyphendata@the\language}%
5822 {\bbbl@info{No hyphenation patterns were set for\%
5823 language '#2'. Reported}}%
5824 {\expandafter\expandafter\expandafter\bbbl@luapatterns
5825 \csname bbl@hyphendata@the\language\endcsname}}}%
5826 \@ifundefined{bbbl@patterns@}{}%
5827 \begingroup
5828 \bbbl@xin@{,\number\language,}{,\bbbl@pttnlist}%
5829 \ifin@\else
5830 \ifx\bbbl@patterns@\@empty\else
5831 \directlua{ Babel.addpatterns(
5832 [[\bbbl@patterns@]], \number\language) }%
5833 \fi
5834 \@ifundefined{bbbl@patterns@#1}%
5835 \@empty
5836 {\directlua{ Babel.addpatterns(
5837 [[\space\csname bbl@patterns@#1\endcsname]],
5838 \number\language) }}%
5839 \xdef\bbbl@pttnlist{\bbbl@pttnlist\number\language,}%
5840 \fi
5841 \endgroup}%
5842 \bbbl@exp{%
5843 \bbbl@ifunset{bbbl@prehc@\languagename}}}%
5844 {\bbbl@ifblank{\bbbl@cs{prehc@\languagename}}}%
5845 {\prehyphenchar=\bbbl@cl{prehc}\relax}}}}

```

\babelpatterns This macro adds patterns. Two macros are used to store them: \bbbl@patterns@ for the global ones and \bbbl@patterns@(*language*) for language ones. We make sure there is a space between words when multiple commands are used.

```

5846 \@onlypreamble\babelpatterns
5847 \AtEndOfPackage{%
5848 \newcommand\babelpatterns[2][\@empty]{%
5849 \ifx\bbbl@patterns@\relax
5850 \let\bbbl@patterns@\@empty
5851 \fi
5852 \ifx\bbbl@pttnlist@\@empty\else

```

```

5853     \bbl@warning{%
5854         You must not intermingle \string\selectlanguage\space and\%
5855         \string\babelpatterns\space or some patterns will not\%
5856         be taken into account. Reported}%
5857 \fi
5858 \ifx\@empty#1%
5859     \protected@edef\bbl@patterns@\bbl@patterns@\space#2}%
5860 \else
5861     \edef\bbl@tempb{\zap@space#1 \@empty}%
5862     \bbl@for\bbl@tempa\bbl@tempb{%
5863         \bbl@fixname\bbl@tempa
5864         \bbl@iflanguage\bbl@tempa{%
5865             \bbl@csarg\protected@edef{patterns@\bbl@tempa}{%
5866                 \@ifundefined{bbl@patterns@\bbl@tempa}%
5867                 \@empty
5868                 {\csname bbl@patterns@\bbl@tempa\endcsname\space}%
5869                 #2}}}%
5870 \fi}}

```

11.6. Southeast Asian scripts

First, some general code for line breaking, used by \babelposthyphenation.

Replace regular (i.e., implicit) discretionaries by spaceskips, based on the previous glyph (which I think makes sense, because the hyphen and the previous char go always together). Other discretionaries are not touched. See Unicode UAX 14.

```

5871 \def\bbl@intraspace#1 #2 #3\@{%
5872     \directlua{
5873         Babel.intraspaces = Babel.intraspaces or {}
5874         Babel.intraspaces['\csname bbl@sbc@language\endcsname'] = %
5875             {b = #1, p = #2, m = #3}
5876         Babel.locale_props[\the\localeid].intraspace = %
5877             {b = #1, p = #2, m = #3}
5878     }}
5879 \def\bbl@intrapenalty#1\@{%
5880     \directlua{
5881         Babel.intrapenalties = Babel.intrapenalties or {}
5882         Babel.intrapenalties['\csname bbl@sbc@language\endcsname'] = #1
5883         Babel.locale_props[\the\localeid].intrapenalty = #1
5884     }}
5885 \begingroup
5886 \catcode`\%=12
5887 \catcode`\&=14
5888 \catcode`\'=12
5889 \catcode`\~ =12
5890 \gdef\bbl@seaintraspace{%
5891     \let\bbl@seaintraspace\relax
5892     \directlua{
5893         Babel.sea_enabled = true
5894         Babel.sea_ranges = Babel.sea_ranges or {}
5895         function Babel.set_chranges (script, chrng)
5896             local c = 0
5897             for s, e in string.gmatch(chrng..' ', '(.-%).%.(.)%s') do
5898                 Babel.sea_ranges[script..c]={tonumber(s,16), tonumber(e,16)}
5899                 c = c + 1
5900             end
5901         end
5902         function Babel.sea_disc_to_space (head)
5903             local sea_ranges = Babel.sea_ranges
5904             local last_char = nil
5905             local quad = 655360      &% 10 pt = 655360 = 10 * 65536
5906             for item in node.traverse(head) do
5907                 local i = item.id
5908                 if i == node.id'glyph' then

```

```

5909         last_char = item
5910     elseif i == 7 and item.subtype == 3 and last_char
5911         and last_char.char > 0x0C99 then
5912         quad = font.getfont(last_char.font).size
5913         for lg, rg in pairs(sea_ranges) do
5914             if last_char.char > rg[1] and last_char.char < rg[2] then
5915                 lg = lg:sub(1, 4) &% Remove trailing number of, e.g., Cyril
5916                 local intraspace = Babel.intraspaces[lg]
5917                 local intrapenalty = Babel.intrapenalties[lg]
5918                 local n
5919                 if intrapenalty ~= 0 then
5920                     n = node.new(14, 0) &% penalty
5921                     n.penalty = intrapenalty
5922                     node.insert_before(head, item, n)
5923                 end
5924                 n = node.new(12, 13) &% (glue, spaceskip)
5925                 node.setglue(n, intraspace.b * quad,
5926                     intraspace.p * quad,
5927                     intraspace.m * quad)
5928                 node.insert_before(head, item, n)
5929                 node.remove(head, item)
5930             end
5931         end
5932     end
5933 end
5934 end
5935 }&
5936 \bbl@luahyphenate}

```

11.7. CJK line breaking

Minimal line breaking for CJK scripts, mainly intended for simple documents and short texts as a secondary language. Only line breaking, with a little stretching for justification, without any attempt to adjust the spacing. It is based on (but does not strictly follow) the Unicode algorithm.

We first need a little table with the corresponding line breaking properties. A few characters have an additional key for the width (fullwidth vs. halfwidth), not yet used. There is a separate file, defined below.

```

5937 \catcode`\%=14
5938 \gdef\bbl@cjkintraspaces{
5939     \let\bbl@cjkintraspaces\relax
5940     \directlua{
5941         require('babel-data-cjk.lua')
5942         Babel.cjk_enabled = true
5943         function Babel.cjk_linebreak(head)
5944             local GLYPH = node.id'glyph'
5945             local last_char = nil
5946             local quad = 655360 % 10 pt = 655360 = 10 * 65536
5947             local last_class = nil
5948             local last_lang = nil
5949             for item in node.traverse(head) do
5950                 if item.id == GLYPH then
5951                     local lang = item.lang
5952                     local LOCALE = node.get_attribute(item,
5953                         Babel.attr_locale)
5954                     local props = Babel.locale_props[LOCALE] or {}
5955                     local class = Babel.cjk_class[item.char].c
5956                     if props.cjk_quotes and props.cjk_quotes[item.char] then
5957                         class = props.cjk_quotes[item.char]
5958                     end
5959                     if class == 'cp' then class = 'cl' % ]] as CL
5960                     elseif class == 'id' then class = 'I'
5961                     elseif class == 'cj' then class = 'I' % loose
5962                     end

```

```

5963     local br = 0
5964     if class and last_class and Babel.cjk_breaks[last_class][class] then
5965         br = Babel.cjk_breaks[last_class][class]
5966     end
5967     if br == 1 and props.linebreak == 'c' and
5968         lang ~= \the\l@nohyphenation\space and
5969         last_lang ~= \the\l@nohyphenation then
5970         local intrapenalty = props.intrapenalty
5971         if intrapenalty ~= 0 then
5972             local n = node.new(14, 0)      % penalty
5973             n.penalty = intrapenalty
5974             node.insert_before(head, item, n)
5975         end
5976         local intraspace = props.intraspace
5977         local n = node.new(12, 13)        % (glue, spaceskip)
5978         node.setglue(n, intraspace.b * quad,
5979             intraspace.p * quad,
5980             intraspace.m * quad)
5981         node.insert_before(head, item, n)
5982     end
5983     if font.getfont(item.font) then
5984         quad = font.getfont(item.font).size
5985     end
5986     last_class = class
5987     last_lang = lang
5988     else % if penalty, glue or anything else
5989         last_class = nil
5990     end
5991 end
5992 lang.hyphenate(head)
5993 end
5994 }%
5995 \bbl@luahyphenate}
5996 \gdef\bbl@luahyphenate{%
5997 \let\bbl@luahyphenate\relax
5998 \directlua{
5999     luatexbase.add_to_callback('hyphenate',
6000     function (head, tail)
6001         if Babel.linebreaking.before then
6002             for k, func in ipairs(Babel.linebreaking.before) do
6003                 func(head)
6004             end
6005         end
6006         lang.hyphenate(head)
6007         if Babel.cjk_enabled then
6008             Babel.cjk_linebreak(head)
6009         end
6010         if Babel.linebreaking.after then
6011             for k, func in ipairs(Babel.linebreaking.after) do
6012                 func(head)
6013             end
6014         end
6015         if Babel.set_hboxed then
6016             Babel.set_hboxed(head)
6017         end
6018         if Babel.sea_enabled then
6019             Babel.sea_disc_to_space(head)
6020         end
6021     end,
6022     'Babel.hyphenate')
6023 }}
6024 \endgroup
6025 %

```

```

6026 \def\bbl@provide@intraspace{%
6027   \bbl@ifunset\bbl@intsp@{language}{}%
6028   {\expandafter\ifx\csname bbl@intsp@{language}\endcsname\@empty\else
6029     \bbl@xin@{/c}{/bbl@cl{lnbrk}}}%
6030   \ifin@           % cjk
6031     \bbl@cjk@intraspace
6032     \directlua{
6033       Babel.locale_props = Babel.locale_props or {}
6034       Babel.locale_props[\the\localeid].linebreak = 'c'
6035     }%
6036     \bbl@exp{\\bbl@intraspace\bbl@cl{intsp}\\\@}%
6037     \ifx\bbl@KVP@intrapenalty\@nnil
6038       \bbl@intrapenalty0\@@
6039     \fi
6040   \else           % sea
6041     \bbl@sea@intraspace
6042     \bbl@exp{\\bbl@intraspace\bbl@cl{intsp}\\\@}%
6043     \directlua{
6044       Babel.sea_ranges = Babel.sea_ranges or {}
6045       Babel.set_chranges('\bbl@cl{sbcpr}',
6046         '\bbl@cl{chrng}')
6047     }%
6048     \ifx\bbl@KVP@intrapenalty\@nnil
6049       \bbl@intrapenalty0\@@
6050     \fi
6051   \fi
6052 \fi
6053 \ifx\bbl@KVP@intrapenalty\@nnil\else
6054   \expandafter\bbl@intrapenalty\bbl@KVP@intrapenalty\@@
6055 \fi}}

```

11.8. Arabic justification

WIP. \bbl@arabicjust is executed with both elongated and kashida. This must be fine tuned. The attribute kashida is set by transforms with kashida.

```

6056 \ifnum\bbl@bidimode>100 \ifnum\bbl@bidimode<200
6057 \def\bblar@chars{%
6058   0628,0629,062A,062B,062C,062D,062E,062F,0630,0631,0632,0633,%
6059   0634,0635,0636,0637,0638,0639,063A,063B,063C,063D,063E,063F,%
6060   0640,0641,0642,0643,0644,0645,0646,0647,0649}
6061 \def\bblar@elongated{%
6062   0626,0628,062A,062B,0633,0634,0635,0636,063B,%
6063   063C,063D,063E,063F,0641,0642,0643,0644,0646,%
6064   0649,064A}
6065 \begingroup
6066   \catcode`\_ =11 \catcode`\:=11
6067   \gdef\bblar@nofswarn{\gdef\msg_warning:nx##1##2##3{}}
6068 \endgroup
6069 \gdef\bbl@arabicjust{%
6070   \let\bbl@arabicjust\relax
6071   \newattribute\bblar@kashida
6072   \directlua{ Babel.attr_kashida = luatexbase.registernumber'bblar@kashida' }%
6073   \bblar@kashida=\z@
6074   \bbl@patchfont{\bbl@parsejalt}}%
6075 \directlua{
6076   Babel.arabic.elong_map = Babel.arabic.elong_map or {}
6077   Babel.arabic.elong_map[\the\localeid] = {}
6078   luatexbase.add_to_callback('post_linebreak_filter',
6079     Babel.arabic.justify, 'Babel.arabic.justify')
6080   luatexbase.add_to_callback('hpack_filter',
6081     Babel.arabic.justify_hbox, 'Babel.arabic.justify_hbox')
6082 }}%

```

Save both node lists to make replacement.

```

6083 \def\bblar@fetchjalt#1#2#3#4{%
6084   \bbl@exp{\bbl@foreach{#1}}{%
6085     \bbl@ifunset{bblar@JE@##1}%
6086     {\setbox\z@\hbox{\texdir TRT ^^^200d\char"##1#2}}%
6087     {\setbox\z@\hbox{\texdir TRT ^^^200d\char"\@nameuse{bblar@JE@##1}#2}}%
6088   \directlua{%
6089     local last = nil
6090     for item in node.traverse(tex.box[0].head) do
6091       if item.id == node.id'glyph' and item.char > 0x600 and
6092         not (item.char == 0x200D) then
6093         last = item
6094       end
6095     end
6096     Babel.arabic.#3['##1#4'] = last.char
6097   }}

```

Elongated forms. Brute force. No rules at all, yet. The ideal: look at jalt table. And perhaps other tables (falt?, cswb?). What about kaf? And diacritic positioning?

```

6098 \gdef\bbl@parsejalt{%
6099   \ifx\addfontfeature\undefined\else
6100     \bbl@xin@{/e}{/\bbl@cl{lnbrk}}%
6101     \ifin@
6102       \directlua{%
6103         if Babel.arabic.elong_map[\the\localeid][\fontid\font] == nil then
6104           Babel.arabic.elong_map[\the\localeid][\fontid\font] = {}
6105           tex.print([[string\cswb\space bbl@parsejalti\endcswb]])
6106         end
6107       }%
6108     \fi
6109   \fi}
6110 \gdef\bbl@parsejalti{%
6111   \begingroup
6112     \let\bbl@parsejalt\relax % To avoid infinite loop
6113     \edef\bbl@tempb{\fontid\font}%
6114     \bblar@nofswarn
6115     \@nameuse{tracinglostchars}\z@
6116     \bblar@fetchjalt\bblar@elongated{}{from}{}%
6117     \bblar@fetchjalt\bblar@chars{^^^064a}{from}{a}% Alef maksura
6118     \bblar@fetchjalt\bblar@chars{^^^0649}{from}{y}% Yeh
6119     \addfontfeature{RawFeature+=jalt}%
6120     % \@namedef{bblar@JE@0643}{06AA}% todo: catch medial kaf
6121     \bblar@fetchjalt\bblar@elongated{}{dest}{}%
6122     \bblar@fetchjalt\bblar@chars{^^^064a}{dest}{a}%
6123     \bblar@fetchjalt\bblar@chars{^^^0649}{dest}{y}%
6124     \directlua{%
6125       for k, v in pairs(Babel.arabic.from) do
6126         if Babel.arabic.dest[k] and
6127           not (Babel.arabic.from[k] == Babel.arabic.dest[k]) then
6128           Babel.arabic.elong_map[\the\localeid][\bbl@tempb]
6129             [Babel.arabic.from[k]] = Babel.arabic.dest[k]
6130         end
6131       end
6132     }%
6133   \endgroup}

```

The actual justification (inspired by CHICKENIZE).

```

6134 \begingroup
6135 \catcode`#=11
6136 \catcode`~=11
6137 \directlua{
6138
6139 Babel.arabic = Babel.arabic or {}

```

```

6140 Babel.arabic.from = {}
6141 Babel.arabic.dest = {}
6142 Babel.arabic.justify_factor = 0.95
6143 Babel.arabic.justify_enabled = true
6144 Babel.arabic.kashida_limit = -1
6145
6146 function Babel.arabic.justify(head)
6147   if not Babel.arabic.justify_enabled then return head end
6148   for line in node.traverse_id(node.id'hlist', head) do
6149     Babel.arabic.justify_hlist(head, line)
6150   end
6151   % In case the very first item is a line (eg, in \vbox):
6152   while head.prev do head = head.prev end
6153   return head
6154 end
6155
6156 function Babel.arabic.justify_hbox(head, gc, size, pack)
6157   local has_inf = false
6158   if Babel.arabic.justify_enabled and pack == 'exactly' then
6159     for n in node.traverse_id(12, head) do
6160       if n.stretch_order > 0 then has_inf = true end
6161     end
6162     if not has_inf then
6163       Babel.arabic.justify_hlist(head, nil, gc, size, pack)
6164     end
6165   end
6166   return head
6167 end
6168
6169 function Babel.arabic.justify_hlist(head, line, gc, size, pack)
6170   local d, new
6171   local k_list, k_item, pos_inline
6172   local width, width_new, full, k_curr, wt_pos, goal, shift
6173   local subst_done = false
6174   local elong_map = Babel.arabic.elong_map
6175   local cnt
6176   local last_line
6177   local GLYPH = node.id'glyph'
6178   local KASHIDA = Babel.attr_kashida
6179   local LOCALE = Babel.attr_locale
6180
6181   if line == nil then
6182     line = {}
6183     line.glue_sign = 1
6184     line.glue_order = 0
6185     line.head = head
6186     line.shift = 0
6187     line.width = size
6188   end
6189
6190   % Exclude last line.
6191   if ((line.next ~= nil or line.glue_sign == 1) and line.glue_order == 0) then
6192     elongs = {} % Stores elongated candidates of each line
6193     k_list = {} % And all letters with kashida
6194     pos_inline = 0 % Not yet used
6195
6196     for n in node.traverse_id(GLYPH, line.head) do
6197       pos_inline = pos_inline + 1 % To find where it is. Not used.
6198
6199       % Elongated glyphs
6200       if elong_map then
6201         local locale = node.get_attribute(n, LOCALE)
6202         if elong_map[locale] and elong_map[locale][n.font] and

```

```

6203         elong_map[locale][n.font][n.char] then
6204         table.insert(elongs, {node = n, locale = locale} )
6205         node.set_attribute(n.prev, KASHIDA, 0)
6206     end
6207 end
6208
6209 % Tatwil. First create a list of nodes marked with kashida. The
6210 % rest of nodes can be ignored. The list of used weights is build
6211 % when transforms with the key kashida= are declared.
6212 if Babel.kashida_wts then
6213     local k_wt = node.get_attribute(n, KASHIDA)
6214     if k_wt > 0 then % todo. parameter for multi inserts
6215         table.insert(k_list, {node = n, weight = k_wt, pos = pos_inline})
6216     end
6217 end
6218
6219 end % of node.traverse_id
6220
6221 if #elongs == 0 and #k_list == 0 then goto next_line end
6222 full = line.width
6223 shift = line.shift
6224 goal = full * Babel.arabic.justify_factor % A bit crude
6225 width = node.dimensions(line.head) % The 'natural' width
6226
6227 % == Elongated ==
6228 % Original idea taken from 'chickenize'
6229 while (#elongs > 0 and width < goal) do
6230     subst_done = true
6231     local x = #elongs
6232     local curr = elongs[x].node
6233     local oldchar = curr.char
6234     curr.char = elong_map[elongs[x].locale][curr.font][curr.char]
6235     width = node.dimensions(line.head) % Check if the line is too wide
6236     % Substitute back if the line would be too wide and break:
6237     if width > goal then
6238         curr.char = oldchar
6239         break
6240     end
6241     % If continue, pop the just substituted node from the list:
6242     table.remove(elongs, x)
6243 end
6244
6245 % == Tatwil ==
6246 % Traverse the kashida node list so many times as required, until
6247 % the line is filled. The first pass adds a tatweel after each
6248 % node with kashida in the line, the second pass adds another one,
6249 % and so on. In each pass, add first the kashida with the highest
6250 % weight, then with lower weight and so on.
6251 if #k_list == 0 then goto next_line end
6252
6253 width = node.dimensions(line.head) % The 'natural' width
6254 k_curr = #k_list % Traverse backwards, from the end
6255 wt_pos = 1
6256
6257 while width < goal do
6258     subst_done = true
6259     k_item = k_list[k_curr].node
6260     if k_list[k_curr].weight == Babel.kashida_wts[wt_pos] then
6261         d = node.copy(k_item)
6262         d.char = 0x0640
6263         d.yoffset = 0 % TODO. From the prev char. But 0 seems safe.
6264         d.xoffset = 0
6265         line.head, new = node.insert_after(line.head, k_item, d)

```

```

6266     width_new = node.dimensions(line.head)
6267     if width > goal or width == width_new then
6268         node.remove(line.head, new) % Better compute before
6269         break
6270     end
6271     if Babel.fix_diacr then
6272         Babel.fix_diacr(k_item.next)
6273     end
6274     width = width_new
6275 end
6276 if k_curr == 1 then
6277     k_curr = #k_list
6278     wt_pos = (wt_pos >= table.getn(Babel.kashida_wts)) and 1 or wt_pos+1
6279 else
6280     k_curr = k_curr - 1
6281 end
6282 end
6283
6284 % Limit the number of tatweel by removing them. Not very efficient,
6285 % but it does the job in a quite predictable way.
6286 if Babel.arabic.kashida_limit > -1 then
6287     cnt = 0
6288     for n in node.traverse_id(GLYPH, line.head) do
6289         if n.char == 0x0640 then
6290             cnt = cnt + 1
6291             if cnt > Babel.arabic.kashida_limit then
6292                 node.remove(line.head, n)
6293             end
6294         else
6295             cnt = 0
6296         end
6297     end
6298 end
6299
6300 ::next_line::
6301
6302 % Must take into account marks and ins, see luatex manual.
6303 % Have to be executed only if there are changes. Investigate
6304 % what's going on exactly.
6305 if subst_done and not gc then
6306     d = node.hpack(line.head, full, 'exactly')
6307     d.shift = shift
6308     node.insert_before(head, line, d)
6309     node.remove(head, line)
6310 end
6311 end % if process line
6312 end
6313 }
6314 \endgroup
6315 \fi\fi % ends Arabic just block: \ifnum\bbl@bidimode>100...

```

11.9. Common stuff

First, a couple of auxiliary macros to set the renderer according to the script. This is done by patching temporarily the low-level fontspec macro containing the current features set with `\defaultfontfeatures`. Admittedly this is somewhat dangerous, but that way the latter command still works as expected, because the renderer is set just before other settings. In xetex they are set to `\relax`.

```

6316 \def\bbl@scr@node@list{%
6317   ,Armenian,Coptic,Cyrillic,Georgian,,Glagolitic,Gothic,%
6318   ,Greek,Latin,Old Church Slavonic Cyrillic,}
6319 \ifnum\bbl@bidimode=102 % bidi-r
6320   \bbl@add\bbl@scr@node@list{Arabic,Hebrew,Syriac}

```

```

6321 \fi
6322 \def\bbl@set@renderer{%
6323   \bbl@xin@{\bbl@cl{sname}}{\bbl@scr@node@list}%
6324   \ifin@
6325     \let\bbl@unset@renderer\relax
6326   \else
6327     \bbl@exp{%
6328       \def\\bbl@unset@renderer{%
6329         \def<g__fontspec_default_fontopts_clist>{%
6330           \[g__fontspec_default_fontopts_clist]}}%
6331       \def<g__fontspec_default_fontopts_clist>{%
6332         Renderer=Harfbuzz,\[g__fontspec_default_fontopts_clist]}}%
6333     \fi}
6334 <@Font selection@>

```

11.10 Automatic fonts and ids switching

After defining the blocks for a number of scripts (must be extended and very likely fine tuned), we define a the function `Babel.locale_map`, which just traverse the node list to carry out the replacements. The table `loc_to_scr` stores the script range for each locale (whose id is the key), copied from this table (so that it can be modified on a locale basis); there is an intermediate table named `chr_to_loc` built on the fly for optimization, which maps a char to the locale. This locale is then used to get the `\language` as stored in `locale_props`, as well as the font (as requested). In the latter table a key starting with / maps the font from the global one (the key) to the local one (the value). Maths are skipped and discretionaries are handled in a special way.

There are two situations where the replacement is not carried out: either the `letters` option has been set and the character is not a letter (in the $\TeX$ sense), or the current script is the same as the new one.

```

6335 \directlua{% DL6
6336 Babel.script_blocks = {
6337   ['dflt'] = {},
6338   ['Arab'] = {{0x0600, 0x06FF}, {0x08A0, 0x08FF}, {0x0750, 0x077F},
6339             {0xFE70, 0xFEFF}, {0xFB50, 0xFDFD}, {0x1EE00, 0x1EEFF}},
6340   ['Armn'] = {{0x0530, 0x058F}},
6341   ['Beng'] = {{0x0980, 0x09FF}},
6342   ['Cher'] = {{0x13A0, 0x13FF}, {0xAB70, 0xABBF}},
6343   ['Copt'] = {{0x03E2, 0x03EF}, {0x2C80, 0x2CFF}, {0x102E0, 0x102FF}},
6344   ['Cyr'] = {{0x0400, 0x04FF}, {0x0500, 0x052F}, {0x1C80, 0x1C8F},
6345             {0x2DE0, 0x2DFF}, {0xA640, 0xA69F}},
6346   ['Deva'] = {{0x0900, 0x097F}, {0xA8E0, 0xA8FF}},
6347   ['Ethi'] = {{0x1200, 0x137F}, {0x1380, 0x139F}, {0x2D80, 0x2DDF},
6348             {0xAB00, 0xAB2F}},
6349   ['Geor'] = {{0x10A0, 0x10FF}, {0x2D00, 0x2D2F}},
6350   % Don't follow strictly Unicode, which places some Coptic letters in
6351   % the 'Greek and Coptic' block
6352   ['Grek'] = {{0x0370, 0x03E1}, {0x03F0, 0x03FF}, {0x1F00, 0x1FFF}},
6353   ['Hans'] = {{0x2E80, 0x2EFF}, {0x3000, 0x303F}, {0x31C0, 0x31EF},
6354             {0x3300, 0x33FF}, {0x3400, 0x4DBF}, {0x4E00, 0x9FFF},
6355             {0xF900, 0xFAFF}, {0xFE30, 0xFE4F}, {0xFF00, 0xFFEF},
6356             {0x20000, 0x2A6DF}, {0x2A700, 0x2B73F},
6357             {0x2B740, 0x2B81F}, {0x2B820, 0x2CEAF},
6358             {0x2CEB0, 0x2EBEF}, {0x2F800, 0x2FA1F}},
6359   ['Hebr'] = {{0x0590, 0x05FF},
6360             {0xFB1F, 0xFB4E}}, % <- Includes some <reserved>
6361   ['Jpan'] = {{0x3000, 0x303F}, {0x3040, 0x309F}, {0x30A0, 0x30FF},
6362             {0x4E00, 0x9FAF}, {0xFF00, 0xFFEF}},
6363   ['Khmr'] = {{0x1780, 0x17FF}, {0x19E0, 0x19FF}},
6364   ['Knda'] = {{0x0C80, 0x0CFF}},
6365   ['Kore'] = {{0x1100, 0x11FF}, {0x3000, 0x303F}, {0x3130, 0x318F},
6366             {0x4E00, 0x9FAF}, {0xA960, 0xA97F}, {0xAC00, 0xD7AF},
6367             {0xD7B0, 0xD7FF}, {0xFF00, 0xFFEF}},
6368   ['Laoo'] = {{0x0E80, 0x0EFF}},
6369   ['Latn'] = {{0x0000, 0x007F}, {0x0080, 0x00FF}, {0x0100, 0x017F},

```

```

6370             {0x0180, 0x024F}, {0x1E00, 0x1EFF}, {0x2C60, 0x2C7F},
6371             {0xA720, 0xA7FF}, {0xAB30, 0xAB6F}},
6372 ['Mahj'] = {{0x11150, 0x1117F}},
6373 ['Mlym'] = {{0x0D00, 0x0D7F}},
6374 ['Mymr'] = {{0x1000, 0x109F}, {0xAA60, 0xAA7F}, {0xA9E0, 0xA9FF}},
6375 ['Orya'] = {{0x0B00, 0x0B7F}},
6376 ['Sinh'] = {{0x0D80, 0x0DFF}, {0x111E0, 0x111FF}},
6377 ['Sycr'] = {{0x0700, 0x074F}, {0x0860, 0x086F}},
6378 ['Taml'] = {{0x0B80, 0x0BFF}},
6379 ['Telu'] = {{0x0C00, 0x0C7F}},
6380 ['Tfng'] = {{0x2D30, 0x2D7F}},
6381 ['Thai'] = {{0x0E00, 0x0E7F}},
6382 ['Tibt'] = {{0x0F00, 0x0FFF}},
6383 ['Vaii'] = {{0xA500, 0xA63F}},
6384 ['Yiii'] = {{0xA000, 0xA48F}, {0xA490, 0xA4CF}}
6385 }
6386
6387 Babel.script_blocks.Cyrs = Babel.script_blocks.Cyrl
6388 Babel.script_blocks.Hant = Babel.script_blocks.Hans
6389 Babel.script_blocks.Kana = Babel.script_blocks.Jpan
6390
6391 function Babel.locale_map(head)
6392   if not Babel.locale_mapped then return head end
6393
6394   local LOCALE = Babel.attr_locale
6395   local GLYPH = node.id('glyph')
6396   local inmath = false
6397   local toloc_save
6398   for item in node.traverse(head) do
6399     local toloc
6400     if not inmath and item.id == GLYPH then
6401       % Optimization: build a table with the chars found
6402       if Babel.chr_to_loc[item.char] then
6403         toloc = Babel.chr_to_loc[item.char]
6404       else
6405         for lc, maps in pairs(Babel.loc_to_scr) do
6406           for _, rg in pairs(maps) do
6407             if item.char >= rg[1] and item.char <= rg[2] then
6408               Babel.chr_to_loc[item.char] = lc
6409               toloc = lc
6410               break
6411             end
6412           end
6413         end
6414         % Treat composite chars in a different fashion, because they
6415         % 'inherit' the previous locale.
6416         if (item.char >= 0x0300 and item.char <= 0x036F) or
6417            (item.char >= 0x1AB0 and item.char <= 0x1AFF) or
6418            (item.char >= 0x1DC0 and item.char <= 0x1DFF) then
6419           Babel.chr_to_loc[item.char] = -2000
6420           toloc = -2000
6421         end
6422         if not toloc then
6423           Babel.chr_to_loc[item.char] = -1000
6424         end
6425       end
6426       if toloc == -2000 then
6427         toloc = toloc_save
6428       elseif toloc == -1000 then
6429         toloc = nil
6430       end
6431       if toloc and Babel.locale_props[toloc] and
6432         Babel.locale_props[toloc].letters and

```

```

6433         tex.getcatcode(item.char) \string~= 11 then
6434         toloc = nil
6435     end
6436     if toloc and Babel.locale_props[toloc].script
6437         and Babel.locale_props[node.get_attribute(item, LOCALE)].script
6438         and Babel.locale_props[toloc].script ==
6439         Babel.locale_props[node.get_attribute(item, LOCALE)].script then
6440         toloc = nil
6441     end
6442     if toloc then
6443         if Babel.locale_props[toloc].lg then
6444             item.lang = Babel.locale_props[toloc].lg
6445             node.set_attribute(item, LOCALE, toloc)
6446         end
6447         if Babel.locale_props[toloc]['/'..item.font] then
6448             item.font = Babel.locale_props[toloc]['/'..item.font]
6449         end
6450     end
6451     toloc_save = toloc
6452     elseif not inmath and item.id == 7 then % Apply recursively
6453         item.replace = item.replace and Babel.locale_map(item.replace)
6454         item.pre      = item.pre and Babel.locale_map(item.pre)
6455         item.post     = item.post and Babel.locale_map(item.post)
6456     elseif item.id == node.id'math' then
6457         inmath = (item.subtype == 0)
6458     end
6459 end
6460 return head
6461 end
6462 }

```

The code for `\babelcharproperty` is straightforward. Just note the modified lua table can be different.

```

6463 \newcommand\babelcharproperty[1]{%
6464   \count@=#1\relax
6465   \ifvmode
6466     \expandafter\bbl@chprop
6467   \else
6468     \bbl@error{charproperty-only-vertical}{}{}{}%
6469   \fi}
6470 \newcommand\bbl@chprop[3][\the\count@]{%
6471   \@tempcnta=#1\relax
6472   \bbl@ifunset{bbl@chprop@#2}% {unknown-char-property}
6473   {\bbl@error{unknown-char-property}{}{#2}{}%
6474   {}%
6475   \loop
6476     \bbl@cs{chprop@#2}{#3}%
6477     \ifnum\count@<\@tempcnta
6478       \advance\count@\@ne
6479     \repeat}
6480 %
6481 \def\bbl@chprop@direction#1{%
6482   \directlua{
6483     Babel.characters[\the\count@] = Babel.characters[\the\count@] or {}
6484     Babel.characters[\the\count@]['d'] = '#1'
6485   }}
6486 \let\bbl@chprop@bc\bbl@chprop@direction
6487 %
6488 \def\bbl@chprop@mirror#1{%
6489   \directlua{
6490     Babel.characters[\the\count@] = Babel.characters[\the\count@] or {}
6491     Babel.characters[\the\count@]['m'] = '\number#1'
6492   }}

```

```

6493 \let\bbl@chprop@bmg\bbl@chprop@mirror
6494 %
6495 \def\bbl@chprop@linebreak#1{%
6496   \directlua{
6497     Babel.cjk_characters[\the\count@] = Babel.cjk_characters[\the\count@] or {}
6498     Babel.cjk_characters[\the\count@]['c'] = '#1'
6499   }}
6500 \let\bbl@chprop@lb\bbl@chprop@linebreak
6501 %
6502 \def\bbl@chprop@locale#1{%
6503   \directlua{
6504     Babel.chr_to_loc = Babel.chr_to_loc or {}
6505     Babel.chr_to_loc[\the\count@] =
6506       \bbl@ifblank{#1}{-1000}{\the\bbl@cs{id@#1}}\space
6507   }}

```

Post-handling hyphenation patterns for non-standard rules, like ff to ff-f. There are still some issues with speed (not very slow, but still slow). The Lua code is below.

```

6508 \directlua{% DL7
6509   Babel.nohyphenation = \the\l@nohyphenation
6510 }

```

Now the T_EX high level interface, which requires the function defined above for converting strings to functions returning a string. These functions handle the {*n*} syntax. For example, pre={1}{1}- becomes function(*m*) return *m*[1]..*m*[1]..'-' end, where *m* are the matches returned after applying the pattern. With a mapped capture the functions are similar to function(*m*) return Babel.capt_map(*m*[1],1) end, where the last argument identifies the mapping to be applied to *m*[1]. The way it is carried out is somewhat tricky, but the effect is not dissimilar to lua load – save the code as string in a TeX macro, and expand this macro at the appropriate place. As \directlua does not take into account the current catcode of @, we just avoid this character in macro names (which explains the internal group, too).

```

6511 \begingroup
6512 \catcode`\~ = 12
6513 \catcode`\% = 12
6514 \catcode`\& = 14
6515 \catcode`\| = 12
6516 \gdef\babelprehyphenation{%&
6517   \@ifnextchar[{\bbl@settransform{0}}{\bbl@settransform{0}}{}}
6518 \gdef\babelposthyphenation{%&
6519   \@ifnextchar[{\bbl@settransform{1}}{\bbl@settransform{1}}{}}
6520 %
6521 \gdef\bbl@settransform#1[#2]#3#4#5{%&
6522   \ifcase#1
6523     \bbl@activateprehyphen
6524   \or
6525     \bbl@activateposthyphen
6526   \fi
6527 \begingroup
6528   \def\babeltempa{\bbl@add@list\babeltempb}%&
6529   \let\babeltempb\empty
6530   \def\bbl@tempa{#5}%&
6531   \bbl@replace\bbl@tempa{,}{ ,}%& TODO. Ugly trick to preserve {}
6532   \expandafter\bbl@foreach\expandafter{\bbl@tempa}{%&
6533     \bbl@ifsamestring{##1}{remove}%&
6534     {\bbl@add@list\babeltempb{nil}}}%&
6535   \directlua{
6536     local rep = [= [#1] =]
6537     local three_args = '%s*=%s*([%-d%.%a{}|]+)%s+([%-d%.%a{}|]+)%s+([%-d%.%a{}|]+)'
6538     & Numeric passes directly: kern, penalty...
6539     rep = rep:gsub('^%s*(remove)%s*$', 'remove = true')
6540     rep = rep:gsub('^%s*(insert)%s*', ', 'insert = true, ')
6541     rep = rep:gsub('^%s*(after)%s*', ', 'after = true, ')
6542     rep = rep:gsub('(string)%s*=%s*([%^s,]*)', Babel.capture_func)
6543     rep = rep:gsub('node%s*=%s*([%a+])%s*([%a+])', Babel.capture_node)

```

```

6544     rep = rep:gsub( '(norule)' .. three_args,
6545         'norule = {' .. '%2, %3, %4' .. '}' )
6546     if #1 == 0 or #1 == 2 then
6547         rep = rep:gsub( '(space)' .. three_args,
6548             'space = {' .. '%2, %3, %4' .. '}' )
6549         rep = rep:gsub( '(spacefactor)' .. three_args,
6550             'spacefactor = {' .. '%2, %3, %4' .. '}' )
6551         rep = rep:gsub( '(kashida)%s*=%s*([^\s,]*)', Babel.capture_kashida)
6552         &% Transform values
6553         rep, n = rep:gsub( '{{[%a%-%.]+}|([%a%_%.]+)}',
6554             function(v,d)
6555                 return string.format (
6556                     '{\the\csname bbl@id@@#3\endcsname,"%s",%s}',
6557                     v,
6558                     load( 'return Babel.locale_props'..
6559                         '[\the\csname bbl@id@@#3\endcsname]. ' .. d)() )
6560             end )
6561         rep, n = rep:gsub( '{{[%a%-%.]+}|([%-d%_.]+)}',
6562             '{\the\csname bbl@id@@#3\endcsname,"%1",%2}')
6563     end
6564     if #1 == 1 then
6565         rep = rep:gsub( '(no)%s*=%s*([^\s,]*)', Babel.capture_func)
6566         rep = rep:gsub( '(pre)%s*=%s*([^\s,]*)', Babel.capture_func)
6567         rep = rep:gsub( '(post)%s*=%s*([^\s,]*)', Babel.capture_func)
6568     end
6569     tex.print([[ \string\babeltempa{}} .. rep .. {}]])
6570 }}&%
6571 \bbl@foreach\babeltempb{&%
6572     \bbl@forkv{##1}}{&%
6573         \in@{,###1,}{,nil,step,data,remove,insert,string,no,pre,no,&%
6574             post,penalty,kashida,space,spacefactor,kern,node,after,norule,}&%
6575         \ifin@else
6576             \bbl@error{bad-transform-option}{###1}{,}{&%
6577                 \fi}}&%
6578 \let\bbl@kv@attribute\relax
6579 \let\bbl@kv@label\relax
6580 \let\bbl@kv@fonts\empty
6581 \let\bbl@kv@prepend\relax
6582 \bbl@forkv{#2}{\bbl@csarg\edef{kv##1}{##2}}&%
6583 \ifx\bbl@kv@fonts\empty\else\bbl@settransfont\fi
6584 \ifx\bbl@kv@attribute\relax
6585     \ifx\bbl@kv@label\relax\else
6586         \bbl@exp{\bbl@trim@def\bbl@kv@fonts{\bbl@kv@fonts}}&%
6587         \bbl@replace\bbl@kv@fonts{ }{,}&%
6588         \edef\bbl@kv@attribute{\bbl@ATR@\bbl@kv@label @#3@\bbl@kv@fonts}&%
6589         \count@ \z@
6590         \def\bbl@elt##1##2##3{&%
6591             \bbl@ifsamestring{#3,\bbl@kv@label}{##1,##2}&%
6592             {\bbl@ifsamestring{\bbl@kv@fonts}{##3}&%
6593                 {\count@ \ne}&%
6594                 {\bbl@error{font-conflict-transforms}{,}{,}}}&%
6595             {}}&%
6596     \bbl@transfont@list
6597     \ifnum\count@=\z@
6598         \bbl@exp{\global\bbl@add\bbl@transfont@list
6599             {\bbl@elt{#3}{\bbl@kv@label}{\bbl@kv@fonts}}}&%
6600     \fi
6601     \bbl@ifunset{\bbl@kv@attribute}&%
6602         {\global\bbl@carg\newattribute{\bbl@kv@attribute}}&%
6603         {}&%
6604     \global\bbl@carg\setattribute{\bbl@kv@attribute}\@ne
6605     \fi
6606 \else

```

```

6607     \edef\bbl@kv@attribute{\expandafter\bbl@stripslash\bbl@kv@attribute}&%
6608     \fi
6609     \directlua{
6610         local lbkr = Babel.linebreaking.replacements[#1]
6611         local u = unicode.utf8
6612         local id, attr, label
6613         if #1 == 0 then
6614             id = \the\csname bbl@id@@#3\endcsname\space
6615         else
6616             id = \the\csname l@#3\endcsname\space
6617         end
6618         \ifx\bbl@kv@attribute\relax
6619             attr = -1
6620         \else
6621             attr = luatexbase.registernumber'\bbl@kv@attribute'
6622         \fi
6623         \ifx\bbl@kv@label\relax\else &% Same refs:
6624             label = [==[\bbl@kv@label]==]
6625         \fi
6626         &% Convert pattern:
6627         local patt = string.gsub([==[#4]==], '%s', '')
6628         if #1 == 0 then
6629             patt = string.gsub(patt, '|', ' ')
6630         end
6631         if not u.find(patt, '()', nil, true) then
6632             patt = '()' .. patt .. '()'
6633         end
6634         patt = string.gsub(patt, '%(%)%^', '^()')
6635         patt = string.gsub(patt, '%$$(%)', '()$')
6636         patt = u.gsub(patt, '{(.)}',
6637             function (n)
6638                 return '%' .. (tonumber(n) and (tonumber(n)+1) or n)
6639             end)
6640         patt = u.gsub(patt, '{(%X%X%X%X+)}',
6641             function (n)
6642                 return u.gsub(u.char(tonumber(n, 16)), '(%p)', '%%1')
6643             end)
6644         lbkr[id] = lbkr[id] or {}
6645         table.insert(lbkr[id], \ifx\bbl@kv@prepend\relax\else 1,\fi
6646             { label=label, attr=attr, pattern=patt, replace={\babeltempb} })
6647     }&%
6648     \endgroup}
6649 \endgroup
6650 %
6651 \let\bbl@transfont@list\@empty
6652 \def\bbl@settransfont{%
6653     \global\let\bbl@settransfont\relax % Execute only once
6654     \gdef\bbl@transfont{%
6655         \def\bbl@elt####1####2####3{%
6656             \bbl@ifblank{####3}%
6657             {\count@tw@}% Do nothing if no fonts
6658             {\count@z@
6659             \bbl@vforeach{####3}{%
6660                 \def\bbl@tempd{#####1}%
6661                 \edef\bbl@tempe{\bbl@transfam/\f@series/\f@shape}%
6662                 \ifx\bbl@tempd\bbl@tempe
6663                     \count@\@ne
6664                 \else\ifx\bbl@tempd\bbl@transfam
6665                     \count@\@ne
6666                 \fi\fi}%
6667                 \ifcase\count@
6668                     \bbl@csarg\unsetattribute{ATR@####2@####1@####3}%
6669                 \or

```

```

6670         \bbl@csarg\setattribute{ATR@###2@###1@###3}\@ne
6671     \fi}}%
6672     \bbl@transfont@list}%
6673     \AddToHook{selectfont}{\bbl@transfont}% Hooks are global.
6674     \gdef\bbl@transfam{-unknown-}%
6675     \bbl@foreach\bbl@font@fams{%
6676         \AddToHook{##1family}{\def\bbl@transfam{##1}}%
6677         \bbl@ifsamestring{\@nameuse{##1default}}\familydefault
6678         {\xdef\bbl@transfam{##1}}%
6679     }}}}
6680 %
6681 \DeclareRobustCommand\enablelocaletransform[1]{%
6682     \bbl@ifunset\bbl@ATR@#1@language\@language @}%
6683     {\bbl@error{transform-not-available}{#1}}}%
6684     {\bbl@csarg\setattribute{ATR@#1@language @}\@language}}
6685 \DeclareRobustCommand\disablelocaletransform[1]{%
6686     \bbl@ifunset\bbl@ATR@#1@language\@language @}%
6687     {\bbl@error{transform-not-available-b}{#1}}}%
6688     {\bbl@csarg\unsetAttribute{ATR@#1@language @}}%

```

The following two macros load the Lua code for transforms, but only once. The only difference is in `add_after` and `add_before`.

```

6689 \def\bbl@activateposthyphen{%
6690     \let\bbl@activateposthyphen\relax
6691     \ifx\bbl@attr@hboxed\undefined
6692         \newattribute\bbl@attr@hboxed
6693     \fi
6694     \directlua{
6695         require('babel-transforms.lua')
6696         Babel.linebreaking.add_after(Babel.post_hyphenate_replace)
6697     }%
6698 \def\bbl@activateprehyphen{%
6699     \let\bbl@activateprehyphen\relax
6700     \ifx\bbl@attr@hboxed\undefined
6701         \newattribute\bbl@attr@hboxed
6702     \fi
6703     \directlua{
6704         require('babel-transforms.lua')
6705         Babel.linebreaking.add_before(Babel.pre_hyphenate_replace)
6706     }%
6707 \newcommand\SetTransformValue[3]{%
6708     \directlua{
6709         Babel.locale_props[\the\csname bbl@id@#1\endcsname].vars["#2"] = #3
6710     }%

```

The code in `babel-transforms.lua` prints at some points the current string being transformed. This macro first make sure this file is loaded. Then, activates temporarily this feature and typeset inside a box the text in the argument.

```

6711 \newcommand\ShowBabelTransforms[1]{%
6712     \bbl@activateprehyphen
6713     \bbl@activateposthyphen
6714     \begingroup
6715         \directlua{ Babel.show_transforms = true }%
6716         \setbox\z@\vbox{#1}%
6717         \directlua{ Babel.show_transforms = false }%
6718     \endgroup}

```

The following experimental (and unfinished) macro applies the prehyphenation transforms for the current locale to a string (characters and spaces) and processes it in a fully expandable way (among other limitations, the string can't contain `]`). The way it operates is admittedly rather cumbersome: it converts the string to a node list, processes it, and converts it back to a string. The lua code is in the lua file below.

```

6719 \newcommand\localeprehyphenation[1]{%
6720     \directlua{ Babel.string_prehyphenation([==#1==], \the\localeid) }}

```

11.11.Bidi

Make sure the `\pagedir` is always the same as the `\bodydir` when the document starts. At this point the main language has been selected, and therefore its `\bodydir` has been set.

```
6721 \AtBeginDocument{\pagedir\bodydir}
```

As a first step, add a handler for bidi and digits (and potentially other processes) just before `luaotfload` is applied, which is loaded by default by `ℒTEX`. Just in case, consider the possibility it has not been loaded.

```
6722 \def\bbl@activate@preotf{%
6723   \let\bbl@activate@preotf\relax % only once
6724   \directlua{
6725     function Babel.pre_otfload_v(head)
6726       if Babel.numbers and Babel.digits_mapped then
6727         head = Babel.numbers(head)
6728       end
6729       if Babel.bidi_enabled then
6730         head = Babel.bidi(head, false, dir)
6731       end
6732       return head
6733     end
6734   %
6735   function Babel.pre_otfload_h(head, gc, sz, pt, dir)
6736     if Babel.numbers and Babel.digits_mapped then
6737       head = Babel.numbers(head)
6738     end
6739     if Babel.bidi_enabled then
6740       head = Babel.bidi(head, false, dir)
6741     end
6742     return head
6743   end
6744   %
6745   luatexbase.add_to_callback('pre_linebreak_filter',
6746     Babel.pre_otfload_v,
6747     'Babel.pre_otfload_v',
6748     Babel.priority_in_callback('pre_linebreak_filter',
6749       'luaotfload.node_processor') or nil)
6750   %
6751   luatexbase.add_to_callback('hpack_filter',
6752     Babel.pre_otfload_h,
6753     'Babel.pre_otfload_h',
6754     Babel.priority_in_callback('hpack_filter',
6755       'luaotfload.node_processor') or nil)
6756   }}
```

The basic setup. The output is modified at a very low level to set the `\bodydir` to the `\pagedir`. Sadly, we have to deal with boxes in math with basic, so the `\bbl@insidemath` hack is activated (set to 1) every math with the package option `bidi=`. The hack for the PUA is no longer necessary with `basic` (24.8), but it's kept in `basic-r`.

```
6757 \ifnum\bbl@bidimode>\@ne % Any bidi= except default (=1)
6758   \let\bbl@beforeforeign\leavevmode
6759   \AtEndOfPackage{\EnableBabelHook{babel-bidi}}
6760 \ifnum\bbl@bidimode>3000
6761   \RequirePackage{unibidi-lua}
6762 \else
6763   \RequirePackage{luatexbase}
6764   \bbl@activate@preotf
6765   \directlua{
6766     require('babel-data-bidi.lua')
6767     \ifcase\expandafter\@gobbletwo\the\bbl@bidimode\or
6768       require('babel-bidi-basic.lua')
6769     \or
6770       require('babel-bidi-basic-r.lua')
6771     table.insert(Babel.ranges, {0xE000, 0xF8FF, 'on'})
6772   }
```

```

6772         table.insert(Babel.ranges, {0xF0000, 0xFFFFD, 'on'})
6773         table.insert(Babel.ranges, {0x100000, 0x10FFFFD, 'on'})
6774     \fi}
6775 \fi
6776 \newattribute\bbl@attr@dir
6777 \directlua{ Babel.attr_dir = luatexbase.registernumber'bbl@attr@dir' }
6778 \bbl@exp{\output{\bodydir\pagedir\the\output}}
6779 \fi
6780 %
6781 \chardef\bbl@thetextdir\z@
6782 \chardef\bbl@thepardir\z@
6783 \def\bbl@setluadir#1#2{% l=\text/pardirection 2= 0:l 1:r 2:a1
6784 \ifcase#2\relax
6785 \ifcase#1\else#1=\z@\fi
6786 \else
6787 \ifcase#1#1=\@ne\fi
6788 \fi}

    \bbl@attr@dir stores the directions with a mask: ..00PPTT, with masks 0xC (PP is the par dir) and
    0x3 (TT is the text dir). These macro names are shared by the 3 engines, with different definitions.

6789 \def\bbl@thedir{0}
6790 \def\bbl@textdir#1{%
6791 \bbl@setluadir\textdirection{#1}%
6792 \chardef\bbl@thetextdir#1\relax
6793 \edef\bbl@thedir{\the\numexpr\bbl@thepardir*4+#1}%
6794 \setattribute\bbl@attr@dir{\numexpr\bbl@thepardir*4+#1}}
6795 \def\bbl@pardir#1{% Used twice
6796 \bbl@setluadir\pardirection{#1}%
6797 \chardef\bbl@thepardir#1\relax}
6798 \def\bbl@bodydir{\bbl@setluadir\bodydirection}% Used once
6799 \def\bbl@dirparastext{\pardirection=\textdirection\relax}% Used once

    RTL text inside math needs special attention. It affects not only to actual math stuff, but also to
    ‘tabular’, which is based on a fake math.

6800 \ifnum\bbl@bidimode>\z@ % Any bidi=
6801 \def\bbl@insidemath{0}%
6802 \def\bbl@everymath{\def\bbl@insidemath{1}}
6803 \def\bbl@everydisplay{\def\bbl@insidemath{2}}
6804 \frozen@everymath\expandafter{%
6805 \expandafter\bbl@everymath\the\frozen@everymath}
6806 \frozen@everydisplay\expandafter{%
6807 \expandafter\bbl@everydisplay\the\frozen@everydisplay}
6808 \AtBeginDocument{
6809 \directlua{
6810     function Babel.math_box_dir(head)
6811         if not (token.get_macro('bbl@insidemath') == '0') then
6812             if Babel.hlist_has_bidi(head) then
6813                 local d = node.new(node.id'dir')
6814                 d.dir = '+TRT'
6815                 for item in node.traverse(head) do
6816                     if item.id == 11 or item.id == node.id'glyph' then
6817                         head = node.insert_before(head, item, d)
6818                         break
6819                     end
6820                 end
6821                 local inmath = false
6822                 for item in node.traverse(head) do
6823                     if item.id == 11 then
6824                         inmath = (item.subtype == 0)
6825                     elseif not inmath then
6826                         node.set_attribute(item,
6827                             Babel.attr_dir, token.get_macro('bbl@thedir'))
6828                     end
6829                 end

```

```

6830         end
6831     end
6832     return head
6833 end
6834 luatexbase.add_to_callback("hpack_filter", Babel.math_box_dir,
6835     "Babel.math_box_dir", 0)
6836 if Babel.unset_atdir then
6837     luatexbase.add_to_callback("pre_linebreak_filter", Babel.unset_atdir,
6838         "Babel.unset_atdir")
6839     luatexbase.add_to_callback("hpack_filter", Babel.unset_atdir,
6840         "Babel.unset_atdir")
6841 end
6842 luatexbase.add_to_callback("post_mlist_to_hlist_filter", function(head)
6843     local dir = tex.mathdirection
6844     local n = node.new("dir")
6845     n.direction = dir
6846     head = node.insert_before(head, head, n)
6847     n = node.new("dir", 1)
6848     n.direction = dir
6849     head = node.insert_after(head, node.tail(head), n)
6850     return head
6851 end, "Babel.ensure_math_dir")
6852 }%
6853 \fi

Experimental. Tentative name.

6854 \DeclareRobustCommand\localebox[1]{%
6855     {\def\bb@insidemath{0}%
6856         \mbox{\foreignlanguage{\language}\{#1\}}}}

```

11.12 Layout

Unlike xetex, luatex requires only minimal changes for right-to-left layouts, particularly in monolingual documents (the engine itself reverses boxes – including column order or headings –, margins, etc.) with `bidibasic`, without having to patch almost any macro where text direction is relevant.

Still, there are three areas deserving special attention, namely, tabular, math, and graphics, text and intrinsically left-to-right elements are intermingled. I’ve made some progress in graphics, but they’re essentially hacks; I’ve also made some progress in ‘tabular’, but when I decided to tackle math (both standard math and ‘amsmath’) the nightmare began. I’m still not sure how ‘amsmath’ should be modified, but the main problem is that, boxes are “generic” containers that can hold text, math, and graphics (even at the same time; remember that inline math is included in the list of text nodes marked with ‘math’ (11) nodes too).

`\@hangfrom` is useful in many contexts and it is redefined always with the `layout` option.

There are, however, a number of issues when the text direction is not the same as the box direction (as set by `\bodydir`), and when `\parbox` and `\hangindent` are involved. Fortunately, latest releases of luatex simplify a lot the solution with `\shapemode`.

With the issue #15 I realized commands are best patched, instead of redefined. With a few lines, a modification could be applied to several classes and packages. Now, `tabular` seems to work (at least in simple cases) with `array`, `tabularx`, `hhline`, `colortbl`, `longtable`, `booktabs`, etc. However, `dcolum` still fails.

```

6857 \bb@trace{Redefinitions for bidi layout}
6858 %
6859 <{*More package options}> ≡
6860 \chardef\bb@eqnpos\z@
6861 \DeclareOption{leqno}{\chardef\bb@eqnpos\@ne}
6862 \DeclareOption{fleqn}{\chardef\bb@eqnpos\tw@}
6863 <{/More package options}>

```

Fixes in luatex are sometimes done with flags. We first activate all all them.

```

6864 \ifnum\bb@bidimode>\z@ % Any bidi=
6865     \matheqdirmode\@ne
6866     \mathemptydisplaymode\@ne

```

```

6867 \breakafterdirmode\@ne
6868 \fixupboxesmode\@ne
6869 \let\bbl@eqnodir\relax
6870 \def\bbl@eqdel{()}
6871 \def\bbl@eqnum{%
6872   {\normalfont\normalcolor
6873     \expandafter\@firstoftwo\bbl@eqdel
6874     \theequation
6875     \expandafter\@secondoftwo\bbl@eqdel}}
6876 \def\bbl@puteqno#1{\eqno\hbox{#1}}
6877 \def\bbl@putleqno#1{\leqno\hbox{#1}}
6878 \def\bbl@eqno@flip#1{% So
6879   \ifdim\predisplaysize=-\maxdimen % For consecutive displays
6880     \eqno
6881     \hb@xt@.01pt{%
6882       \hb@xt@\displaywidth{\hss{#1}\glet\bbl@upset\@currentlabel}}\hss}%
6883   \else
6884     \leqno\hbox{#1}\glet\bbl@upset\@currentlabel}%
6885   \fi
6886   \bbl@exp{\def\\@currentlabel{\bbl@upset}}}}
6887 \def\bbl@leqno@flip#1{%
6888   \ifdim\predisplaysize=-\maxdimen % For consecutive displays
6889     \leqno
6890     \hb@xt@.01pt{%
6891       \hss\hb@xt@\displaywidth{#1}\glet\bbl@upset\@currentlabel}\hss}%
6892   \else
6893     \eqno\hbox{#1}\glet\bbl@upset\@currentlabel}%
6894   \fi
6895   \bbl@exp{\def\\@currentlabel{\bbl@upset}}}}
6896 %
6897 \AtBeginDocument{%
6898   \ifx\bbl@noamsmath\relax\else
6899     \ifx\maketag@@@\@undefined % Normal equation, eqnarray
6900       \ifnum\bbl@eqnpos=\tw@
6901         \bbl@replace\equation{\hb@xt@\linewidth}%
6902           {\hbox bdir\mathdirection to\linewidth}%
6903         \bbl@carg\bbl@sreplace{[ ]}{\hb@xt@\linewidth}%
6904           {\hbox bdir\mathdirection to\linewidth}%
6905       \fi
6906       \AddToHook{env/equation/begin}{%
6907         \ifnum\bbl@thetextdir>z@
6908           \let\@eqnnum\bbl@eqnum
6909           \edef\bbl@eqnodir{\noexpand\bbl@textdir{\the\bbl@thetextdir}}%
6910           \chardef\bbl@thetextdir\z@
6911           \bbl@add\normalfont{\bbl@eqnodir}%
6912           \ifcase\bbl@eqnpos
6913             \let\bbl@puteqno\bbl@eqno@flip
6914           \or
6915             \let\bbl@puteqno\bbl@leqno@flip
6916           \fi
6917         \fi}%
6918       \ifnum\bbl@eqnpos=\tw@\else
6919         \def\endequation{\bbl@puteqno{\@eqnnum}$$\@ignoretrue}%
6920       \fi
6921       \AddToHook{env/eqnarray/begin}{%
6922         \ifnum\bbl@thetextdir>z@
6923           \edef\bbl@eqnodir{\noexpand\bbl@textdir{\the\bbl@thetextdir}}%
6924           \chardef\bbl@thetextdir\z@
6925           \bbl@add\normalfont{\bbl@eqnodir}%
6926         \ifnum\bbl@eqnpos=\@ne
6927           \def\@eqnnum{%
6928             \setbox\z@\hbox{\bbl@eqnum}%
6929             \hbox to0.01pt{\hss\hbox to\displaywidth{\box\z@\hss}}}%

```

```

6930         \else
6931         \let\@eqnnum\bbledqnum
6932         \fi
6933     \fi}
6934 \else % amstex
6935     \bbl@exp{% Hack to hide maybe undefined conditionals:
6936         \chardef\bbledqnpos=0<\iftagsleft@>1<\else>\<\if@fleqn>2<\fi>\<\fi>\relax}%
6937     \ifnum\bbledqnpos=\@ne
6938         \let\bbledams@lap\hbox
6939     \else
6940         \let\bbledams@lap\llap
6941     \fi
6942     \ExplSyntaxOn % Required by \bbl@sreplace with \intertext@
6943     \bbl@sreplace\intertext@\normalbaselines}%
6944     {\normalbaselines
6945     \ifx\bbledqnodir\relax\else\bbl@pdir\@ne\bbl@eqnodir\fi}%
6946     \ExplSyntaxOff
6947     \def\bbledams@tagbox#1#2{\#1{\bbledqnodir#2}}% #1=hbox|@lap|flip
6948     \ifx\bbledams@lap\hbox % leqno
6949         \def\bbledams@flip#1{%
6950             \hbox to 0.01pt{\hss\hbox to\displaywidth{\#1}\hss}}%
6951     \else % eqno
6952         \def\bbledams@flip#1{%
6953             \hbox to 0.01pt{\hbox to\displaywidth{\hss{\#1}\hss}}%
6954         \fi
6955     \def\bbledams@preset#1{%
6956         \ifnum\bbledthetextdir>\z@
6957             \edef\bbledqnodir{\noexpand\bbledtextdir{\the\bbledthetextdir}}%
6958             \bbl@sreplace\textdef@{\hbox}{\bbledams@tagbox\hbox}%
6959             \bbl@sreplace\maketag@@@{\hbox}{\bbledams@tagbox#1}%
6960         \fi}%
6961     \def\bbledams@equation{%
6962         \ifnum\bbledthetextdir>\z@
6963             \bbledams@preset\hbox
6964             \chardef\bbledthetextdir\z@
6965             \bbl@add\normalfont{\bbledqnodir}%
6966             \ifcase\bbledqnpos
6967                 \def\veqno##1##2{\bbledqno@flip{##1##2}}%
6968             \or
6969                 \def\veqno##1##2{\bbledqno@flip{##1##2}}%
6970             \fi
6971         \fi}%
6972     \AddToHook{env/equation/begin}{\bbledams@equation}%
6973     \AddToHook{env/equation*/begin}{\bbledams@equation}%
6974     \AddToHook{env/cases/begin}{\bbledams@preset\bbledams@lap}%
6975     \AddToHook{env/multline/begin}{\bbledams@preset\hbox}%
6976     \AddToHook{env/gather/begin}{\bbledams@preset\bbledams@lap}%
6977     \AddToHook{env/gather*/begin}{\bbledams@preset\bbledams@lap}%
6978     \AddToHook{env/align/begin}{\bbledams@preset\bbledams@lap}%
6979     \AddToHook{env/align*/begin}{\bbledams@preset\bbledams@lap}%
6980     \AddToHook{env/alignat/begin}{\bbledams@preset\bbledams@lap}%
6981     \AddToHook{env/alignat*/begin}{\bbledams@preset\bbledams@lap}%
6982     \AddToHook{env/eqnalign/begin}{\bbledams@preset\hbox}%
6983     % Hackish, for proper alignment. Don't ask me why it works!:
6984     \bbl@exp{% Avoid a 'visible' conditional
6985         \\AddToHook{env/align*/end}{\<\iftag@>\<\else>\\tag*{\<\fi>}}%
6986         \\AddToHook{env/alignat*/end}{\<\iftag@>\<\else>\\tag*{\<\fi>}}%
6987     \AddToHook{env/flalign/begin}{\bbledams@preset\hbox}%
6988     \AddToHook{env/split/before}{%
6989         \ifnum\bbledthetextdir>\z@
6990             \bbl@ifsamestring\@currentvir{equation}%
6991             {\ifx\bbledams@lap\hbox % leqno
6992                 \def\bbledams@flip#1{%

```

```

6993         \hbox to 0.01pt{\hbox to\displaywidth{#{#1}\hss}\hss}}%
6994     \else
6995         \def\bbl@ams@flip#1{%
6996             \hbox to 0.01pt{\hss\hbox to\displaywidth{\hss{#1}}}%
6997         \fi}%
6998     {}%
6999     \fi}%
7000 \fi\fi}
7001 \fi

Declarations specific to lua, called by \babelprovide.

7002 \def\bbl@provide@extra#1{%
7003     % == onchar ==
7004     \ifx\bbl@KVP@onchar\@nnil\else
7005         \bbl@luahyphenate
7006         \bbl@exp{%
7007             \\AddToHook{env/document/before}{%
7008                 {\let\\bbl@ifrestoring\\@firstoftwo
7009                     \\select@language{#1}}}%
7010         \directlua{
7011             if Babel.locale_mapped == nil then
7012                 Babel.locale_mapped = true
7013                 Babel.linebreaking.add_before(Babel.locale_map, 1)
7014                 Babel.loc_to_scr = {}
7015                 Babel.chr_to_loc = Babel.chr_to_loc or {}
7016             end
7017             Babel.locale_props[\the\localeid].letters = false
7018         }%
7019         \bbl@xin@{ letters }{ \bbl@KVP@onchar\space}%
7020     \ifin@
7021         \directlua{
7022             Babel.locale_props[\the\localeid].letters = true
7023         }%
7024     \fi
7025     \bbl@xin@{ ids }{ \bbl@KVP@onchar\space}%
7026     \ifin@
7027         \ifx\bbl@starthyphens\@undefined % Needed if no explicit selection
7028             \AddBabelHook{babel-onchar}{beforestart}{\bbl@starthyphens}}%
7029         \fi
7030         \bbl@exp{\\bbl@add\\bbl@starthyphens
7031             {\\bbl@patterns@lua{\language#1}}}%
7032         \directlua{
7033             if Babel.script_blocks['\bbl@cl{sbc}'] then
7034                 Babel.loc_to_scr[\the\localeid] = Babel.script_blocks['\bbl@cl{sbc}']
7035                 Babel.locale_props[\the\localeid].lg = \the\@nameuse{l\language#1}\space
7036             end
7037         }%
7038     \fi
7039     \bbl@xin@{ fonts }{ \bbl@KVP@onchar\space}%
7040     \ifin@
7041         \bbl@ifunset{bbl@sys@\language#1}{\bbl@provide@sys{\language#1}}{%
7042             \bbl@ifunset{bbl@wdir@\language#1}{\bbl@provide@dirs{\language#1}}{%
7043                 \directlua{
7044                     if Babel.script_blocks['\bbl@cl{sbc}'] then
7045                         Babel.loc_to_scr[\the\localeid] =
7046                             Babel.script_blocks['\bbl@cl{sbc}']
7047                     end}%
7048                 \ifx\bbl@mapselect\@undefined
7049                     \AtBeginDocument{%
7050                         \bbl@patchfont{\bbl@mapselect}}%
7051                     {\selectfont}}%
7052                 \def\bbl@mapselect{%
7053                     \let\bbl@mapselect\relax

```

```

7054     \edef\bbl@prefontid{\fontid\font}}%
7055 \def\bbl@mapdir##1{%
7056     \begingroup
7057     \setbox\z@\hbox{% Force text mode
7058     \def\language{##1}%
7059     \let\bbl@ifrestoring\@firstoftwo % To avoid font warning
7060     \bbl@switchfont
7061     \ifnum\fontid\font>\z@ % A hack, for the pgf nullfont hack
7062     \directlua{
7063         Babel.locale_props[\the\csname bbl@id@##1\endcsname]%
7064         ['/bbl@prefontid'] = \fontid\font\space}%
7065     \fi}%
7066     \endgroup}%
7067 \fi
7068 \bbl@exp{\bbl@add\bbl@mapselect{\bbl@mapdir{\language}}}%
7069 \fi
7070 \fi
7071 % == mapfont ==
7072 % For bidi texts, to switch the font based on direction. Deprecated
7073 \ifx\bbl@KVP@mapfont\@nnil\else
7074     \bbl@ifsamestring{\bbl@KVP@mapfont}{direction}}{%
7075         {\bbl@error{unknown-mapfont}}}%
7076     \bbl@ifunset{\bbl@lsys\language}{\bbl@provide@lsys{\language}}{%
7077     \bbl@ifunset{\bbl@wdir\language}{\bbl@provide@dirs{\language}}{%
7078     \ifx\bbl@mapselect\@undefined
7079         \AtBeginDocument{%
7080             \bbl@patchfont{\bbl@mapselect}}%
7081             {\selectfont}}%
7082     \def\bbl@mapselect{%
7083         \let\bbl@mapselect\relax
7084         \edef\bbl@prefontid{\fontid\font}}%
7085     \def\bbl@mapdir##1{%
7086         {\def\language{##1}%
7087         \let\bbl@ifrestoring\@firstoftwo % avoid font warning
7088         \bbl@switchfont
7089         \directlua{Babel.fontmap
7090             [\the\csname bbl@wdir@##1\endcsname]%
7091             [\bbl@prefontid]=\fontid\font}}}%
7092     \fi
7093     \bbl@exp{\bbl@add\bbl@mapselect{\bbl@mapdir{\language}}}%
7094 \fi
7095 % == Line breaking: CJK quotes ==
7096 \ifcase\bbl@engine\or
7097     \bbl@xin{/c}{\bbl@cl{\lnbrk}}%
7098     \ifin@
7099         \bbl@ifunset{\bbl@quote\language}}{%
7100         {\directlua{
7101             Babel.locale_props[\the\localeid].cjk_quotes = {}
7102             local cs = 'op'
7103             for c in string.utfvalues(
7104                 [[\csname bbl@quote\language\endcsname]]) do
7105                 if Babel.cjk_characters[c].c == 'qu' then
7106                     Babel.locale_props[\the\localeid].cjk_quotes[c] = cs
7107                 end
7108                 cs = ( cs == 'op') and 'cl' or 'op'
7109             end
7110             }}%
7111         \fi
7112     \fi
7113 % == Counters: mapdigits ==
7114 % Native digits
7115 \ifx\bbl@KVP@mapdigits\@nnil\else
7116     \bbl@ifunset{\bbl@dgnat\language}}{%

```

```

7117     {\bbl@activate@preotf
7118     \directlua{
7119         Babel.digits_mapped = true
7120         Babel.digits = Babel.digits or {}
7121         Babel.digits[\the\localeid] =
7122             table.pack(string.utfvalue('\bbl@cl{dgnat}'))
7123         if not Babel.numbers then
7124             function Babel.numbers(head)
7125                 local LOCALE = Babel.attr_locale
7126                 local GLYPH = node.id'glyph'
7127                 local inmath = false
7128                 for item in node.traverse(head) do
7129                     if not inmath and item.id == GLYPH then
7130                         local temp = node.get_attribute(item, LOCALE)
7131                         if Babel.digits[temp] then
7132                             local chr = item.char
7133                             if chr > 47 and chr < 58 then
7134                                 item.char = Babel.digits[temp][chr-47]
7135                             end
7136                         end
7137                     elseif item.id == node.id'math' then
7138                         inmath = (item.subtype == 0)
7139                     end
7140                 end
7141                 return head
7142             end
7143         end
7144     } }%
7145 \fi
7146 % == transforms ==
7147 \ifx\bbl@KVP@transforms\@nnil\else
7148     \def\bbl@elt##1##2##3{%
7149         \in@{$transforms.}{##1}%
7150         \ifin@
7151             \def\bbl@tempa{##1}%
7152             \bbl@replace\bbl@tempa{transforms.}{}%
7153             \bbl@carg\bbl@transforms{babel\bbl@tempa}{##2}{##3}%
7154         \fi}%
7155 \bbl@exp{%
7156     \\\bbl@ifblank{\bbl@cl{dgnat}}%
7157     {\let\\bbl@tempa\relax}%
7158     {\def\\bbl@tempa{%
7159         \\\bbl@elt{transforms.prehyphenation}%
7160         {digits.native.1.0}{([0-9])}%
7161         \\\bbl@elt{transforms.prehyphenation}%
7162         {digits.native.1.1}{string={1|string|0123456789|string|\bbl@cl{dgnat}}}}}%
7163 \ifx\bbl@tempa\relax\else
7164     \toks@\expandafter\expandafter\expandafter{%
7165         \csname bbl@inidata@\language\endcsname}%
7166     \bbl@csarg\edef{inidata@\language}{%
7167         \unexpanded\expandafter{\bbl@tempa}%
7168         \the\toks@}%
7169 \fi
7170 \csname bbl@inidata@\language\endcsname
7171 \bbl@release@transforms\relax % \relax closes the last item.
7172 \fi}

```

Start tabular here:

```

7173 \def\localerestoredirs{%
7174     \ifcase\bbl@thetextdir
7175     \ifnum\textdirection=\z@\else\textdirection=\z@\fi
7176     \else
7177         \ifnum\textdirection=\@ne\else\textdirection=\@ne\fi

```

```

7178 \fi
7179 \ifcase\bbl@thepardir
7180 \ifnum\pardirection=\z@\else\pardirection=\z@\bodydirection=\z@\fi
7181 \else
7182 \ifnum\pardirection=\@ne\else\pardirection=\@ne\bodydirection=\@ne\fi
7183 \fi}
7184 %
7185 \IfBabelLayout{tabular}%
7186 {\chardef\bbl@tabular@mode\z@}% All RTL
7187 {\IfBabelLayout{lrtabular}%
7188 {\chardef\bbl@tabular@mode\@ne}%
7189 {\chardef\bbl@tabular@mode\z@}}% Mixed, with LTR cols
7190 %
7191 \ifnum\bbl@bidimode>\@ne % Any lua bidi= except default=1
7192 % Redefine: vrules mess up dirs (why?).
7193 \AtBeginDocument{\def\@arstrut{\relax\copy\@arstrutbox}}%
7194 \ifcase\bbl@tabular@mode\else % 1 = Mixed
7195 \let\bbl@parabefore\relax
7196 \AddToHook{para/before}{\bbl@parabefore}
7197 \AtBeginDocument{%
7198 \bbl@replace\@tabular{\hbox\bgroup}{\hbox bdir\z@\bgroup}%
7199 \ifnum\bbl@tabular@mode=\@ne
7200 \bbl@ifunset{\@tabclassz}{\}%
7201 \bbl@exp{% Hide conditionals
7202 \\\bbl@sreplace\\\@tabclassz
7203 {\<ifcase>\\\@chnum}%
7204 {\\\localerestoredirs\<ifcase>\\\@chnum}}}%
7205 \@ifpackageloaded{colortbl}%
7206 {\bbl@sreplace\@classz
7207 {\hbox\bgroup\bgroup}{\hbox\bgroup\bgroup\localerestoredirs}}}%
7208 {\@ifpackageloaded{array}%
7209 {\bbl@exp{% Hide conditionals
7210 \\\bbl@sreplace\\\@classz
7211 {\<ifcase>\\\@chnum}%
7212 {\bgroup\\\localerestoredirs\<ifcase>\\\@chnum}%
7213 \\\bbl@sreplace\\\@classz
7214 {\\\do@row@strut\<fi>}{\\do@row@strut\<fi>\egroup}}}%
7215 {}}}%
7216 \fi}%
7217 \fi

```

Very likely the `\output` routine must be patched in a quite general way to make sure the `\bodydir` is set to `\pagedir`. Note outside `\output` they can be different (and often are). For the moment, two *ad hoc* changes.

```

7218 \AtBeginDocument{%
7219 \@ifpackageloaded{multicol}%
7220 {\toks\expandafter{\multi@column@out}%
7221 \edef\multi@column@out{\bodydir\pagedir\the\toks@}}%
7222 {}%
7223 \@ifpackageloaded{paracol}%
7224 {\edef\pcol@output{%
7225 \bodydir\pagedir\unexpanded\expandafter{\pcol@output}}}%
7226 {}%
7227 \fi

```

Finish here if there is no layout.

```
7228 \ifx\bbl@opt@layout\@nnil\endinput\fi
```

`\hangindent` does not honour direction changes by default, so we need to redefine `\@hangfrom`.

```

7229 \chardef\bbl@trace\vboxdir\z@
7230 \ifnum\bbl@bidimode>\z@ % Any bidi=
7231 \IfBabelLayout{nopars}{}
7232 {\edef\bbl@opt@layout{\bbl@opt@layout.pars.}}%
7233 \IfBabelLayout{pars}

```

```

7234 {\chardef\bbL@trace@vboxdir\@ne
7235 \def\@hangfrom#1{%
7236 \setbox\@tempboxa\hbox{\{#1\}}%
7237 \hangindent\wd\@tempboxa
7238 \ifnum\bbL@vbox@bodydir=\pardirection\else
7239 \shapemode\@ne
7240 \fi
7241 \noindent\box\@tempboxa}}
7242 {}
7243 \fi

```

We need to patch lists in documents with both LTR and RTL paragraphs. See issue #395 in GitHub. There was a partial solution, but a better one has been devised by Udi Fogiel (in 26.4).

```

7244 \IfBabelLayout{lists}
7245 {\chardef\bbL@trace@vboxdir\@ne
7246 \let\bbL@OL@list\list
7247 \bbL@sreplace\list{\parshape}{\bbL@listparshape}%
7248 \let\bbL@NL@list\list
7249 \def\bbL@listparshape#1#2#3{%
7250 \parshape #1 #2 #3 %
7251 \ifnum\bbL@vbox@bodydir=\pardirection\else
7252 \shapemode\tw@
7253 \fi}}
7254 {}
7255 %
7256 \ifcase\bbL@trace@vboxdir\else
7257 \AtBeginDocument{\chardef\bbL@vbox@bodydir\pagedirection}%
7258 \def\bbL@vbox@lists{\chardef\bbL@vbox@bodydir\bodydirection}%
7259 \let\bbL@bidi@vbox\everyvbox
7260 \@nameuse{newtoks}\everyvbox % \outer in Plain
7261 \everyvbox\expandafter{\the\bbL@bidi@vbox}%
7262 \bbL@bidi@vbox{\bbL@vbox@lists\the\everyvbox}%
7263 \fi
7264 %
7265 \IfBabelLayout{graphics}
7266 {\let\bbL@pictresetdir\relax
7267 \def\bbL@pictsetdir#1{%
7268 \ifcase\bbL@thetextdir
7269 \let\bbL@pictresetdir\relax
7270 \else
7271 \ifcase#1\bodydir TLT % Remember this sets the inner boxes
7272 \or\textdir TLT
7273 \else\bodydir TLT \textdir TLT
7274 \fi
7275 % \(\text|par)dir required in pgf:
7276 \def\bbL@pictresetdir{\bodydir TRT\pardir TRT\textdir TRT\relax}%
7277 \fi}%
7278 \AddToHook{env/picture/begin}{\bbL@pictsetdir\tw@}%
7279 \directlua{
7280 Babel.get_picture_dir = true
7281 Babel.picture_has_bidi = 0
7282 %
7283 function Babel.picture_dir (head)
7284 if not Babel.get_picture_dir then return head end
7285 if Babel.hlist_has_bidi(head) then
7286 Babel.picture_has_bidi = 1
7287 end
7288 return head
7289 end
7290 luatexbase.add_to_callback("hpack_filter", Babel.picture_dir,
7291 "Babel.picture_dir")
7292 }%
7293 \AddToHook{package/graphics/after}{%

```

```

7294 \bbl@exp{%
7295   \\bbl@sreplace\\rotatebox{\hbox{{\string#2}}}
7296   {\hbox bdir\z@\hbox bdir<ifcase>\bbl@thetextdir\z@<else>\@ne<fi>{\string#2}}}%
7297 \AddToHook{package/graphicx/after}{%
7298   \bbl@exp{%
7299     \\bbl@sreplace\\Grot@box@std{\hbox{{\string#2}}}
7300     {\hbox bdir\z@\hbox bdir<ifcase>\bbl@thetextdir\z@<else>\@ne<fi>{\string#2}}}%
7301     \\bbl@sreplace\\Grot@box@kv{\hbox{\string#3}}
7302     {\hbox bdir\z@\hbox bdir<ifcase>\bbl@thetextdir\z@<else>\@ne<fi>{\string#3}}}%
7303     \\bbl@sreplace\\Grot@box{\hbox}{\hbox bdir\z@}}
7304 \AtBeginDocument{%
7305   \long\def\put(#1,#2)#3{%
7306     \@killglue
7307     % Try:
7308     \ifx\bbl@pictresetdir\relax
7309       \def\bbl@tempc{0}%
7310     \else
7311       \directlua{
7312         Babel.get_picture_dir = true
7313         Babel.picture_has_bidi = 0
7314       }%
7315       \setbox\z@\hb@xt\z@{%
7316         \@defaultunitsset\@tempdimc{#1}\unitlength
7317         \kern\@tempdimc
7318         #3\hss}%
7319       \edef\bbl@tempc{\directlua{tex.print(Babel.picture_has_bidi)}}%
7320     \fi
7321     % Do:
7322     \@defaultunitsset\@tempdimc{#2}\unitlength
7323     \raise\@tempdimc\hb@xt\z@{%
7324       \@defaultunitsset\@tempdimc{#1}\unitlength
7325       \kern\@tempdimc
7326       {\ifnum\bbl@tempc>\z@\bbl@pictresetdir\fi#3}\hss}%
7327     \ignorespaces}%
7328   \MakeRobust\put}%
7329 \AtBeginDocument
7330 {\AddToHook{cmd/diagbox@pict/before}{\let\bbl@pictsetdir\@gobble}%
7331 \ifx\pgfpicture\undefined\else
7332   \AddToHook{env/pgfpicture/begin}{\bbl@pictsetdir\@ne}%
7333   \bbl@add\pgfinterruptpicture{\bbl@pictresetdir}%
7334   \bbl@add\pgfsys@beginpicture{\bbl@pictsetdir\z@}%
7335 \fi
7336 \ifx\tikzpicture\undefined\else
7337   \AddToHook{env/tikzpicture/begin}{\bbl@pictsetdir\tw}%
7338   \bbl@add\tikz@atbegin@node{\bbl@pictresetdir}%
7339   \bbl@sreplace\tikz{\begingroup}{\begingroup\bbl@pictsetdir\tw}%
7340   \bbl@sreplace\tikzpicture{\begingroup}{\begingroup\bbl@pictsetdir\tw}%
7341 \fi
7342 }}
7343 {}

```

Implicitly reverses sectioning labels in bidi=basic-r, because the full stop is not in contact with L numbers any more. I think there must be a better way. Assumes bidi=basic, but there are some additional readjustments for bidi=default.

```

7344 \IfBabelLayout{counters*}%
7345 {\bbl@add\bbl@opt@layout{.counters.}}%
7346 \directlua{
7347   luatexbase.add_to_callback("process_output_buffer",
7348     Babel.discard_sublr , "Babel.discard_sublr") }%
7349 {}
7350 \IfBabelLayout{counters}%
7351 {\let\bbl@latinarabic=\@arabic
7352 \let\bbl@0L@@arabic\@arabic

```

```

7353 \def\@arabic#1{\babelsublr{\bbl@latinarabic#1}}%
7354 \@ifpackagewith{babel}{bidi=default}%
7355 {\let\bbl@asciroman=\@roman
7356 \let\bbl@OL@roman\@roman
7357 \def\@roman#1{\babelsublr{\ensureascii{\bbl@asciroman#1}}}%
7358 \let\bbl@asciiRoman=\@Roman
7359 \let\bbl@OL@roman\@Roman
7360 \def\@Roman#1{\babelsublr{\ensureascii{\bbl@asciiRoman#1}}}%
7361 \let\bbl@OL@labelenumii\labelenumii
7362 \def\labelenumii{\theenumii}%
7363 \let\bbl@OL@p@enumiii\p@enumiii
7364 \def\p@enumiii{\p@enumii}\theenumii{}}{}{}

```

Some $\TeX$ macros use internally the math mode for text formatting. They have very little in common and are grouped here, as a single option.

```

7365 \IfBabelLayout{extras}%
7366 {\bbl@ncarg\let\bbl@OL@underline{underline }%
7367 \bbl@carg\bbl@sreplace{underline }{\hbox}%
7368 {\def\bbl@insidemath{0}\hbox bdir\ifcase\bbl@thetextdir\z@else\@ne\fi}%
7369 \let\bbl@OL@LaTeXe\LaTeXe
7370 \DeclareRobustCommand{\LaTeXe}{\mbox{\m@th
7371 \if b\expandafter\@car\@series\@nil\boldmath\fi
7372 \babelsublr{\LaTeX\kern.15em2$_{\textstyle\varepsilon}$}}}
7373 {}
7374 {/luatex}

```

11.13 Lua: transforms

After declaring the table containing the patterns with their replacements, we define some auxiliary functions: `str_to_nodes` converts the string returned by a function to a node list, taking the node at base as a model (font, language, etc.); `fetch_word` fetches a series of glyphs and discretionary, which pattern is matched against (if there is a match, it is called again before trying other patterns, and this is very likely the main bottleneck).

`post_hyphenate_replace` is the callback applied after `lang.hyphenate`. This means the automatic hyphenation points are known. As empty captures return a byte position (as explained in the `luatex` manual), we must convert it to a utf8 position. With `first`, the last byte can be the leading byte in a utf8 sequence, so we just remove it and add 1 to the resulting length. With `last` we must take into account the capture position points to the next character. Here `word_head` points to the starting node of the text to be matched.

```

7375 {*transforms}
7376 Babel.linebreaking.replacements = {}
7377 Babel.linebreaking.replacements[0] = {} -- pre
7378 Babel.linebreaking.replacements[1] = {} -- post
7379
7380 function Babel.tovalue(v)
7381   if type(v) == 'table' then
7382     return Babel.locale_props[v[1]].vars[v[2]] or v[3]
7383   else
7384     return v
7385   end
7386 end
7387
7388 Babel.attr_hboxed = luatexbase.registernumber'bbl@attr@hboxed'
7389
7390 function Babel.set_hboxed(head, gc)
7391   for item in node.traverse(head) do
7392     node.set_attribute(item, Babel.attr_hboxed, 1)
7393   end
7394   return head
7395 end
7396
7397 Babel.fetch_subtext = {}
7398

```

```

7399 Babel.ignore_pre_char = function(node)
7400   return (node.lang == Babel.nohyphenation)
7401 end
7402
7403 Babel.show_transforms = false
7404
7405 -- Merging both functions doesn't seem feasible, because there are too
7406 -- many differences.
7407 Babel.fetch_subtext[0] = function(head)
7408   local word_string = ''
7409   local word_nodes = {}
7410   local lang
7411   local item = head
7412   local inmath = false
7413
7414   while item do
7415     if item.id == 11 then
7416       inmath = (item.subtype == 0)
7417     end
7418
7419     if inmath then
7420       -- pass
7421     end
7422
7423     elseif item.id == 29 then
7424       local locale = node.get_attribute(item, Babel.attr_locale)
7425
7426       if lang == locale or lang == nil then
7427         lang = locale
7428         if Babel.ignore_pre_char(item) then
7429           word_string = word_string .. Babel.us_char
7430         else
7431           if node.has_attribute(item, Babel.attr_hboxed) then
7432             word_string = word_string .. Babel.us_char
7433           else
7434             word_string = word_string .. unicode.utf8.char(item.char)
7435           end
7436         end
7437         word_nodes[#word_nodes+1] = item
7438       else
7439         break
7440       end
7441
7442     elseif item.id == 12 and item.subtype == 13 then
7443       if node.has_attribute(item, Babel.attr_hboxed) then
7444         word_string = word_string .. Babel.us_char
7445       else
7446         word_string = word_string .. ' '
7447       end
7448       word_nodes[#word_nodes+1] = item
7449
7450       -- Ignore leading unrecognized nodes, too.
7451       elseif word_string ~= '' then
7452         word_string = word_string .. Babel.us_char
7453         word_nodes[#word_nodes+1] = item -- Will be ignored
7454       end
7455
7456     item = item.next
7457   end
7458
7459   -- Here and above we remove some trailing chars but not the
7460   -- corresponding nodes. But they aren't accessed.
7461   if word_string:sub(-1) == ' ' then

```

```

7462     word_string = word_string:sub(1,-2)
7463 end
7464 if Babel.show_transforms then texio.write_nl(word_string) end
7465 word_string = unicode.utf8.gsub(word_string, Babel.us_char .. '+$', '')
7466 return word_string, word_nodes, item, lang
7467 end
7468
7469 Babel.fetch_subtext[1] = function(head)
7470     local word_string = ''
7471     local word_nodes = {}
7472     local lang
7473     local item = head
7474     local inmath = false
7475
7476     while item do
7477
7478         if item.id == 11 then
7479             inmath = (item.subtype == 0)
7480         end
7481
7482         if inmath then
7483             -- pass
7484
7485         elseif item.id == 29 then
7486             if item.lang == lang or lang == nil then
7487                 lang = lang or item.lang
7488                 if node.has_attribute(item, Babel.attr_hboxed) then
7489                     word_string = word_string .. Babel.us_char
7490                 elseif (item.char == 124) or (item.char == 61) then -- not =, not |
7491                     word_string = word_string .. Babel.us_char
7492                 else
7493                     word_string = word_string .. unicode.utf8.char(item.char)
7494                 end
7495                 word_nodes[#word_nodes+1] = item
7496             else
7497                 break
7498             end
7499
7500         elseif item.id == 7 and item.subtype == 2 then
7501             if node.has_attribute(item, Babel.attr_hboxed) then
7502                 word_string = word_string .. Babel.us_char
7503             else
7504                 word_string = word_string .. '='
7505             end
7506             word_nodes[#word_nodes+1] = item
7507
7508         elseif item.id == 7 and item.subtype == 3 then
7509             if node.has_attribute(item, Babel.attr_hboxed) then
7510                 word_string = word_string .. Babel.us_char
7511             else
7512                 word_string = word_string .. '|'
7513             end
7514             word_nodes[#word_nodes+1] = item
7515
7516         -- (1) Go to next word if nothing was found, and (2) implicitly
7517         -- remove leading USs.
7518         elseif word_string == '' then
7519             -- pass
7520
7521         -- This is the responsible for splitting by words.
7522         elseif (item.id == 12 and item.subtype == 13) then
7523             break
7524

```

```

7525     else
7526         word_string = word_string .. Babel.us_char
7527         word_nodes[#word_nodes+1] = item -- Will be ignored
7528     end
7529
7530     item = item.next
7531 end
7532 if Babel.show_transforms then texio.write_nl(word_string) end
7533 word_string = unicode.utf8.gsub(word_string, Babel.us_char .. '+$', '')
7534 return word_string, word_nodes, item, lang
7535 end
7536
7537 function Babel.pre_hyphenate_replace(head)
7538     Babel.hyphenate_replace(head, 0)
7539 end
7540
7541 function Babel.post_hyphenate_replace(head)
7542     Babel.hyphenate_replace(head, 1)
7543 end
7544
7545 Babel.us_char = string.char(31)
7546
7547 function Babel.hyphenate_replace(head, mode)
7548     local u = unicode.utf8
7549     local lbkr = Babel.linebreaking.replacements[mode]
7550     local tovalue = Babel.tovalue
7551
7552     local word_head = head
7553
7554     if Babel.show_transforms then
7555         texio.write_nl('\n==== Showing ' .. (mode == 0 and 'pre' or 'post') .. 'hyphenation ====')
7556     end
7557
7558     while true do -- for each subtext block
7559
7560         local w, w_nodes, nw, lang = Babel.fetch_subtext[mode](word_head)
7561
7562         if Babel.debug then
7563             print()
7564             print((mode == 0) and '@@@<' or '@@@>', w)
7565         end
7566
7567         if nw == nil and w == '' then break end
7568
7569         if not lang then goto next end
7570         if not lbkr[lang] then goto next end
7571
7572         -- For each saved (pre|post)hyphenation. TODO. Reconsider how
7573         -- loops are nested.
7574         for k=1, #lbkr[lang] do
7575             local p = lbkr[lang][k].pattern
7576             local r = lbkr[lang][k].replace
7577             local attr = lbkr[lang][k].attr or -1
7578
7579             if Babel.debug then
7580                 print('*****', p, mode)
7581             end
7582
7583             -- This variable is set in some cases below to the first *byte*
7584             -- after the match, either as found by u.match (faster) or the
7585             -- computed position based on sc if w has changed.
7586             local last_match = 0
7587             local step = 0

```

```

7588
7589 -- For every match.
7590 while true do
7591     if Babel.debug then
7592         print('====')
7593     end
7594     local new -- used when inserting and removing nodes
7595     local dummy_node -- used by after
7596
7597     local matches = { u.match(w, p, last_match) }
7598
7599     if #matches < 2 then break end
7600
7601     -- Get and remove empty captures (with ()'s, which return a
7602     -- number with the position), and keep actual captures
7603     -- (from (...)), if any, in matches.
7604     local first = table.remove(matches, 1)
7605     local last = table.remove(matches, #matches)
7606     -- Non re-fetched substrings may contain \31, which separates
7607     -- subsubstrings.
7608     if string.find(w:sub(first, last-1), Babel.us_char) then break end
7609
7610     local save_last = last -- with A()BC()D, points to D
7611
7612     -- Fix offsets, from bytes to unicode. Explained above.
7613     first = u.len(w:sub(1, first-1)) + 1
7614     last = u.len(w:sub(1, last-1)) -- now last points to C
7615
7616     -- This loop stores in a small table the nodes
7617     -- corresponding to the pattern. Used by 'data' to provide a
7618     -- predictable behavior with 'insert' (w_nodes is modified on
7619     -- the fly), and also access to 'remove'd nodes.
7620     local sc = first-1 -- Used below, too
7621     local data_nodes = {}
7622
7623     local enabled = true
7624     for q = 1, last-first+1 do
7625         data_nodes[q] = w_nodes[sc+q]
7626         if enabled
7627             and attr > -1
7628             and not node.has_attribute(data_nodes[q], attr)
7629         then
7630             enabled = false
7631         end
7632     end
7633
7634     -- This loop traverses the matched substring and takes the
7635     -- corresponding action stored in the replacement list.
7636     -- sc = the position in substr nodes / string
7637     -- rc = the replacement table index
7638     local rc = 0
7639
7640     ----- TODO. dummy_node?
7641     while rc < last-first+1 or dummy_node do -- for each replacement
7642         if Babel.debug then
7643             print('.....', rc + 1)
7644         end
7645         sc = sc + 1
7646         rc = rc + 1
7647
7648         if Babel.debug then
7649             Babel.debug_hyph(w, w_nodes, sc, first, last, last_match)
7650             local ss = ''

```

```

7651         for itt in node.traverse(head) do
7652             if itt.id == 29 then
7653                 ss = ss .. unicode.utf8.char(itt.char)
7654             else
7655                 ss = ss .. '{' .. itt.id .. '}'
7656             end
7657         end
7658         print('*****', ss)
7659
7660     end
7661
7662     local crep = r[rc]
7663     local item = w_nodes[sc]
7664     local item_base = item
7665     local placeholder = Babel.us_char
7666     local d
7667
7668     if crep and crep.data then
7669         item_base = data_nodes[crep.data]
7670     end
7671
7672     if crep then
7673         step = crep.step or step
7674     end
7675
7676     if crep and crep.after then
7677         crep.insert = true
7678         if dummy_node then
7679             item = dummy_node
7680         else -- TODO. if there is a node after?
7681             d = node.copy(item_base)
7682             head, item = node.insert_after(head, item, d)
7683             dummy_node = item
7684         end
7685     end
7686
7687     if crep and not crep.after and dummy_node then
7688         node.remove(head, dummy_node)
7689         dummy_node = nil
7690     end
7691
7692     if not enabled then
7693         last_match = save_last
7694         goto next
7695
7696     elseif crep and next(crep) == nil then -- = {}
7697         if step == 0 then
7698             last_match = save_last -- Optimization
7699         else
7700             last_match = utf8.offset(w, sc+step)
7701         end
7702         goto next
7703
7704     elseif crep == nil or crep.remove then
7705         node.remove(head, item)
7706         table.remove(w_nodes, sc)
7707         w = u.sub(w, 1, sc-1) .. u.sub(w, sc+1)
7708         sc = sc - 1 -- Nothing has been inserted.
7709         last_match = utf8.offset(w, sc+1+step)
7710         goto next
7711
7712     elseif crep and crep.kashida then
7713         node.set_attribute(item,

```

```

7714         Babel.attr_kashida,
7715         crep.kashida)
7716     last_match = utf8.offset(w, sc+1+step)
7717     goto next
7718
7719 elseif crep and crep.string then
7720     local str = crep.string(matches)
7721     if str == '' then -- Gather with nil
7722         node.remove(head, item)
7723         table.remove(w_nodes, sc)
7724         w = u.sub(w, 1, sc-1) .. u.sub(w, sc+1)
7725         sc = sc - 1 -- Nothing has been inserted.
7726     else
7727         local loop_first = true
7728         for s in string.utfvalues(str) do
7729             d = node.copy(item_base)
7730             d.char = s
7731             if loop_first then
7732                 loop_first = false
7733                 head, new = node.insert_before(head, item, d)
7734                 if sc == 1 then
7735                     word_head = head
7736                 end
7737                 w_nodes[sc] = d
7738                 w = u.sub(w, 1, sc-1) .. u.char(s) .. u.sub(w, sc+1)
7739             else
7740                 sc = sc + 1
7741                 head, new = node.insert_before(head, item, d)
7742                 table.insert(w_nodes, sc, new)
7743                 w = u.sub(w, 1, sc-1) .. u.char(s) .. u.sub(w, sc)
7744             end
7745             if Babel.debug then
7746                 print('.....', 'str')
7747                 Babel.debug_hyph(w, w_nodes, sc, first, last, last_match)
7748             end
7749             end -- for
7750             node.remove(head, item)
7751         end -- if ''
7752         last_match = utf8.offset(w, sc+1+step)
7753         goto next
7754
7755 elseif mode == 1 and crep and (crep.pre or crep.no or crep.post) then
7756     d = node.new(7, 3) -- (disc, regular)
7757     d.pre = Babel.str_to_nodes(crep.pre, matches, item_base)
7758     d.post = Babel.str_to_nodes(crep.post, matches, item_base)
7759     d.replace = Babel.str_to_nodes(crep.no, matches, item_base)
7760     d.attr = item_base.attr
7761     if crep.pre == nil then -- TeXbook p96
7762         d.penalty = tovalue(crep.penalty) or tex.hyphenpenalty
7763     else
7764         d.penalty = tovalue(crep.penalty) or tex.exhyphenpenalty
7765     end
7766     placeholder = '|'
7767     head, new = node.insert_before(head, item, d)
7768
7769 elseif mode == 0 and crep and (crep.pre or crep.no or crep.post) then
7770     -- ERROR
7771
7772 elseif crep and crep.penalty then
7773     d = node.new(14, 0) -- (penalty, userpenalty)
7774     d.attr = item_base.attr
7775     d.penalty = tovalue(crep.penalty)
7776     head, new = node.insert_before(head, item, d)

```

```

7777
7778 elseif crep and crep.space then
7779     -- 655360 = 10 pt = 10 * 65536 sp
7780     d = node.new(12, 13)      -- (glue, spaceskip)
7781     local quad = font.getfont(item_base.font).size or 655360
7782     node.setglue(d, tovalue(crep.space[1]) * quad,
7783                  tovalue(crep.space[2]) * quad,
7784                  tovalue(crep.space[3]) * quad)
7785     if mode == 0 then
7786         placeholder = ' '
7787     end
7788     head, new = node.insert_before(head, item, d)
7789
7790 elseif crep and crep.norule then
7791     -- 655360 = 10 pt = 10 * 65536 sp
7792     d = node.new(2, 3)        -- (rule, empty) = \no*rule
7793     local quad = font.getfont(item_base.font).size or 655360
7794     d.width  = tovalue(crep.norule[1]) * quad
7795     d.height = tovalue(crep.norule[2]) * quad
7796     d.depth  = tovalue(crep.norule[3]) * quad
7797     head, new = node.insert_before(head, item, d)
7798
7799 elseif crep and crep.spacefactor then
7800     d = node.new(12, 13)      -- (glue, spaceskip)
7801     local base_font = font.getfont(item_base.font)
7802     node.setglue(d,
7803                  tovalue(crep.spacefactor[1]) * base_font.parameters['space'],
7804                  tovalue(crep.spacefactor[2]) * base_font.parameters['space_stretch'],
7805                  tovalue(crep.spacefactor[3]) * base_font.parameters['space_shrink'])
7806     if mode == 0 then
7807         placeholder = ' '
7808     end
7809     head, new = node.insert_before(head, item, d)
7810
7811 elseif mode == 0 and crep and crep.space then
7812     -- ERROR
7813
7814 elseif crep and crep.kern then
7815     d = node.new(13, 1)      -- (kern, user)
7816     local quad = font.getfont(item_base.font).size or 655360
7817     d.attr = item_base.attr
7818     d.kern = tovalue(crep.kern) * quad
7819     head, new = node.insert_before(head, item, d)
7820
7821 elseif crep and crep.node then
7822     d = node.new(crep.node[1], crep.node[2])
7823     d.attr = item_base.attr
7824     head, new = node.insert_before(head, item, d)
7825
7826 end -- i.e., replacement cases
7827
7828 -- Shared by disc, space(factor), kern, node and penalty.
7829 if sc == 1 then
7830     word_head = head
7831 end
7832 if crep.insert then
7833     w = u.sub(w, 1, sc-1) .. placeholder .. u.sub(w, sc)
7834     table.insert(w_nodes, sc, new)
7835     last = last + 1
7836 else
7837     w_nodes[sc] = d
7838     node.remove(head, item)
7839     w = u.sub(w, 1, sc-1) .. placeholder .. u.sub(w, sc+1)

```

```

7840         end
7841
7842         last_match = utf8.offset(w, sc+l+step)
7843
7844         ::next::
7845
7846     end -- for each replacement
7847
7848     if Babel.show_transforms then texio.write_nl('> ' .. w) end
7849     if Babel.debug then
7850         print('.....', '/')
7851         Babel.debug_hyph(w, w_nodes, sc, first, last, last_match)
7852     end
7853
7854     if dummy_node then
7855         node.remove(head, dummy_node)
7856         dummy_node = nil
7857     end
7858
7859     end -- for match
7860
7861     end -- for patterns
7862
7863     ::next::
7864     word_head = nw
7865 end -- for substring
7866
7867 if Babel.show_transforms then texio.write_nl(string.rep('-', 32) .. '\n') end
7868 return head
7869 end
7870
7871 -- This table stores capture maps, numbered consecutively
7872 Babel.capture_maps = {}
7873
7874 function Babel.esc_hex_to_char(h)
7875     if tex.getcatcode(tonumber(h, 16)) ~= 11 and
7876         tex.getcatcode(tonumber(h, 16)) ~= 12 then
7877         return string.format([[\\Uchar"%X ]], tonumber(h,16))
7878     else
7879         return unicode.utf8.char(tonumber(h, 16))
7880     end
7881 end
7882
7883 -- The following functions belong to the next macro
7884 function Babel.capture_func(key, cap)
7885     local ret = "[[" .. cap:gsub('{{([0-9])}}', "")..m[%1]..[""] .. "]"
7886     local cnt
7887     local u = unicode.utf8
7888     ret = u.gsub(ret, '{(%x%x%x%x+)}', '\\x01%1\\x04')
7889     ret, cnt = ret:gsub('{{([0-9])|([^\^]|+)|(.-)}', Babel.capture_func_map)
7890     ret = u.gsub(ret, '\\x01(%x%x%x%x+)%x04', Babel.esc_hex_to_char)
7891     ret = ret:gsub("%[%[%]%]%.", '')
7892     ret = ret:gsub("%.%[%[%]%]%", '')
7893     return key .. [[=function(m) return ]] .. ret .. [[ end]]
7894 end
7895
7896 function Babel.capt_map(from, mapno)
7897     return Babel.capture_maps[mapno][from] or from
7898 end
7899
7900 -- Handle the {n|abc|ABC} syntax in captures
7901 function Babel.capture_func_map(capno, from, to)
7902     local u = unicode.utf8

```

```

7903 from = u.gsub(from, '\x01(%x%x%x%x+)\x04',
7904     function (n)
7905         return u.char(tonumber(n, 16))
7906     end)
7907 to = u.gsub(to, '\x01(%x%x%x%x+)\x04',
7908     function (n)
7909         return u.char(tonumber(n, 16))
7910     end)
7911 local froms = {}
7912 for s in string.utfcharacters(from) do
7913     table.insert(froms, s)
7914 end
7915 local cnt = 1
7916 table.insert(Babel.capture_maps, {})
7917 local mlen = table.getn(Babel.capture_maps)
7918 for s in string.utfcharacters(to) do
7919     Babel.capture_maps[mlen][froms[cnt]] = s
7920     cnt = cnt + 1
7921 end
7922 return "]]..Babel.capt_map(m[" .. capno .. "]," ..
7923     (mlen) .. " ).." .. "["
7924 end
7925
7926 -- Create/Extend reversed sorted list of kashida weights:
7927 function Babel.capture_kashida(key, wt)
7928     wt = tonumber(wt)
7929     if Babel.kashida_wts then
7930         for p, q in ipairs(Babel.kashida_wts) do
7931             if wt == q then
7932                 break
7933             elseif wt > q then
7934                 table.insert(Babel.kashida_wts, p, wt)
7935                 break
7936             elseif table.getn(Babel.kashida_wts) == p then
7937                 table.insert(Babel.kashida_wts, wt)
7938             end
7939         end
7940     else
7941         Babel.kashida_wts = { wt }
7942     end
7943     return 'kashida = ' .. wt
7944 end
7945
7946 function Babel.capture_node(id, subtype)
7947     local sbt = 0
7948     for k, v in pairs(node.subtypes(id)) do
7949         if v == subtype then sbt = k end
7950     end
7951     return 'node = {' .. node.id(id) .. ', ' .. sbt .. '}'
7952 end
7953
7954 -- Experimental: applies prehyphenation transforms to a string (letters
7955 -- and spaces).
7956 function Babel.string_prehyphenation(str, locale)
7957     local n, head, last, res
7958     head = node.new(8, 0) -- dummy (hack just to start)
7959     last = head
7960     for s in string.utfvalues(str) do
7961         if s == 20 then
7962             n = node.new(12, 0)
7963         else
7964             n = node.new(29, 0)
7965             n.char = s

```

```

7966     end
7967     node.set_attribute(n, Babel.attr_locale, locale)
7968     last.next = n
7969     last = n
7970 end
7971 head = Babel.hyphenate_replace(head, 0)
7972 res = ''
7973 for n in node.traverse(head) do
7974     if n.id == 12 then
7975         res = res .. ' '
7976     elseif n.id == 29 then
7977         res = res .. unicode.utf8.char(n.char)
7978     end
7979 end
7980 tex.print(res)
7981 end
7982 (/transforms)

```

11.14 Lua: Auto bidi with basic and basic-r

The file `babel-data-bidi.lua` currently only contains data. It is a large and boring file and it is not shown here (see the generated file), but here is a sample:

```

% [0x25]={d='et'},
% [0x26]={d='on'},
% [0x27]={d='on'},
% [0x28]={d='on', m=0x29},
% [0x29]={d='on', m=0x28},
% [0x2A]={d='on'},
% [0x2B]={d='es'},
% [0x2C]={d='cs'},
%

```

For the meaning of these codes, see the Unicode standard.

Now the `basic-r` bidi mode. One of the aims is to implement a fast and simple bidi algorithm, with a single loop. I managed to do it for R texts, with a second smaller loop for a special case. The code is still somewhat chaotic, but its behavior is essentially correct. I cannot resist copying the following text from Emacs `bidi.c` (which also attempts to implement the bidi algorithm with a single loop):

Arrrrgh!! The UAX#9 algorithm is too deeply entrenched in the assumption of batch-style processing [...]. May the fleas of a thousand camels infest the armpits of those who design supposedly general-purpose algorithms by looking at their own implementations, and fail to consider other possible implementations!

Well, it took me some time to guess what the batch rules in UAX#9 actually mean (in other word, *what* they do and *why*, and not only *how*), but I think (or I hope) I've managed to understand them.

In some sense, there are two bidi modes, one for numbers, and the other for text. Furthermore, setting just the direction in R text is not enough, because there are actually *two* R modes (set explicitly in Unicode with RLM and ALM). In `babel` the `dir` is set by a higher protocol based on the language/script, which in turn sets the correct `dir` (<l>, <r> or <al>).

From UAX#9: “Where available, markup should be used instead of the explicit formatting characters”. So, this simple version just ignores formatting characters. Actually, most of that annex is devoted to how to handle them.

BD14-BD16 are not implemented. Unicode (and the W3C) are making a great effort to deal with some special problematic cases in “streamed” plain text. I don’t think this is the way to go – particular issues should be fixed by a high level interface taking into account the needs of the document. And here is where `luatex` excels, because everything related to bidi writing is under our control.

```

7983 (*basic-r)
7984 Babel.bidi_enabled = true
7985
7986 require('babel-data-bidi.lua')
7987
7988 local characters = Babel.characters
7989 local ranges = Babel.ranges

```

```

7990
7991 local DIR = node.id("dir")
7992
7993 local function dir_mark(head, from, to, outer)
7994   dir = (outer == 'r') and 'TLT' or 'TRT' -- i.e., reverse
7995   local d = node.new(DIR)
7996   d.dir = '+' .. dir
7997   node.insert_before(head, from, d)
7998   d = node.new(DIR)
7999   d.dir = '-' .. dir
8000   node.insert_after(head, to, d)
8001 end
8002
8003 function Babel.bidi(head, ispar)
8004   local first_n, last_n          -- first and last char with nums
8005   local last_es                  -- an auxiliary 'last' used with nums
8006   local first_d, last_d          -- first and last char in L/R block
8007   local dir, dir_real

   Next also depends on script/lang (<al>/<r>). To be set by babel.tex.pardir is dangerous, could be
   (re)set but it should be changed only in vmode. There are two strong's – strong = l/al/r and
   strong_lr = l/r (there must be a better way):

8008   local strong = ('TRT' == tex.pardir) and 'r' or 'l'
8009   local strong_lr = (strong == 'l') and 'l' or 'r'
8010   local outer = strong
8011
8012   local new_dir = false
8013   local first_dir = false
8014   local inmath = false
8015
8016   local last_lr
8017
8018   local type_n = ''
8019
8020   for item in node.traverse(head) do
8021
8022     -- three cases: glyph, dir, otherwise
8023     if item.id == node.id'glyph'
8024       or (item.id == 7 and item.subtype == 2) then
8025
8026       local itemchar
8027       if item.id == 7 and item.subtype == 2 then
8028         itemchar = item.replace.char
8029       else
8030         itemchar = item.char
8031       end
8032       local chardata = characters[itemchar]
8033       dir = chardata and chardata.d or nil
8034       if not dir then
8035         for nn, et in ipairs(ranges) do
8036           if itemchar < et[1] then
8037             break
8038           elseif itemchar <= et[2] then
8039             dir = et[3]
8040             break
8041           end
8042         end
8043       end
8044       dir = dir or 'l'
8045       if inmath then dir = ('TRT' == tex.mathdir) and 'r' or 'l' end

```

Next is based on the assumption babel sets the language *and* switches the script with its dir. We treat a language block as a separate Unicode sequence. The following piece of code is executed at the first glyph after a 'dir' node. We don't know the current language until then. This is not exactly true,

as the math mode may insert explicit dirs in the node list, so, for the moment there is a hack by brute force (just above).

```

8046     if new_dir then
8047         attr_dir = 0
8048         for at in node.traverse(item.attr) do
8049             if at.number == Babel.attr_dir then
8050                 attr_dir = at.value & 0x3
8051             end
8052         end
8053         if attr_dir == 1 then
8054             strong = 'r'
8055         elseif attr_dir == 2 then
8056             strong = 'al'
8057         else
8058             strong = 'l'
8059         end
8060         strong_lr = (strong == 'l') and 'l' or 'r'
8061         outer = strong_lr
8062         new_dir = false
8063     end
8064
8065     if dir == 'nsm' then dir = strong end          -- W1

```

Numbers. The dual `<al>/<r>` system for R is somewhat cumbersome.

```

8066     dir_real = dir          -- We need dir_real to set strong below
8067     if dir == 'al' then dir = 'r' end -- W3

```

By W2, there are no `<en>` `<et>` `<es>` if `strong == <al>`, only `<an>`. Therefore, there are not `<et en>` nor `<en et>`, W5 can be ignored, and W6 applied:

```

8068     if strong == 'al' then
8069         if dir == 'en' then dir = 'an' end          -- W2
8070         if dir == 'et' or dir == 'es' then dir = 'on' end -- W6
8071         strong_lr = 'r'                             -- W3
8072     end

```

Once finished the basic setup for glyphs, consider the two other cases: dir node and the rest.

```

8073     elseif item.id == node.id'dir' and not inmath then
8074         new_dir = true
8075         dir = nil
8076     elseif item.id == node.id'math' then
8077         inmath = (item.subtype == 0)
8078     else
8079         dir = nil          -- Not a char
8080     end

```

Numbers in R mode. A sequence of `<en>`, `<et>`, `<an>`, `<es>` and `<cs>` is typeset (with some rules) in L mode. We store the starting and ending points, and only when anything different is found (including nil, i.e., a non-char), the `textdir` is set. This means you cannot insert, say, a `whatsit`, but this is what I would expect (with `luacolor` you may colorize some digits). Anyway, this behavior could be changed with a switch in the future. Note in the first branch only `<an>` is relevant if `<al>`.

```

8081     if dir == 'en' or dir == 'an' or dir == 'et' then
8082         if dir ~= 'et' then
8083             type_n = dir
8084         end
8085         first_n = first_n or item
8086         last_n = last_es or item
8087         last_es = nil
8088     elseif dir == 'es' and last_n then -- W3+W6
8089         last_es = item
8090     elseif dir == 'cs' then          -- it's right - do nothing
8091     elseif first_n then -- & if dir = any but en, et, an, es, cs, inc nil
8092         if strong_lr == 'r' and type_n ~= '' then
8093             dir_mark(head, first_n, last_n, 'r')

```

```

8094     elseif strong_lr == 'l' and first_d and type_n == 'an' then
8095         dir_mark(head, first_n, last_n, 'r')
8096         dir_mark(head, first_d, last_d, outer)
8097         first_d, last_d = nil, nil
8098     elseif strong_lr == 'l' and type_n ~= '' then
8099         last_d = last_n
8100     end
8101     type_n = ''
8102     first_n, last_n = nil, nil
8103 end

```

R text in L, or L text in R. Order of dir_ mark's are relevant: d goes outside n, and therefore it's emitted after. See dir_mark to understand why (but is the nesting actually necessary or is a flat dir structure enough?). Only L, R (and AL) chars are taken into account – everything else, including spaces, whatsits, etc., are ignored:

```

8104 if dir == 'l' or dir == 'r' then
8105     if dir ~= outer then
8106         first_d = first_d or item
8107         last_d = item
8108     elseif first_d and dir ~= strong_lr then
8109         dir_mark(head, first_d, last_d, outer)
8110         first_d, last_d = nil, nil
8111     end
8112 end

```

Mirroring. Each chunk of text in a certain language is considered a “closed” sequence. If <r on r> and <l on l>, it's clearly <r> and <l>, resp'tly, but with other combinations depends on outer. From all these, we select only those resolving <on> → <r>. At the beginning (when last_lr is nil) of an R text, they are mirrored directly. Numbers in R mode are processed. It should not be done, but it doesn't hurt.

```

8113 if dir and not last_lr and dir ~= 'l' and outer == 'r' then
8114     item.char = characters[item.char] and
8115         characters[item.char].m or item.char
8116 elseif (dir or new_dir) and last_lr ~= item then
8117     local mir = outer .. strong_lr .. (dir or outer)
8118     if mir == 'rrr' or mir == 'lrr' or mir == 'rrl' or mir == 'rlr' then
8119         for ch in node.traverse(node.next(last_lr)) do
8120             if ch == item then break end
8121             if ch.id == node.id'glyph' and characters[ch.char] then
8122                 ch.char = characters[ch.char].m or ch.char
8123             end
8124         end
8125     end
8126 end

```

Save some values for the next iteration. If the current node is 'dir', open a new sequence. Since dir could be changed, strong is set with its real value (dir_real).

```

8127 if dir == 'l' or dir == 'r' then
8128     last_lr = item
8129     strong = dir_real -- Don't search back - best save now
8130     strong_lr = (strong == 'l') and 'l' or 'r'
8131 elseif new_dir then
8132     last_lr = nil
8133 end
8134 end

```

Mirror the last chars if they are no directed. And make sure any open block is closed, too.

```

8135 if last_lr and outer == 'r' then
8136     for ch in node.traverse_id(node.id'glyph', node.next(last_lr)) do
8137         if characters[ch.char] then
8138             ch.char = characters[ch.char].m or ch.char
8139         end
8140     end
8141 end

```

```

8142 if first_n then
8143   dir_mark(head, first_n, last_n, outer)
8144 end
8145 if first_d then
8146   dir_mark(head, first_d, last_d, outer)
8147 end

```

In boxes, the dir node could be added before the original head, so the actual head is the previous node.

```

8148 return node.prev(head) or head
8149 end
8150 {/basic-r}

```

And here the Lua code for bidi=basic:

```

8151 {*basic}
8152 -- e.g., Babel.fontmap[1][<prefontid>]=<dirfontid>
8153
8154 Babel.fontmap = Babel.fontmap or {}
8155 Babel.fontmap[0] = {} -- l
8156 Babel.fontmap[1] = {} -- r
8157 Babel.fontmap[2] = {} -- al/an
8158
8159 -- To cancel mirroring. Also OML, OMS, U?
8160 Babel.symbol_fonts = Babel.symbol_fonts or {}
8161 Babel.symbol_fonts[font.id('tenln')] = true
8162 Babel.symbol_fonts[font.id('tenlnw')] = true
8163 Babel.symbol_fonts[font.id('tencirc')] = true
8164 Babel.symbol_fonts[font.id('tencircw')] = true
8165
8166 Babel.bidi_enabled = true
8167 Babel.mirroring_enabled = true
8168
8169 require('babel-data-bidi.lua')
8170
8171 local characters = Babel.characters
8172 local ranges = Babel.ranges
8173
8174 local DIR = node.id('dir')
8175 local GLYPH = node.id('glyph')
8176
8177 local function insert_implicit(head, state, outer)
8178   local new_state = state
8179   if state.sim and state.eim and state.sim ~= state.eim then
8180     dir = ((outer == 'r') and 'TLT' or 'TRT') -- i.e., reverse
8181     local d = node.new(DIR)
8182     d.dir = '+' .. dir
8183     node.insert_before(head, state.sim, d)
8184     local d = node.new(DIR)
8185     d.dir = '-' .. dir
8186     node.insert_after(head, state.eim, d)
8187   end
8188   new_state.sim, new_state.eim = nil, nil
8189   return head, new_state
8190 end
8191
8192 local function insert_numeric(head, state)
8193   local new
8194   local new_state = state
8195   if state.san and state.ean and state.san ~= state.ean then
8196     local d = node.new(DIR)
8197     d.dir = '+TLT'
8198     _, new = node.insert_before(head, state.san, d)
8199     if state.san == state.sim then state.sim = new end
8200     local d = node.new(DIR)

```

```

8201     d.dir = '-TLT'
8202     _, new = node.insert_after(head, state.ean, d)
8203     if state.ean == state.eim then state.eim = new end
8204 end
8205 new_state.san, new_state.ean = nil, nil
8206 return head, new_state
8207 end
8208
8209 local function glyph_not_symbol_font(node)
8210   if node.id == GLYPH then
8211     return not Babel.symbol_fonts[node.font]
8212   else
8213     return false
8214   end
8215 end
8216
8217 -- TODO - \hbox with an explicit dir can lead to wrong results
8218 -- <R \hbox dir TLT{<R>}> and <L \hbox dir TRT{<L>}>. A small attempt
8219 -- was made to improve the situation, but the problem is the 3-dir
8220 -- model in babel/Unicode and the 2-dir model in LuaTeX don't fit
8221 -- well.
8222
8223 function Babel.bidi(head, ispar, hdir)
8224   local d    -- d is used mainly for computations in a loop
8225   local prev_d = ''
8226   local new_d = false
8227
8228   local nodes = {}
8229   local outer_first = nil
8230   local inmath = false
8231
8232   local glue_d = nil
8233   local glue_i = nil
8234
8235   local has_en = false
8236   local first_et = nil
8237
8238   local has_hyperlink = false
8239
8240   local ATDIR = Babel.attr_dir
8241   local attr_d, temp
8242   local locale_d
8243
8244   local save_outer
8245   local locale_d = node.get_attribute(head, ATDIR)
8246   if locale_d then
8247     locale_d = locale_d & 0x3
8248     save_outer = (locale_d == 0 and 'l') or
8249                 (locale_d == 1 and 'r') or
8250                 (locale_d == 2 and 'al')
8251   elseif ispar then -- Or error? Shouldn't happen
8252     -- when the callback is called, we are just _after_ the box,
8253     -- and the textdir is that of the surrounding text
8254     save_outer = ('TRT' == tex.pardir) and 'r' or 'l'
8255   else -- Empty box
8256     save_outer = ('TRT' == hdir) and 'r' or 'l'
8257   end
8258   local outer = save_outer
8259   local last = outer
8260   -- 'al' is only taken into account in the first, current loop
8261   if save_outer == 'al' then save_outer = 'r' end
8262
8263   local fontmap = Babel.fontmap

```

```

8264
8265 for item in node.traverse(head) do
8266
8267     -- Mask: DxxxPPTT (Done, Paddir [0-2], Textdir [0-2])
8268     locale_d = node.get_attribute(item, ATDIR)
8269     node.set_attribute(item, ATDIR, 0x80)
8270
8271     -- In what follows, #node is the last (previous) node, because the
8272     -- current one is not added until we start processing the neutrals.
8273     -- three cases: glyph, dir, otherwise
8274     if glyph_not_symbol_font(item)
8275         or (item.id == 7 and item.subtype == 2) then
8276
8277         if inmath then goto nextnode end
8278
8279         if locale_d == 0x80 then goto nextnode end
8280         locale_d = locale_d or ((save_outer=='l') and 0 or 1)
8281
8282         local d_font = nil
8283         local item_r
8284         if item.id == 7 and item.subtype == 2 then
8285             item_r = item.replace -- automatic discs have just 1 glyph
8286         else
8287             item_r = item
8288         end
8289
8290         local chardata = characters[item_r.char]
8291         d = chardata and chardata.d or nil
8292         if not d or d == 'nsm' then
8293             for nn, et in ipairs(ranges) do
8294                 if item_r.char < et[1] then
8295                     break
8296                 elseif item_r.char <= et[2] then
8297                     if not d then d = et[3]
8298                     elseif d == 'nsm' then d_font = et[3]
8299                     end
8300                     break
8301                 end
8302             end
8303         end
8304         d = d or 'l'
8305
8306         -- A short 'pause' in bidi for mapfont
8307         -- %%% TODO. move if fontmap here
8308         d_font = d_font or d
8309         d_font = (d_font == 'l' and 0) or
8310             (d_font == 'nsm' and 0) or
8311             (d_font == 'r' and 1) or
8312             (d_font == 'al' and 2) or
8313             (d_font == 'an' and 2) or nil
8314         if d_font and fontmap and fontmap[d_font][item_r.font] then
8315             item_r.font = fontmap[d_font][item_r.font]
8316         end
8317
8318         if new_d then
8319             table.insert(nodes, {nil, (outer == 'l') and 'l' or 'r', nil})
8320             if inmath then
8321                 attr_d = 0
8322             else
8323                 attr_d = locale_d & 0x3
8324             end
8325             if attr_d == 1 then
8326                 outer_first = 'r'

```

```

8327         last = 'r'
8328     elseif attr_d == 2 then
8329         outer_first = 'r'
8330         last = 'al'
8331     else
8332         outer_first = 'l'
8333         last = 'l'
8334     end
8335     outer = last
8336     has_en = false
8337     first_et = nil
8338     new_d = false
8339 end
8340
8341 if glue_d then
8342     if (d == 'l' and 'l' or 'r') ~= glue_d then
8343         table.insert(nodes, {glue_i, 'on', nil})
8344     end
8345     glue_d = nil
8346     glue_i = nil
8347 end
8348
8349 elseif item.id == DIR then
8350     if inmath then goto nextnode end
8351     d = nil
8352     new_d = true
8353
8354 elseif item.id == node.id'glue' and item.subtype == 13 then
8355     if inmath then goto nextnode end
8356     glue_d = d
8357     glue_i = item
8358     d = nil
8359
8360 elseif item.id == node.id'math' then
8361     if item.subtype == 0 then -- mathon
8362         inmath = true
8363         d = 'on'
8364         table.insert(nodes, {item, 'on', outer_first})
8365         outer_first = nil
8366         goto nextnode
8367     else -- mathoff
8368         inmath = false
8369     end
8370
8371 elseif item.id == 8 and item.subtype == 19 then
8372     has_hyperlink = true
8373
8374 else
8375     d = nil
8376 end
8377
8378 -- AL <= EN/ET/ES      -- W2 + W3 + W6
8379 if last == 'al' and d == 'en' then
8380     d = 'an'          -- W3
8381 elseif last == 'al' and (d == 'et' or d == 'es') then
8382     d = 'on'          -- W6
8383 end
8384
8385 -- EN + CS/ES + EN      -- W4
8386 if d == 'en' and #nodes >= 2 then
8387     if (nodes[#nodes][2] == 'es' or nodes[#nodes][2] == 'cs')
8388         and nodes[#nodes-1][2] == 'en' then
8389         nodes[#nodes][2] = 'en'

```

```

8390     end
8391 end
8392
8393 -- AN + CS + AN          -- W4 too, because uax9 mixes both cases
8394 if d == 'an' and #nodes >= 2 then
8395     if (nodes[#nodes][2] == 'cs')
8396         and nodes[#nodes-1][2] == 'an' then
8397         nodes[#nodes][2] = 'an'
8398     end
8399 end
8400
8401 -- ET/EN                  -- W5 + W7->l / W6->on
8402 if d == 'et' then
8403     first_et = first_et or (#nodes + 1)
8404 elseif d == 'en' then
8405     has_en = true
8406     first_et = first_et or (#nodes + 1)
8407 elseif first_et then      -- d may be nil here !
8408     if has_en then
8409         if last == 'l' then
8410             temp = 'l'    -- W7
8411         else
8412             temp = 'en'   -- W5
8413         end
8414     else
8415         temp = 'on'      -- W6
8416     end
8417     for e = first_et, #nodes do
8418         if glyph_not_symbol_font(nodes[e][1]) then nodes[e][2] = temp end
8419     end
8420     first_et = nil
8421     has_en = false
8422 end
8423
8424 -- Force mathdir in math if ON (currently works as expected only
8425 -- with 'l')
8426
8427 if inmath and d == 'on' then
8428     d = ('TRT' == tex.mathdir) and 'r' or 'l'
8429 end
8430
8431 if d then
8432     if d == 'al' then
8433         d = 'r'
8434         last = 'al'
8435     elseif d == 'l' or d == 'r' then
8436         last = d
8437     end
8438     prev_d = d
8439     table.insert(nodes, {item, d, outer_first})
8440 end
8441
8442 outer_first = nil
8443
8444 ::nextnode::
8445
8446 end -- for each node
8447
8448 -- TODO -- repeated here in case EN/ET is the last node. Find a
8449 -- better way of doing things:
8450 if first_et then      -- dir may be nil here !
8451     if has_en then
8452         if last == 'l' then

```

```

8453         temp = 'l'      -- W7
8454     else
8455         temp = 'en'      -- W5
8456     end
8457 else
8458     temp = 'on'          -- W6
8459 end
8460 for e = first_et, #nodes do
8461     if glyph_not_symbol_font(nodes[e][1]) then nodes[e][2] = temp end
8462 end
8463 end
8464
8465 -- dummy node, to close things
8466 table.insert(nodes, {nil, (outer == 'l') and 'l' or 'r', nil})
8467
8468 ----- NEUTRAL -----
8469
8470 outer = save_outer
8471 last = outer
8472
8473 local first_on = nil
8474
8475 for q = 1, #nodes do
8476     local item
8477
8478     local outer_first = nodes[q][3]
8479     outer = outer_first or outer
8480     last = outer_first or last
8481
8482     local d = nodes[q][2]
8483     if d == 'an' or d == 'en' then d = 'r' end
8484     if d == 'cs' or d == 'et' or d == 'es' then d = 'on' end --- W6
8485
8486     if d == 'on' then
8487         first_on = first_on or q
8488     elseif first_on then
8489         if last == d then
8490             temp = d
8491         else
8492             temp = outer
8493         end
8494         for r = first_on, q - 1 do
8495             nodes[r][2] = temp
8496             item = nodes[r][1]      -- MIRRORING
8497             if Babel.mirroring_enabled and glyph_not_symbol_font(item)
8498                 and temp == 'r' and characters[item.char] then
8499                 local font_mode = ''
8500                 if item.font > 0 and font.fonts[item.font].properties then
8501                     font_mode = font.fonts[item.font].properties.mode
8502                 end
8503                 if font_mode ~= 'harf' and font_mode ~= 'plug' then
8504                     item.char = characters[item.char].m or item.char
8505                 end
8506             end
8507         end
8508         first_on = nil
8509     end
8510
8511     if d == 'r' or d == 'l' then last = d end
8512 end
8513
8514 ----- IMPLICIT, REORDER -----
8515

```

```

8516 outer = save_outer
8517 last = outer
8518
8519 local state = {}
8520 state.has_r = false
8521
8522 for q = 1, #nodes do
8523
8524     local item = nodes[q][1]
8525
8526     outer = nodes[q][3] or outer
8527
8528     local d = nodes[q][2]
8529
8530     if d == 'nsm' then d = last end          -- W1
8531     if d == 'en' then d = 'an' end
8532     local isdir = (d == 'r' or d == 'l')
8533
8534     if outer == 'l' and d == 'an' then
8535         state.san = state.san or item
8536         state.ean = item
8537     elseif state.san then
8538         head, state = insert_numeric(head, state)
8539     end
8540
8541     if outer == 'l' then
8542         if d == 'an' or d == 'r' then      -- im -> implicit
8543             if d == 'r' then state.has_r = true end
8544             state.sim = state.sim or item
8545             state.eim = item
8546         elseif d == 'l' and state.sim and state.has_r then
8547             head, state = insert_implicit(head, state, outer)
8548         elseif d == 'l' then
8549             state.sim, state.eim, state.has_r = nil, nil, false
8550         end
8551     else
8552         if d == 'an' or d == 'l' then
8553             if nodes[q][3] then -- nil except after an explicit dir
8554                 state.sim = item -- so we move sim 'inside' the group
8555             else
8556                 state.sim = state.sim or item
8557             end
8558             state.eim = item
8559         elseif d == 'r' and state.sim then
8560             head, state = insert_implicit(head, state, outer)
8561         elseif d == 'r' then
8562             state.sim, state.eim = nil, nil
8563         end
8564     end
8565
8566     if isdir then
8567         last = d          -- Don't search back - best save now
8568     elseif d == 'on' and state.san then
8569         state.san = state.san or item
8570         state.ean = item
8571     end
8572
8573 end
8574
8575 head = node.prev(head) or head
8576 % \end{macrocode}
8577 %
8578 % Now direction nodes has been distributed with relation to characters

```

```

8579% and spaces, we need to take into account \TeX\-specific elements in
8580% the node list, to move them at an appropriate place. Firstly, with
8581% hyperlinks. Secondly, we avoid them between penalties and spaces, so
8582% that the latter are still discardable.
8583%
8584% \begin{macrocode}
8585 --- FIXES ---
8586 if has_hyperlink then
8587   local flag, linking = 0, 0
8588   for item in node.traverse(head) do
8589     if item.id == DIR then
8590       if item.dir == '+TRT' or item.dir == '+TLT' then
8591         flag = flag + 1
8592       elseif item.dir == '-TRT' or item.dir == '-TLT' then
8593         flag = flag - 1
8594       end
8595     elseif item.id == 8 and item.subtype == 19 then
8596       linking = flag
8597     elseif item.id == 8 and item.subtype == 20 then
8598       if linking > 0 then
8599         if item.prev.id == DIR and
8600            (item.prev.dir == '-TRT' or item.prev.dir == '-TLT') then
8601           d = node.new(DIR)
8602           d.dir = item.prev.dir
8603           node.remove(head, item.prev)
8604           node.insert_after(head, item, d)
8605         end
8606       end
8607       linking = 0
8608     end
8609   end
8610 end
8611
8612 for item in node.traverse_id(10, head) do
8613   local p = item
8614   local flag = false
8615   while p.prev and p.prev.id == 14 do
8616     flag = true
8617     p = p.prev
8618   end
8619   if flag then
8620     node.insert_before(head, p, node.copy(item))
8621     node.remove(head, item)
8622   end
8623 end
8624
8625 return head
8626 end
8627
8627 function Babel.unset_atdir(head)
8628   local ATDIR = Babel.attr_dir
8629   for item in node.traverse(head) do
8630     node.set_attribute(item, ATDIR, 0x80)
8631   end
8632   return head
8633 end

```

The following code has been contributed by Udi Fogiel. It fixes misplaced glues when combined with penalties at line breaks.

```

8634 local GLUE    = node.id'glue'
8635 local PENALTY  = node.id'penalty'
8636 local HLIST    = node.id'hlist'
8637 local LINE    = table.swapped(node.subtypes("hlist")).line
8638

```

```

8639 local characters = Babel.characters
8640 local ranges      = Babel.ranges
8641
8642 -- L1's whitespace set: WS + isolate formats + (per the UBA reference) the
8643 -- explicit embedding/override formats + BN.
8644 -- Babel does not use embedding/override/isolates. In fact, their use
8645 -- is discouraged by the UAX#9. To be removed.
8646 local ll_whitespace = {
8647   ws = true, bn = true,
8648   lre = true, rle = true, lro = true, rlo = true, pdf = true,
8649   lri = true, rli = true, fsi = true, pdi = true,
8650 }
8651
8652 local function bidi_class(c)
8653   local cd = characters[c]
8654   local d = cd and cd.d
8655   if not d then
8656     for _, et in ipairs(ranges) do
8657       if c < et[1] then
8658         break
8659       elseif c <= et[2] then
8660         d = et[3]
8661         break
8662       end
8663     end
8664   end
8665   return d or 'l'
8666 end
8667
8668 -- Nodes we step over as part of a trailing-whitespace run: glue,
8669 -- penalties, and a whitespace glyph.
8670 local function skippable(n)
8671   local id = n.id
8672   if id == GLUE or id == PENALTY then
8673     return true
8674   elseif id == GLYPH then
8675     return ll_whitespace[bidi_class(n.char)] == true
8676   end
8677   return false
8678 end
8679
8680 local function is_enddir(n)
8681   return n.id == DIR and n.subtype == 1
8682 end
8683
8684 -- Rule L1, clause 4, for one line: pull its trailing whitespace out past the
8685 -- closing dir run so it renders in the box (base) direction.
8686 local function reset_line(head)
8687   local last = node.slide(head) -- tail, fixes prev links
8688   -- 1. skip the tail that follows the closing dir run (rightskip/parfillskip/...)
8689   local q = last
8690   while q and skippable(q) do q = q.prev end
8691   if not (q and is_enddir(q)) then return head end
8692   local d_last = q
8693   -- 2. step back over the whole closing dir run
8694   while q and q.id == DIR do q = q.prev end
8695   -- 3. what precedes the dirs must be the trailing whitespace run
8696   if not (q and skippable(q)) then return head end
8697   local w_last = q
8698   while q and skippable(q) do q = q.prev end
8699   local before_w = q
8700   if not before_w then return head end -- whole line is whitespace; leave it
8701   local w_first = before_w.next

```

```

8702 local d_first = w_last.next
8703 local after_d = d_last.next
8704 -- detach W = [w_first .. w_last]
8705 before_w.next = d_first
8706 if d_first then d_first.prev = before_w end
8707 -- reinsert W after the dir run (outside every dir => box/base direction)
8708 d_last.next = w_first
8709 w_first.prev = d_last
8710 w_last.next = after_d
8711 if after_d then after_d.prev = w_last end
8712 return head
8713 end
8714
8715 function Babel.reset_lines(head)
8716 for line, subtype in node.traverse_id(HLIST, head) do
8717   if subtype == LINE then
8718     line.list = reset_line(line.list)
8719   end
8720 end
8721 return head
8722 end
8723
8724 luatexbase.add_to_callback('post_linebreak_filter',
8725   function(head, gc)
8726     if Babel.bidi_enabled then head = Babel.reset_lines(head) end
8727     return head
8728   end,
8729   'Babel.reset_lines')
8730 (/basic)

```

12. Data for CJK

It is a boring file and it is not shown here (see the generated file), but here is a sample:

```

% [0x0021]={c='ex'},
% [0x0024]={c='pr'},
% [0x0025]={c='po'},
% [0x0028]={c='op'},
% [0x0029]={c='cp'},
% [0x002B]={c='pr'},
%

```

For the meaning of these codes, see the Unicode standard.

13. The ‘nil’ language

This ‘language’ does nothing, except setting the hyphenation patterns to nohyphenation. For this language currently no special definitions are needed or available.

The macro `\LdfInit` takes care of preventing that this file is loaded more than once, checking the category code of the `@` sign, etc.

```

8731 (*nil)
8732 \ProvidesLanguage{nil}[<@date@> v<@version@> Nil language]
8733 \LdfInit{nil}{datenil}

```

When this file is read as an option, i.e., by the `\usepackage` command, `nil` could be an ‘unknown’ language in which case we have to make it known.

```

8734 \ifx\l@nil\undefined
8735   \newlanguage\l@nil
8736   \@namedef{bbl@hyphendata@the\l@nil}{{}}}% Remove warning
8737   \let\bbl@elt\relax
8738   \edef\bbl@languages{% Add it to the list of languages

```

```

8739 \bbl@languages\bbl@elt{nil}{\the\l@nil}{}}
8740 \fi

```

This macro is used to store the values of the hyphenation parameters `\lefthyphenmin` and `\righthyphenmin`.

```

8741 \providehyphenmins{\CurrentOption}{\m@ne\m@ne}

```

The next step consists of defining commands to switch to (and from) the ‘nil’ language.

\captionnil

\datenil

```

8742 \let\captionnil\@empty
8743 \let\datenil\@empty

```

There is no locale file for this pseudo-language, so the corresponding fields are defined here.

```

8744 \def\bbl@inidata@nil{%
8745 \bbl@elt{identification}{tag.ini}{und}%
8746 \bbl@elt{identification}{load.level}{0}%
8747 \bbl@elt{identification}{charset}{utf8}%
8748 \bbl@elt{identification}{version}{1.0}%
8749 \bbl@elt{identification}{date}{2022-05-16}%
8750 \bbl@elt{identification}{name.local}{nil}%
8751 \bbl@elt{identification}{name.english}{nil}%
8752 \bbl@elt{identification}{name.babel}{nil}%
8753 \bbl@elt{identification}{tag.bcp47}{und}%
8754 \bbl@elt{identification}{language.tag.bcp47}{und}%
8755 \bbl@elt{identification}{tag.opentype}{dflt}%
8756 \bbl@elt{identification}{script.name}{Latin}%
8757 \bbl@elt{identification}{script.tag.bcp47}{Latn}%
8758 \bbl@elt{identification}{script.tag.opentype}{DFLT}%
8759 \bbl@elt{identification}{level}{1}%
8760 \bbl@elt{identification}{encodings}{}%
8761 \bbl@elt{identification}{derivate}{no}}
8762 \@namedef{bbl@tbc@nil}{und}
8763 \@namedef{bbl@lbc@nil}{und}
8764 \@namedef{bbl@casing@nil}{und}
8765 \@namedef{bbl@lotf@nil}{dflt}
8766 \@namedef{bbl@elname@nil}{nil}
8767 \@namedef{bbl@lname@nil}{nil}
8768 \@namedef{bbl@esname@nil}{Latin}
8769 \@namedef{bbl@sname@nil}{Latin}
8770 \@namedef{bbl@sbc@nil}{Latn}
8771 \@namedef{bbl@sotf@nil}{latn}

```

The macro `\ldf@finish` takes care of looking for a configuration file, setting the main language to be switched on at `\begin{document}` and resetting the category code of `@` to its original value.

```

8772 \ldf@finish{nil}
8773 {/nil}

```

14. Calendars

The code for specific calendars are placed in the specific files, loaded when requested by an ini file in the identification section with `require.calendars`.

Start with function to compute the Julian day. It's based on the little library `calendar.js`, by John Walker, in the public domain.

```

8774 ((*Compute Julian day)) ≡
8775 \def\bbl@fpmmod#1#2{(#1-#2*floor((#1)/(#2)))}
8776 \def\bbl@cs@gregleap#1{%
8777 (\bbl@fpmmod{#1}{4} == 0) &&
8778 (!((\bbl@fpmmod{#1}{100} == 0) && (\bbl@fpmmod{#1}{400} != 0)))}
8779 \def\bbl@cs@jd#1#2#3{% year, month, day
8780 \fpeval{ 1721424.5 + (365 * (#1 - 1)) +
8781 floor((#1 - 1) / 4) + (-floor((#1 - 1) / 100)) +

```

```

8782 floor((#1 - 1) / 400) + floor((((367 * #2) - 362) / 12) +
8783 ((#2 <= 2) ? 0 : (\bbl@cs@gregleap{#1} ? -1 : -2)) + #3) }}
8784 ({/Compute Julian day})

```

14.1. Islamic

The code for the Civil calendar is based on it, too.

```

8785 (*ca-islamic)
8786 <@Compute Julian day@>
8787 % == islamic (default)
8788 % Not yet implemented
8789 \def\bbl@ca@islamic#1-#2-#3\@#4#5#6{

  The Civil calendar.

8790 \def\bbl@cs@isltojd#1#2#3{ % year, month, day
8791 ((#3 + ceil(29.5 * (#2 - 1)) +
8792 (#1 - 1) * 354 + floor((3 + (11 * #1)) / 30) +
8793 1948439.5) - 1) }
8794 \@namedef\bbl@ca@islamic-civil++{\bbl@ca@islamicvl@x{+2}}
8795 \@namedef\bbl@ca@islamic-civil+{\bbl@ca@islamicvl@x{+1}}
8796 \@namedef\bbl@ca@islamic-civil{\bbl@ca@islamicvl@x{}}
8797 \@namedef\bbl@ca@islamic-civil-{\bbl@ca@islamicvl@x{-1}}
8798 \@namedef\bbl@ca@islamic-civil--{\bbl@ca@islamicvl@x{-2}}
8799 \def\bbl@ca@islamicvl@x#1#2-#3-#4\@#5#6#7{%
8800 \edef\bbl@tempa{%
8801 \fpeval{ floor(\bbl@cs@jd{#2}{#3}{#4})+0.5 #1}}%
8802 \edef#5{%
8803 \fpeval{ floor(((30*(\bbl@tempa-1948439.5)) + 10646)/10631) }}%
8804 \edef#6{\fpeval{
8805 min(12,ceil((\bbl@tempa-(29+\bbl@cs@isltojd{#5}{1}{1}))/29.5)+1) }}%
8806 \edef#7{\fpeval{ \bbl@tempa - \bbl@cs@isltojd{#5}{#6}{1} + 1} }}

```

The Umm al-Qura calendar, used mainly in Saudi Arabia, is based on moment-hijri, by Abdullah Alsigar (license MIT).

Since the main aim is to provide a suitable \today, and maybe some close dates, data just covers Hijri ~1435/~1460 (Gregorian ~2014/~2038).

```

8807 \def\bbl@cs@umalqura@data{56660, 56690,56719,56749,56778,56808,%
8808 56837,56867,56897,56926,56956,56985,57015,57044,57074,57103,%
8809 57133,57162,57192,57221,57251,57280,57310,57340,57369,57399,%
8810 57429,57458,57487,57517,57546,57576,57605,57634,57664,57694,%
8811 57723,57753,57783,57813,57842,57871,57901,57930,57959,57989,%
8812 58018,58048,58077,58107,58137,58167,58196,58226,58255,58285,%
8813 58314,58343,58373,58402,58432,58461,58491,58521,58551,58580,%
8814 58610,58639,58669,58698,58727,58757,58786,58816,58845,58875,%
8815 58905,58934,58964,58994,59023,59053,59082,59111,59141,59170,%
8816 59200,59229,59259,59288,59318,59348,59377,59407,59436,59466,%
8817 59495,59525,59554,59584,59613,59643,59672,59702,59731,59761,%
8818 59791,59820,59850,59879,59909,59939,59968,59997,60027,60056,%
8819 60086,60115,60145,60174,60204,60234,60264,60293,60323,60352,%
8820 60381,60411,60440,60469,60499,60528,60558,60588,60618,60648,%
8821 60677,60707,60736,60765,60795,60824,60853,60883,60912,60942,%
8822 60972,61002,61031,61061,61090,61120,61149,61179,61208,61237,%
8823 61267,61296,61326,61356,61385,61415,61445,61474,61504,61533,%
8824 61563,61592,61621,61651,61680,61710,61739,61769,61799,61828,%
8825 61858,61888,61917,61947,61976,62006,62035,62064,62094,62123,%
8826 62153,62182,62212,62242,62271,62301,62331,62360,62390,62419,%
8827 62448,62478,62507,62537,62566,62596,62625,62655,62685,62715,%
8828 62744,62774,62803,62832,62862,62891,62921,62950,62980,63009,%
8829 63039,63069,63099,63128,63157,63187,63216,63246,63275,63305,%
8830 63334,63363,63393,63423,63453,63482,63512,63541,63571,63600,%
8831 63630,63659,63689,63718,63747,63777,63807,63836,63866,63895,%
8832 63925,63955,63984,64014,64043,64073,64102,64131,64161,64190,%
8833 64220,64249,64279,64309,64339,64368,64398,64427,64457,64486,%

```

```

8834 64515,64545,64574,64603,64633,64663,64692,64722,64752,64782,%
8835 64811,64841,64870,64899,64929,64958,64987,65017,65047,65076,%
8836 65106,65136,65166,65195,65225,65254,65283,65313,65342,65371,%
8837 65401,65431,65460,65490,65520}
8838 \namedef{bbl@ca@islamic-umalqura+}{\bbl@ca@islamcuqr@x{+1}}
8839 \namedef{bbl@ca@islamic-umalqura}{\bbl@ca@islamcuqr@x{}}
8840 \@namedef{bbl@ca@islamic-umalqura-}{\bbl@ca@islamcuqr@x{-1}}
8841 \def\bbl@ca@islamcuqr@x#1#2-#3-#4\@@#5#6#7{%
8842 \ifnum#2>2014 \ifnum#2<2038
8843 \bbl@afterfi\expandafter\@gobble
8844 \fi\fi
8845 {\bbl@error{year-out-range}{2014-2038}}{}}%
8846 \edef\bbl@tempd{\fpeval{ % (Julian) day
8847 \bbl@cs@jd{#2}{#3}{#4} + 0.5 - 2400000 #1}}%
8848 \count@\@ne
8849 \bbl@foreach\bbl@cs@umalqura@data{%
8850 \advance\count@\@ne
8851 \ifnum##1>\bbl@tempd\else
8852 \edef\bbl@tempe{\the\count@}%
8853 \edef\bbl@tempb{##1}%
8854 \fi}%
8855 \edef\bbl@templ{\fpeval{ \bbl@tempe + 16260 + 949 }}% month~lunar
8856 \edef\bbl@tempa{\fpeval{ floor((\bbl@templ - 1) / 12) }}% annus
8857 \edef#5{\fpeval{ \bbl@tempa + 1 }}%
8858 \edef#6{\fpeval{ \bbl@templ - (12 * \bbl@tempa) }}%
8859 \edef#7{\fpeval{ \bbl@tempd - \bbl@tempb + 1 }}%
8860 \bbl@add\bbl@precalendar{%
8861 \bbl@replace\bbl@ld@calendar{-civil}}}%
8862 \bbl@replace\bbl@ld@calendar{-umalqura}}}%
8863 \bbl@replace\bbl@ld@calendar{+}}}%
8864 \bbl@replace\bbl@ld@calendar{-}}}%
8865 (/ca-islamic)

```

14.2. Hebrew

This is basically the set of macros written by Michail Rozman in 1991, with corrections and adaptations by Rama Porrat, Misha, Dan Haran and Boris Lavva. This must be eventually replaced by computations with l3fp. An explanation of what's going on can be found in `hebcsl.sty`

```

8866 (*ca-hebrew)
8867 \newcount\bbl@cntcommon
8868 \def\bbl@remainder#1#2#3{%
8869 #3=#1\relax
8870 \divide #3 by #2\relax
8871 \multiply #3 by -#2\relax
8872 \advance #3 by #1\relax}%
8873 \newif\ifbbl@divisible
8874 \def\bbl@checkifdivisible#1#2{%
8875 {\countdef\tmp=0
8876 \bbl@remainder{#1}{#2}{\tmp}%
8877 \ifnum \tmp=0
8878 \global\bbl@divisibletrue
8879 \else
8880 \global\bbl@divisiblefalse
8881 \fi}}
8882 \newif\ifbbl@gregleap
8883 \def\bbl@ifgregleap#1{%
8884 \bbl@checkifdivisible{#1}{4}%
8885 \ifbbl@divisible
8886 \bbl@checkifdivisible{#1}{100}%
8887 \ifbbl@divisible
8888 \bbl@checkifdivisible{#1}{400}%
8889 \ifbbl@divisible
8890 \bbl@gregleaptrue

```

```

8891         \else
8892             \bbl@gregleapfalse
8893         \fi
8894     \else
8895         \bbl@gregleaptrue
8896     \fi
8897 \else
8898     \bbl@gregleapfalse
8899 \fi
8900 \ifbbl@gregleap}
8901 \def\bbl@gregdayspriormonths#1#2#3{%
8902     {#3=\ifcase #1 0 \or 0 \or 31 \or 59 \or 90 \or 120 \or 151 \or
8903         181 \or 212 \or 243 \or 273 \or 304 \or 334 \fi
8904     \bbl@ifgregleap{#2}%
8905     \ifnum #1 > 2
8906         \advance #3 by 1
8907     \fi
8908     \fi
8909     \global\bbl@cntcommon=#3}%
8910 #3=\bbl@cntcommon}
8911 \def\bbl@gregdaysprioryears#1#2{%
8912     {\countdef\tmpc=4
8913     \countdef\tmpb=2
8914     \tmpb=#1\relax
8915     \advance \tmpb by -1
8916     \tmpc=\tmpb
8917     \multiply \tmpc by 365
8918     #2=\tmpc
8919     \tmpc=\tmpb
8920     \divide \tmpc by 4
8921     \advance #2 by \tmpc
8922     \tmpc=\tmpb
8923     \divide \tmpc by 100
8924     \advance #2 by -\tmpc
8925     \tmpc=\tmpb
8926     \divide \tmpc by 400
8927     \advance #2 by \tmpc
8928     \global\bbl@cntcommon=#2\relax}%
8929 #2=\bbl@cntcommon}
8930 \def\bbl@absfromgreg#1#2#3#4{%
8931     {\countdef\tmpd=0
8932     #4=#1\relax
8933     \bbl@gregdayspriormonths{#2}{#3}{\tmpd}%
8934     \advance #4 by \tmpd
8935     \bbl@gregdaysprioryears{#3}{\tmpd}%
8936     \advance #4 by \tmpd
8937     \global\bbl@cntcommon=#4\relax}%
8938 #4=\bbl@cntcommon}
8939 \newif\ifbbl@hebrleap
8940 \def\bbl@checkleaphebryear#1{%
8941     {\countdef\tmpa=0
8942     \countdef\tmpb=1
8943     \tmpa=#1\relax
8944     \multiply \tmpa by 7
8945     \advance \tmpa by 1
8946     \bbl@remainder{\tmpa}{19}{\tmpb}%
8947     \ifnum \tmpb < 7
8948         \global\bbl@hebrleaptrue
8949     \else
8950         \global\bbl@hebrleapfalse
8951     \fi}}
8952 \def\bbl@hebrlapsedmonths#1#2{%
8953     {\countdef\tmpa=0

```

```

8954 \countdef\tmpb=1
8955 \countdef\tmpc=2
8956 \tmpa=#1\relax
8957 \advance \tmpa by -1
8958 #2=\tmpa
8959 \divide #2 by 19
8960 \multiply #2 by 235
8961 \bbl@remainder{\tmpa}{19}{\tmpb}% \tmpa=years%19-years this cycle
8962 \tmpc=\tmpb
8963 \multiply \tmpb by 12
8964 \advance #2 by \tmpb
8965 \multiply \tmpc by 7
8966 \advance \tmpc by 1
8967 \divide \tmpc by 19
8968 \advance #2 by \tmpc
8969 \global\bbl@cntcommon=#2}%
8970 #2=\bbl@cntcommon}
8971 \def\bbl@hebreleapseddays#1#2{%
8972 {\countdef\tmpa=0
8973 \countdef\tmpb=1
8974 \countdef\tmpc=2
8975 \bbl@hebreleapsedmonths{#1}{#2}%
8976 \tmpa=#2\relax
8977 \multiply \tmpa by 13753
8978 \advance \tmpa by 5604
8979 \bbl@remainder{\tmpa}{25920}{\tmpc}% \tmpc == ConjunctionParts
8980 \divide \tmpa by 25920
8981 \multiply #2 by 29
8982 \advance #2 by 1
8983 \advance #2 by \tmpa
8984 \bbl@remainder{#2}{7}{\tmpa}%
8985 \ifnum \tmpc < 19440
8986 \ifnum \tmpc < 9924
8987 \else
8988 \ifnum \tmpa=2
8989 \bbl@checkleaphebyear{#1}% of a common year
8990 \ifbbl@hebrleap
8991 \else
8992 \advance #2 by 1
8993 \fi
8994 \fi
8995 \fi
8996 \ifnum \tmpc < 16789
8997 \else
8998 \ifnum \tmpa=1
8999 \advance #1 by -1
9000 \bbl@checkleaphebyear{#1}% at the end of leap year
9001 \ifbbl@hebrleap
9002 \advance #2 by 1
9003 \fi
9004 \fi
9005 \fi
9006 \else
9007 \advance #2 by 1
9008 \fi
9009 \bbl@remainder{#2}{7}{\tmpa}%
9010 \ifnum \tmpa=0
9011 \advance #2 by 1
9012 \else
9013 \ifnum \tmpa=3
9014 \advance #2 by 1
9015 \else
9016 \ifnum \tmpa=5

```

```

9017         \advance #2 by 1
9018     \fi
9019 \fi
9020 \fi
9021 \global\bbl@cntcommon=#2\relax}%
9022 #2=\bbl@cntcommon}
9023 \def\bbl@daysinhebrewyear#1#2{%
9024     {\countdef\tmpe=12
9025     \bbl@hebreleapseddays{#1}{\tmpe}%
9026     \advance #1 by 1
9027     \bbl@hebreleapseddays{#1}{#2}%
9028     \advance #2 by -\tmpe
9029     \global\bbl@cntcommon=#2}%
9030 #2=\bbl@cntcommon}
9031 \def\bbl@hebrdayspriormonths#1#2#3{%
9032     {\countdef\tmpf= 14
9033     #3=\ifcase #1
9034         0 \or
9035         0 \or
9036         30 \or
9037         59 \or
9038         89 \or
9039         118 \or
9040         148 \or
9041         148 \or
9042         177 \or
9043         207 \or
9044         236 \or
9045         266 \or
9046         295 \or
9047         325 \or
9048         400
9049     \fi
9050     \bbl@checkleaphebrewyear{#2}%
9051     \ifbbl@hebrleap
9052         \ifnum #1 > 6
9053             \advance #3 by 30
9054         \fi
9055     \fi
9056     \bbl@daysinhebrewyear{#2}{\tmpf}%
9057     \ifnum #1 > 3
9058         \ifnum \tmpf=353
9059             \advance #3 by -1
9060         \fi
9061         \ifnum \tmpf=383
9062             \advance #3 by -1
9063         \fi
9064     \fi
9065     \ifnum #1 > 2
9066         \ifnum \tmpf=355
9067             \advance #3 by 1
9068         \fi
9069         \ifnum \tmpf=385
9070             \advance #3 by 1
9071         \fi
9072     \fi
9073     \global\bbl@cntcommon=#3\relax}%
9074 #3=\bbl@cntcommon}
9075 \def\bbl@absfromhebr#1#2#3#4{%
9076     {#4=#1\relax
9077     \bbl@hebrdayspriormonths{#2}{#3}{#1}%
9078     \advance #4 by #1\relax
9079     \bbl@hebreleapseddays{#3}{#1}%

```

```

9080 \advance #4 by #1\relax
9081 \advance #4 by -1373429
9082 \global\bbl@cntcommon=#4\relax}%
9083 #4=\bbl@cntcommon}
9084 \def\bbl@hebrfromgreg#1#2#3#4#5#6{%
9085 {\countdef\tmpx= 17
9086 \countdef\tmpy= 18
9087 \countdef\tmpz= 19
9088 #6=#3\relax
9089 \global\advance #6 by 3761
9090 \bbl@absfromgreg{#1}{#2}{#3}{#4}%
9091 \tmpz=1 \tmpy=1
9092 \bbl@absfromhebr{\tmpz}{\tmpy}{#6}{\tmpx}%
9093 \ifnum \tmpx > #4\relax
9094 \global\advance #6 by -1
9095 \bbl@absfromhebr{\tmpz}{\tmpy}{#6}{\tmpx}%
9096 \fi
9097 \advance #4 by -\tmpx
9098 \advance #4 by 1
9099 #5=#4\relax
9100 \divide #5 by 30
9101 \loop
9102 \bbl@hebrdayspriormonths{#5}{#6}{\tmpx}%
9103 \ifnum \tmpx < #4\relax
9104 \advance #5 by 1
9105 \tmpy=\tmpx
9106 \repeat
9107 \global\advance #5 by -1
9108 \global\advance #4 by -\tmpy}}
9109 \newcount\bbl@hebrday \newcount\bbl@hebrmonth \newcount\bbl@hebryear
9110 \newcount\bbl@gregday \newcount\bbl@gregmonth \newcount\bbl@gregyear
9111 \def\bbl@ca@hebrew#1-#2-#3\@@#4#5#6{%
9112 \bbl@gregday=#3\relax \bbl@gregmonth=#2\relax \bbl@gregyear=#1\relax
9113 \bbl@hebrfromgreg
9114 {\bbl@gregday}{\bbl@gregmonth}{\bbl@gregyear}%
9115 {\bbl@hebrday}{\bbl@hebrmonth}{\bbl@hebryear}%
9116 \edef#4{\the\bbl@hebryear}%
9117 \edef#5{\the\bbl@hebrmonth}%
9118 \edef#6{\the\bbl@hebrday}}
9119 (/ca-hebrew)

```

14.3. Persian

There is an algorithm written in TeX by Jabri, Abolhassani, Pournader and Esfahbod, created for the first versions of the FarsiTeX system (no longer available), but the original license is GPL, so its use with LPL is problematic. The code here follows loosely that by John Walker, which is free and accurate, but sadly very complex, so the relevant data for the years 2013-2050 have been pre-calculated and stored. Actually, all we need is the first day (either March 20 or March 21).

```

9120 (*ca-persian)
9121 <@Compute Julian day@>
9122 \def\bbl@cs@firstjal@xx{2012,2016,2020,2024,2028,2029,% March 20
9123 2032,2033,2036,2037,2040,2041,2044,2045,2048,2049}
9124 \def\bbl@ca@persian#1-#2-#3\@@#4#5#6{%
9125 \edef\bbl@tempa{#1}% 20XX-03-\bbl@tempe = 1 farvardin:
9126 \ifnum\bbl@tempa>2012 \ifnum\bbl@tempa<2051
9127 \bbl@afterfi\expandafter\@gobble
9128 \fi\fi
9129 {\bbl@error{year-out-range}{2013-2050}{}}}%
9130 \bbl@xin{\bbl@tempa}{\bbl@cs@firstjal@xx}%
9131 \ifin@def\bbl@tempe{20}\else\def\bbl@tempe{21}\fi
9132 \edef\bbl@tempc{\fpeval{\bbl@cs@jd{\bbl@tempa}{#2}{#3}+.5}}% current
9133 \edef\bbl@tempb{\fpeval{\bbl@cs@jd{\bbl@tempa}{03}{\bbl@tempe}+.5}}% begin
9134 \ifnum\bbl@tempc<\bbl@tempb

```

```

9135 \edef\bbl@tempa{\fpeval{\bbl@tempa-1}}% go back 1 year and redo
9136 \bbl@xin@{\bbl@tempa}{\bbl@cs@firstjal@xx}%
9137 \ifin@def\bbl@tempe{20}\else\def\bbl@tempe{21}\fi
9138 \edef\bbl@tempb{\fpeval{\bbl@cs@jd{\bbl@tempa}{03}{\bbl@tempe}+.5}}%
9139 \fi
9140 \edef#4{\fpeval{\bbl@tempa-621}}% set Jalali year
9141 \edef#6{\fpeval{\bbl@tempc-\bbl@tempb+1}}% days from 1 farvardin
9142 \edef#5{\fpeval{% set Jalali month
9143   (#6 <= 186) ? ceil(#6 / 31) : ceil((#6 - 6) / 30)}}
9144 \edef#6{\fpeval{% set Jalali day
9145   (#6 - ((#5 <= 7) ? ((#5 - 1) * 31) : ((#5 - 1) * 30) + 6))}}
9146 (/ca-persian)

```

14.4. Coptic and Ethiopic

Adapted from `jquery.calendars.package-1.1.4`, written by Keith Wood, 2010. Dual license: GPL and MIT. The only difference is the epoch.

```

9147 (*ca-coptic)
9148 <@Compute Julian day@>
9149 \def\bbl@ca@coptic#1-#2-#3\@@#4#5#6{%
9150 \edef\bbl@tempd{\fpeval{floor(\bbl@cs@jd{#1}{#2}{#3}) + 0.5}}%
9151 \edef\bbl@tempc{\fpeval{\bbl@tempd - 1825029.5}}%
9152 \edef#4{\fpeval{%
9153   floor((\bbl@tempc - floor((\bbl@tempc+366) / 1461)) / 365) + 1}}%
9154 \edef\bbl@tempc{\fpeval{%
9155   \bbl@tempd - (#4-1) * 365 - floor(#4/4) - 1825029.5}}%
9156 \edef#5{\fpeval{floor(\bbl@tempc / 30) + 1}}%
9157 \edef#6{\fpeval{\bbl@tempc - (#5 - 1) * 30 + 1}}
9158 (/ca-coptic)
9159 (*ca-ethiopic)
9160 <@Compute Julian day@>
9161 \def\bbl@ca@ethiopic#1-#2-#3\@@#4#5#6{%
9162 \edef\bbl@tempd{\fpeval{floor(\bbl@cs@jd{#1}{#2}{#3}) + 0.5}}%
9163 \edef\bbl@tempc{\fpeval{\bbl@tempd - 1724220.5}}%
9164 \edef#4{\fpeval{%
9165   floor((\bbl@tempc - floor((\bbl@tempc+366) / 1461)) / 365) + 1}}%
9166 \edef\bbl@tempc{\fpeval{%
9167   \bbl@tempd - (#4-1) * 365 - floor(#4/4) - 1724220.5}}%
9168 \edef#5{\fpeval{floor(\bbl@tempc / 30) + 1}}%
9169 \edef#6{\fpeval{\bbl@tempc - (#5 - 1) * 30 + 1}}
9170 (/ca-ethiopic)

```

14.5. Julian

Based on [ReinDersh].

```

9171 (*ca-julian)
9172 <@Compute Julian day@>
9173 \def\bbl@ca@julian#1-#2-#3\@@#4#5#6{%
9174 \edef\bbl@tempj{\fpeval{floor(\bbl@cs@jd{#1}{#2}{#3}) + .5}}%
9175 \edef\bbl@tempa{\fpeval{\bbl@tempj + 32082.5}}%
9176 \edef\bbl@tempb{\fpeval{floor((4 * \bbl@tempa + 3) / 1461)}}%
9177 \edef\bbl@tempc{\fpeval{\bbl@tempa - floor(1461*\bbl@tempb/4)}}%
9178 \edef\bbl@tempd{\fpeval{floor((5 * \bbl@tempc + 2) / 153)}}%
9179 \edef#6{\fpeval{\bbl@tempc - floor((153*\bbl@tempd+2) / 5) + 1}}%
9180 \edef#5{\fpeval{\bbl@tempd + 3 - 12 * floor(\bbl@tempd / 10)}}%
9181 \edef#4{\fpeval{\bbl@tempb - 4800 + floor(\bbl@tempd / 10)}}
9182 (/ca-julian)

```

The Indian National Calendar, Saka era.

```

9183 (*ca-indian)
9184 <@Compute Julian day@>
9185 \def\bbl@sakachaitra#1{%

```

```

9186 \bbl@cs@jd{#1}{3}{%
9187 \fpeval{ ((\bbl@fpmmod{#1}{4})==0 && \bbl@fpmmod{#1}{100}!=0)
9188 ||\bbl@fpmmod{#1}{400}==0) ? 21 : 22 }}
9189 \def\bbl@ca@indian#1-#2-#3\@#4#5#6{%
9190 \edef\bbl@tempa{\fpeval{\bbl@cs@jd{#1}{#2}{#3}}}%
9191 \edef\bbl@tempb{\fpeval{\bbl@sakachaitra{#1}}}%
9192 \edef\bbl@tempc{\fpeval{\bbl@tempa < \bbl@tempb ? #1-1 : #1}}%
9193 \edef\bbl@tempd{\fpeval{\bbl@tempa-\bbl@sakachaitra{\bbl@tempc}}}%
9194 \edef\bbl@tempe{\fpeval{ ((\bbl@fpmmod{\bbl@tempc}{4})==0 &&
9195 \bbl@fpmmod{\bbl@tempc}{100}!=0) || \bbl@fpmmod{\bbl@tempc}{400}==0)?31:30}}%
9196 \edef#4{\fpeval{\bbl@tempc-78}}%
9197 \edef#5{\fpeval{\bbl@tempd<\bbl@tempe ? 1 :
9198 (\bbl@tempd-\bbl@tempe < 155 ?
9199 2+trunc((\bbl@tempd-\bbl@tempe)/31,0) :
9200 7+trunc((\bbl@tempd-\bbl@tempe-155)/30,0))}}%
9201 \edef#6{\fpeval{\bbl@tempd<\bbl@tempe ? \bbl@tempd+1 :
9202 (\bbl@tempd-\bbl@tempe < 155 ?
9203 \bbl@fpmmod{\bbl@tempd-\bbl@tempe}{31}+1 :
9204 \bbl@fpmmod{\bbl@tempd-\bbl@tempe-155}{30}+1)}}
9205 {/ca-indian}
9206 %

```

14.6. Buddhist

That's very simple.

```

9207 {*ca-buddhist}
9208 \def\bbl@ca@buddhist#1-#2-#3\@#4#5#6{%
9209 \edef#4{\number\numexpr#1+543\relax}%
9210 \edef#5{#2}%
9211 \edef#6{#3}}
9212 {/ca-buddhist}
9213 %
9214 % \subsection{Chinese}
9215 %
9216 % Brute force, with the Julian day of first day of each month. The
9217 % table has been computed with the help of \textsf{python-lunardate} by
9218 % Ricky Yeung, GPLv2 (but the code itself has not been used). The range
9219 % is 2015-2044.
9220 %
9221 % \begin{macrocode}
9222 {*ca-chinese}
9223 \ExplSyntaxOn
9224 <@Compute Julian day@>
9225 \def\bbl@ca@chinese#1-#2-#3\@#4#5#6{%
9226 \edef\bbl@tempd{\fpeval{%
9227 \bbl@cs@jd{#1}{#2}{#3} - 2457072.5 }}%
9228 \count@ \z@
9229 \@tempcnta=2015
9230 \bbl@foreach\bbl@cs@chinese@data{%
9231 \ifnum##1>\bbl@tempd\else
9232 \advance\count@\@ne
9233 \ifnum\count@>12
9234 \count@\@ne
9235 \advance\@tempcnta\@ne\fi
9236 \bbl@xin@{,##1,},{,\bbl@cs@chinese@leap,}%
9237 \ifin@
9238 \advance\count@\m@ne
9239 \edef\bbl@tempe{\the\numexpr\count@+12\relax}%
9240 \else
9241 \edef\bbl@tempe{\the\count@}%
9242 \fi
9243 \edef\bbl@tempb{##1}%
9244 \fi}%

```

```

9245 \edef#4{\the\@tempcnta}%
9246 \edef#5{\bbl@tempe}%
9247 \edef#6{\the\numexpr\bbl@tempd-\bbl@tempb+1\relax}}
9248 \def\bbl@cs@chinese@leap{%
9249 885,1920,2953,3809,4873,5906,6881,7825,8889,9893,10778}
9250 \def\bbl@cs@chinese@data{0,29,59,88,117,147,176,206,236,266,295,325,
9251 354,384,413,443,472,501,531,560,590,620,649,679,709,738,%
9252 768,797,827,856,885,915,944,974,1003,1033,1063,1093,1122,%
9253 1152,1181,1211,1240,1269,1299,1328,1358,1387,1417,1447,1477,%
9254 1506,1536,1565,1595,1624,1653,1683,1712,1741,1771,1801,1830,%
9255 1860,1890,1920,1949,1979,2008,2037,2067,2096,2126,2155,2185,%
9256 2214,2244,2274,2303,2333,2362,2392,2421,2451,2480,2510,2539,%
9257 2569,2598,2628,2657,2687,2717,2746,2776,2805,2835,2864,2894,%
9258 2923,2953,2982,3011,3041,3071,3100,3130,3160,3189,3219,3248,%
9259 3278,3307,3337,3366,3395,3425,3454,3484,3514,3543,3573,3603,%
9260 3632,3662,3691,3721,3750,3779,3809,3838,3868,3897,3927,3957,%
9261 3987,4016,4046,4075,4105,4134,4163,4193,4222,4251,4281,4311,%
9262 4341,4370,4400,4430,4459,4489,4518,4547,4577,4606,4635,4665,%
9263 4695,4724,4754,4784,4814,4843,4873,4902,4931,4961,4990,5019,%
9264 5049,5079,5108,5138,5168,5197,5227,5256,5286,5315,5345,5374,%
9265 5403,5433,5463,5492,5522,5551,5581,5611,5640,5670,5699,5729,%
9266 5758,5788,5817,5846,5876,5906,5935,5965,5994,6024,6054,6083,%
9267 6113,6142,6172,6201,6231,6260,6289,6319,6348,6378,6408,6437,%
9268 6467,6497,6526,6556,6585,6615,6644,6673,6703,6732,6762,6791,%
9269 6821,6851,6881,6910,6940,6969,6999,7028,7057,7087,7116,7146,%
9270 7175,7205,7235,7264,7294,7324,7353,7383,7412,7441,7471,7500,%
9271 7529,7559,7589,7618,7648,7678,7708,7737,7767,7796,7825,7855,%
9272 7884,7913,7943,7972,8002,8032,8062,8092,8121,8151,8180,8209,%
9273 8239,8268,8297,8327,8356,8386,8416,8446,8475,8505,8534,8564,%
9274 8593,8623,8652,8681,8711,8740,8770,8800,8829,8859,8889,8918,%
9275 8948,8977,9007,9036,9066,9095,9124,9154,9183,9213,9243,9272,%
9276 9302,9331,9361,9391,9420,9450,9479,9508,9538,9567,9597,9626,%
9277 9656,9686,9715,9745,9775,9804,9834,9863,9893,9922,9951,9981,%
9278 10010,10040,10069,10099,10129,10158,10188,10218,10247,10277,%
9279 10306,10335,10365,10394,10423,10453,10483,10512,10542,10572,%
9280 10602,10631,10661,10690,10719,10749,10778,10807,10837,10866,%
9281 10896,10926,10956,10986,11015,11045,11074,11103}
9282 \ExplSyntaxOff
9283 (/ca-chinese)

```

15. Support for Plain T_EX (plain.def)

15.1. Not renaming hyphen.tex

As Don Knuth has declared that the filename `hyphen.tex` may only be used to designate *his* version of the american English hyphenation patterns, a new solution has to be found in order to be able to load hyphenation patterns for other languages in a plain-based T_EX-format. When asked he responded:

That file name is “sacred”, and if anybody changes it they will cause severe upward/downward compatibility headaches.

People can have a file `locallyhyphen.tex` or whatever they like, but they mustn’t diddle with `hyphen.tex` (or `plain.tex` except to preload additional fonts).

The files `bplain.tex` and `blplain.tex` can be used as replacement wrappers around `plain.tex` and `lplain.tex` to achieve the desired effect, based on the `babel` package. If you load each of them with `iniTEX`, you will get a file called either `bplain.fmt` or `blplain.fmt`, which you can use as replacements for `plain.fmt` and `lplain.fmt`.

As these files are going to be read as the first thing `iniTEX` sees, we need to set some category codes just to be able to change the definition of `\input`.

```

9284 (*bplain|blplain)
9285 \catcode`\{=1 % left brace is begin-group character
9286 \catcode`\}=2 % right brace is end-group character
9287 \catcode`\#=6 % hash mark is macro parameter character

```

If a file called `hyphen.cfg` can be found, we make sure that *it* will be read instead of the file `hyphen.tex`. We do this by first saving the original meaning of `\input` (and I use a one letter control sequence for that so as not to waste multi-letter control sequence on this in the format).

```
9288 \openin 0 hyphen.cfg
9289 \ifeof0
9290 \else
9291   \let\input
```

Then `\input` is defined to forget about its argument and load `hyphen.cfg` instead. Once that's done the original meaning of `\input` can be restored and the definition of `\a` can be forgotten.

```
9292   \def\input #1 {%
9293     \let\input\input
9294     \a hyphen.cfg
9295     \let\input\undefined
9296   }
9297 \fi
9298 (/bplain|bplain)
```

Now that we have made sure that `hyphen.cfg` will be loaded at the right moment it is time to load `plain.tex`.

```
9299 (bplain)\a plain.tex
9300 (bplain)\a lplain.tex
```

Finally we change the contents of `\fmtname` to indicate that this is *not* the plain format, but a format based on plain with the `babel` package preloaded.

```
9301 (bplain)\def\fmtname{babel-plain}
9302 (bplain)\def\fmtname{babel-lplain}
```

When you are using a different format, based on `plain.tex` you can make a copy of `blplain.tex`, rename it and replace `plain.tex` with the name of your format file.

15.2. Emulating some $\text{\LaTeX}$ features

The file `babel.def` expects some definitions made in the $\text{\LaTeX} 2_{\epsilon}$ style file. So, in Plain we must provide at least some predefined values as well some tools to set them (even if not all options are available). There are no package options, and therefore and alternative mechanism is provided. For the moment, only `\babeloptionstrings` and `\babeloptionmath` are provided, which can be defined before loading `babel`. `\BabelModifiers` can be set too (but not sure it works).

```
9303 ((*Emulate LaTeX)) ≡
9304 \def\@empty{}
9305 \def\loadlocalcfg#1{%
9306   \openin0#1.cfg
9307   \ifeof0
9308     \closein0
9309   \else
9310     \closein0
9311     {\immediate\writel6{*****}%
9312      \immediate\writel6{* Local config file #1.cfg used}%
9313      \immediate\writel6{*}%
9314     }
9315     \input #1.cfg\relax
9316   \fi
9317   \@endoflfd}
```

15.3. General tools

A number of $\text{\LaTeX}$ macro's that are needed later on.

```
9318 \long\def\@firstofone#1{#1}
9319 \long\def\@firstoftwo#1#2{#1}
9320 \long\def\@secondoftwo#1#2{#2}
9321 \def\@nnil{\nil}
9322 \def\@gobbletwo#1#2{}
9323 \def\@ifstar#1{\@ifnextchar *{\@firstoftwo{#1}}}
```

```

9324 \def\@star@or@long#1{%
9325   \ifstar
9326     {\let\l@ngrel@x\relax#1}%
9327     {\let\l@ngrel@x\long#1}}
9328 \let\l@ngrel@x\relax
9329 \def\@car#1#2\@nil{#1}
9330 \def\@cdr#1#2\@nil{#2}
9331 \let\@typeset@protect\relax
9332 \let\protected@edef\edef
9333 \long\def\@gobble#1{}
9334 \edef\@backslashchar{\expandafter\@gobble\string\}
9335 \def\strip@prefix#1>{}
9336 \def\g@addto@macro#1#2{%
9337   \toks@\expandafter{#1#2}%
9338   \xdef#1{\the\toks@}}
9339 \def\@namedef#1{\expandafter\def\csname #1\endcsname}
9340 \def\@nameuse#1{\csname #1\endcsname}
9341 \def\@ifundefined#1{%
9342   \expandafter\ifx\csname#1\endcsname\relax
9343     \expandafter\@firstoftwo
9344   \else
9345     \expandafter\@secondoftwo
9346   \fi}
9347 \def\@expandtwoargs#1#2#3{%
9348   \edef\reserved@a{\noexpand#1{#2}{#3}}\reserved@a}
9349 \def\zap@space#1 #2{%
9350   #1%
9351   \ifx#2\@empty\else\expandafter\zap@space\fi
9352   #2}
9353 \let\bbl@trace\@gobble
9354 \def\bbl@error#1{% Implicit #2#3#4
9355   \begingroup
9356     \catcode`\=0 \catcode`\==12 \catcode`\`=12
9357     \catcode`\^M=5 \catcode`\%=14
9358     \input errbabel.def
9359   \endgroup
9360   \bbl@error{#1}}
9361 \def\bbl@warning#1{%
9362   \begingroup
9363     \newlinechar=`^^J
9364     \def\{^^J(babel) }%
9365     \message{\{#1}%
9366   \endgroup}
9367 \let\bbl@infowarn\bbl@warning
9368 \def\bbl@info#1{%
9369   \begingroup
9370     \newlinechar=`^^J
9371     \def\{^^J}%
9372     \wlog{#1}%
9373   \endgroup}

```

$\LaTeX 2_{\epsilon}$ has the command `\onlypreamble` which adds commands to a list of commands that are no longer needed after `\begin{document}`.

```

9374 \ifx\@preamblecmds\undefined
9375   \def\@preamblecmds{}
9376 \fi
9377 \def\@onlypreamble#1{%
9378   \expandafter\gdef\expandafter\@preamblecmds\expandafter{%
9379     \@preamblecmds\do#1}}
9380 \@onlypreamble\@onlypreamble

```

Mimic $\LaTeX$'s `\AtBeginDocument`; for this to work the user needs to add `\begindocument` to his file.

```

9381 \def\begindocument{%
9382   \@begindocumenthook

```

```

9383 \global\let\@begindocumenthook\@undefined
9384 \def\do##1{\global\let##1\@undefined}%
9385 \@preamblecmds
9386 \global\let\do\noexpand}

9387 \ifx\@begindocumenthook\@undefined
9388 \def\@begindocumenthook{}
9389 \fi
9390 \@onlypreamble\@begindocumenthook
9391 \def\AtBeginDocument{\g@addto@macro\@begindocumenthook}

```

We also have to mimic $\LaTeX$'s `\AtEndOfPackage`. Our replacement macro is much simpler; it stores its argument in `\@endoflfd`.

```

9392 \def\AtEndOfPackage#1{\g@addto@macro\@endoflfd{#1}}
9393 \@onlypreamble\AtEndOfPackage
9394 \def\@endoflfd{}
9395 \@onlypreamble\@endoflfd
9396 \let\bbl@afterlang\@empty
9397 \chardef\bbl@opt@hyphenmap\z@

```

$\LaTeX$ needs to be able to switch off writing to its auxiliary files; plain doesn't have them by default. There is a trick to hide some conditional commands from the outer `\ifx`. The same trick is applied below.

```

9398 \catcode`\&=\z@
9399 \ifx&\if@files\@undefined
9400 \expandafter\let\csname if@files\expandafter\endcsname
9401 \csname iffalse\endcsname
9402 \fi
9403 \catcode`\&=4

```

Mimic $\LaTeX$'s commands to define control sequences.

```

9404 \def\newcommand{\@star@or@long\new@command}
9405 \def\new@command#1{%
9406 \testopt{\@newcommand#1}0}
9407 \def\@newcommand#1[#2]{%
9408 \ifnextchar [{\@xargdef#1[#2]}%
9409 {\@argdef#1[#2]}}
9410 \long\def\@argdef#1[#2]#3{%
9411 \@yargdef#1\@ne{#2}{#3}}
9412 \long\def\@xargdef#1[#2][#3]#4{%
9413 \expandafter\def\expandafter#1\expandafter{%
9414 \expandafter\@protected@testopt\expandafter #1%
9415 \csname\string#1\expandafter\endcsname{#3}}}%
9416 \expandafter\@yargdef \csname\string#1\endcsname
9417 \tw@{#2}{#4}}
9418 \long\def\@yargdef#1#2#3{%
9419 \@tempcnta#3\relax
9420 \advance \@tempcnta \@ne
9421 \let\@hash@\relax
9422 \edef\reserved@a{\ifx#2\tw@ [\@hash@1]\fi}%
9423 \@tempcntb #2%
9424 \@whilenum\@tempcntb <\@tempcnta
9425 \do{%
9426 \edef\reserved@a{\reserved@a\@hash@\the\@tempcntb}%
9427 \advance\@tempcntb \@ne}%
9428 \let\@hash@##%
9429 \l@ngrelx\expandafter\def\expandafter#1\reserved@a}
9430 \def\providecommand{\@star@or@long\provide@command}
9431 \def\provide@command#1{%
9432 \begingroup
9433 \escapechar\m@ne\def\@gtempa{\string#1}%
9434 \endgroup
9435 \expandafter\ifundefined\@gtempa
9436 {\def\reserved@a{\new@command#1}}%

```

```

9437     {\let\reserved@a\relax
9438     \def\reserved@a{\new@command\reserved@a}}}%
9439     \reserved@a}%

9440 \def\DeclareRobustCommand{\@star@or@long\declare@robustcommand}
9441 \def\declare@robustcommand#1{%
9442     \edef\reserved@a{\string#1}%
9443     \def\reserved@b{#1}%
9444     \edef\reserved@b{\expandafter\strip@prefix\meaning\reserved@b}%
9445     \edef#1{%
9446         \ifx\reserved@a\reserved@b
9447             \noexpand\x@protect
9448             \noexpand#1%
9449         \fi
9450         \noexpand\protect
9451         \expandafter\noexpand\csname
9452             \expandafter\@gobble\string#1 \endcsname
9453     }%
9454     \expandafter\new@command\csname
9455         \expandafter\@gobble\string#1 \endcsname
9456 }
9457 \def\x@protect#1{%
9458     \ifx\protect\@typeset@protect\else
9459         \x@protect#1%
9460     \fi
9461 }
9462 \catcode`\&=\z@ % Trick to hide conditionals
9463 \def\x@protect#1&fi#2#3{&fi\protect#1}

```

The following little macro `\in@` is taken from `latex.ltx`; it checks whether its first argument is part of its second argument. It uses the boolean `\in@`; allocating a new boolean inside conditionally executed code is not possible, hence the construct with the temporary definition of `\bbl@tempa`.

```

9464 \def\bbl@tempa{\csname newif\endcsname&ifin@}
9465 \catcode`\&=4
9466 \ifx\in@\@undefined
9467     \def\in@#1#2{%
9468         \def\in@@##1#1##2##3\in@@{%
9469             \ifx\in@@##2\in@false\else\in@true\fi}%
9470         \in@@##2#1\in@\in@@}
9471 \else
9472     \let\bbl@tempa\@empty
9473 \fi
9474 \bbl@tempa

```

$\TeX$ has a macro to check whether a certain package was loaded with specific options. The command has two extra arguments which are code to be executed in either the true or false case. This is used to detect whether the document needs one of the accents to be activated (activegrave and activeacute). For plain $\TeX$ we assume that the user wants them to be active by default. Therefore the only thing we do is execute the third argument (the code for the true case).

```

9475 \def\@ifpackagewith#1#2#3#4{#3}

```

The $\TeX$ macro `\@ifl@aded` checks whether a file was loaded. This functionality is not needed for plain $\TeX$ but we need the macro to be defined as a no-op.

```

9476 \def\@ifl@aded#1#2#3#4{}

```

For the following code we need to make sure that the commands `\newcommand` and `\providecommand` exist with some sensible definition. They are not fully equivalent to their $\TeX 2_{\epsilon}$ versions; just enough to make things work in plain $\TeX$ environments.

```

9477 \ifx\@tempcnta\@undefined
9478     \csname newcount\endcsname\@tempcnta\relax
9479 \fi
9480 \ifx\@tempcntb\@undefined
9481     \csname newcount\endcsname\@tempcntb\relax
9482 \fi

```

To prevent wasting two counters in $\TeX$ (because counters with the same name are allocated later by it) we reset the counter that holds the next free counter (`\count10`).

```

9483 \ifx\bye\@undefined
9484   \advance\count10 by -2\relax
9485 \fi
9486 \ifx\@ifnextchar\@undefined
9487   \def\@ifnextchar#1#2#3{%
9488     \let\reserved@d=#1%
9489     \def\reserved@a{#2}\def\reserved@b{#3}%
9490     \futurelet\@let@token\@ifnch}
9491 \def\@ifnch{%
9492   \ifx\@let@token\@sptoken
9493     \let\reserved@c\@xifnch
9494   \else
9495     \ifx\@let@token\reserved@d
9496       \let\reserved@c\reserved@a
9497     \else
9498       \let\reserved@c\reserved@b
9499     \fi
9500   \fi
9501   \reserved@c}
9502 \def:{\let\@sptoken= } \: % this makes \@sptoken a space token
9503 \def:{\@xifnch} \expandafter\def: {\futurelet\@let@token\@ifnch}
9504 \fi
9505 \def\@testopt#1#2{%
9506   \@ifnextchar[#{1}{#1[#2]}}
9507 \def\@protected@testopt#1{%
9508   \ifx\protect\@typeset@protect
9509     \expandafter\@testopt
9510   \else
9511     \@x@protect#1%
9512   \fi}
9513 \long\def\@whilenum#1\do #2{\ifnum #1\relax #2\relax\@iwhilenum{#1\relax
9514   #2\relax}\fi}
9515 \long\def\@iwhilenum#1{\ifnum #1\expandafter\@iwhilenum
9516   \else\expandafter\@gobble\fi{#1}}

```

15.4. Encoding related macros

Code from `ltoutenc.dtx`, adapted for use in the plain $\TeX$ environment.

```

9517 \def\DeclareTextCommand{%
9518   \@dec@text@cmd\providecommand
9519 }
9520 \def\ProvideTextCommand{%
9521   \@dec@text@cmd\providecommand
9522 }
9523 \def\DeclareTextSymbol#1#2#3{%
9524   \@dec@text@cmd\chardef#1{#2}#3\relax
9525 }
9526 \def\@dec@text@cmd#1#2#3{%
9527   \expandafter\def\expandafter#2%
9528     \expandafter{%
9529       \csname#3-cmd\expandafter\endcsname
9530       \expandafter#2%
9531       \csname#3\string#2\endcsname
9532     }%
9533 %   \let\@ifdefinable\@rc@ifdefinable
9534   \expandafter#1\csname#3\string#2\endcsname
9535 }
9536 \def\@current@cmd#1{%
9537   \ifx\protect\@typeset@protect\else
9538     \noexpand#1\expandafter\@gobble

```

```

9539 \fi
9540 }
9541 \def\@changed@cmd#1#2{%
9542 \ifx\protect\@typeset@protect
9543 \expandafter\ifx\csname\cf@encoding\string#1\endcsname\relax
9544 \expandafter\ifx\csname ?\string#1\endcsname\relax
9545 \expandafter\def\csname ?\string#1\endcsname{%
9546 \@changed@x@err{#1}%
9547 }%
9548 \fi
9549 \global\expandafter\let
9550 \csname\cf@encoding\string#1\endcsname\expandafter\endcsname
9551 \csname ?\string#1\endcsname
9552 \fi
9553 \csname\cf@encoding\string#1%
9554 \expandafter\endcsname
9555 \else
9556 \noexpand#1%
9557 \fi
9558 }
9559 \def\@changed@x@err#1{%
9560 \errhelp{Your command will be ignored, type <return> to proceed}%
9561 \errmessage{Command \protect#1 undefined in encoding \cf@encoding}}
9562 \def\DeclareTextCommandDefault#1{%
9563 \DeclareTextCommand#1?%
9564 }
9565 \def\ProvideTextCommandDefault#1{%
9566 \ProvideTextCommand#1?%
9567 }
9568 \expandafter\let\csname OT1-cmd\endcsname\@current@cmd
9569 \expandafter\let\csname?-cmd\endcsname\@changed@cmd
9570 \def\DeclareTextAccent#1#2#3{%
9571 \DeclareTextCommand#1{#2}[1]{\accent#3 #1}
9572 }
9573 \def\DeclareTextCompositeCommand#1#2#3#4{%
9574 \expandafter\let\expandafter\reserved@a\csname#2\string#1\endcsname
9575 \edef\reserved@b{\string##1}%
9576 \edef\reserved@c{%
9577 \expandafter\@strip@args\meaning\reserved@a:-\@strip@args}%
9578 \ifx\reserved@b\reserved@c
9579 \expandafter\expandafter\expandafter\ifx
9580 \expandafter\@car\reserved@a\relax\relax\@nil
9581 \@text@composite
9582 \else
9583 \edef\reserved@b##1{%
9584 \def\expandafter\noexpand
9585 \csname#2\string#1\endcsname###1{%
9586 \noexpand\@text@composite
9587 \expandafter\noexpand\csname#2\string#1\endcsname
9588 ###1\noexpand\@empty\noexpand\@text@composite
9589 {##1}%
9590 }%
9591 }%
9592 \expandafter\reserved@b\expandafter{\reserved@a{##1}}%
9593 \fi
9594 \expandafter\def\csname\expandafter\string\csname
9595 #2\endcsname\string#1-\string#3\endcsname{#4}
9596 \else
9597 \errhelp{Your command will be ignored, type <return> to proceed}%
9598 \errmessage{\string\DeclareTextCompositeCommand\space used on
9599 inappropriate command \protect#1}
9600 \fi
9601 }

```

```

9602 \def\@text@composite#1#2#3\@text@composite{%
9603   \expandafter\@text@composite@x
9604     \csname\string#1-\string#2\endcsname
9605 }
9606 \def\@text@composite@x#1#2{%
9607   \ifx#1\relax
9608     #2%
9609   \else
9610     #1%
9611   \fi
9612 }
9613 %
9614 \def\@strip@args#1:#2-#3\@strip@args{#2}
9615 \def\DeclareTextComposite#1#2#3#4{%
9616   \def\reserved@a{\DeclareTextCompositeCommand#1{#2}{#3}}%
9617   \bgroup
9618     \lccode\@=#4%
9619     \lowercase{%
9620   \egroup
9621     \reserved@a \@%
9622   }%
9623 }
9624 %
9625 \def\UseTextSymbol#1#2{#2}
9626 \def\UseTextAccent#1#2#3{}
9627 \def\@use@text@encoding#1{}
9628 \def\DeclareTextSymbolDefault#1#2{%
9629   \DeclareTextCommandDefault#1{\UseTextSymbol{#2}#1}%
9630 }
9631 \def\DeclareTextAccentDefault#1#2{%
9632   \DeclareTextCommandDefault#1{\UseTextAccent{#2}#1}%
9633 }
9634 \def\cf@encoding{OT1}

```

Currently we only use the $\text{\LaTeX 2\epsilon}$ method for accents for those that are known to be made active in *some* language definition file.

```

9635 \DeclareTextAccent{"}{OT1}{127}
9636 \DeclareTextAccent{'}{OT1}{19}
9637 \DeclareTextAccent{^}{OT1}{94}
9638 \DeclareTextAccent{\`}{OT1}{18}
9639 \DeclareTextAccent{\~}{OT1}{126}

```

The following control sequences are used in `babel.def` but are not defined for PLAIN $\text{\TeX}$.

```

9640 \DeclareTextSymbol{\textquotedblleft}{OT1}{92}
9641 \DeclareTextSymbol{\textquotedblright}{OT1}{'\'}
9642 \DeclareTextSymbol{\textquoteleft}{OT1}{'\'}
9643 \DeclareTextSymbol{\textquoteright}{OT1}{'\'}
9644 \DeclareTextSymbol{\i}{OT1}{16}
9645 \DeclareTextSymbol{\ss}{OT1}{25}

```

For a couple of languages we need the $\text{\LaTeX}$ -control sequence `\scriptsize` to be available. Because plain $\text{\TeX}$ doesn't have such a sophisticated font mechanism as $\text{\LaTeX}$ has, we just `\let` it to `\sevenrm`.

```

9646 \ifx\scriptsize\undefined
9647   \let\scriptsize\sevenrm
9648 \fi

```

And a few more “dummy” definitions.

```

9649 \def\language{english}%
9650 \let\bbl@opt@shorthands\@nnil
9651 \def\bbl@ifshorthand#1#2#3#2{%
9652   \let\bbl@language@opts\@empty
9653   \let\bbl@provide@locale\relax
9654   \ifx\babeloptionstrings\undefined
9655     \let\bbl@opt@strings\@nnil

```

```

9656 \else
9657 \let\bbl@opt@strings\babeloptionstrings
9658 \fi
9659 \def\BabelStringsDefault{generic}
9660 \def\bbl@tempa{normal}
9661 \ifx\babeloptionmath\bbl@tempa
9662 \def\bbl@mathnormal{\noexpand\textormath}
9663 \fi
9664 \def\AfterBabelLanguage#1#2{}
9665 \ifx\BabelModifiers\@undefined\let\BabelModifiers\relax\fi
9666 \let\bbl@afterlang\relax
9667 \def\bbl@opt@safe{BR}
9668 \ifx@uclclist\@undefined\let@uclclist\@empty\fi
9669 \ifx\bbl@trace\@undefined\def\bbl@trace#1{}\fi
9670 \expandafter\newif\csname ifbbl@single\endcsname
9671 \chardef\bbl@bidimode\z@
9672 \let\bbl@ifmetadata\gobble
9673 ({/Emulate LaTeX})

A proxy file:
9674 ({*plain})
9675 \input babel.def
9676 ({/plain})

```

16. Acknowledgements

In the initial stages of the development of babel, Bernd Raichle provided many helpful suggestions and Michel Goossens supplied contributions for many languages. Ideas from Nico Poppelier, Piet van Oostrum and many others have been used. Paul Wackers and Werenfried Spit helped find and repair bugs.

More recently, there are significant contributions by Salim Bou, Ulrike Fischer, Loren Davis and Udi Fogiel.

Barbara Beeton has helped in improving the manual.

There are also many contributors for specific languages, which are mentioned in the respective files. Without them, babel just wouldn't exist.

References

- [1] Huda Smitshuijzen Abifares, *Arabic Typography*, Saqi, 2001.
- [2] Johannes Braams, Victor Eijkhout and Nico Poppelier, *The development of national $\TeX$ styles*, *TUGboat* 10 (1989) #3, pp. 401–406.
- [3] Yannis Haralambous, *Fonts & Encodings*, O'Reilly, 2007.
- [4] Donald E. Knuth, *The $\TeX$ book*, Addison-Wesley, 1986.
- [5] Jukka K. Korpela, *Unicode Explained*, O'Reilly, 2006.
- [6] Leslie Lamport, *$\TeX$, A document preparation System*, Addison-Wesley, 1986.
- [7] Leslie Lamport, in: $\TeX$ hax Digest, Volume 89, #13, 17 February 1989.
- [8] Ken Lunde, *CJKV Information Processing*, O'Reilly, 2nd ed., 2009.
- [9] Edward M. Reingold and Nachum Dershowitz, *Calendrical Calculations: The Ultimate Edition*, Cambridge University Press, 2018.
- [10] Hubert Partl, *German $\TeX$* , *TUGboat* 9 (1988) #1, pp. 70–72.
- [11] Joachim Schrod, *International $\TeX$ is ready to use*, *TUGboat* 11 (1990) #1, pp. 87–90.
- [12] Apostolos Syropoulos, Antonis Tsolomitis and Nick Sofroniu, *Digital typography using $\TeX$* , Springer, 2002, pp. 301–373.
- [13] K.F. Treebus. *Tekstwijzer, een gids voor het grafisch verwerken van tekst*, SDU Uitgeverij ('s-Gravenhage, 1988).